

Lời nói đầu

Cuốn **Giáo trình C++ — Sách giáo viên — Quyển 1** được biên soạn đồng bộ cùng chương trình đào tạo lập trình thuật toán C++ cơ bản iKHEDU, dành riêng cho quý thầy cô giáo và các huấn luyện viên chuyên trách đội tuyển tin học. Tài liệu này cung cấp trọn bộ lời giải chi tiết, mã nguồn C++ tối ưu và hệ thống **phương pháp luận giảng dạy thực chiến chuyên sâu** (ý tưởng và phân tích thuật toán, bảng mô phỏng chạy tay từng bước trên số liệu mẫu thực tế, lưu ý và các bẫy lỗi kinh điển thường gặp) cho các bài toán thực hành thuộc các chủ đề nền tảng: Nền tảng lập trình C++, Thuật toán sắp xếp & kỹ thuật mảng, Kỹ thuật hai con trỏ, Kỹ thuật cửa sổ trượt, Mảng tiền tố & mảng hiệu, cùng Thuật toán tìm kiếm nhị phân.

Mỗi bài toán trong sách giáo viên được cấu trúc thống nhất và tinh gọn qua 4 mục trọng tâm: (1) Ý tưởng & Phân tích thuật toán gắn liền với bản chất toán học; (2) Bảng chạy tay trực quan (Dry Run Table) trên các giá trị số thực tế của mẫu thử (Sample); (3) Lưu ý & các bẫy lỗi kinh điển học sinh thường gặp trong môi trường lập trình; và (4) Mã nguồn C++ tham khảo tối ưu nhất (Fast I/O, Safe Input, 0 std:::). Tài liệu là cẩm nang đặc lực hỗ trợ thầy cô tổ chức các giờ dạy chất lượng cao, phân tích sai sót của học sinh và đồng hành hiệu quả trong từng bài học.

CHƯƠNG 01: NỀN TẢNG LẬP TRÌNH C++

Bài 01: Biến, Kiểu Dữ Liệu, Toán Tử & Nhập Xuất An Toàn

Bài 01 [cppb_10_01_tinh_tong_hai_so]: Tính Tổng Hai Số

Bối cảnh: Trong giờ học lập trình C++ đầu tiên, thầy giáo giao cho mỗi bạn học sinh hai số nguyên và yêu cầu viết chương trình tính tổng của chúng. Bạn An muốn hoàn thành thật nhanh bài toán này để làm quen với việc đọc và in dữ liệu trong C++.

Nhiệm vụ: Cho hai số nguyên a và b . Hãy lập trình tính và in ra tổng $a + b$.

Đầu vào (Input):

- Một dòng duy nhất chứa hai số nguyên a và b ($-10^9 \leq a, b \leq 10^9$), cách nhau bởi một khoảng trắng.

Đầu ra (Output):

- In ra một số nguyên duy nhất là tổng $a + b$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 5	8

Giải thích: Ta có hai số nguyên $a = 3$ và $b = 5$. Tổng của hai số là: $3 + 5 = 8$. Chương trình in ra kết quả: 8.

1. Mục Tiêu Học Tập & Chuẩn Đầu Ra

- Khai báo và sử dụng biến số nguyên.
- Biết cách đọc dữ liệu bằng cin và xuất dữ liệu bằng cout.
- Rèn luyện phản xạ sử dụng long long khi tổng hai số có thể vượt quá 2×10^9 .

2. Phân Tích Đề Bài & Bản Chất Toán Học

- Bản chất:** Tính tổng số học cơ bản $S = a + b$.
- Edge cases:** Các số có thể mang dấu âm (ví dụ: $-10^9 + (-10^9) = -2 \times 10^9$). Nếu dùng int 32-bit có dấu, giá trị lớn nhất là $2^{31} - 1 \approx 2.14 \times 10^9$, khi cộng hai số lớn dễ tiệm cận ngưỡng tràn số (Overflow).

3. Câu Hỏi Gợi Mở Dẫn Dắt

- Trong C++, khi cần lưu số nguyên có giá trị lên tới hàng tỷ, kiểu dữ liệu int có an toàn không?
- Lệnh nào giúp đọc lần lượt hai số cách nhau bởi dấu cách từ bàn phím?

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Đọc trực tiếp a, b , tính tổng và xuất ra màn hình.
- Bất biến:** Mọi phép toán cộng đều thực hiện trên miền giá trị long long. Độ phức tạp thời gian $\mathcal{O}(1)$, bộ nhớ $\mathcal{O}(1)$.

5. Mô Phỏng Từng Bước Trên Sample

Bước	Biến a	Biến b	Biểu thức $a + b$	Kết quả in
1	3	5	$3 + 5 = 8$	8

6. Phân Tích Độ Phức Tạp

- Thời gian: $\mathcal{O}(1)$ (chỉ gồm 1 phép đọc, 1 phép cộng, 1 phép in).
- Không gian bộ nhớ: $\mathcal{O}(1)$ (chỉ sử dụng 2 ô nhớ kiểu long long).

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- **Bẫy 1:** Dùng int thay vì long long khi test đạt cực đại $a = 10^9, b = 10^9$.
- **Bẫy 2:** Quên ngắt dòng $\backslash n$ hoặc viết nhầm toán tử $>>$ của cin thành $<<$.

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long a, b;
    if (!(cin >> a >> b)) return 0;

    cout << a + b << '\n';
    return 0;
}
```

9. Bài Toán Mở Rộng & Chuyển Giao

- Tính hiệu $a - b$ và tích $a \times b$ (lưu ý tích hai số 10^9 chắc chắn tràn long long nếu không kiểm soát).
- Mở rộng tính tổng của một dãy nhiều số nguyên.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long a, b;
    if (!(cin >> a >> b)) return 0;

    cout << a + b << '\n';
    return 0;
}
```

Bài 02 [cppb_10_02_dien_tich_hinh_chu_nhat]: Diện Tích & Chu Vi Hình Chữ Nhật

Bối cảnh: Bác An vừa mua một mảnh đất hình chữ nhật để trồng rau sạch. Bác cần tính chu vi mảnh đất để mua lưới rào xung quanh, đồng thời tính diện tích để mua phân bón hữu cơ phù hợp. Bạn hãy lập trình giúp bác An tính toán nhanh chóng.

Nhiệm vụ: Cho chiều dài a và chiều rộng b của hình chữ nhật. Hãy lập trình tính và in ra chu vi cùng diện tích của hình chữ nhật đó trên cùng một dòng.

Đầu vào (Input):

- Một dòng duy nhất chứa hai số nguyên dương a và b ($1 \leq a, b \leq 10^9$), cách nhau bởi một khoảng trắng.

Đầu ra (Output):

- In ra hai số nguyên cách nhau bởi một khoảng trắng lần lượt là chu vi và diện tích của hình chữ nhật.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 7	22 28

Giải thích: Hình chữ nhật có chiều dài $a = 4$ và chiều rộng $b = 7$:

- Chu vi: $(4 + 7) \times 2 = 22$.
- Diện tích: $4 \times 7 = 28$. Kết quả in ra: 22 28.

1. Mục Tiêu Học Tập & Chuẩn Đầu Ra

- Nắm vững công thức chu vi và diện tích hình học cơ bản.
- Hiểu rõ hiện tượng tràn số khi thực hiện phép nhân hai số lớn.
- Bắt buộc dùng long long cho biến diện tích vì $a \times b$ có thể lên tới 10^{18} .

2. Phân Tích Đề Bài & Bản Chất Toán Học

- Chu vi $P = 2 \times (a + b)$. Khi $a, b \leq 10^9$, $P \leq 4 \times 10^9$ (vượt quá kiểu int có dấu).
- Diện tích $S = a \times b$. Khi $a, b = 10^9$, $S = 10^{18}$, bắt buộc phải dùng kiểu số nguyên 64-bit long long (chứa được tới xấp xỉ 9×10^{18}).

3. Câu Hỏi Gợi Mở Dẫn Dắt

- Nếu khai báo int a, b; long long s = a * b; thì kết quả có bị sai không (Gợi ý: Phép nhân $a \times b$ xảy ra trên miền int trước rồi mới gán sang s).
- Làm thế nào để đảm bảo an toàn tuyệt đối ngay từ khâu khai báo

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Khai báo cả a và b dưới kiểu long long.
- Tính $P = 2 \times (a + b)$ và $S = a \times b$. In ra theo định dạng yêu cầu.

5. Mô Phỏng Từng Bước Trên Sample

Biến a	Biến b	Chu vi $2 \times (a + b)$	Diện tích $a \times b$	Output
4	7	$2 * (4 + 7) = 22$	$4 * 7 = 28$	22 28

6. Phân Tích Độ Phức Tạp

- Thời gian: $\mathcal{O}(1)$.
- Không gian: $\mathcal{O}(1)$.

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- Khai báo a, b là int. Khi $a = 10^6, b = 10^6$, diện tích $a \times b = 10^{12}$ sẽ bị tràn số sinh ra số âm ngẫu nhiên.
- In ngược thứ tự: in diện tích trước chu vi sau.

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long a, b;
    if (!(cin >> a >> b)) return 0;

    long long perimeter = 2 * (a + b);
    long long area = a * b;

    cout << perimeter << ' ' << area << '\n';
    return 0;
}
```

9. Bài Toán Mở Rộng & Chuyển Giao

- Tính diện tích tam giác với công thức Heron hoặc đường cao.
- Mở rộng tính chu vi diện tích hình tròn có số thực π .

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long a, b;
    if (!(cin >> a >> b)) return 0;

    long long perimeter = 2 * (a + b);
    long long area = a * b;
```

```
cout << perimeter << ' ' << area << '\n';
return 0;
}
```

Bài 03 [cppb_10_03_trung_binh_ba_so]: Điểm Trung Bình Ba Môn

Bối cảnh: Kết thúc kỳ thi học kỳ, bạn Bình nhận được điểm số của ba môn thi: Toán, Văn và Ngoại ngữ. Để biết được học lực tổng kết sơ bộ, Bình cần tính điểm trung bình cộng của ba môn này và làm tròn đến đúng hai chữ số thập phân.

Nhiệm vụ: Cho ba số nguyên a, b, c lần lượt là điểm số của ba môn thi. Hãy lập trình tính và in ra điểm trung bình cộng của ba môn, lấy đúng 2 chữ số sau dấu phẩy thập phân.

Đầu vào (Input):

- Một dòng duy nhất chứa ba số nguyên a, b, c ($0 \leq a, b, c \leq 10$), cách nhau bởi một khoảng trắng.

Đầu ra (Output):

- In ra một số thực là điểm trung bình cộng của ba môn thi, định dạng đúng 2 chữ số thập phân.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
8 7 9	8.00

Giải thích: Tổng điểm ba môn: $8 + 7 + 9 = 24$. Điểm trung bình cộng: $24/3 = 8.0$. In ra đúng 2 chữ số thập phân: 8.00.

Ví dụ mẫu (Sample 2):

Đầu vào (Input)	Đầu ra (Output)
7 8 8	7.67

Giải thích: Tổng điểm: $7 + 8 + 8 = 23$. Điểm trung bình: $23/3 = 7.6666...$ làm tròn 2 chữ số thập phân là 7.67.

1. Mục Tiêu Học Tập & Chuẩn Đầu Ra

- Làm quen với kiểu dữ liệu số thực double.
- Nắm vững cú pháp định dạng số thập phân: `cout << fixed << setprecision(k)`.
- Nhận biết bẫy phép chia nguyên giữa hai số nguyên ($\text{sum} / 3$).

2. Phân Tích Đề Bài & Bản Chất Toán Học

- Công thức trung bình: $\text{avg} = \frac{a+b+c}{3}$.
- Phép chia trong C++: Nếu tử số và mẫu số đều là số nguyên, máy tính sẽ thực hiện phép chia lấy phần nguyên (bỏ qua toàn bộ phần thập phân). Do đó, cần ép kiểu số thực bằng cách nhân 1.0 hoặc chia cho 3.0.

3. Câu Hỏi Gọi Mở Dẫn Dắt

- Nếu $a = 7, b = 8, c = 8$, tổng là 23. Trong C++, kết quả của $23 / 3$ bằng bao nhiêu (Bảng 7 chứ không phải 7.666...).
- Làm thế nào để máy tính hiểu ta muốn lấy kết quả dưới dạng số thực

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Đọc 3 số, tính $avg = (a + b + c) / 3.0$.
- Sử dụng cặp cờ fixed và setprecision(2) trong thư viện chuẩn để in ra đúng 2 chữ số sau dấu chấm.

5. Mô Phỏng Từng Bước Trên Sample

Dữ liệu a, b, c	Tổng $a + b + c$	Phép chia thực	Định dạng setprecision(2)
8 7 9	24	$24 / 3.0 = 8.0$	8.00
7 8 8	23	$23 / 3.0 = 7.6666...$	7.67

6. Phân Tích Độ Phức Tạp

- Thời gian: $O(1)$.
- Không gian: $O(1)$.

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- Viết `int sum = a + b + c; double avg = sum / 3;` (sai vì `sum / 3` đã bị mất phần thập phân trước khi gán cho `avg`).
- Quên thêm `fixed`, chỉ dùng `setprecision(2)` sẽ khiến máy in 2 chữ số có nghĩa thay vì 2 chữ số sau dấu phẩy.

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    double a, b, c;
    if (!(cin >> a >> b >> c)) return 0;

    double avg = (a + b + c) / 3.0;

    cout << fixed << setprecision(2) << avg << '\n';
    return 0;
}
```

9. Bài Toán Mở Rộng & Chuyển Giao

- Tính điểm trung bình có hệ số (Toán nhân 2, Văn nhân 2, Ngoại ngữ nhân 1).

- Tính điểm trung bình của một mảng gồm N môn học.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    double a, b, c;
    if (!(cin >> a >> b >> c)) return 0;

    double avg = (a + b + c) / 3.0;

    cout << fixed << setprecision(2) << avg << '\n';
    return 0;
}
```

Bài 04 [cppb_10_04_chu_so_hang_don_vi]: Chữ Số Hàng Đơn Vị & Hàng Chục

Bối cảnh: Trong trò chơi bốc thăm may mắn tại hội chợ xuân, mỗi người tham gia nhận được một tấm vé có in một mã số nguyên dương. Hai chữ số cuối cùng của mã vé (chữ số hàng chục và chữ số hàng đơn vị) sẽ quyết định giải thưởng mà người đó nhận được.

Nhiệm vụ: Cho một số nguyên dương N có ít nhất hai chữ số. Hãy lập trình tìm và in ra chữ số hàng chục cùng chữ số hàng đơn vị của N , cách nhau bởi một khoảng trắng.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($10 \leq N \leq 10^9$).

Đầu ra (Output):

- In ra hai chữ số cách nhau bởi một khoảng trắng lần lượt là chữ số hàng chục và chữ số hàng đơn vị của N .

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
357	5 7

Giải thích: Số $N = 357$:

- Chữ số hàng đơn vị là: $357 \% 10 = 7$.
- Để lấy chữ số hàng chục: chia nguyên bỏ chữ số cuối $357 / 10 = 35$, sau đó lấy phần dư $35 \% 10 = 5$. Kết quả in ra: 5 7.

Ví dụ mẫu (Sample 2):

Đầu vào (Input)	Đầu ra (Output)
80	8 0

Giải thích: Số $N = 80$ có chữ số hàng chục là 8 và chữ số hàng đơn vị là 0.

1. Mục Tiêu Học Tập & Chuẩn Đầu Ra

- Thành thạo toán tử chia lấy phần dư % và chia lấy phần nguyên /.
- Nắm vững kỹ thuật trích xuất các chữ số tận cùng của một số nguyên hệ thập phân.

2. Phân Tích Đề Bài & Bản Chất Toán Học

- Chữ số hàng đơn vị của N luôn là $N \% 10$.
- Muốn lấy chữ số hàng chục: Ta thực hiện phép chia nguyên $N/10$ để gọt bỏ chữ số hàng đơn vị, khi đó chữ số hàng chục cũ trở thành chữ số hàng đơn vị mới, ta chỉ cần lấy $(N/10) \% 10$.

3. Câu Hỏi Gợi Mở Dẫn Dắt

- Muốn lấy chữ số cuối cùng của số 12345 ta làm phép toán gì
- Sau khi bỏ chữ số cuối, số còn lại là gì Làm sao để lấy tiếp chữ số đứng ngay trước nó

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Đọc số N .
- Áp dụng công thức: $don_vi = n \% 10$, $hang_chuc = (n / 10) \% 10$.
- In theo đúng thứ tự: hàng chục rồi đến hàng đơn vị.

5. Mô Phỏng Từng Bước Trên Sample

Giá trị N	$n \% 10$ (Hàng đơn vị)	$n / 10$	$(n / 10) \% 10$ (Hàng chục)	Output
357	7	35	5	5 7
80	0	8	8	8 0

6. Phân Tích Độ Phức Tạp

- Thời gian: $O(1)$.
- Không gian: $O(1)$.

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- In nhầm thứ tự hàng đơn vị trước hàng chục sau.
- Nhầm lẫn giữa phép chia nguyên / và phép chia dư %.

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long n;
    if (!(cin >> n)) return 0;
```

```
int don_vi = n % 10;
int hang_chuc = (n / 10) % 10;

cout << hang_chuc << ' ' << don_vi << '\n';
return 0;
}
```

9. Bài Toán Mở Rộng & Chuyển Giao

- Trích xuất chữ số hàng trăm: $(n / 100) \% 10$.
- Tách toàn bộ các chữ số của một số nguyên bằng vòng lặp while ($n > 0$).

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long n;
    if (!(cin >> n)) return 0;

    int don_vi = n % 10;
    int hang_chuc = (n / 10) % 10;

    cout << hang_chuc << ' ' << don_vi << '\n';
    return 0;
}
```

Bài 05 [cppb_10_05_tong_chu_so]: Tổng Chữ Số Của Số Có Ba Chữ Số

Bối cảnh: Trong môn Toán tư duy, các bạn nhỏ được làm quen với khái niệm “Tổng các chữ số”. Cô giáo đó bạn Nam tính thật nhanh tổng các chữ số của một số nguyên dương có đúng 3 chữ số mà không cần dùng đến vòng lặp.

Nhiệm vụ: Cho một số nguyên dương N có đúng ba chữ số ($100 \leq N \leq 999$). Hãy lập trình tính và in ra tổng của ba chữ số tạo nên số N .

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($100 \leq N \leq 999$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là tổng các chữ số của N .

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
357	15

Giải thích: Số $N = 357$ có 3 chữ số:

- Chữ số hàng trăm: $357/100 = 3$.
- Chữ số hàng chục: $(357/10)\%10 = 5$.

- Chữ số hàng đơn vị: $357 \% 10 = 7$. Tổng ba chữ số là: $3 + 5 + 7 = 15$.

Ví dụ mẫu (Sample 2):

Đầu vào (Input)	Đầu ra (Output)
505	10

Giải thích: Tổng các chữ số: $5 + 0 + 5 = 10$.

1. Mục Tiêu Học Tập & Chuẩn Đầu Ra

- Áp dụng thành thạo phép chia lấy phần nguyên / và chia lấy phần dư % để tách rời các chữ số hàng trăm, hàng chục, hàng đơn vị.
- Rèn luyện kỹ năng kết hợp các biểu thức toán học trong cùng một chương trình.

2. Phân Tích Đề Bài & Bản Chất Toán Học

- Với số có 3 chữ số $N = \overline{abc}$:
- Chữ số hàng trăm $a = N / 100$.
- Chữ số hàng chục $b = (N / 10) \% 10$.
- Chữ số hàng đơn vị $c = N \% 10$.
- Tổng các chữ số cần tìm là $S = a + b + c$.

3. Câu Hỏi Gợi Mở Dẫn Dắt

- Số 357 chia cho 100 được phần nguyên bằng bao nhiêu
- Làm sao để lấy được chữ số ở giữa (hàng chục)

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Đọc số nguyên N .
- Tách từng biến tram, chuc, don_vi theo công thức toán học.
- In ra tổng tram + chuc + don_vi. Độ phức tạp $O(1)$.

5. Mô Phỏng Từng Bước Trên Sample

Số N	Hàng trăm $n / 100$	Hàng chục $(n / 10) \% 10$	Hàng đơn vị $n \% 10$	Tổng chữ số
357	3	5	7	$3 + 5 + 7 = 15$
505	5	0	5	$5 + 0 + 5 = 10$

6. Phân Tích Độ Phức Tạp

- Thời gian: $O(1)$.
- Không gian: $O(1)$.

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- Nhầm công thức hàng chục thành $n \% 100$ (kết quả là số có 2 chữ số chứ không phải 1 chữ số).
- Quên rằng đề bài có định $100 \leq N \leq 999$ nên có thể giải trực tiếp bằng công thức mà chưa cần vòng lặp.

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    int tram = n / 100;
    int chuc = (n / 10) % 10;
    int don_vi = n % 10;

    cout << tram + chuc + don_vi << '\n';
    return 0;
}
```

9. Bài Toán Mở Rộng & Chuyển Giao

- Mở rộng tính tổng chữ số của số có 4 chữ số hoặc số có độ dài bất kỳ bằng vòng lặp (sẽ học ở Bài 02).

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    int tram = n / 100;
    int chuc = (n / 10) % 10;
    int don_vi = n % 10;

    cout << tram + chuc + don_vi << '\n';
    return 0;
}
```

Bài 06 [cppb_10_06_chia_keo_trung_thu]: Chia Kẹo Trung Thu

Bối cảnh: Nhân dịp Tết Trung Thu, cô giáo chủ nhiệm mang đến lớp một gói kẹo gồm N chiếc kẹo để chia đều cho K bạn học sinh trong lớp. Mỗi bạn học sinh đều nhận được số lượng kẹo bằng nhau. Số kẹo còn dư lại (nếu có) sẽ được cho vào hòm quà chung của lớp.

Nhiệm vụ: Cho hai số nguyên dương N và K lần lượt là số lượng kẹo và số bạn học sinh. Hãy lập trình tính và in ra số kẹo mỗi bạn nhận được và số kẹo còn dư vào hòm quà chung.

Đầu vào (Input):

- Một dòng duy nhất chứa hai số nguyên dương N và K ($1 \leq N, K \leq 10^9$), cách nhau bởi một khoảng trắng.

Đầu ra (Output):

- In ra hai số nguyên cách nhau bởi một khoảng trắng: số thứ nhất là số kẹo mỗi bạn nhận được, số thứ hai là số kẹo còn dư.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
23 5	4 3

Giải thích: Có 23 cái kẹo chia cho 5 bạn:

- Mỗi bạn nhận được: $23/5 = 4$ cái kẹo.
- Số kẹo đã chia: $4 \times 5 = 20$ cái.
- Số kẹo còn dư: $23\%5 = 3$ cái. Kết quả in ra: 4 3.

Ví dụ mẫu (Sample 2):

Đầu vào (Input)	Đầu ra (Output)
30 6	5 0

Giải thích: 30 chia hết cho 6, mỗi bạn nhận đúng 5 cái và không dư cái nào.

1. Mục Tiêu Học Tập & Chuẩn Đầu Ra

- Áp dụng ý nghĩa thực tế của phép chia nguyên / (thương số) và phép chia dư % (phần dư).
- Nhận thức về điều kiện dữ liệu lên tới 10^9 cần dùng kiểu long long.

2. Phân Tích Đề Bài & Bản Chất Toán Học

- Định lý chia có dư: Với mọi số nguyên dương N, K , luôn tồn tại duy nhất cặp số q, r sao cho:

$$N = q \times K + r \quad (0 \leq r < K)$$

- Trong đó $q = N/K$ là số kẹo mỗi bạn nhận được, $r = N\%K$ là số kẹo dư.

3. Câu Hỏi Gợi Mở Dẫn Dắt

- Khi chia đều cho K bạn, số kẹo dư tối đa có thể là bao nhiêu (Luôn nhỏ hơn K).
- Phép toán nào trong C++ cho ta thương số và phép toán nào cho ta số dư

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Đọc N, K kiểu long long.
- Tính $each = n / k$ và $rem = n \% k$. In ra kết quả.

5. Mô Phỏng Từng Bước Trên Sample

Tổng số kẹo N	Số học sinh K	Thương N/K	Số dư $N\%K$	Kết quả
23	5	4	3	4 3
30	6	5	0	5 0

6. Phân Tích Độ Phức Tạp

- Thời gian: $\mathcal{O}(1)$.
- Không gian: $\mathcal{O}(1)$.

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- Chia cho 0 (trong bài này đề bài cho $K \geq 1$, nhưng cần luôn cảnh báo học sinh không bao giờ chia cho số 0).
- Nhầm thứ tự in: in số dư trước thương số sau.

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long n, k;
    if (!(cin >> n >> k)) return 0;

    long long each = n / k;
    long long rem = n % k;

    cout << each << ' ' << rem << '\n';
    return 0;
}
```

9. Bài Toán Mở Rộng & Chuyển Giao

- Bài toán tìm số chuyến xe tối thiểu để chở hết N người biết mỗi xe chở tối đa K người (công thức làm tròn lên: $(n + k - 1) / k$).

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long n, k;
    if (!(cin >> n >> k)) return 0;

    long long each = n / k;
    long long rem = n % k;
```

```
cout << each << ' ' << rem << '\n';
return 0;
}
```

Bài 07 [cppb_l0_07_doi_thoi_gian_giay]: Đổi Giây Sang Giờ Phút Giây

Bối cảnh: Trong một giải chạy marathon quốc tế, thiết bị đo thời gian tự động của ban tổ chức ghi nhận thành tích của vận động viên dưới dạng tổng số giây kể từ thời điểm xuất phát. Để hiển thị lên bảng điện tử cho khán giả dễ theo dõi, ban tổ chức cần chuyển đổi tổng số giây này thành định dạng giờ, phút và giây.

Nhiệm vụ: Cho một số nguyên không âm T là tổng số giây. Hãy lập trình đổi T thành h giờ, m phút và s giây, in ra theo định dạng h:m:s.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên không âm T ($0 \leq T \leq 10^9$).

Đầu ra (Output):

- In ra một dòng duy nhất theo định dạng h:m:s.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3665	1:1:5

Giải thích: Ta có 1 giờ = 3600 giây, 1 phút = 60 giây:

- Số giờ: $3665/3600 = 1$ giờ.
- Số giây còn lại sau khi trừ giờ: $3665\%3600 = 65$ giây.
- Số phút: $65/60 = 1$ phút.
- Số giây còn lại: $65\%60 = 5$ giây. Định dạng kết quả: 1:1:5.

Ví dụ mẫu (Sample 2):

Đầu vào (Input)	Đầu ra (Output)
125	0:2:5

Giải thích: 125 giây gồm 0 giờ, 2 phút và 5 giây. Kết quả: 0:2:5.

1. Mục Tiêu Học Tập & Chuẩn Đầu Ra

- Vận dụng phối hợp nhiều lần phép chia nguyên và chia dư để đổi đơn vị đo lường.
- Xuất dữ liệu kèm các ký tự đặc biệt như dấu hai chấm :.

2. Phân Tích Đề Bài & Bản Chất Toán Học

- 1 giờ = 3600 giây.
- Số giờ $h = T/3600$.
- Phần giây dư chưa đủ 1 giờ là $R = T\%3600$.

- Từ R giây dư, số phút là $m = R/60$.
- Số giây còn lại sau khi trừ phút là $s = R\%60$.

3. Câu Hỏi Gợi Mở Dẫn Dắt

- Trong 1 giờ có bao nhiêu giây
- Khi tính xong số giờ, ta dùng phép toán nào để biết còn dư bao nhiêu giây

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Đọc T kiểu long long (do T có thể lên tới 10^9).
- Lần lượt tính h, m, s và in theo định dạng h:m:s.

5. Mô Phỏng Từng Bước Trên Sample

T	$h = T/3600$	$R = T\%3600$	$m = R/60$	$s = R\%60$	Định dạng in
3665	1	65	1	5	1:1:5
125	0	125	2	5	0:2:5

6. Phân Tích Độ Phức Tạp

- Thời gian: $\mathcal{O}(1)$.
- Không gian: $\mathcal{O}(1)$.

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- Tính số phút bằng $t / 60$ (sai vì chưa trừ đi số giờ đã quy đổi).
- Dùng kiểu int cho biến $T = 10^9$ (mặc dù 10^9 vừa vặn int, nhưng việc dùng long long là thói quen an toàn).

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long t;
    if (!(cin >> t)) return 0;

    long long h = t / 3600;
    long long rem = t % 3600;
    long long m = rem / 60;
    long long s = rem % 60;

    cout << h << ':' << m << ':' << s << '\n';
    return 0;
}
```

9. Bài Toán Mở Rộng & Chuyển Giao

- In định dạng đồng hồ điện tử đủ 2 chữ số (ví dụ 01:01:05) bằng cách thêm setw(2) và setfill('0').

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long t;
    if (!(cin >> t)) return 0;

    long long h = t / 3600;
    long long rem = t % 3600;
    long long m = rem / 60;
    long long s = rem % 60;

    cout << h << ':' << m << ':' << s << '\n';
    return 0;
}
```

Bài 08 [cppb_10_08_mua_vo_khuyen_mai]: Mua Vở Có Khuyến Mãi

Bối cảnh: Đầu năm học mới, một nhà sách tổ chức chương trình khuyến mãi đặc biệt cho học sinh: Mỗi quyển vở có giá niêm yết là P đồng. Cứ mỗi khi khách hàng mua 5 quyển vở thì sẽ được tặng thêm 1 quyển vở hoàn toàn miễn phí. Bạn Nam cần có đúng N quyển vở để phục vụ việc ghi chép các môn học. Bạn hãy tính xem Nam cần trả ít nhất bao nhiêu tiền để có đủ N quyển vở.

Nhiệm vụ: Cho hai số nguyên dương N (số quyển vở Nam cần có) và P (giá niêm yết một quyển vở). Hãy lập trình tính và in ra số tiền tối thiểu mà Nam phải thanh toán.

Đầu vào (Input):

- Một dòng duy nhất chứa hai số nguyên dương N và P ($1 \leq N \leq 10^9, 1 \leq P \leq 10^6$), cách nhau bởi một khoảng trắng.

Đầu ra (Output):

- In ra một số nguyên duy nhất là số tiền tối thiểu Nam cần thanh toán.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
13 5000	55000

Giải thích: Nam cần có 13 quyển vở:

- Cứ mỗi nhóm 6 quyển (gồm mua 5 tặng 1), Nam chỉ cần trả tiền cho 5 quyển.
- Với 13 quyển, Nam có thể chia thành: 2 nhóm 6 quyển (được 12 quyển, trong đó mua 10 tặng 2) và còn thiếu 1 quyển cần mua thêm.
- Tổng số quyển Nam phải trả tiền là: $2 \times 5 + 1 = 11$ quyển.

- Số tiền thanh toán: $11 \times 5000 = 55000$ đồng.

Ví dụ mẫu (Sample 2):

Đầu vào (Input)	Đầu ra (Output)
5 6000	30000

Giải thích: Nam mua đúng 5 quyền (chưa đủ 6 quyền để nhận quà tặng kèm lúc lấy), cần thanh toán $5 \times 6000 = 30000$ đồng (Nam sẽ nhận thêm 1 quyền quà tặng là 6 quyền, vẫn thỏa mãn có đủ ít nhất 5 quyền).

1. Mục Tiêu Học Tập & Chuẩn Đầu Ra

- Áp dụng tư duy gom nhóm số học (Block/Group division) bằng phép chia / và chia dư %.
- Nhận thức về việc sử dụng long long khi giá trị tiền bạc đạt tới 10^{15} .

2. Phân Tích Đề Bài & Bản Chất Toán Học

- Mỗi khối 6 quyền (mua 5 tặng 1) chỉ cần trả tiền 5 quyền.
- Số nhóm 6 quyền đầy đủ là $g = N/6$. Số quyền lẻ còn lại là $r = N\%6$.
- Trong phần lẻ r quyền, số quyền phải trả tiền là $\min(r, 5)$ (nếu $r = 5$, ta mua 5 quyền là đủ).
- Tổng số quyền cần trả tiền: $to_buy = g \times 5 + \min(r, 5)$.
- Tổng tiền: $total = to_buy \times P$. Với $N \leq 10^9, P \leq 10^6$, tổng tiền có thể lên tới 10^{15} , bắt buộc dùng long long.

3. Câu Hỏi Gợi Mở Dẫn Dắt

- Cứ mỗi lần nhận được 6 quyền trên tay, thực chất ta chỉ phải bỏ tiền mua bao nhiêu quyền
- Phép toán nào giúp tìm số lượng nhóm 6 quyền hoàn chỉnh

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Phân tích số quyền thành các nhóm 6 và phần dư.
- Tính trực tiếp trong $O(1)$ bằng công thức số học, không cần dùng vòng lặp.

5. Mô Phỏng Từng Bước Trên Sample

N	P	Nhóm $g = N/6$	Dư $r = N\%6$	Số quyền mua: $g \times 5 + \min(r, 5)$	Tổng tiền thanh toán
13	5000	2	1	$2 \times 5 + 1 = 11$	$11 \times 5000 = 55000$
5	6000	0	5	$0 \times 5 + 5 = 5$	$5 \times 6000 = 30000$

6. Phân Tích Độ Phức Tạp

- Thời gian: $\mathcal{O}(1)$.
- Không gian: $\mathcal{O}(1)$.

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- Dùng int cho biến kết quả tiền, dẫn đến tràn số nghiêm trọng khi nhân với $P = 10^6$.
- Chia nhầm cho 5 thay vì chia cho 6 (vì mỗi nhóm ưu đãi trọn gói gồm $5 + 1 = 6$ quyền).

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long n, p;
    if (!(cin >> n >> p)) return 0;

    long long groups = n / 6;
    long long rem = n % 6;

    long long to_buy = groups * 5 + min(rem, 5LL);
    long long total_cost = to_buy * p;

    cout << total_cost << '\n';
    return 0;
}
```

9. Bài Toán Mở Rộng & Chuyển Giao

- Bài toán khuyến mãi mua K tặng M tổng quát.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long n, p;
    if (!(cin >> n >> p)) return 0;

    long long groups = n / 6;
    long long rem = n % 6;

    long long to_buy = groups * 5 + min(rem, 5LL);
    long long total_cost = to_buy * p;

    cout << total_cost << '\n';
    return 0;
}
```

Bài 02: Cấu Trúc Rẽ Nhánh & Cấu Trúc Vòng Lặp

Bài 01 [cppb_10_09_so_lon_hon_trong_hai_so]: Tìm Số Lớn Hơn Trong Hai Số

Bối cảnh: Trong cuộc thi đo chiều cao tại phòng y tế học đường, bác sĩ cần so sánh chiều cao của hai bạn An và Bình để ghi nhận bạn nào có chiều cao nhỉnh hơn. Nếu hai bạn cao bằng nhau thì bác sĩ ghi nhận kết quả là bằng nhau.

Nhiệm vụ: Cho hai số nguyên a và b . Hãy lập trình so sánh hai số:

- Nếu $a > b$, in ra giá trị của a .
- Nếu $b > a$, in ra giá trị của b .
- Nếu $a = b$, in ra chuỗi BANG NHAU.

Đầu vào (Input):

- Một dòng duy nhất chứa hai số nguyên a và b ($-10^9 \leq a, b \leq 10^9$), cách nhau bởi một khoảng trắng.

Đầu ra (Output):

- In ra một dòng duy nhất theo yêu cầu của đề bài.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
7 12	12

Giải thích: Vì $12 > 7$ nên chương trình in ra số lớn hơn là 12.

Ví dụ mẫu (Sample 2):

Đầu vào (Input)	Đầu ra (Output)
9 9	BANG NHAU

Giải thích: Vì hai số bằng nhau nên chương trình in ra BANG NHAU.

1. Mục Tiêu Học Tập & Chuẩn Đầu Ra

- Làm quen với cấu trúc rẽ nhánh cơ bản if, else if, else.
- Hiểu và sử dụng các toán tử so sánh $>$, $<$, $==$.
- Rèn luyện kỹ năng xử lý trường hợp đặc biệt (hai số bằng nhau).

2. Phân Tích Đề Bài & Bản Chất Toán Học

- Có 3 trường hợp độc lập loại trừ nhau (trichotomy property): $a > b$, $a < b$, hoặc $a = b$.
- Cấu trúc if - else if - else đảm bảo chỉ có đúng một nhánh được thực thi.

3. Câu Hỏi Gợi Mở Dẫn Dắt

- Khi so sánh hai số bất kỳ, có bao nhiêu trường hợp có thể xảy ra
- Trong C++, toán tử so sánh bằng được viết như thế nào (Chú ý $==$ chứ không phải $=$).

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Đọc a, b . Dùng cấu trúc rẽ nhánh 3 trường hợp để in kết quả tương ứng.

5. Mô Phỏng Từng Bước Trên Sample

a	b	Điều kiện thỏa mãn	In ra màn hình
7	12	$b > a$ ($12 > 7$)	12
9	9	$a == b$ ($9 == 9$)	BANG NHAU

6. Phân Tích Độ Phức Tạp

- Thời gian: $\mathcal{O}(1)$.
- Không gian: $\mathcal{O}(1)$.

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- Viết nhầm `if (a = b)` thay vì `if (a == b)`.
- Thiếu dấu ngoặc nhọn hoặc in sai chính tả chuỗi BANG NHAU.

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long a, b;
    if (!(cin >> a >> b)) return 0;

    if (a > b) {
        cout << a << '\n';
    } else if (b > a) {
        cout << b << '\n';
    } else {
        cout << "BANG NHAU\n";
    }

    return 0;
}
```

9. Bài Toán Mở Rộng & Chuyển Giao

- Tìm số lớn nhất trong ba số a, b, c .

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```

long long a, b;
if (!(cin >> a >> b)) return 0;

if (a > b) {
    cout << a << '\n';
} else if (b > a) {
    cout << b << '\n';
} else {
    cout << "BANG NHAU\n";
}

return 0;
}

```

Bài 02 [cppb_10_10_kiem_tra_so_chan_le]: Kiểm Tra Số Chẵn Lẻ

Bối cảnh: Trong các bài toán số học, tính chẵn lẻ (parity) là một tính chất căn bản. Một số nguyên chia hết cho 2 được gọi là số chẵn, ngược lại được gọi là số lẻ. Bạn hãy lập trình kiểm tra tính chẵn lẻ của một số nguyên được nhập từ bàn phím.

Nhiệm vụ: Cho một số nguyên N . Hãy lập trình kiểm tra:

- Nếu N là số chẵn, in ra CHAN.
- Nếu N là số lẻ, in ra LE.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên N ($-10^{18} \leq N \leq 10^{18}$).

Đầu ra (Output):

- In ra một dòng duy nhất chữ CHAN hoặc LE.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
8	CHAN

Giải thích: Số 8 chia hết cho 2 (phần dư bằng 0) nên là số chẵn. In ra CHAN.

Ví dụ mẫu (Sample 2):

Đầu vào (Input)	Đầu ra (Output)
-7	LE

Giải thích: Số -7 không chia hết cho 2 nên là số lẻ. In ra LE.

1. Mục Tiêu Học Tập & Chuẩn Đầu Ra

- Áp dụng toán tử chia dư $\%$ 2 kết hợp với cấu trúc if - else.
- Xử lý đúng số âm: trong C++, số âm lẻ $\%$ 2 có thể ra -1, nhưng điều kiện $n \% 2 == 0$ luôn đúng cho mọi số chẵn (dương, âm và số 0).

2. Phân Tích Đề Bài & Bản Chất Toán Học

- Số chẵn là số có dạng $2k$ ($k \in \mathbb{Z}$). Số chẵn luôn có phần dư bằng 0 khi chia cho 2.

- Viết điều kiện $(n \% 2 == 0)$ là phương pháp an toàn nhất cho cả số âm lẫn số dương.

3. Câu Hỏi Gọi Mở Dẫn Dắt

- Số 0 là số chẵn hay số lẻ ($0 \% 2 == 0$ nên là số chẵn).
- Nếu viết $\text{if } (n \% 2 == 1)$ thì với số âm như -7 điều kiện có đúng không (Không, vì $-7 \% 2$ bằng -1).

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Đọc N kiểu long long.
- Kiểm tra $n \% 2 == 0$. In CHAN nếu đúng, ngược lại in LE.

5. Mô Phỏng Từng Bước Trên Sample

N	$n \% 2$	So sánh với 0	Kết quả in
8	0	$0 == 0$ (Đúng)	CHAN
-7	-1	$-1 == 0$ (Sai)	LE

6. Phân Tích Độ Phức Tạp

- Thời gian: $O(1)$.
- Không gian: $O(1)$.

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- Bẫy số âm khi dùng $n \% 2 == 1$ để kiểm tra số lẻ.
- Dùng int khi N lên tới 10^{18} .

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long n;
    if (!(cin >> n)) return 0;

    if (n % 2 == 0) {
        cout << "CHAN\n";
    } else {
        cout << "LE\n";
    }

    return 0;
}
```

9. Bài Toán Mở Rộng & Chuyển Giao

- Kiểm tra tính chia hết cho 3, 5, 10.

- Kiểm tra số có chữ số tận cùng là chữ số chẵn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long n;
    if (!(cin >> n)) return 0;

    if (n % 2 == 0) {
        cout << "CHAN\n";
    } else {
        cout << "LE\n";
    }

    return 0;
}
```

Bài 03 [cppb_l0_11_kiem_tra_ba_canh_tam_giac]: Kiểm Tra Ba Cạnh Tam Giác

Bối cảnh: Trong giờ thực hành môn Hình học, thầy giáo phát cho mỗi nhóm học sinh ba thanh gỗ có độ dài lần lượt là a, b, c . Các bạn cần kiểm tra xem ba thanh gỗ này có thể ghép lại thành một hình tam giác hay không.

Nhiệm vụ: Cho ba số nguyên dương a, b, c . Hãy lập trình kiểm tra xem a, b, c có thể là độ dài ba cạnh của một tam giác hợp lệ hay không:

- Nếu tạo thành tam giác, in ra YES.
- Nếu không tạo thành tam giác, in ra NO.

Đầu vào (Input):

- Một dòng duy nhất chứa ba số nguyên dương a, b, c ($1 \leq a, b, c \leq 10^9$), cách nhau bởi một khoảng trắng.

Đầu ra (Output):

- In ra một dòng duy nhất chữ YES hoặc NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 4 5	YES

Giải thích: Ba số 3,4,5 thỏa mãn bất đẳng thức tam giác:

- $3 + 4 = 7 > 5$
- $3 + 5 = 8 > 4$
- $4 + 5 = 9 > 3$ Do đó tạo thành tam giác hợp lệ. In ra YES.

Ví dụ mẫu (Sample 2):

Đầu vào (Input)	Đầu ra (Output)
1 2 5	NO

Giải thích: Ta có $1 + 2 = 3 < 5$, tổng hai cạnh không lớn hơn cạnh còn lại nên không thể tạo thành tam giác. In ra NO.

1. Mục Tiêu Học Tập & Chuẩn Đầu Ra

- Áp dụng toán tử logic && (VÀ) để kết hợp đồng thời nhiều điều kiện.
- Nắm vững bất đẳng thức tam giác trong hình học phẳng.
- Phòng tránh tràn số khi cộng hai số 10^9 bằng cách sử dụng kiểu long long.

2. Phân Tích Đề Bài & Bản Chất Toán Học

- Định lý bất đẳng thức tam giác: Ba đoạn thẳng độ dài dương a, b, c tạo thành tam giác khi và chỉ khi tổng độ dài hai cạnh bất kỳ luôn lớn hơn độ dài cạnh còn lại:

$$\begin{cases} a + b > c \\ a + c > b \\ b + c > a \end{cases}$$

- Khi $a, b \leq 10^9$, tổng $a + b$ có thể lên tới 2×10^9 , nếu dùng int có nguy cơ tràn số khi cộng, do đó cần dùng long long.

3. Câu Hỏi Gợi Mở Dẫn Dắt

- Để ba đoạn thẳng ghép thành tam giác, điều kiện về tổng hai cạnh bất kỳ phải như thế nào so với cạnh thứ ba
- Trong C++, làm sao để biểu diễn việc cả 3 điều kiện đều phải đồng thời xảy ra (Toán tử &&).

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Đọc a, b, c kiểu long long.
- Kiểm tra $(a + b > c \ \&\& \ a + c > b \ \&\& \ b + c > a)$. In YES hoặc NO.

5. Mô Phỏng Từng Bước Trên Sample

a, b, c	$a + b > c$	$a + c > b$	$b + c > a$	Kết luận logic	Kết quả in
3 4 5	$7 > 5$ (Đúng)	$8 > 4$ (Đúng)	$9 > 3$ (Đúng)	Đúng cả 3	YES
1 2 5	$3 > 5$ (Sai)	—	—	Sai ít nhất 1	NO

6. Phân Tích Độ Phức Tạp

- Thời gian: $\mathcal{O}(1)$.
- Không gian: $\mathcal{O}(1)$.

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- Dùng toán tử || thay vì &&.

- Viết điều kiện $\geq$ thay vì $>$ (trường hợp suy biến $a + b = c$ tạo thành một đoạn thẳng chứ không phải tam giác).

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long a, b, c;
    if (!(cin >> a >> b >> c)) return 0;

    if (a + b > c && a + c > b && b + c > a) {
        cout << "YES\n";
    } else {
        cout << "NO\n";
    }

    return 0;
}
```

9. Bài Toán Mở Rộng & Chuyển Giao

- Phân loại tam giác: tam giác đều ($a = b = c$), tam giác cân ($a = b \vee b = c \vee c = a$), tam giác vuông (Định lý Pytago $a^2 + b^2 = c^2$).

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long a, b, c;
    if (!(cin >> a >> b >> c)) return 0;

    if (a + b > c && a + c > b && b + c > a) {
        cout << "YES\n";
    } else {
        cout << "NO\n";
    }

    return 0;
}
```

Bài 04 [cppb_10_12_xep_loai_hoc_luc]: Xếp Loại Học Sinh & Kiểm Tra Điểm

Bối cảnh: Cuối năm học, nhà trường tiến hành phân loại danh hiệu cho học sinh dựa trên điểm trung bình tổng kết môn học S . Điểm số hợp lệ phải nằm trong thang điểm từ 0.0 đến 10.0. Bạn hãy lập trình tự động hóa việc kiểm tra điểm và xếp loại học sinh.

Nhiệm vụ: Cho một số thực S là điểm trung bình của học sinh:

- Nếu $S < 0.0$ hoặc $S > 10.0$: In ra DIEM KHONG HOP LE.

- Ngược lại, xếp loại theo thang điểm:
- Nếu $8.0 \leq S \leq 10.0$: In ra GIOI.
- Nếu $6.5 \leq S < 8.0$: In ra KHA.
- Nếu $5.0 \leq S < 6.5$: In ra TRUNG BINH.
- Nếu $S < 5.0$: In ra CHUA DAT.

Đầu vào (Input):

- Một dòng duy nhất chứa số thực S ($-10.0 \leq S \leq 20.0$).

Đầu ra (Output):

- In ra một dòng duy nhất kết quả xếp loại tương ứng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
8.5	GIOI

Giải thích: Điểm $S = 8.5$ nằm trong khoảng $[8.0, 10.0]$ nên đạt loại GIOI.

Ví dụ mẫu (Sample 2):

Đầu vào (Input)	Đầu ra (Output)
11.5	DIEM KHONG HOP LE

Giải thích: Điểm $11.5 > 10.0$ vượt quá thang điểm quy định, in ra DIEM KHONG HOP LE.

1. Mục Tiêu Học Tập & Chuẩn Đầu Ra

- Thành thạo chuỗi câu lệnh rẽ nhánh nhiều tầng if - else if - else.
- Hiểu nguyên lý kiểm tra tính hợp lệ trước (Validation First) để loại trừ sớm dữ liệu sai.
- Nhận thức về thứ tự các điều kiện trong chuỗi rẽ nhánh loại trừ.

2. Phân Tích Đề Bài & Bản Chất Toán Học

- Kiểm tra tính hợp lệ: $S \in [0.0, 10.0]$. Nếu không thỏa mãn, in thông báo lỗi và kết thúc ngay.
- Khi điểm hợp lệ, do đã qua nhánh $s \geq 8.0$, nên ở nhánh else if ($s \geq 6.5$) ta ngầm hiểu rằng $S < 8.0$ mà không cần viết thêm $s < 8.0 \ \&\& \ s \geq 6.5$. Đây là kỹ thuật viết code gọn gàng và chuẩn mực.

3. Câu Hỏi Gợi Mở Dẫn Dắt

- Nếu ta đặt điều kiện else if ($s \geq 5.0$) lên trước else if ($s \geq 8.0$) thì điều gì sẽ xảy ra khi một bạn được 9.0 điểm
- Vì sao bước kiểm tra điểm hợp lệ nên được đưa lên đầu tiên

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Đọc số thực S .
- Chuỗi rẽ nhánh từ thang điểm cao xuống thấp. In kết quả.

5. Mô Phỏng Từng Bước Trên Sample

Điểm S	Kiểm tra hợp lệ $s < 0 \parallel s > 10$	Nhánh thỏa mãn	Kết quả in
8.5	Sai (Điểm hợp lệ)	$s \geq 8.0$	GIOI
11.5	Đúng (Vượt ngưỡng 10)	Nhánh đầu tiên	DIEM KHONG HOP LE

6. Phân Tích Độ Phức Tạp

- Thời gian: $\mathcal{O}(1)$.
- Không gian: $\mathcal{O}(1)$.

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- Đảo lộn thứ tự các nhánh so sánh điểm khiến học sinh giỏi bị phân loại thành trung bình.
- Bỏ sót trường hợp biên đúng bằng ngưỡng 8.0 hoặc 5.0.

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    double s;
    if (!(cin >> s)) return 0;

    if (s < 0.0 || s > 10.0) {
        cout << "DIEM KHONG HOP LE\n";
    } else if (s >= 8.0) {
        cout << "GIOI\n";
    } else if (s >= 6.5) {
        cout << "KHA\n";
    } else if (s >= 5.0) {
        cout << "TRUNG BINH\n";
    } else {
        cout << "CHUA DAT\n";
    }

    return 0;
}
```

9. Bài Toán Mở Rộng & Chuyển Giao

- Tính cước dịch vụ Internet hoặc biểu phí điện thoại theo các mức tiêu dùng.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```
double s;
if (!(cin >> s)) return 0;

if (s < 0.0 || s > 10.0) {
    cout << "DIEM KHONG HOP LE\n";
} else if (s >= 8.0) {
    cout << "GIOI\n";
} else if (s >= 6.5) {
    cout << "KHA\n";
} else if (s >= 5.0) {
    cout << "TRUNG BINH\n";
} else {
    cout << "CHUA DAT\n";
}

return 0;
}
```

Bài 05 [cppb_l0_13_tong_tu_1_den_n]: Bảng Nhân & Tính Tổng Từ 1 Đến N

Bối cảnh: Trong tuần lễ rèn luyện tính toán nhanh, các bạn học sinh lớp 6 được yêu cầu tính tổng dãy số nguyên liên tiếp $S = 1 + 2 + 3 + \dots + N$. Bạn An muốn viết một chương trình C++ sử dụng vòng lặp để vừa kiểm tra kết quả tính tay, vừa hiểu rõ cách máy tính thực hiện các phép tính tích lũy lặp đi lặp lại.

Nhiệm vụ: Cho một số nguyên dương N . Hãy lập trình sử dụng vòng lặp tính và in ra tổng của tất cả các số nguyên từ 1 đến N .

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 10^6$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là tổng $S = 1 + 2 + \dots + N$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5	15

Giải thích: Tổng các số từ 1 đến 5 là: $1 + 2 + 3 + 4 + 5 = 15$. Chương trình in ra 15.

Ví dụ mẫu (Sample 2):

Đầu vào (Input)	Đầu ra (Output)
100	5050

Giải thích: Tổng các số từ 1 đến 100 là 5050.

1. Mục Tiêu Học Tập & Chuẩn Đầu Ra

- Nắm vững cú pháp vòng lặp for (khởi tạo; điều kiện; bước nhảy).
- Thực hành mẫu tích lũy (Accumulation pattern) cơ bản nhất: cộng dồn vào biến sum.
- Nhận biết nguy cơ tràn số khi $N = 10^6$ thì $S \approx \frac{10^6 \times 10^6}{2} = 5 \times 10^{11}$, vượt quá giới hạn int.

2. Phân Tích Đề Bài & Bản Chất Toán Học

- Công thức Gauss: $S = \frac{N(N+1)}{2}$.
- Với bài tập này, mục tiêu sư phạm là rèn luyện cho học sinh cách điều khiển vòng lặp for và biến tích lũy $sum = 0$.
- Khởi tạo long long $sum = 0$; là bắt buộc vì nếu để rác bộ nhớ thì kết quả sẽ sai lệch hoàn toàn.

3. Câu Hỏi Gợi Mở Dẫn Dắt

- Trước khi cộng, biến sum phải có giá trị ban đầu là bao nhiêu
- Vòng lặp bắt đầu từ số mấy và kết thúc ở số mấy
- Tại sao $N = 10^6$ lại cần long long cho biến sum

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Đọc N . Khởi tạo long long $sum = 0$;
- Vòng lặp for ($int\ i = 1; i \leq n; i++$) $sum += i$;
- Xuất giá trị sum.

5. Mô Phỏng Từng Bước Trên Sample ($N = 5$)

Bước lặp i	Thao tác cộng dồn	Giá trị biến sum
Khởi tạo	—	0
$i = 1$	$sum = 0 + 1$	1
$i = 2$	$sum = 1 + 2$	3
$i = 3$	$sum = 3 + 3$	6
$i = 4$	$sum = 6 + 4$	10
$i = 5$	$sum = 10 + 5$	15

6. Phân Tích Độ Phức Tạp

- Thời gian: $\mathcal{O}(N)$ (với $N \leq 10^6$, máy tính chạy trong khoảng 0.005s, hoàn toàn dưới giới hạn 1.0s).
- Không gian: $\mathcal{O}(1)$.

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- Khai báo $int\ sum = 0$; dẫn đến tràn số thành số âm khi $N = 10^6$.
- Quên khởi tạo $sum = 0$, để biến chứa giá trị rác ngẫu nhiên của bộ nhớ RAM.
- Điều kiện dừng viết $i < n$ thay vì $i \leq n$ (bỏ sót số cuối cùng).

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long n;
    if (!(cin >> n)) return 0;

    long long sum = 0;
    for (int i = 1; i <= n; i++) {
        sum += i;
    }

    cout << sum << '\n';
    return 0;
}
```

9. Bài Toán Mở Rộng & Chuyển Giao

- Mở rộng tính tổng bình phương: $S = 1^2 + 2^2 + \dots + N^2$.
- Tính tổng các số lẻ trong đoạn từ 1 đến N .

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long n;
    if (!(cin >> n)) return 0;

    long long sum = 0;
    for (int i = 1; i <= n; i++) {
        sum += i;
    }

    cout << sum << '\n';
    return 0;
}
```

Bài 06 [cppb_l0_14_dem_uoc_so_nguyen_duong]: Đếm Số Lượng Ước Số Nguyên Dương

Bối cảnh: Trong các bài kiểm tra toán học, việc tìm và đếm các ước số của một số nguyên dương N là nền tảng để kiểm tra số nguyên tố và số hoàn hảo. Bạn hãy lập trình đếm xem số nguyên dương N có bao nhiêu ước số nguyên dương.

Nhiệm vụ: Cho một số nguyên dương N ($1 \leq N \leq 10^5$). Hãy lập trình đếm và in ra số lượng các ước số nguyên dương của N .

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 10^5$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là số lượng ước số nguyên dương của N .

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
12	6

Giải thích: Các ước số nguyên dương của 12 là: 1,2,3,4,6,12. Tổng cộng có 6 ước số.

Ví dụ mẫu (Sample 2):

Đầu vào (Input)	Đầu ra (Output)
7	2

Giải thích: Số 7 là số nguyên tố, chỉ có đúng 2 ước là 1 và 7.

1. Mục Tiêu Học Tập & Chuẩn Đầu Ra

- Thành thạo mẫu tích lũy dạng Đếm (cnt++) kết hợp cấu trúc if bên trong vòng lặp for.
- Nắm vững định nghĩa ước số: i là ước của N khi và chỉ khi $N \% i == 0$.

2. Phân Tích Đề Bài & Bản Chất Toán Học

- Với $N \leq 10^5$, cách tiếp cận tự nhiên và cơ bản nhất cho học sinh mới bắt đầu là duyệt toàn bộ các số i từ 1 đến N , kiểm tra nếu $N \% i == 0$ thì tăng biến đếm lên 1 đơn vị.
- Khởi tạo biến đếm ban đầu cnt = 0.

3. Câu Hỏi Gợi Mở Dẫn Dắt

- Một số i được gọi là ước của N khi phép chia N cho i có phần dư bằng mấy
- Khi tìm thấy một ước số, ta làm gì với biến đếm

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Duyệt vòng lặp i từ 1 đến N .
- Nếu $n \% i == 0$, thực hiện cnt++. Sau vòng lặp in ra cnt.

5. Mô Phỏng Từng Bước Trên Sample ($N = 12$)

i	Kiểm tra $12 \% i == 0$	Thao tác biến đếm	Giá trị cnt
1	$12 \% 1 == 0$ (Đúng)	cnt++	1
2	$12 \% 2 == 0$ (Đúng)	cnt++	2
3	$12 \% 3 == 0$ (Đúng)	cnt++	3
4	$12 \% 4 == 0$ (Đúng)	cnt++	4

5	$12\%5 \neq 0$ (Sai)	Bỏ qua	4
6	$12\%6 == 0$ (Đúng)	cnt++	5
7..11	Không chia hết	Bỏ qua	5
12	$12\%12 == 0$ (Đúng)	cnt++	6

6. Phân Tích Độ Phức Tạp

- Thời gian: $\mathcal{O}(N)$. Với $N \leq 10^5$, số bước lặp là 10^5 , thời gian chạy $< 0.001s$.
- Không gian: $\mathcal{O}(1)$.

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- Bắt đầu vòng lặp từ $i = 0$ dẫn đến lỗi chia cho 0 (Floating point exception / Runtime Error).
- Quên khởi tạo $cnt = 0$.

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    int cnt = 0;
    for (int i = 1; i <= n; i++) {
        if (n % i == 0) {
            cnt++;
        }
    }

    cout << cnt << '\n';
    return 0;
}
```

9. Bài Toán Mở Rộng & Chuyển Giao

- Tính tổng các ước số nguyên dương của N .
- Tối ưu thuật toán đếm ước trong $\mathcal{O}(\sqrt{N})$ khi học ở Chương Số học (Chương 04).

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
```

```
int cnt = 0;
for (int i = 1; i <= n; i++) {
    if (n % i == 0) {
        cnt++;
    }
}

cout << cnt << '\n';
return 0;
}
```

Bài 07 [cppb_l0_15_kiem_tra_nam_nhuan]: Kiểm Tra Năm Nhuận & Số Ngày Trong Tháng

Bối cảnh: Theo Dương lịch, một năm thông thường có 365 ngày, riêng năm nhuận có 366 ngày với tháng Hai có 29 ngày thay vì 28 ngày. Quy tắc xác định năm nhuận là: Năm đó phải chia hết cho 400, HOẶC chia hết cho 4 nhưng KHÔNG chia hết cho 100. Bạn hãy lập trình giúp xác định xem một năm có phải là năm nhuận hay không và cho biết tháng Hai của năm đó có bao nhiêu ngày.

Nhiệm vụ: Cho một số nguyên dương Y là năm cần kiểm tra. Hãy lập trình:

- Nếu Y là năm nhuận: In ra NHUAN 29.
- Nếu Y không phải năm nhuận: In ra KHONG NHUAN 28.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương Y ($1 \leq Y \leq 10^5$).

Đầu ra (Output):

- In ra một dòng duy nhất theo định dạng yêu cầu của đề bài.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
2024	NHUAN 29

Giải thích: Năm 2024 chia hết cho 4 và không chia hết cho 100, do đó là năm nhuận và tháng Hai có 29 ngày.

Ví dụ mẫu (Sample 2):

Đầu vào (Input)	Đầu ra (Output)
1900	KHONG NHUAN 28

Giải thích: Năm 1900 chia hết cho 4 và chia hết cho 100, nhưng không chia hết cho 400 nên không phải năm nhuận. Tháng Hai có 28 ngày.

Ví dụ mẫu (Sample 3):

Đầu vào (Input)	Đầu ra (Output)
2000	NHUAN 29

Giải thích: Năm 2000 chia hết cho 400 nên là năm nhuận.

1. Mục Tiêu Học Tập & Chuẩn Đầu Ra

- Phối hợp thành thạo các toán tử logic $\parallel$ (HOẶC) và $\&\&$ (VÀ) với thứ tự ưu tiên dấu ngoặc.
- Rèn luyện kỹ năng lưu biểu thức điều kiện phức tạp vào một biến kiểu bool giúp mã nguồn sáng rõ.

2. Phân Tích Đề Bài & Bản Chất Toán Học

- Biểu thức logic chuẩn xác cho năm nhuận:

$$\text{is_leap} = (Y \% 400 == 0) \vee (Y \% 4 == 0 \wedge Y \% 100 \neq 0)$$

- Năm 1900, 2100 chia hết cho 4 nhưng chia hết cho 100 và không chia hết cho 400 nên KHÔNG phải năm nhuận.
- Năm 2000, 2400 chia hết cho 400 nên LÀ năm nhuận.

3. Câu Hỏi Gợi Mở Dẫn Dắt

- Năm 1900 chia hết cho 4, vậy nó có phải năm nhuận không Tại sao
- Để gom nhóm điều kiện “chia hết cho 4 và không chia hết cho 100”, ta dùng cặp ngoặc tròn như thế nào

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Đọc Y.
- Tính biến cờ bool $\text{is_leap} = (y \% 400 == 0) \parallel (y \% 4 == 0 \&\& y \% 100 != 0);$.
- Rẽ nhánh in kết quả tương ứng.

5. Mô Phỏng Từng Bước Trên Sample

Năm Y	$y \% 400 == 0$	$y \% 4 == 0$	$y \% 100 != 0$	Biến is_leap	Output
2024	Sai	Đúng	Đúng	Đúng	NHUAN 29
1900	Sai	Đúng	Sai (vì $1900 \% 100 == 0$)	Sai	KHONG NHUAN 28
2000	Đúng	Đúng	Sai	Đúng	NHUAN 29

6. Phân Tích Độ Phức Tạp

- Thời gian: $\mathcal{O}(1)$.
- Không gian: $\mathcal{O}(1)$.

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- Viết điều kiện đơn giản $y \% 4 == 0$ (bị sai ở các năm tròn thế kỷ như 1900, 2100).
- Thiếu dấu ngoặc phân tách giữa $\parallel$ và $\&\&$.

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int y;
    if (!(cin >> y)) return 0;

    bool is_leap = (y % 400 == 0) || (y % 4 == 0 && y % 100 != 0);

    if (is_leap) {
        cout << "NHUAN 29\n";
    } else {
        cout << "KHONG NHUAN 28\n";
    }

    return 0;
}
```

9. Bài Toán Mở Rộng & Chuyển Giao

- Nhập tháng M và năm Y , in ra chính xác số ngày của tháng M trong năm Y .

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int y;
    if (!(cin >> y)) return 0;

    bool is_leap = (y % 400 == 0) || (y % 4 == 0 && y % 100 != 0);

    if (is_leap) {
        cout << "NHUAN 29\n";
    } else {
        cout << "KHONG NHUAN 28\n";
    }

    return 0;
}
```

Bài 08 [cppb_l0_16_dem_va_tong_chu_so]: Đếm Chữ Số & Tổng Chữ Số Của Số Nguyên

Bối cảnh: Trong khoa học mật mã và lý thuyết số, tổng chữ số và số lượng chữ số của một số nguyên lớn đóng vai trò quan trọng trong việc tính toán mã kiểm tra (checksum) để phát hiện sai sót dữ liệu truyền qua mạng.

Nhiệm vụ: Cho một số nguyên không âm N . Hãy lập trình đếm số lượng chữ số và tính tổng tất cả các chữ số của N .

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên không âm N ($0 \leq N \leq 10^{18}$).

Đầu ra (Output):

- In ra hai số nguyên cách nhau bởi một khoảng trắng: số thứ nhất là số lượng chữ số, số thứ hai là tổng các chữ số của N .

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
12345	5 15

Giải thích: Số 12345 có 5 chữ số. Tổng các chữ số là: $1 + 2 + 3 + 4 + 5 = 15$. Kết quả in ra: 5 15.

Ví dụ mẫu (Sample 2):

Đầu vào (Input)	Đầu ra (Output)
0	1 0

Giải thích: Số 0 có đúng 1 chữ số và tổng các chữ số bằng 0.

1. Mục Tiêu Học Tập & Chuẩn Đầu Ras

- Thành thạo vòng lặp while ($n > 0$) để trích xuất từng chữ số bằng $\% 10$ và $/= 10$.
- Xử lý trọn vẹn trường hợp biên đặc biệt $N = 0$.
- Phối hợp cùng lúc 2 mẫu tích lũy: mẫu Đếm (`count_digits++`) và mẫu Tổng (`sum_digits += n % 10`).

2. Phân Tích Đề Bài & Bản Chất Toán Học

- Với số nguyên $N > 0$:
- Chữ số cuối cùng là $N \% 10$.
- Sau khi lấy chữ số cuối, ta bỏ nó đi bằng phép chia nguyên $N = N / 10$.
- Quá trình này lặp lại cho đến khi $N = 0$.
- Số lần lặp chính là số lượng chữ số của N .
- **Edge case:** Khi $N = 0$, nếu chạy trực tiếp vào while ($n > 0$) thì vòng lặp không chạy lần nào, dẫn đến đếm ra 0 chữ số (sai). Vì vậy cần kiểm tra riêng trường hợp $N = 0$.

3. Câu Hỏi Gợi Mở Dẫn Dắt

- Khi nào vòng lặp rút trích chữ số sẽ dừng lại (Khi N giảm về bằng 0).
- Nếu người dùng nhập vào số 0 thì vòng lặp while ($n > 0$) có chạy không Ta xử lý thế nào

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Đọc N kiểu long long.
- Nếu $N = 0$, in 1 0 và thoát ngay.
- Khởi tạo `count_digits = 0`, `sum_digits = 0`.

- Dừng vòng lặp while ($n > 0$) cập nhật biến đếm và biến tổng.

5. Mô Phỏng Từng Bước Trên Sample ($N = 12345$)

Bước	Giá trị N	$n \% 10$	sum_digits	count_digits	$n /= 10$ mới
Ban đầu	12345	—	0	0	—
Lần 1	12345	5	5	1	1234
Lần 2	1234	4	9	2	123
Lần 3	123	3	12	3	12
Lần 4	12	2	14	4	1
Lần 5	1	1	15	5	0 (Dừng)

6. Phân Tích Độ Phức Tạp

- Thời gian: $\mathcal{O}(\log_{10} N)$ (với $N \leq 10^{18}$, số bước lặp tối đa là 19 vòng lặp, chạy trong 0.000001s).
- Không gian: $\mathcal{O}(1)$.

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- Bẫy $N = 0$ (cho ra 0 chữ số thay vì 1 chữ số).
- Dùng kiểu int cho biến N dẫn đến tràn số khi $N > 2 \times 10^9$.

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long n;
    if (!(cin >> n)) return 0;

    if (n == 0) {
        cout << "1 0\n";
        return 0;
    }

    int count_digits = 0;
    long long sum_digits = 0;

    while (n > 0) {
        sum_digits += n % 10;
        count_digits++;
        n /= 10;
    }

    cout << count_digits << ' ' << sum_digits << '\n';
}
```

```
return 0;
}
```

9. Bài Toán Mở Rộng & Chuyển Giao

- Tìm chữ số đầu tiên của số nguyên N .
- Kiểm tra số siêu nguyên tố hoặc số đối xứng số học.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long n;
    if (!(cin >> n)) return 0;

    if (n == 0) {
        cout << "1 0\n";
        return 0;
    }

    int count_digits = 0;
    long long sum_digits = 0;

    while (n > 0) {
        sum_digits += n % 10;
        count_digits++;
        n /= 10;
    }

    cout << count_digits << ' ' << sum_digits << '\n';
    return 0;
}
```

Bài 03: Mảng 1 Chiều, Vector, Xâu Ký Tự & Tổ Chức Hàm

Bài 01 [cppb_10_17_doc_va_in_mang_nguoc]: Đọc Và In Mảng Theo Thứ Tự Ngược Lại

Bối cảnh: Trong trò chơi ghi nhớ chuỗi số, người quản trò đọc ra một dãy gồm N số nguyên. Người chơi có nhiệm vụ đọc lại dãy số đó theo thứ tự ngược lại từ số cuối cùng về số đầu tiên. Bạn An muốn viết một chương trình sử dụng vector trong C++ để ghi lại dãy số và in ngược thật chính xác. Nhiệm vụ: Cho một dãy gồm N số nguyên. Hãy lập trình đọc dữ liệu vào một vector<int> và in các phần tử theo thứ tự ngược lại từ phần tử cuối cùng về phần tử đầu tiên, cách nhau bởi một khoảng trắng.

Đầu vào (Input):

- Dòng thứ nhất chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng thứ hai chứa N số nguyên $a_1, a_2, \dots, a_N$ ($-10^9 \leq a_i \leq 10^9$), cách nhau bởi một khoảng trắng.

Đầu ra (Output):

- In ra trên một dòng duy nhất N số nguyên theo thứ tự từ cuối về đầu, cách nhau bởi một khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 1 3 5 7 9	9 7 5 3 1

Giải thích: Dãy số ban đầu là [1,3,5,7,9]. Thứ tự in ngược lại từ cuối về đầu là: 9,7,5,3,1.

1. Mục Tiêu Học Tập & Chuẩn Đầu Ra

- Làm quen với cấu trúc dữ liệu mảng động vector<int> a(n).
- Nắm vững quy tắc chỉ số trong C++: bắt đầu từ 0 đến $n - 1$.
- Thành thạo vòng lặp duyệt ngược: for (int i = n - 1; i >= 0; i--).

2. Phân Tích Đề Bài & Bản Chất Toán Học

- Mảng có N phần tử: phần tử đầu tiên nằm tại chỉ số 0, phần tử cuối cùng nằm tại chỉ số $n - 1$.
- Để duyệt ngược, ta cho biến chỉ số i chạy từ $n - 1$ giảm dần về 0 với bước nhảy $i--$.

3. Câu Hỏi Gợi Mở Dẫn Dắt

- Nếu mảng có 5 phần tử thì phần tử cuối cùng mang chỉ số mấy (Chỉ số 4).
- Vòng lặp duyệt ngược bắt đầu từ đâu và điều kiện dừng là gì

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Khởi tạo vector<int> a(n).
- Đọc từng phần tử qua vòng lặp.

- Duyệt ngược từ $n - 1$ về 0 và in ra. Độ phức tạp $O(N)$.

5. Mô Phỏng Từng Bước Trên Sample ($N = 5$)

Chỉ số i	4	3	2	1	0
Giá trị $a[i]$	9	7	5	3	1
Thứ tự in	1	2	3	4	5

6. Phân Tích Độ Phức Tạp

- Thời gian: $O(N)$.
- Không gian: $O(N)$.

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- Bẫy truy cập ngoài phạm vi (Out of bounds): Bắt đầu duyệt từ $i = n$ (phần tử $a[n]$ không tồn tại, gây lỗi bộ nhớ).
- Viết điều kiện dừng $i > 0$ thay vì $i \geq 0$ (bỏ quên phần tử đầu tiên $a[0]$).

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<int> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    for (int i = n - 1; i >= 0; i--) {
        cout << a[i] << (i == 0 ? '\n' : ' ');
    }

    return 0;
}
```

9. Bài Toán Mở Rộng & Chuyển Giao

- Đảo ngược mảng trực tiếp bằng hàm `reverse(a.begin(), a.end())` hoặc kỹ thuật hoán đổi 2 con trỏ `swap(a[l], a[r])`.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
```

```
cin.tie(nullptr);

int n;
if (!(cin >> n)) return 0;

vector<int> a(n);
for (int i = 0; i < n; i++) {
    cin >> a[i];
}

for (int i = n - 1; i >= 0; i--) {
    cout << a[i] << (i == 0 ? '\n' : ' ');
}

return 0;
}
```

Bài 02 [cppb_10_18_dem_so_chan_trong_vector]: Đếm Số Phần Tử Chẵn Trong Vector

Bối cảnh: Thầy giáo thể dục ghi lại số lần nhảy dây của N học sinh trong lớp thành một danh sách số nguyên. Thầy muốn thống kê xem có bao nhiêu bạn đạt số lần nhảy dây là một số chẵn. Bạn hãy lập trình giúp thầy giải quyết bài toán này.

Nhiệm vụ: Cho một dãy gồm N số nguyên. Hãy lập trình sử dụng vector để lưu trữ và đếm xem trong dãy có bao nhiêu số chẵn.

Đầu vào (Input):

- Dòng thứ nhất chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng thứ hai chứa N số nguyên $a_1, a_2, \dots, a_N$ ($-10^9 \leq a_i \leq 10^9$), cách nhau bởi một khoảng trắng.

Đầu ra (Output):

- In ra một số nguyên duy nhất là số lượng phần tử chẵn trong dãy.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 4 7 2 9 8 5	3

Giải thích: Trong dãy có 3 số chẵn là: 4, 2, 8. Kết quả in ra: 3.

Ví dụ mẫu (Sample 2):

Đầu vào (Input)	Đầu ra (Output)
4 1 3 5 7	0

Giải thích: Không có số chẵn nào trong dãy, in ra 0.

1. Mục Tiêu Học Tập & Chuẩn Đầu Ra

- Kết hợp duyệt vector với kiểm tra điều kiện phần tử $a[i] \% 2 == 0$.
- Rèn luyện kỹ thuật đếm trực tiếp ngay trong quá trình nhập dữ liệu (Stream processing) để tối ưu code.

2. Phân Tích Đề Bài & Bản Chất Toán Học

- Duyệt qua từng phần tử của mảng, nếu phần tử chia hết cho 2 ($x \% 2 == 0$), tăng biến đếm `count_even++`.

3. Câu Hỏi Gợi Mở Dẫn Dắt

- Khi một số âm chẵn (như -4) chia dư cho 2 thì kết quả là mấy ($-4 \% 2 == 0$, điều kiện vẫn đúng).
- Ta có thể kiểm tra chẵn lẻ ngay lúc đọc cin >> `a[i]` mà không cần duyệt vòng lặp thứ hai được không

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Khởi tạo `count_even = 0`.
- Duyệt N phần tử, kiểm tra $a[i] \% 2 == 0$.
- In kết quả. Độ phức tạp thời gian $O(N)$, bộ nhớ $O(N)$.

5. Mô Phỏng Từng Bước Trên Sample

Phần tử a_i	4	7	2	9	8	5
$a[i] \% 2 == 0$	Đúng	Sai	Đúng	Sai	Đúng	Sai
<code>count_even</code>	1	1	2	2	3	3

6. Phân Tích Độ Phức Tạp

- Thời gian: $O(N)$.
- Không gian: $O(N)$.

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- Kiểm tra số lẻ bằng $a[i] \% 2 == 1$ gây lỗi nếu có số âm lẻ (vì $-3 \% 2 == -1$).
- Quên khởi tạo `count_even = 0`.

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<int> a(n);
    int count_even = 0;

    for (int i = 0; i < n; i++) {
        cin >> a[i];
        if (a[i] % 2 == 0) {
```

```
count_even++;
}
}

cout << count_even << '\n';
return 0;
}
```

9. Bài Toán Mở Rộng & Chuyển Giao

- Tính tổng các phần tử chẵn trong mảng.
- Tách mảng ban đầu thành hai vector: một vector chứa các số chẵn, một vector chứa các số lẻ.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<int> a(n);
    int count_even = 0;

    for (int i = 0; i < n; i++) {
        cin >> a[i];
        if (a[i] % 2 == 0) {
            count_even++;
        }
    }

    cout << count_even << '\n';
    return 0;
}
```

Bài 03 [cppb_l0_19_tìm_max_va_vị_trị]: Tìm Giá Trị Lớn Nhất & Vị Trí Xuất Hiện

Bối cảnh: Trong cuộc thi bắn cung, ban giám khảo ghi nhận điểm số của N lượt bắn vào một bảng điện tử. Bạn hãy viết chương trình giúp bạn tổ chức tìm ra điểm số cao nhất đạt được và vị trí của lượt bắn đạt điểm số cao nhất đó (tính theo thứ tự từ 1 đến N). Nếu có nhiều lượt bắn cùng đạt điểm cao nhất, hãy in ra vị trí xuất hiện đầu tiên.

Nhiệm vụ: Cho một dãy gồm N số nguyên. Hãy lập trình tìm giá trị lớn nhất trong dãy và chỉ số 1-based của vị trí xuất hiện đầu tiên của giá trị lớn nhất đó.

Đầu vào (Input):

- Dòng thứ nhất chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng thứ hai chứa N số nguyên $a_1, a_2, \dots, a_N$ ($-10^9 \leq a_i \leq 10^9$), cách nhau bởi một khoảng trắng.

Đầu ra (Output):

- In ra hai số nguyên cách nhau bởi một khoảng trắng: số thứ nhất là giá trị lớn nhất, số thứ hai là vị trí (bắt đầu từ 1) xuất hiện đầu tiên.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 7 2 7 5	7 2

Giải thích: Giá trị lớn nhất trong dãy là 7. Giá trị 7 xuất hiện ở vị trí thứ 2 và vị trí thứ 4. Vị trí xuất hiện đầu tiên là 2. Kết quả: 7 2.

Ví dụ mẫu (Sample 2):

Đầu vào (Input)	Đầu ra (Output)
4 -5 -2 -8 -2	-2 2

Giải thích: Giá trị lớn nhất là -2 tại vị trí thứ 2.

1. Mục Tiêu Học Tập & Chuẩn Đầu Ra

- Thành thạo mẫu tích lũy Cực trị (Finding Maximum Pattern).
- Biết cách khởi tạo biến `max_val` an toàn bằng chính phần tử đầu tiên `a[0]`.
- Nắm vững kỹ thuật chuyển đổi chỉ số giữa 0-based (nội bộ C++) và 1-based (yêu cầu đề bài).

2. Phân Tích Đề Bài & Bản Chất Toán Học

- Khởi tạo: Gán `max_val = a[0]` và `best_pos = 1`.
- Duyệt từ $i = 1$ đến $N - 1$: Nếu gặp $a[i] > \text{max_val}$ thì cập nhật `max_val = a[i]` và `best_pos = i + 1`.
- Chú ý: Dùng dấu $>$ ngặt (chứ không dùng $\geq$) để đảm bảo giữ nguyên vị trí xuất hiện đầu tiên khi có nhiều phần tử bằng nhau.

3. Câu Hỏi Gợi Mở Dẫn Dắt

- Nếu tất cả các phần tử đều là số âm, khởi tạo `max_val = 0` có đúng không (Sai hoàn toàn vì 0 lớn hơn mọi số âm).
- Để lưu vị trí xuất hiện đầu tiên, ta dùng điều kiện so sánh $>$ hay $\geq$

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Khởi tạo giá trị lớn nhất bằng `a[0]`.
- Duyệt tuyến tính $O(N)$ cập nhật cực trị.

5. Mô Phỏng Từng Bước Trên Sample

Chỉ số i	Giá trị $a[i]$	So sánh với <code>max_val</code>	Cập nhật <code>max_val</code>	Cập nhật <code>best_pos</code>
0	3	Khởi tạo ban đầu	3	1

1	7	$7 > 3$ (Đúng)	7	2
2	2	$2 > 7$ (Sai)	7	2
3	7	$7 > 7$ (Sai, vì dấu $>$ ngặt)	7	2
4	5	$5 > 7$ (Sai)	7	2

6. Phân Tích Độ Phức Tạp

- Thời gian: $\mathcal{O}(N)$.
- Không gian: $\mathcal{O}(N)$.

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- Khởi tạo `max_val = 0` thay vì `a[0]` (sai với mảng toàn số âm).
- Dùng `>=` khiến vị trí bị cập nhật thành vị trí xuất hiện cuối cùng.
- In ra chỉ số 0-based thay vì 1-based.

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    long long max_val = a[0];
    int best_pos = 1;

    for (int i = 1; i < n; i++) {
        if (a[i] > max_val) {
            max_val = a[i];
            best_pos = i + 1;
        }
    }

    cout << max_val << ' ' << best_pos << '\n';
    return 0;
}
```

9. Bài Toán Mở Rộng & Chuyển Giao

- Tìm giá trị lớn thứ nhì trong mảng.
- Tìm giá trị nhỏ nhất và khoảng cách giữa vị trí nhỏ nhất và lớn nhất.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    long long max_val = a[0];
    int best_pos = 1; // 1-based index

    for (int i = 1; i < n; i++) {
        if (a[i] > max_val) {
            max_val = a[i];
            best_pos = i + 1;
        }
    }

    cout << max_val << ' ' << best_pos << '\n';
    return 0;
}
```

Bài 04 [cppb_10_20_viet_ham_kiem_tra_mang_tang_dan]: Viết Hàm Kiểm Tra Mảng Tăng Dần

Bối cảnh: Trước khi áp dụng thuật toán Tìm kiếm nhị phân (sẽ học ở các chương sau), một điều kiện tiên quyết bắt buộc là dãy số phải được sắp xếp theo thứ tự không giảm (tăng dần). Bạn hãy lập trình viết một hàm chuyên dụng để kiểm tra xem một dãy số đã được sắp xếp tăng dần hay chưa. Nhiệm vụ: Cho một dãy gồm N số nguyên. Hãy viết hàm `bool isSorted(const vector<int>& a)` kiểm tra tính tăng dần của mảng:

- Nếu mảng đã sắp xếp không giảm ($a_0 \leq a_1 \leq \dots \leq a_{n-1}$), in ra YES.
- Ngược lại, in ra NO.

Đầu vào (Input):

- Dòng thứ nhất chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng thứ hai chứa N số nguyên $a_1, a_2, \dots, a_N$ ($-10^9 \leq a_i \leq 10^9$), cách nhau bởi một khoảng trắng.

Đầu ra (Output):

- In ra một dòng duy nhất chữ YES hoặc NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 2 4 4 7 9	YES

Giải thích: Dãy $2 \leq 4 \leq 4 \leq 7 \leq 9$ thỏa mãn thứ tự không giảm. In ra YES.

Ví dụ mẫu (Sample 2):

Đầu vào (Input)	Đầu ra (Output)
4 1 5 3 8	NO

Giải thích: Tại vị trí 5 và 3, ta có $5 > 3$ vi phạm tính chất tăng dần. In ra NO.

1. Mục Tiêu Học Tập & Chuẩn Đầu Ra

- Học cách tổ chức mã nguồn sạch bằng cách tách bài toán thành Hàm con (function).
- Hiểu và áp dụng kỹ thuật truyền tham chiếu hằng `const vector<int>&` a để tránh sao chép tốn bộ nhớ.
- Nắm vững tư duy phản chứng (Early Exit): chỉ cần tìm thấy 1 cặp $a[i] > a[i + 1]$ vi phạm là lập tức return false.

2. Phân Tích Đề Bài & Bản Chất Toán Học

- Mảng tăng dần (không giảm) khi và chỉ khi mọi cặp phần tử kế nhau đều thỏa mãn $a[i] \leq a[i + 1]$.
- Nếu tồn tại bất kỳ chỉ số i nào mà $a[i] > a[i + 1]$, mảng chắc chắn không tăng dần.

3. Câu Hỏi Gợi Mở Dẫn Dắt

- Để biết một mảng có tăng dần không, ta cần so sánh bao nhiêu cặp phần tử kế nhau ($N - 1$ cặp).
- Khi phát hiện một cặp vi phạm, có cần kiểm tra tiếp các phần tử phía sau không (Không, dừng và trả về false ngay).
- Tại sao nên viết `const vector<int>&` thay vì `vector<int>`

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Viết hàm `isSorted(const vector<int>& a)`.
- Duyệt từ $i = 0$ đến $n - 2$. Nếu $a[i] > a[i + 1]$ thì return false. Nếu duyệt hết mà không vi phạm thì return true.

5. Mô Phỏng Từng Bước Trên Sample

Cặp kế nhau ($a[i], a[i + 1]$)	Kiểm tra $a[i] > a[i + 1]$	Hành động
(2, 4)	$2 > 4$ (Sai)	Tiếp tục duyệt
(4, 4)	$4 > 4$ (Sai, vì bằng nhau vẫn thỏa mãn không giảm)	Tiếp tục duyệt
(4, 7)	$4 > 7$ (Sai)	Tiếp tục duyệt
(7, 9)	$7 > 9$ (Sai)	Hết mảng $\Rightarrow$ return true

6. Phân Tích Độ Phức Tạp

- Thời gian: $\mathcal{O}(N)$ trong trường hợp xấu nhất, trung bình dừng sớm hơn.
- Không gian: $\mathcal{O}(1)$ phụ trội nhờ truyền tham chiếu &.

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- Duyệt vòng lặp đến $i < n$ và kiểm tra $a[i] > a[i + 1]$ dẫn đến lỗi truy cập $a[n]$ ngoài phạm vi.
- Viết nhầm điều kiện bằng nhau $a[i] == a[i+1]$ là vi phạm (đề bài cho phép các phần tử bằng nhau).

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

bool isSorted(const vector<int>& a) {
    int n = a.size();
    for (int i = 0; i < n - 1; i++) {
        if (a[i] > a[i + 1]) {
            return false;
        }
    }
    return true;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<int> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    if (isSorted(a)) {
        cout << "YES\n";
    } else {
        cout << "NO\n";
    }

    return 0;
}
```

9. Bài Toán Mở Rộng & Chuyển Giao

- Kiểm tra mảng giảm dần hoặc kiểm tra mảng cấp số cộng.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

// Hàm kiểm tra mảng tăng dần, truyền tham chiếu hằng để tối ưu bộ nhớ
bool isSorted(const vector<int>& a) {
    int n = a.size();
```

```

    for (int i = 0; i < n - 1; i++) {
        if (a[i] > a[i + 1]) {
            return false; // Phát hiện cặp nghịch thế, kết luận ngay
        }
    }
    return true; // Tất cả các cặp kề nhau đều thỏa mãn
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<int> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    if (isSorted(a)) {
        cout << "YES\n";
    } else {
        cout << "NO\n";
    }

    return 0;
}

```

Bài 05 [cppb_10_21_dem_ky_tu_trong_xau]: Đếm Số Lần Xuất Hiện Của Ký Tự Trong Xâu

Bối cảnh: Trong việc xử lý văn bản và công cụ tìm kiếm, thao tác đếm số lần xuất hiện của một ký tự cụ thể trong một đoạn văn bản là thao tác cơ bản nhất. Bạn hãy lập trình đọc một chuỗi ký tự và đếm xem một ký tự mục tiêu xuất hiện bao nhiêu lần trong chuỗi đó.

Nhiệm vụ: Cho một xâu ký tự S (không chứa khoảng trắng) và một ký tự C . Hãy lập trình đếm và in ra số lần ký tự C xuất hiện trong xâu S .

Đầu vào (Input):

- Dòng thứ nhất chứa xâu ký tự S ($1 \leq |S| \leq 10^5$), chỉ gồm các chữ cái tiếng Anh.
- Dòng thứ hai chứa một ký tự C .

Đầu ra (Output):

- In ra một số nguyên duy nhất là số lần xuất hiện của ký tự C trong xâu S .

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
banana a	3

Giải thích: Trong xâu banana, ký tự a xuất hiện 3 lần (tại các vị trí chỉ số 1, 3, 5). In ra 3.

Ví dụ mẫu (Sample 2):

Đầu vào (Input)	Đầu ra (Output)
-----------------	-----------------

Programming z

0

Giải thích: Ký tự z không xuất hiện lần nào trong xâu, in ra 0.

1. Mục Tiêu Học Tập & Chuẩn Đầu Ra

- Làm quen với kiểu dữ liệu xâu ký tự string trong C++.
- Sử dụng cú pháp vòng lặp duyệt từng ký tự for (char x : s).
- Phân biệt kiểu dữ liệu ký tự đơn char (dấu nháy đơn 'a') và xâu ký tự string (dấu nháy kép "banana").

2. Phân Tích Đề Bài & Bản Chất Toán Học

- Duyệt qua từng ký tự của xâu từ trái sang phải, so sánh $x == c$. Nếu bằng nhau thì tăng biến đếm.

3. Câu Hỏi Gợi Mở Dẫn Dắt

- Trong C++, chuỗi ký tự được lưu bằng kiểu dữ liệu nào
- Ký tự C được biểu diễn bằng dấu nháy đơn hay nháy kép

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Đọc xâu s và ký tự c.
- Dùng vòng lặp duyệt qua xâu trong thời gian $\mathcal{O}(|S|)$.

5. Mô Phỏng Từng Bước Trên Sample (banana, a)

Ký tự x	b	a	n	a	n	a
$x == 'a'$	Sai	Đúng	Sai	Đúng	Sai	Đúng
count_c	0	1	1	2	2	3

6. Phân Tích Độ Phức Tạp

- Thời gian: $\mathcal{O}(|S|)$.
- Không gian: $\mathcal{O}(1)$ phụ trội.

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- Nhầm dấu nháy đơn 'c' và nháy kép "c".
- Không phân biệt chữ hoa và chữ thường ('A' khác 'a').

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```

string s;
char c;
if (!(cin >> s >> c)) return 0;

int count_c = 0;
for (char x : s) {
    if (x == c) {
        count_c++;
    }
}

cout << count_c << '\n';
return 0;
}

```

9. Bài Toán Mở Rộng & Chuyển Giao

- Đếm số lượng chữ số, chữ cái hoa, chữ cái thường trong một xâu.

4. Lời giải tham khảo

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    char c;
    if (!(cin >> s >> c)) return 0;

    int count_c = 0;
    for (char x : s) {
        if (x == c) {
            count_c++;
        }
    }

    cout << count_c << '\n';
    return 0;
}

```

Bài 06 [cppb_l0_22_bang_dem_tan_suat_ky_tu]: Bảng Đếm Tần Suất Ký Tự Trong Xâu

Bối cảnh: Trong cuộc thi đánh vần tiếng Anh, ban tổ chức đưa ra một từ vựng và đố các thí sinh: “Ký tự nào xuất hiện nhiều lần nhất trong từ này và xuất hiện bao nhiêu lần”. Bạn hãy lập trình xây dựng bảng đếm tần suất để giải quyết bài toán một cách nhanh chóng và chính xác nhất.

Nhiệm vụ: Cho một xâu ký tự S chỉ gồm các chữ cái tiếng Anh in thường (a–z). Hãy lập trình tìm ký tự xuất hiện nhiều nhất cùng số lần xuất hiện của nó. Nếu có nhiều ký tự có cùng số lần xuất hiện lớn nhất, hãy in ra ký tự có thứ tự bảng chữ cái nhỏ nhất.

Đầu vào (Input):

- Một dòng duy nhất chứa xâu ký tự S ($1 \leq |S| \leq 10^5$), chỉ gồm các chữ cái tiếng Anh in thường.

Đầu ra (Output):

- In ra trên một dòng duy nhất: ký tự xuất hiện nhiều nhất và số lần xuất hiện, cách nhau bởi một khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
abracadabra	a 5

Giải thích: Xâu abracadabra có 11 ký tự. Thống kê tần suất:

- a: xuất hiện 5 lần.
- b: xuất hiện 2 lần.
- r: xuất hiện 2 lần.
- c: xuất hiện 1 lần.
- d: xuất hiện 1 lần. Ký tự a xuất hiện nhiều nhất (5 lần). Kết quả in ra: a 5.

Ví dụ mẫu (Sample 2):

Đầu vào (Input)	Đầu ra (Output)
baab	a 2

Giải thích: Cả a và b đều xuất hiện 2 lần. Vì ký tự a đứng trước b trong bảng chữ cái nên ta chọn a.

1. Mục Tiêu Học Tập & Chuẩn Đầu Ra

- Nắm vững kỹ thuật ánh xạ ký tự sang chỉ số mảng bằng mã ASCII: $\text{index} = c - 'a'$.
- Sử dụng mảng đếm tần suất (Frequency Array) kích thước cố định 26 phần tử.
- Xử lý điều kiện ưu tiên thứ tự từ điển: duyệt từ $i = 0$ đến 25 với điều kiện $>$ ngắt để tự động ưu tiên ký tự có mã nhỏ hơn.

2. Phân Tích Đề Bài & Bản Chất Toán Học

- Bảng chữ cái tiếng Anh in thường có 26 ký tự từ 'a' đến 'z'.
- Khi lấy ký tự $c - 'a'$, ta nhận được giá trị từ 0 đến 25 (với $'a' - 'a' = 0$, $'b' - 'a' = 1$, ..., $'z' - 'a' = 25$).
- Mảng `freq[26]` khởi tạo toàn 0. Mỗi khi gặp ký tự c , tăng `freq[c - 'a']++`.

3. Câu Hỏi Gợi Mở Dẫn Dắt

- Trong bảng mã ASCII, ký tự 'a' và 'b' cách nhau bao nhiêu đơn vị
- Làm thế nào để dùng một mảng số nguyên để đếm số lần xuất hiện của 26 chữ cái

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Duyệt qua xâu cập nhật bảng đếm: $\mathcal{O}(|S|)$.
- Tìm giá trị lớn nhất trong mảng 26 phần tử: $\mathcal{O}(26) = \mathcal{O}(1)$.
- Tổng thời gian: $\mathcal{O}(|S|)$ cực kỳ tối ưu so với cách đếm từng phần tử tốn $\mathcal{O}(|S|^2)$.

5. Mô Phỏng Từng Bước Trên Sample (abracadabra)

Ký tự	Chỉ số c - 'a'	Tần suất đếm được
a	0	5
b	1	2
c	2	1
d	3	1
r	17	2
Max	0 (a)	5

6. Phân Tích Độ Phức Tạp

- Thời gian: $\mathcal{O}(|S| + 26) = \mathcal{O}(|S|)$.
- Không gian: $\mathcal{O}(26) = \mathcal{O}(1)$ bộ nhớ phụ trợ.

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- Quên khởi tạo mảng tần suất bằng 0 (`int freq[26] = {};`).
- Ánh xạ sai chỉ số thành c thay vì c - 'a'.
- Dùng `>=` khi so sánh tần suất làm mất tính ưu tiên thứ tự từ điển nhỏ hơn.

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    if (!(cin >> s)) return 0;

    int freq[26] = {};
    for (char c : s) {
        freq[c - 'a']++;
    }

    int best_char_idx = 0;
    for (int i = 1; i < 26; i++) {
        if (freq[i] > freq[best_char_idx]) {
            best_char_idx = i;
        }
    }

    char ans_char = (char)('a' + best_char_idx);
    cout << ans_char << ' ' << freq[best_char_idx] << '\n';
    return 0;
}
```

9. Bài Toán Mở Rộng & Chuyển Giao

- Kiểm tra hai xâu có phải là đảo từ (Anagram) của nhau không.
- Tìm ký tự đầu tiên không bị lặp lại trong xâu.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    if (!(cin >> s)) return 0;

    int freq[26] = {};
    for (char c : s) {
        freq[c - 'a']++;
    }

    int best_char_idx = 0;
    for (int i = 1; i < 26; i++) {
        if (freq[i] > freq[best_char_idx]) {
            best_char_idx = i;
        }
    }

    char ans_char = (char)('a' + best_char_idx);
    cout << ans_char << ' ' << freq[best_char_idx] << '\n';
    return 0;
}
```

Bài 07 [cppb_10_23_dao_nguoc_mang_swap]: Đảo Ngược Mảng Bằng Kỹ Thuật Hoán Đổi

Bối cảnh: Trong tối ưu thuật toán, thay vì tạo ra một mảng mới tốn thêm bộ nhớ để chứa kết quả đảo ngược, các lập trình viên thường dùng kỹ thuật Hai con trỏ đối xứng từ hai đầu mảng và hoán đổi trực tiếp các cặp phần tử (in-place reversal). Bạn hãy viết một hàm thực hiện công việc này.

Nhiệm vụ: Cho một dãy gồm N số nguyên. Hãy viết hàm void reverseArray(vector<int>& a) sử dụng kỹ thuật hoán đổi (swap) để đảo ngược dãy số ngay trên mảng ban đầu, sau đó in ra dãy số sau khi đã đảo ngược.

Đầu vào (Input):

- Dòng thứ nhất chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng thứ hai chứa N số nguyên $a_1, a_2, \dots, a_N$ ($-10^9 \leq a_i \leq 10^9$), cách nhau bởi một khoảng trắng.

Đầu ra (Output):

- In ra một dòng duy nhất gồm N số nguyên của mảng sau khi đã đảo ngược, cách nhau bởi một khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 1 2 3 4 5	5 4 3 2 1

Giải thích: - Bước 1: Hoán đổi $a[0]$ và $a[4] \Rightarrow [5,2,3,4,1]$. - Bước 2: Hoán đổi $a[1]$ và $a[3] \Rightarrow [5,4,3,2,1]$. - Bước 3: Hai con trỏ gặp nhau tại phần tử giữa $a[2]$, kết thúc. Kết quả: 5 4 3 2 1.

Ví dụ mẫu (Sample 2):

Đầu vào (Input)	Đầu ra (Output)
4 10 20 30 40	40 30 20 10

Giải thích: Hoán đổi (10,40) và (20,30) thu được 40 30 20 10.

1. Mục Tiêu Học Tập & Chuẩn Đầu Ra

- Thành thạo hàm hoán đổi $\text{swap}(x, y)$.
- Hiểu sâu sắc cơ chế truyền tham chiếu $\text{vector}<\text{int}>\& a$ (thay đổi giá trị thực của mảng sau khi gọi hàm).
- Làm quen với mô hình tư duy Hai con trỏ (Two Pointers) cơ bản nhất để chuẩn bị cho Chương 02.

2. Phân Tích Đề Bài & Bản Chất Toán Học

- Hai con trỏ: $l = 0$ (đầu mảng) và $r = n - 1$ (cuối mảng).
- Chừng nào $l < r$, ta hoán đổi giá trị của $a[l]$ và $a[r]$, sau đó tăng $l++$ và giảm $r--$.
- Khi $l \geq r$, toàn bộ mảng đã được đảo ngược đối xứng hoàn hảo.

3. Câu Hỏi Gợi Mở Dẫn Dắt

- Nếu hàm viết $\text{void reverseArray}(\text{vector}<\text{int}> a)$ (thiếu dấu $\&$) thì mảng ở hàm main có bị thay đổi không (Không, vì chỉ thay đổi trên bản sao).
- Vì sao điều kiện lặp lại là $l < r$ mà không phải $l \leq r$

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Đảo ngược tại chỗ (in-place) với $\mathcal{O}(1)$ bộ nhớ phụ trội và $\mathcal{O}(N/2) = \mathcal{O}(N)$ phép toán.

5. Mô Phỏng Từng Bước Trên Sample ([1,2,3,4,5])

Bước	Con trỏ l	Con trỏ r	Cặp hoán đổi	Mảng sau hoán đổi
1	0 (gt: 1)	4 (gt: 5)	$\text{swap}(a[0], a[4])$	[5, 2, 3, 4, 1]
2	1 (gt: 2)	3 (gt: 4)	$\text{swap}(a[1], a[3])$	[5, 4, 3, 2, 1]
3	2 (gt: 3)	2 (gt: 3)	$l = r$ (Dừng)	[5, 4, 3, 2, 1]

6. Phân Tích Độ Phức Tạp

- Thời gian: $\mathcal{O}(N)$.

- Không gian: $O(1)$ bộ nhớ phụ trợ.

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- Quên dấu tham chiếu & trong tham số hàm.
- Cho hai con trỏ chạy quá đà (chạy từ 0 đến $n - 1$ dẫn đến hoán đổi hai lần khiến mảng quay trở lại như cũ).

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

void reverseArray(vector<int>& a) {
    int l = 0;
    int r = (int)a.size() - 1;
    while (l < r) {
        swap(a[l], a[r]);
        l++;
        r--;
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<int> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    reverseArray(a);

    for (int i = 0; i < n; i++) {
        cout << a[i] << (i == n - 1 ? '\n' : ' ');
    }

    return 0;
}
```

9. Bài Toán Mở Rộng & Chuyển Giao

- Đảo ngược một đoạn con từ vị trí L đến vị trí R trong mảng.
- Thuật toán xoay mảng sang phải K vị trí bằng 3 lần đảo ngược mảng.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

// Hàm đảo ngược mảng in-place bằng tham chiếu vector<int>& a
void reverseArray(vector<int>& a) {
    int l = 0;
    int r = (int)a.size() - 1;
    while (l < r) {
        swap(a[l], a[r]);
    }
}
```

```

        l++;
        r--;
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<int> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    reverseArray(a);

    for (int i = 0; i < n; i++) {
        cout << a[i] << (i == n - 1 ? '\n' : ' ');
    }

    return 0;
}

```

Bài 08 [cppb_10_24_kiem_tra_xau_doi_xung_palindrome]: Kiểm Tra Xâu Đối Xứng Palindrome

Bối cảnh: Một xâu ký tự được gọi là đối xứng (Palindrome) nếu đọc từ trái sang phải hay từ phải sang trái đều thu được chuỗi ký tự hoàn toàn giống nhau (ví dụ: radar, madam, racecar). Kiểm tra xâu đối xứng là một bài toán kinh điển trong xử lý chuỗi và là tiền đề cho các thuật toán quy hoạch động chuỗi sau này.

Nhiệm vụ: Cho một xâu ký tự S chỉ gồm các chữ cái tiếng Anh in thường. Hãy viết hàm bool `isPalindrome(const string& s)` để kiểm tra:

- Nếu S là xâu đối xứng, in ra YES.
- Nếu S không phải xâu đối xứng, in ra NO.

Đầu vào (Input):

- Một dòng duy nhất chứa xâu ký tự S ($1 \leq |S| \leq 10^5$), chỉ gồm các chữ cái in thường không chứa khoảng trắng.

Đầu ra (Output):

- In ra một dòng duy nhất chữ YES hoặc NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
racecar	YES

Giải thích: Xâu racecar đọc xuôi hay ngược đều là racecar. Kết quả in ra: YES.

Ví dụ mẫu (Sample 2):

Đầu vào (Input)	Đầu ra (Output)
algorithm	NO

Giải thích: Xâu algorithm đọc ngược là mihtirogla, không trùng khớp với xâu ban đầu. Kết quả: NO.

1. Mục Tiêu Học Tập & Chuẩn Đầu Ra

- Áp dụng kỹ thuật hai con trỏ đối xứng trên cấu trúc string.
- Rèn luyện kỹ năng viết hàm trả về kiểu bool với kỹ thuật Early Exit (dừng ngay khi tìm thấy điểm sai lệch).
- Hiểu bản chất đối xứng: $s[i] == s[n - 1 - i]$ với mọi $0 \leq i < n/2$.

2. Phân Tích Đề Bài & Bản Chất Toán Học

- Một xâu có độ dài N là palindrome khi và chỉ khi:

$$s[i] = s[N - 1 - i] \quad \forall 0 \leq i < [N/2]$$
- Bằng cách dùng hai con trỏ $l = 0$ và $r = N - 1$, ta so sánh từng cặp ký tự đối xứng. Nếu có bất kỳ cặp nào khác nhau thì kết luận NO ngay.

3. Câu Hỏi Gợi Mở Dẫn Dắt

- Một từ có 1 chữ cái (như "a") có phải là xâu đối xứng không (Có, vì $l = r$ không vi phạm).
- Khi nào thì hai con trỏ sẽ dừng lại (Khi $l \geq r$).

4. Chiến Lược Tối Ưu & Bất Biến Thuật Toán

- Sử dụng hai con trỏ quét từ ngoài vào trong: độ phức tạp thời gian $\mathcal{O}(|S|/2) = \mathcal{O}(|S|)$, bộ nhớ phụ trợ $\mathcal{O}(1)$.

5. Mô Phỏng Từng Bước Trên Sample (racecar)

Bước	Con trỏ l	Con trỏ r	So sánh $s[l]$ và $s[r]$	Kết quả so sánh
1	0 ('r')	6 ('r')	'r' == 'r'	Khớp
2	1 ('a')	5 ('a')	'a' == 'a'	Khớp
3	2 ('c')	4 ('c')	'c' == 'c'	Khớp
4	3 ('e')	3 ('e')	$l = r$ (Dừng)	Khớp toàn bộ $\Rightarrow$ YES

6. Phân Tích Độ Phức Tạp

- Thời gian: $\mathcal{O}(|S|)$.
- Không gian: $\mathcal{O}(1)$ bộ nhớ phụ trợ nhờ truyền const string&.

7. Các Bẫy Lỗi Lập Trình Kinh Điển

- Đảo ngược chuỗi bằng phép cộng chuỗi $t = s[i] + t$ trong vòng lặp gây độ phức tạp $\mathcal{O}(|S|^2)$ dẫn đến lỗi chạy quá thời gian (TLE) khi $|S| = 10^5$.
- Không dùng tham chiếu & mà truyền tham trị cả chuỗi lớn 10^5 làm giảm hiệu năng.

8. Mã Nguồn Tham Chiếu

```
#include <bits/stdc++.h>
using namespace std;

bool isPalindrome(const string& s) {
    int l = 0;
    int r = (int)s.size() - 1;
    while (l < r) {
        if (s[l] != s[r]) {
            return false;
        }
        l++;
        r--;
    }
    return true;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    if (!(cin >> s)) return 0;

    if (isPalindrome(s)) {
        cout << "YES\n";
    } else {
        cout << "NO\n";
    }

    return 0;
}
```

9. Bài Toán Mở Rộng & Chuyển Giao

- Kiểm tra palindrome có bỏ qua dấu cách và dấu câu (Palindrome câu văn).
- Bài toán tìm chuỗi con đối xứng dài nhất (LPS - sẽ học trong Chương Quy hoạch động).

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

// Hàm kiểm tra chuỗi đối xứng bằng kỹ thuật hai con trỏ đối xứng
bool isPalindrome(const string& s) {
    int l = 0;
    int r = (int)s.size() - 1;
    while (l < r) {
        if (s[l] != s[r]) {
            return false; // Khác nhau tại vị trí đối xứng
        }
        l++;
        r--;
    }
    return true;
}
```

```
    }  
    return true; // Tất cả các cặp đối xứng đều khớp  
}  
  
int main() {  
    ios::sync_with_stdio(false);  
    cin.tie(nullptr);  
  
    string s;  
    if (!(cin >> s)) return 0;  
  
    if (isPalindrome(s)) {  
        cout << "YES\n";  
    } else {  
        cout << "NO\n";  
    }  
  
    return 0;  
}
```

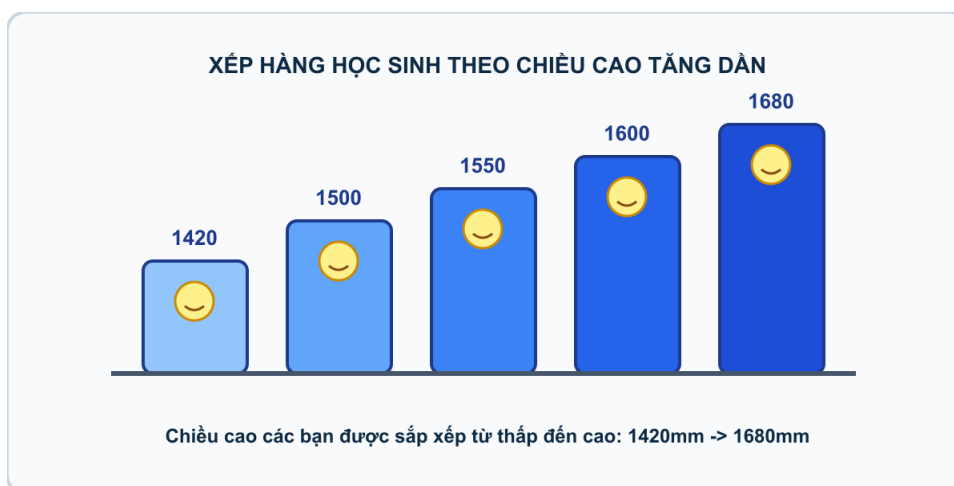


CHƯƠNG 02: THUẬT TOÁN SẮP XẾP & KỸ THUẬT MẢNG

Bài 04: Thuật toán sắp xếp

Bài 01 [cppb_sx_01_xep_hang_diem_danh]: Xếp Hàng Điểm Danh

Bối cảnh: Trong buổi học thể dục đầu năm, thầy giáo muốn xếp hàng N bạn học sinh theo thứ tự chiều cao từ thấp đến cao để chuẩn bị cho bài tập đồng diễn.



Hình minh họa: Mô hình sắp xếp thứ tự và tối ưu hóa vị trí

Nhiệm vụ: Cho danh sách chiều cao của N bạn học sinh. Hãy in ra danh sách chiều cao sau khi đã xếp hàng theo thứ tự tăng dần.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 1000$).
- Dòng 2: Chứa N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^6$).

Đầu ra (Output):

- In ra trên một dòng gồm N số nguyên biểu diễn chiều cao sau khi sắp xếp tăng dần, cách nhau bởi khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 1550 1420 1680 1500 1600	1420 1500 1550 1600 1680

Giải thích: Chiều cao ban đầu của 5 bạn học sinh lần lượt là: 1550, 1420, 1680, 1500, 1600 (đơn vị: mm). Sau khi sắp xếp theo thứ tự chiều cao tăng dần từ thấp đến cao, thứ tự đứng vào hàng chuẩn xác sẽ là: $1420 \leq 1500 \leq 1550 \leq 1600 \leq 1680$.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho danh sách chiều cao của N bạn học sinh. Hãy in ra danh sách chiều cao sau khi đã xếp hàng theo thứ tự tăng dần.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nhập dữ liệu vào mảng/biến | Input: 5 1550 1420 1680 1500 1600 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Chiều cao ban đầu của 5 bạn học sinh lần lượt là: 1550,1420,1680,1500,1600 (đơn vị: mm). Sau khi sắp xếp theo thứ ... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 1420 1500 1550 1600 1680 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Chiều cao ban đầu của 5 bạn học sinh lần lượt là: 1550,1420,1680,1500,1600 (đơn vị: mm). Sau khi sắp xếp theo thứ tự chiều cao tăng dần từ thấp đến cao, thứ tự đứng vào hàng chuẩn xác sẽ là: $1420 \leq 1500 \leq 1550 \leq 1600 \leq 1680$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<int> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    sort(a.begin(), a.end());

    for (int i = 0; i < n; ++i) {
        cout << a[i] << (i == n - 1 ? "" : " ");
    }
    cout << "\n";
    return 0;
}
```

Bài 02 [cppb_sx_02_khoang_cach_nho_nhat]: Khoảng Cách Nhỏ Nhất

Bối cảnh: Trên trục đường chính của một khu đô thị thông minh, ban quản lý đã cho lắp đặt N trạm cảm biến đo lường môi trường tại các vị trí có tọa độ $A_1, A_2, \dots, A_N$. Nhằm đảm bảo vùng phủ sóng không bị can nhiễu tín hiệu tần số vô tuyến giữa hai trạm kề sát nhau, trung tâm điều hành cần xác định khoảng cách ngắn nhất giữa hai trạm cảm biến bất kỳ trên toàn tuyến.



Hình minh họa: Mô hình sắp xếp thứ tự và tối ưu hóa vị trí

Nhiệm vụ: Cho danh sách tọa độ của N trạm cảm biến. Hãy tính và in ra khoảng cách nhỏ nhất giữa hai trạm cảm biến bất kỳ trong hệ thống.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($2 \leq N \leq 10^5$) — số lượng trạm cảm biến.
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$) — tọa độ các trạm cảm biến.

Đầu ra (Output):

- In ra một số nguyên duy nhất là khoảng cách nhỏ nhất giữa hai trạm cảm biến bất kỳ.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 8 3 14 6 10	2

Giải thích: Tọa độ các trạm cảm biến ban đầu là: 8,3,14,6,10. Sau khi sắp xếp lại theo chiều tăng dần của vị trí trên trục đường: 3,6,8,10,14. Khoảng cách giữa các cặp trạm liên kề nhau:

- Giữa trạm 3 và 6: khoảng cách là $6 - 3 = 3$.
- Giữa trạm 6 và 8: khoảng cách là $8 - 6 = 2$.
- Giữa trạm 8 và 10: khoảng cách là $10 - 8 = 2$.
- Giữa trạm 10 và 14: khoảng cách là $14 - 10 = 4$.

Do đó, khoảng cách nhỏ nhất giữa hai trạm bất kỳ là 2 (đạt được giữa trạm 6 và 8, hoặc trạm 8 và 10).

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách tọa độ của N trạm cảm biến. Hãy tính và in ra khoảng cách nhỏ nhất giữa hai trạm cảm biến bất kỳ trong hệ thống.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 8 3 14 6 10 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Tọa độ các trạm cảm biến ban đầu là: 8,3,14,6,10. Sau khi sắp

xếp lại theo chiều tăng dần của vị trí trên trục đườ... | Tính toán từng bước trạng thái || 3 | Xuất kết quả | Output: 2 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Tọa độ các trạm cảm biến ban đầu là: 8,3,14,6,10. Sau khi sắp xếp lại theo chiều tăng dần của vị trí trên trục đường: 3,6,8,10,14. Khoảng cách giữa các cặp trạm liên kề nhau:

- Giữa trạm 3 và 6: khoảng cách là $6 - 3 = 3$.
- Giữa trạm 6 và 8: khoảng cách là $8 - 6 = 2$.
- Giữa trạm 8 và 10: khoảng cách là $10 - 8 = 2$.
- Giữa trạm 10 và 14: khoảng cách là $14 - 10 = 4$.

Do đó, khoảng cách nhỏ nhất giữa hai trạm bất kỳ là 2 (đạt được giữa trạm 6 và 8, hoặc trạm 8 và 10).

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm main() để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    sort(a.begin(), a.end());

    long long ans = a[1] - a[0];
    for (int i = 1; i < n - 1; ++i) {
        ans = min(ans, a[i + 1] - a[i]);
    }

    cout << ans << "\n";
    return 0;
}
```

Bài 03 [cppb_sx_03_tri_tuyet_doi]: Sắp Xếp Theo Trị Tuyệt Đối

Bối cảnh: Trong phòng thí nghiệm địa chất, các chuyên gia đang theo dõi sự biến thiên độ lệch nhiệt độ của N mẫu khoáng thạch so với ngưỡng tiêu chuẩn 0°C . Độ lệch được ghi nhận dưới dạng các

số nguyên: giá trị dương thể hiện mẫu bị nóng lên, giá trị âm thể hiện mẫu bị làm lạnh, và 0 nghĩa là ổn định. Để ưu tiên kiểm định các mẫu có độ lệch biên độ nhỏ trước (ít biến dạng cấu trúc nhất), chuyên gia muốn sắp xếp danh sách các mẫu theo độ lớn biên thiên (tức giá trị tuyệt đối). Trong trường hợp hai mẫu có cùng độ lớn biên độ biên thiên, mẫu bị làm lạnh (số âm) cần được xử lý trước mẫu bị nóng lên (số dương).



Hình minh họa: Mô hình sắp xếp thứ tự và tối ưu hóa vị trí

Nhiệm vụ: Cho một dãy gồm N số nguyên $A_1, A_2, \dots, A_N$. Hãy sắp xếp các phần tử theo giá trị tuyệt đối tăng dần. Nếu hai phần tử có cùng giá trị tuyệt đối, phần tử mang dấu âm phải đứng trước phần tử mang dấu dương.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 10^5$) — số lượng mẫu thí nghiệm.
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$) — độ lệch nhiệt độ của các mẫu.

Đầu ra (Output):

- In ra dãy số sau khi sắp xếp trên một dòng, các phần tử cách nhau bởi một khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 5 -8 2 -3 8	2 -3 5 -8 8

Giải thích: Xét dãy số ban đầu: 5, -8, 2, -3, 8.

- Giá trị tuyệt đối của các phần tử lần lượt là: $|5| = 5$, $|-8| = 8$, $|2| = 2$, $|-3| = 3$, $|8| = 8$.
- Sắp xếp theo thứ tự độ lớn tăng dần:
- $|2| = 2 \Rightarrow 2$ đứng đầu.
- $|-3| = 3 \Rightarrow -3$ đứng tiếp theo.
- $|5| = 5 \Rightarrow 5$ đứng tiếp theo.
- Với hai phần tử có độ lớn bằng nhau là -8 và 8 (cùng có trị tuyệt đối là 8): theo quy tắc ưu tiên, số âm -8 phải đứng trước số dương 8. Kết quả thu được: 2 -3 5 -8 8.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho một dãy gồm N số nguyên $A_1, A_2, \dots, A_N$. Hãy sắp xếp các phần tử theo giá trị tuyệt đối tăng dần. Nếu hai phần tử có cùng giá trị tuyệt đối, phần tử mang dấu âm phải đứng trước phần tử mang dấu dương.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—|—| 1 | Nhập dữ liệu vào mảng/biến | Input: 5 5 -8 2 -3 8 | Khởi tạo cấu trúc dữ liệu ban đầu
| 2 | Thực thi thuật toán tối ưu | Xét dãy số ban đầu: 5, -8, 2, -3, 8. - Giá trị tuyệt đối của các phần tử lần lượt là: $|5| = 5$, $|-8| = 8$, $|2| = 2$... | Tính toán từng bước trạng thái | 3 | Xuất kết quả |
Output: 2 -3 5 -8 8 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Xét dãy số ban đầu: 5, -8, 2, -3, 8.

- Giá trị tuyệt đối của các phần tử lần lượt là: $|5| = 5$, $|-8| = 8$, $|2| = 2$, $|-3| = 3$, $|8| = 8$.
- Sắp xếp theo thứ tự độ lớn tăng dần:
- $|2| = 2 \Rightarrow 2$ đứng đầu.
- $|-3| = 3 \Rightarrow -3$ đứng tiếp theo.
- $|5| = 5 \Rightarrow 5$ đứng tiếp theo.
- Với hai phần tử có độ lớn bằng nhau là -8 và 8 (cùng có trị tuyệt đối là 8): theo quy tắc ưu tiên, số âm -8 phải đứng trước số dương 8. Kết quả thu được: 2 -3 5 -8 8.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

bool cmp(long long u, long long v) {
    if (abs(u) != abs(v)) return abs(u) < abs(v);
    return u < v;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<long long> a(n);
```

```
for (int i = 0; i < n; ++i) cin >> a[i];

sort(a.begin(), a.end(), cmp);

for (int i = 0; i < n; ++i) {
    cout << a[i] << (i == n - 1 ? "" : " ");
}
cout << "\n";
return 0;
}
```

Bài 04 [cppb_sx_04_dem_gia_tri_phan_biet]: Đếm Giá Trị Phân Biệt

Bối cảnh: Tại một cổng kiểm soát vé vào hội chợ công nghệ, hệ thống quét mã vạch ghi nhận mã số thẻ của N lượt khách ra vào trong ngày. Do một số khách hàng có thể quét thẻ nhiều lần khi di chuyển qua lại giữa các khu vực triển lãm, danh sách các mã vé thu thập được có hiện tượng lặp lại. Ban tổ chức cần thống kê chính xác số lượng khách tham quan thực tế (tức số lượng mã số thẻ độc nhất, phân biệt nhau) đã đến dự hội chợ.



Hình minh họa: Mô hình sắp xếp thứ tự và tối ưu hóa vị trí

Nhiệm vụ: Cho danh sách N mã số thẻ nguyên $A_1, A_2, \dots, A_N$. Hãy đếm và in ra số lượng giá trị phân biệt trong dãy số đó.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 2 \cdot 10^5$) — số lượt quét mã vé.
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$) — danh sách mã vé được ghi nhận.

Đầu ra (Output):

- In ra một số nguyên duy nhất là số lượng giá trị phân biệt xuất hiện trong dãy.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 2 3 2 1 3 5	4

Giải thích: Danh sách mã vé ghi nhận là: 2,3,2,1,3,5. Sau khi sắp xếp tăng dần: 1,2,2,3,3,5. Các nhóm giá trị trùng nhau được gom liền kề:

- Giá trị 1 (xuất hiện 1 lần)

- Giá trị 2 (xuất hiện 2 lần)
- Giá trị 3 (xuất hiện 2 lần)
- Giá trị 5 (xuất hiện 1 lần) Tổng cộng có 4 giá trị phân biệt khác nhau là $\{1,2,3,5\}$. Do đó in ra kết quả là 4.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách N mã số thẻ nguyên $A_1, A_2, \dots, A_N$. Hãy đếm và in ra số lượng giá trị phân biệt trong dãy số đó.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 6 2 3 2 1 3 5 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Danh sách mã vé ghi nhận là: 2,3,2,1,3,5. Sau khi sắp xếp tăng dần: 1,2,2,3,3,5. Các nhóm giá trị trùng nh... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 4 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Danh sách mã vé ghi nhận là: 2,3,2,1,3,5. Sau khi sắp xếp tăng dần: 1,2,2,3,3,5. Các nhóm giá trị trùng nhau được gom liền kè:

- Giá trị 1 (xuất hiện 1 lần)
- Giá trị 2 (xuất hiện 2 lần)
- Giá trị 3 (xuất hiện 2 lần)
- Giá trị 5 (xuất hiện 1 lần) Tổng cộng có 4 giá trị phân biệt khác nhau là $\{1,2,3,5\}$. Do đó in ra kết quả là 4.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];
```

```
sort(a.begin(), a.end());

int cnt = 1;
for (int i = 1; i < n; ++i) {
    if (a[i] != a[i - 1]) ++cnt;
}

cout << cnt << "\n";
return 0;
}
```

Bài 05 [cppb_sx_05_hai_tram_kiem_soat]: Hai Trạm Kiểm Soát Gần Nhau Nhất

Bối cảnh: Trên một tuyến hành lang vận tải đường sắt cao tốc thẳng tắp kéo dài, ban điều hành dự án bố trí N trạm kiểm soát tự động tại các vị trí có tọa độ $X_1, X_2, \dots, X_N$ (tính theo mét so với điểm gốc 0). Vì tuyến đường rất dài (tọa độ có thể lên đến 10^{12} m), các thiết bị cảm biến liên lạc giữa các trạm đòi hỏi một khoảng cách đệm an toàn tối thiểu để tránh xung đột kênh truyền. Bộ phận kỹ thuật cần tìm ra khoảng cách nhỏ nhất giữa hai trạm bất kỳ để kiểm tra mức độ an toàn kỹ thuật.



Hình minh họa: Mô hình sắp xếp thứ tự và tối ưu hóa vị trí

Nhiệm vụ: Cho danh sách tọa độ của N trạm kiểm soát. Hãy tìm và in ra khoảng cách ngắn nhất giữa hai trạm kiểm soát bất kỳ trên tuyến đường.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($2 \leq N \leq 10^5$) — số lượng trạm kiểm soát.
- Dòng 2: N số nguyên không âm $X_1, X_2, \dots, X_N$ ($0 \leq X_i \leq 10^{12}$) — tọa độ của các trạm.

Đầu ra (Output):

- In ra một số nguyên duy nhất là khoảng cách ngắn nhất giữa hai trạm.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 1500 300 2800 800 1200 3150	300

Giải thích: Tọa độ ban đầu của 6 trạm là: 1500,300,2800,800,1200,3150. Sắp xếp các trạm theo thứ tự tăng dần của tọa độ dọc tuyến đường: 300,800,1200,1500,2800,3150. Khoảng cách giữa các trạm liên tiếp:

- $800 - 300 = 500$
- $1200 - 800 = 400$
- $1500 - 1200 = 300$
- $2800 - 1500 = 1300$
- $3150 - 2800 = 350$

Khoảng cách ngắn nhất đạt được là 300 mét (giữa hai trạm tại tọa độ 1200 và 1500).

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách tọa độ của N trạm kiểm soát. Hãy tìm và in ra khoảng cách ngắn nhất giữa hai trạm kiểm soát bất kỳ trên tuyến đường.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 6 1500 300 2800 800 1200 3150 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Tọa độ ban đầu của 6 trạm là: 1500,300,2800,800,1200,3150. Sắp xếp các trạm theo thứ tự tăng dần của tọa độ dọc t... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 300 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Tọa độ ban đầu của 6 trạm là: 1500,300,2800,800,1200,3150. Sắp xếp các trạm theo thứ tự tăng dần của tọa độ dọc tuyến đường: 300,800,1200,1500,2800,3150. Khoảng cách giữa các trạm liên tiếp:

- $800 - 300 = 500$
- $1200 - 800 = 400$
- $1500 - 1200 = 300$
- $2800 - 1500 = 1300$
- $3150 - 2800 = 350$

Khoảng cách ngắn nhất đạt được là 300 mét (giữa hai trạm tại tọa độ 1200 và 1500).

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<long long> x(n);
    for (int i = 0; i < n; ++i) cin >> x[i];

    sort(x.begin(), x.end());

    long long min_dist = x[1] - x[0];
    for (int i = 1; i < n - 1; ++i) {
        min_dist = min(min_dist, x[i + 1] - x[i]);
    }

    cout << min_dist << "\n";
    return 0;
}
```

Bài 06 [cppb_sx_06_khoang_trong_lon_nhat]: Khoảng Trống Lớn Nhất Trên Trục Tọa Độ

Bối cảnh: Trong một trò chơi thám hiểm vũ trụ ảo, một con tàu không gian phải di chuyển dọc theo một hành lang hẹp trên trục tọa độ 1 chiều. Trên hành lang này xuất hiện N chương ngại vật thiên thạch tại các tọa độ $A_1, A_2, \dots, A_N$. Để con tàu có thể kích hoạt động cơ siêu tốc một cách an toàn, nó cần tìm ra khoảng không gian trống lớn nhất giữa hai chương ngại vật liên tiếp nhau dọc theo hành trình.

KHOẢNG CÁCH NGĂN NHẤT GIỮA HAI TRẠM QUAN SÁT



Sau khi sắp xếp, khoảng cách nhỏ nhất luôn nằm ở hai phần tử kề nhau

Hình minh họa: Mô hình sắp xếp thứ tự và tối ưu hóa vị trí

Nhiệm vụ: Cho danh sách tọa độ của N chương ngại vật. Hãy tìm và in ra khoảng cách lớn nhất giữa hai chương ngại vật liên tiếp sau khi sắp xếp vị trí của chúng theo thứ tự tăng dần trên trục tọa độ.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($2 \leq N \leq 10^5$) — số lượng chương ngại vật.

- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($-10^{18} \leq A_i \leq 10^{18}$) — tọa độ các chương ngại vật.

Đầu ra (Output):

- In ra một số nguyên duy nhất là khoảng cách lớn nhất giữa hai chương ngại vật liên tiếp.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 10 3 25 8 12	13

Giải thích: Tọa độ các chương ngại vật ban đầu là: 10,3,25,8,12. Sau khi sắp xếp tăng dần theo chiều dọc hàng lang: 3,8,10,12,25. Khoảng cách giữa các chương ngại vật liên tiếp lần lượt là:

- $8 - 3 = 5$
- $10 - 8 = 2$
- $12 - 10 = 2$
- $25 - 12 = 13$

Khoảng trống lớn nhất giữa hai chương ngại vật liên tiếp là 13 (giữa vị trí 12 và 25).

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách tọa độ của N chương ngại vật. Hãy tìm và in ra khoảng cách lớn nhất giữa hai chương ngại vật liên tiếp sau khi sắp xếp vị trí của chúng theo thứ tự tăng dần trên trục tọa độ.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 10 3 25 8 12 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Tọa độ các chương ngại vật ban đầu là: 10,3,25,8,12. Sau khi sắp xếp tăng dần theo chiều dọc hàng lang: 3, 8, 10,... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 13 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Tọa độ các chương ngại vật ban đầu là: 10,3,25,8,12. Sau khi sắp xếp tăng dần theo chiều dọc hàng lang: 3,8,10,12,25. Khoảng cách giữa các chương ngại vật liên tiếp lần lượt là:

- $8 - 3 = 5$
- $10 - 8 = 2$
- $12 - 10 = 2$
- $25 - 12 = 13$

Khoảng trống lớn nhất giữa hai chương ngại vật liên tiếp là 13 (giữa vị trí 12 và 25).

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.

- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    sort(a.begin(), a.end());

    long long max_gap = 0;
    for (int i = 0; i < n - 1; ++i) {
        max_gap = max(max_gap, a[i + 1] - a[i]);
    }

    cout << max_gap << "\n";
    return 0;
}
```

Bài 07 [cppb_sx_07_sap_xep_tong_chu_so]: Sắp Xếp Theo Tổng Chữ Số

Bối cảnh: Trong một trò chơi giải mật mã cổ xưa, người chơi nhận được N phiến đá mang các con số nguyên dương bí ẩn $A_1, A_2, \dots, A_N$. Quy luật mở cánh cổng cổ thành yêu cầu người chơi phải đặt các phiến đá vào rãnh theo thứ tự tăng dần của **tổng các chữ số** cấu thành nên con số đó. Trong trường hợp có nhiều con số có cùng tổng chữ số, phiến đá có giá trị nhỏ hơn sẽ được ưu tiên đặt trước.



Hình minh họa: Mô hình sắp xếp thứ tự và tối ưu hóa vị trí

Nhiệm vụ: Cho danh sách N số nguyên dương. Hãy sắp xếp dãy số theo quy tắc:

1. Tổng các chữ số tăng dần.
2. Nếu hai số có cùng tổng các chữ số, số có giá trị nguyên nhỏ hơn sẽ đứng trước.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 10^5$) — số lượng phiên đá.
- Dòng 2: N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$) — giá trị ghi trên các phiên đá.

Đầu ra (Output):

- In ra dãy số sau khi sắp xếp trên một dòng, các số cách nhau bởi một khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 13 20 4 103 11	11 20 4 13 103

Giải thích: Tính tổng chữ số của từng số trong dãy ban đầu:

- Số 13: tổng chữ số là $1 + 3 = 4$.
- Số 20: tổng chữ số là $2 + 0 = 2$.
- Số 4: tổng chữ số là 4.
- Số 103: tổng chữ số là $1 + 0 + 3 = 4$.
- Số 11: tổng chữ số là $1 + 1 = 2$.

Sắp xếp theo tổng chữ số tăng dần:

- Nhóm có tổng chữ số bằng 2: gồm {20,11}. Vì $11 < 20$ nên 11 đứng trước 20.
- Nhóm có tổng chữ số bằng 4: gồm {13,4,103}. Sắp xếp theo giá trị tăng dần: $4 < 13 < 103$.

Kết quả cuối cùng: 11 20 4 13 103.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách N số nguyên dương. Sắp xếp lại dãy số theo quy tắc:
 1. Số nào có **tổng các chữ số** nhỏ hơn sẽ đứng trước.
 2. Nếu hai số có cùng tổng chữ số, số có **giá trị nhỏ hơn** sẽ đứng trước.
- **Phương pháp tiếp cận — Hàm tính tổng chữ số & Custom Comparator:**
- Viết hàm phụ trợ `int sum_digits(long long n)`: Dùng vòng lặp `while (n > 0)` lấy `n % 10` cộng dồn vào tổng, sau đó `n /= 10`.
- Định nghĩa hàm so sánh `bool cmp(long long u, long long v)`:

```
int su = sum_digits(u), sv = sum_digits(v);
if (su != sv) return su < sv;
return u < v;
```

- Gọi `sort(a.begin(), a.end(), cmp)`. Độ phức tạp thời gian: $\mathcal{O}(N \log N \cdot \log_{10}(\max A))$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử: $N = 5$, mảng ban đầu $a = [15, 20, 9, 32, 11]$.

Phần tử x	Tổng chữ số $S(x)$	Thứ tự ưu tiên sau phân tích
-------------	--------------------	------------------------------

20	$2 + 0 = 2$	Ưu tiên 1 (tổng chữ số nhỏ nhất là 2)
11	$1 + 1 = 2$	Ưu tiên 2 (cùng tổng 2, nhưng $20 > 11$ nên 11 đứng trước 20)
32	$3 + 2 = 5$	Ưu tiên 3 (tổng 5)
15	$1 + 5 = 6$	Ưu tiên 4 (tổng 6)
9	9	Ưu tiên 5 (tổng chữ số lớn nhất là 9)

Kết quả sau khi sắp xếp chuẩn xác: 11 20 32 15 9.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tính lại tổng chữ số quá nhiều lần:** Trong hàm cmp, việc gọi sum_digits liên tục trong mỗi phép so sánh vẫn chấp nhận được khi $N \leq 10^5$. Tuy nhiên, để tối ưu tốc độ tối đa, có thể tiền tính tổng chữ số và lưu dưới dạng pair<int, long long> (tổng chữ số, giá trị gốc).
- **Bẫy 2 — Số 0:** Nếu số có thể bằng 0, hàm tính tổng chữ số phải xử lý đúng trường hợp này (tổng bằng 0).

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

long long sum_digits(long long x) {
    long long s = 0;
    while (x > 0) {
        s += x % 10;
        x /= 10;
    }
    return s;
}

bool cmp(long long a, long long b) {
    long long sa = sum_digits(a);
    long long sb = sum_digits(b);
    if (sa != sb) return sa < sb;
    return a < b;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

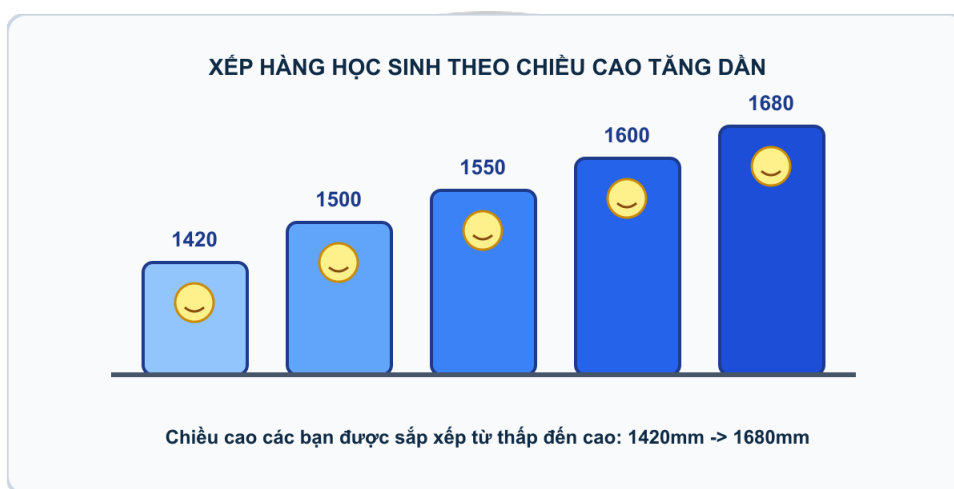
    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    sort(a.begin(), a.end(), cmp);
}
```

```
for (int i = 0; i < n; ++i) {
    cout << a[i] << (i == n - 1 ? "" : " ");
}
cout << "\n";
return 0;
}
```

Bài 08 [cppb_sx_08_gom_cum_chenh_lech_k]: Gom Cụm Chênh Lệch Không Quá K

Bối cảnh: Để chuẩn bị cho kỳ thi học sinh giỏi, câu lạc bộ Tin học có N bạn học sinh tham gia với mức điểm năng lực khảo sát ban đầu lần lượt là $A_1, A_2, \dots, A_N$. Huấn luyện viên muốn chia các bạn học sinh thành các nhóm học tập sao cho trình độ trong mỗi nhóm tương đối đồng đều: chênh lệch điểm số giữa bạn có điểm cao nhất và bạn có điểm thấp nhất trong cùng một nhóm không được vượt quá K . Để việc giảng dạy và quản lý lớp học đạt hiệu quả cao nhất, huấn luyện viên mong muốn số lượng nhóm cần chia là ít nhất có thể.



Hình minh họa: Mô hình sắp xếp thứ tự và tối ưu hóa vị trí

Nhiệm vụ: Cho điểm năng lực của N bạn học sinh và số nguyên K . Hãy tìm số lượng nhóm ít nhất để phân chia toàn bộ N học sinh thỏa mãn điều kiện chênh lệch tối đa giữa bạn cao nhất và bạn thấp nhất trong mỗi nhóm không quá K .

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên dương N và K ($1 \leq N \leq 2 \cdot 10^5, 0 \leq K \leq 10^9$).
- Dòng 2: N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$) — điểm năng lực của các học sinh.

Đầu ra (Output):

- In ra một số nguyên duy nhất là số lượng nhóm ít nhất cần chia.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 3 1 10 3 4 12 15	3

Giải thích: Sắp xếp điểm năng lực của 6 bạn học sinh theo thứ tự tăng dần: 1,3,4,10,12,15. Với $K = 3$, ta có thể gom tối ưu thành 3 nhóm như sau:

- Nhóm 1: {1,3,4} (Điểm cao nhất là 4, thấp nhất là 1, chênh lệch $4 - 1 = 3 \leq 3$).
- Nhóm 2: {10,12} (Điểm cao nhất là 12, thấp nhất là 10, chênh lệch $12 - 10 = 2 \leq 3$).
- Nhóm 3: {15} (Chỉ gồm 1 bạn, chênh lệch bằng $0 \leq 3$).

Không thể chia thành ít hơn 3 nhóm mà vẫn thỏa mãn điều kiện. Vì vậy kết quả là 3.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho điểm năng lực của N bạn học sinh và số nguyên K . Hãy tìm số lượng nhóm ít nhất để phân chia toàn bộ N học sinh thỏa mãn điều kiện chênh lệch tối đa giữa bạn cao nhất và bạn thấp nhất trong mỗi nhóm không quá K .

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 6 3 1 10 3 4 12 15 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Sắp xếp điểm năng lực của 6 bạn học sinh theo thứ tự tăng dần: 1,3,4,10,12,15. Với $K = 3$, ta có thể gom tối ưu ... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 3 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Sắp xếp điểm năng lực của 6 bạn học sinh theo thứ tự tăng dần: 1,3,4,10,12,15. Với $K = 3$, ta có thể gom tối ưu thành 3 nhóm như sau:

- Nhóm 1: {1,3,4} (Điểm cao nhất là 4, thấp nhất là 1, chênh lệch $4 - 1 = 3 \leq 3$).
- Nhóm 2: {10,12} (Điểm cao nhất là 12, thấp nhất là 10, chênh lệch $12 - 10 = 2 \leq 3$).
- Nhóm 3: {15} (Chỉ gồm 1 bạn, chênh lệch bằng $0 \leq 3$).

Không thể chia thành ít hơn 3 nhóm mà vẫn thỏa mãn điều kiện. Vì vậy kết quả là 3.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 .. N-1$) và 1-based ($1 .. N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long k;
    if (!(cin >> n >> k)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];
```

```

sort(a.begin(), a.end());

int groups = 1;
long long min_val = a[0];

for (int i = 1; i < n; ++i) {
    if (a[i] - min_val > k) {
        ++groups;
        min_val = a[i];
    }
}

cout << groups << "\n";
return 0;
}

```

Bài 09 [cppb_sx_09_phan_tu_xuat_hien_nhieu_nhat]: Tìm Phần Tử Xuất Hiện Nhiều Nhất

Bối cảnh: Trong một đợt khảo sát ý kiến bình chọn sản phẩm được yêu thích nhất của một trang thương mại điện tử, ban tổ chức thu thập được N phiếu bầu có mã định danh sản phẩm lần lượt là $A_1, A_2, \dots, A_N$. Ban tổ chức cần công bố sản phẩm chiến thắng — tức sản phẩm nhận được số lượng phiếu bầu nhiều nhất. Trong trường hợp có nhiều sản phẩm cùng đạt được số phiếu bầu cao nhất, ban tổ chức sẽ ưu tiên trao giải cho sản phẩm có mã số định danh nhỏ nhất.



Hình minh họa: Mô hình sắp xếp thứ tự và tối ưu hóa vị trí

Nhiệm vụ: Cho danh sách N số nguyên đại diện cho các mã phiếu bầu. Hãy tìm phần tử có tần suất xuất hiện nhiều nhất trong dãy. Nếu có nhiều phần tử có cùng tần suất cực đại, hãy in ra phần tử có giá trị nhỏ nhất cùng với số lần xuất hiện của nó.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 2 \cdot 10^5$) — số lượng phiếu bầu.
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$) — danh sách mã phiếu bầu.

Đầu ra (Output):

- In ra hai số nguyên cách nhau bởi khoảng trắng: số đầu tiên là giá trị của phần tử xuất hiện nhiều nhất, số thứ hai là số lần xuất hiện của phần tử đó.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
7 3 5 2 3 5 3 2	3 3

Giải thích: Danh sách các phiếu bầu là: 3,5,2,3,5,3,2. Thống kê tần suất xuất hiện của từng giá trị:

- Mã số 2: xuất hiện 2 lần.
- Mã số 3: xuất hiện 3 lần.
- Mã số 5: xuất hiện 2 lần.

Mã số xuất hiện nhiều nhất là 3 với số lần xuất hiện là 3. Do đó kết quả in ra là 3 3.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách N số nguyên đại diện cho các mã phiếu bầu. Hãy tìm phần tử có tần suất xuất hiện nhiều nhất trong dãy. Nếu có nhiều phần tử có cùng tần suất cực đại, hãy in ra phần tử có giá trị nhỏ nhất cùng với số lần xuất hiện của nó.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 7 3 5 2 3 5 3 2 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Danh sách các phiếu bầu là: 3,5,2,3,5,3,2. Thống kê tần suất xuất hiện của từng giá trị: - Mã số 2: xuất hiện ... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 3 3 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Danh sách các phiếu bầu là: 3,5,2,3,5,3,2. Thống kê tần suất xuất hiện của từng giá trị:

- Mã số 2: xuất hiện 2 lần.
- Mã số 3: xuất hiện 3 lần.
- Mã số 5: xuất hiện 2 lần.

Mã số xuất hiện nhiều nhất là 3 với số lần xuất hiện là 3. Do đó kết quả in ra là 3 3.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```

int n;
if (!(cin >> n)) return 0;

vector<long long> a(n);
for (int i = 0; i < n; ++i) cin >> a[i];

sort(a.begin(), a.end());

long long best_val = a[0];
int max_freq = 1;

long long cur_val = a[0];
int cur_freq = 1;

for (int i = 1; i < n; ++i) {
    if (a[i] == cur_val) {
        ++cur_freq;
    } else {
        if (cur_freq > max_freq) {
            max_freq = cur_freq;
            best_val = cur_val;
        }
        cur_val = a[i];
        cur_freq = 1;
    }
}
if (cur_freq > max_freq) {
    max_freq = cur_freq;
    best_val = cur_val;
}

cout << best_val << " " << max_freq << "\n";
return 0;
}

```

Bài 10 [cppb_sx_10_sap_xep_luu_vi_tri]: Sắp Xếp Lưu Vị Trí Ban Đầu

Bối cảnh: Trong một cuộc thi marathon tiếp sức, ban tổ chức gắn chip định vị ghi nhận tốc độ chạy trung bình của N vận động viên. Mỗi vận động viên xuất phát từ một vị trí tương ứng với số thứ tự đăng ký ban đầu từ 1 đến N , với tốc độ đạt được lần lượt là $A_1, A_2, \dots, A_N$. Để phục vụ công tác xếp hạng phân loại và trao huy chương sau chặng đua, ban trọng tài cần sắp xếp lại các vận động viên theo thứ tự tốc độ tăng dần, đồng thời phải truy vết chính xác số thứ tự xuất phát ban đầu của từng người. Nếu có hai vận động viên đạt cùng một mức tốc độ, người nào xuất phát trước (có số thứ tự ban đầu nhỏ hơn) sẽ được xếp đứng trước.



Hình minh họa: Mô hình sắp xếp thứ tự và tối ưu hóa vị trí

Nhiệm vụ: Cho một dãy gồm N số nguyên $A_1, A_2, \dots, A_N$. Hãy sắp xếp các phần tử theo thứ tự giá trị tăng dần, đồng thời in ra giá trị và vị trí ban đầu (chỉ số 1-indexed) của mỗi phần tử trong mảng gốc. Nếu hai phần tử có cùng giá trị, phần tử xuất hiện trước trong mảng gốc sẽ đứng trước.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 10^5$) — số lượng phần tử.
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$) — giá trị của các phần tử.

Đầu ra (Output):

- In ra N dòng, mỗi dòng gồm 2 số nguyên cách nhau bởi khoảng trắng: số đầu tiên là giá trị phần tử sau khi sắp xếp, số thứ hai là chỉ số ban đầu của nó trong mảng gốc.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 40 10 20 10 30	10 2 10 4 20 3 30 5 40 1

Giải thích: Dãy ban đầu cùng vị trí gốc (1-indexed) là:

- Vị trí 1: 40
- Vị trí 2: 10
- Vị trí 3: 20
- Vị trí 4: 10
- Vị trí 5: 30

Sau khi sắp xếp theo giá trị tăng dần:

- Giá trị 10: có ở vị trí 2 và vị trí 4. Vì $2 < 4$ nên in 10 2 trước, sau đó in 10 4.
- Giá trị 20: ở vị trí 3 $\Rightarrow$ in 20 3.
- Giá trị 30: ở vị trí 5 $\Rightarrow$ in 30 5.
- Giá trị 40: ở vị trí 1 $\Rightarrow$ in 40 1.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho một dãy gồm N số nguyên $A_1, A_2, \dots, A_N$. Hãy sắp xếp các phần tử theo thứ tự giá trị tăng dần, đồng thời in ra giá trị và vị trí ban đầu (chỉ số 1-indexed) của mỗi phần tử trong mảng gốc. Nếu hai phần tử có cùng giá trị, phần tử xuất hiện trước trong mảng gốc sẽ đứng trước.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 40 10 20 10 30 | Khởi tạo cấu trúc dữ liệu ban đầu | 2 | Thực thi thuật toán tối ưu | Dãy ban đầu cùng vị trí gốc (1-indexed) là: - Vị trí 1: 40 - Vị trí 2: 10 - Vị trí 3: 20 - Vị trí 4: 10 - Vị trí... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 10 2 10 4 20 3 30 5 40 1 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Dãy ban đầu cùng vị trí gốc (1-indexed) là:

- Vị trí 1: 40
- Vị trí 2: 10
- Vị trí 3: 20
- Vị trí 4: 10
- Vị trí 5: 30

Sau khi sắp xếp theo giá trị tăng dần:

- Giá trị 10: có ở vị trí 2 và vị trí 4. Vì $2 < 4$ nên in 10 2 trước, sau đó in 10 4.
- Giá trị 20: ở vị trí 3 $\Rightarrow$ in 20 3.
- Giá trị 30: ở vị trí 5 $\Rightarrow$ in 30 5.
- Giá trị 40: ở vị trí 1 $\Rightarrow$ in 40 1.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
```

```
vector<vector<long long>> a(n, vector<long long>(2));
for (int i = 0; i < n; ++i) {
    cin >> a[i][0];
    a[i][1] = i + 1;
}

sort(a.begin(), a.end());

for (int i = 0; i < n; ++i) {
    cout << a[i][0] << " " << a[i][1] << "\n";
}

return 0;
}
```

Bài 11 [cppb_sx_11_ghep_so_lon_nhat]: Ghép Chuỗi Tạo Số Lớn Nhất

Bối cảnh: Tại một triển lãm công nghệ robot tương tác, một thiết bị điều khiển cần thiết lập mã khóa bảo mật cấp cao bằng cách ghép N mảnh thẻ từ chứa các chữ số $S_1, S_2, \dots, S_N$. Mỗi mảnh thẻ là một chuỗi ký tự số không âm. Để hệ thống đạt mức bảo mật tối đa, mã số cuối cùng thu được sau khi ghép toàn bộ N mảnh thẻ lại với nhau thành một con số duy nhất phải là số có giá trị lớn nhất có thể.



Hình minh họa: Mô hình sắp xếp thứ tự và tối ưu hóa vị trí

Nhiệm vụ: Cho danh sách N chuỗi số nguyên không âm. Hãy tìm cách sắp xếp và ghép nối toàn bộ N chuỗi số này lại với nhau để tạo thành một số có giá trị lớn nhất.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 10^4$) — số lượng chuỗi số.
- Dòng 2: N chuỗi số $S_1, S_2, \dots, S_N$ (độ dài mỗi chuỗi không quá 10 ký tự).

Đầu ra (Output):

- In ra chuỗi số lớn nhất có thể tạo được. Nếu kết quả chỉ gồm toàn các chữ số 0, chỉ cần in ra đúng một chữ số 0.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 3 30 34 5	534330

Giải thích: Xét 4 mảnh thẻ: "3", "30", "34", "5".

- Khi so sánh giữa "3" và "30", ghép "3" + "30" = "330", còn "30" + "3" = "303". Vì "330" > "303" nên "3" phải đứng trước "30".
- Tương tự, "34" + "3" = "343" lớn hơn "3" + "34" = "334", nên "34" phải đứng trước "3".
- Mảnh "5" khi ghép với bất kỳ mảnh nào khác ở đầu luôn tạo ra tiền tố 5... lớn nhất.

Thứ tự ghép tối ưu từ trước ra sau là: "5", "34", "3", "30". Chuỗi kết quả thu được là 534330.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách N số nguyên không âm. Ghép toàn bộ các số này lại thành một số nguyên lớn nhất có thể.
- **Tại sao so sánh thông thường bị sai**
- Nếu so sánh theo thứ tự từ điển thông thường ("9" > "34" đúng, nhưng "3" > "30" thì "30" lại dài hơn "3"). Nếu xếp "30" trước "3" ta được "303", trong khi "3" trước "30" cho "330" lớn hơn!
- **Tính chất bắc cầu của phép ghép (Greedy Comparator):**
- Để quyết định giữa hai chuỗi u và v chuỗi nào nên đứng trước, ta so sánh trực tiếp kết quả của hai cách ghép:

$$\text{Nếu } u + v > v + u \Rightarrow u \text{ phải đứng trước } v.$$
- Quan hệ này thỏa mãn tính chất phản đối xứng và bắc cầu (Strict Weak Ordering), cho phép hàm sort định hình đúng toàn bộ dãy ghép.

2. Bảng chạy tay trên số liệu mẫu

Mẫu thử: Danh sách gồm 4 chuỗi ["3", "30", "34", "5", "9"].

Cặp so sánh (u, v)	Ghép $u + v$	Ghép $v + u$	Quyết định
"9" và "5"	"95"	"59"	"9" đứng trước "5"
"34" và "3"	"343"	"334"	"34" đứng trước "3"
"3" và "30"	"330"	"303"	"3" đứng trước "30"

Sau khi sắp xếp: ["9", "5", "34", "3", "30"]. Ghép lại được số lớn nhất: 9534330.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Toàn số 0:** Nếu input gồm các số [0, 0, 0], kết quả ghép sẽ là "000". Đáp số hợp lệ của bài toán khi đó chỉ là một số "0". Cần kiểm tra nếu phần tử đầu tiên sau khi sắp xếp là "0" thì in ngay "0" và kết thúc.
- **Bẫy 2 — Dùng dấu $\geq$ trong comparator:** Viết return $u + v \geq v + u$; sẽ gây lỗi vi phạm Strict Weak Ordering dẫn đến crash chương trình khi gặp hai số giống hệt nhau.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

bool cmp(const string &a, const string &b) {
    return a + b > b + a;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<string> s(n);
    for (int i = 0; i < n; ++i) cin >> s[i];

    sort(s.begin(), s.end(), cmp);

    if (s[0] == "") {
        cout << 0 << "\n";
        return 0;
    }

    for (int i = 0; i < n; ++i) {
        cout << s[i];
    }
    cout << "\n";
    return 0;
}
```

Bài 12 [cppb_sx_12_bang_diem_hoc_sinh]: Bảng Điểm Học Sinh Đa Trường

Bối cảnh: Tại kỳ thi Olympic Khoa học Trẻ liên trường, hội đồng chấm thi tổng hợp kết quả của N thí sinh tham gia hai môn thi bắt buộc là Toán và Tin học. Mỗi thí sinh có một mã định danh ID duy nhất, cùng điểm số môn Toán (*Math*) và điểm số môn Tin học (*Info*). Để công bố danh sách tuyên dương và xét học bổng, hội đồng thi cần xây dựng bảng xếp hạng chung toàn cuộc thi dựa trên nhiều tiêu chí chặt chẽ.



Hình minh họa: Mô hình sắp xếp thứ tự và tối ưu hóa vị trí

Nhiệm vụ: Cho danh sách N học sinh cùng điểm số của hai môn thi. Hãy sắp xếp danh sách học sinh theo các quy tắc ưu tiên sau:

1. Tổng điểm hai môn ($Math + Info$) giảm dần.
2. Nếu bằng tổng điểm, thí sinh có điểm môn Tin học ($Info$) cao hơn sẽ đứng trước.
3. Nếu vẫn bằng nhau cả về điểm Tin học, thí sinh có mã số định danh ID nhỏ hơn sẽ đứng trước.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 10^5$) — số lượng thí sinh.
- N dòng tiếp theo: Mỗi dòng gồm 3 số nguyên $ID, Math, Info$ ($1 \leq ID \leq 10^9, 0 \leq Math, Info \leq 100$) lần lượt là mã số, điểm Toán và điểm Tin học của thí sinh.

Đầu ra (Output):

- In ra danh sách học sinh sau khi đã sắp xếp thứ tự, mỗi học sinh trên một dòng gồm 3 số nguyên $ID, Math, Info$ cách nhau bởi khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 101 8 9 102 9 8 103 10 10	103 10 10 101 8 9 102 9 8

Giải thích: Thống kê điểm của 3 thí sinh:

- Thí sinh 103: Điểm Toán = 10, Điểm Tin = 10 $\Rightarrow$ Tổng điểm = 20.
- Thí sinh 101: Điểm Toán = 8, Điểm Tin = 9 $\Rightarrow$ Tổng điểm = 17.
- Thí sinh 102: Điểm Toán = 9, Điểm Tin = 8 $\Rightarrow$ Tổng điểm = 17.

Xếp hạng theo các tiêu chí:

- Thí sinh 103 có tổng điểm cao nhất (20) nên đứng vị trí số 1.
- Giữa hai thí sinh 101 và 102 có cùng tổng điểm là 17: xét tiêu chí phụ điểm Tin học, thí sinh 101 có điểm Tin $9 > 8$ của thí sinh 102, do đó thí sinh 101 xếp trước thí sinh 102.

Kết quả in ra đúng thứ tự: 103 10 10, tiếp theo là 101 8 9, và cuối cùng là 102 9 8.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách N học sinh cùng điểm số của hai môn thi. Hãy sắp xếp danh sách học sinh theo các quy tắc ưu tiên sau:
 1. Tổng điểm hai môn ($Math + Info$) giảm dần.
 2. Nếu bằng tổng điểm, thí sinh có điểm môn Tin học ($Info$) cao hơn sẽ đứng trước.
 3. Nếu vẫn bằng nhau cả về điểm Tin học, thí sinh có mã số định danh ID nhỏ hơn sẽ đứng trước.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 101 8 9 102 9 8 103 10 10 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Thống kê điểm của 3 thí sinh: - Thí sinh 103: Điểm Toán = 10, Điểm Tin = 10 $\Rightarrow$ Tổng điểm = 20. - Thí sinh ... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 103 10 10 101 8 9 102 9 8 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Thống kê điểm của 3 thí sinh:

- Thí sinh 103: Điểm Toán = 10, Điểm Tin = 10 $\Rightarrow$ Tổng điểm = 20.
- Thí sinh 101: Điểm Toán = 8, Điểm Tin = 9 $\Rightarrow$ Tổng điểm = 17.
- Thí sinh 102: Điểm Toán = 9, Điểm Tin = 8 $\Rightarrow$ Tổng điểm = 17.

Xếp hạng theo các tiêu chí:

- Thí sinh 103 có tổng điểm cao nhất (20) nên đứng vị trí số 1.
- Giữa hai thí sinh 101 và 102 có cùng tổng điểm là 17: xét tiêu chí phụ điểm Tin học, thí sinh 101 có điểm Tin 9 > 8 của thí sinh 102, do đó thí sinh 101 xếp trước thí sinh 102.

Kết quả in ra đúng thứ tự: 103 10 10, tiếp theo là 101 8 9, và cuối cùng là 102 9 8.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

bool cmp(const vector<long long> &a, const vector<long long> &b) {
    long long total_a = a[1] + a[2];
    long long total_b = b[1] + b[2];
    if (total_a != total_b) return total_a > total_b;
    if (a[2] != b[2]) return a[2] > b[2];
    return a[0] < b[0];
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<vector<long long>> a(n, vector<long long>(3));
    for (int i = 0; i < n; ++i) {
        cin >> a[i][0] >> a[i][1] >> a[i][2];
    }
}
```

```
sort(a.begin(), a.end(), cmp);

for (int i = 0; i < n; ++i) {
    cout << a[i][0] << " " << a[i][1] << " " << a[i][2] << "\n";
}
return 0;
}
```

Bài 13 [cppb_sx_13_bang_xep_hang_the_thao]: Bảng Xếp Hạng Giải Đấu Thể Thao

Bối cảnh: Vòng bảng giải bóng đá thanh thiếu niên quy tụ N đội tuyển tranh tài vừa kết thúc. Ban tổ chức đã tổng hợp đầy đủ các thông số kỹ thuật của từng đội bóng để xác định thứ hạng vào vòng loại trực tiếp, bao gồm: Mã đội ID , Điểm số tích lũy $Points$, Hiệu số bàn thắng bại $GoalDiff$ (bàn thắng trừ bàn thua), và Tổng số bàn thắng ghi được $Goals$. Theo điều lệ thi đấu chuẩn quốc tế, bảng xếp hạng phải được phân định dựa trên một chuỗi các chỉ số phụ công bằng.



Hình minh họa: Mô hình sắp xếp thứ tự và tối ưu hóa vị trí

Nhiệm vụ: Cho thông số thi đấu của N đội bóng. Hãy xếp hạng các đội theo thứ tự ưu tiên sau:

1. Điểm số tích lũy ($Points$) giảm dần.
2. Nếu bằng điểm số, đội có Hiệu số bàn thắng bại ($GoalDiff$) lớn hơn sẽ đứng trước.
3. Nếu vẫn bằng cả điểm số và hiệu số, đội ghi được Tổng số bàn thắng ($Goals$) nhiều hơn sẽ đứng trước.
4. Nếu cả 3 chỉ số trên đều hoàn toàn bằng nhau, đội có Mã số định danh (ID) nhỏ hơn sẽ đứng trước.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 10^5$) — số lượng đội bóng.
- N dòng tiếp theo: Mỗi dòng gồm 4 số nguyên $ID, Points, GoalDiff, Goals$ ($1 \leq ID \leq 10^9, 0 \leq Points, Goals \leq 100, -100 \leq GoalDiff \leq 100$).

Đầu ra (Output):

- In ra danh sách mã đội ID sau khi đã sắp xếp hoàn tất thứ hạng từ cao xuống thấp trên một dòng, cách nhau bởi một khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 1 10 5 12 2 10 5 15 3 12 2 8	3 2 1

Giải thích: Xét thông số của 3 đội bóng:

- Đội 3: có 12 điểm (cao nhất) $\Rightarrow$ xếp vị trí số 1.
- Đội 1 và Đội 2: đều có 10 điểm và cùng có hiệu số bàn thắng bại là 5.
- Xét chỉ số phụ số bàn thắng ghi được: Đội 2 ghi được 15 bàn, trong khi Đội 1 chỉ ghi được 12 bàn ($15 > 12$).
- Do đó Đội 2 xếp thứ nhì, Đội 1 xếp thứ ba.

Thứ tự mã đội trên bảng xếp hạng là: 3 2 1.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho thông số thi đấu của N đội bóng. Hãy xếp hạng các đội theo thứ tự ưu tiên sau:
 1. Điểm số tích lũy (*Points*) giảm dần.
 2. Nếu bằng điểm số, đội có Hiệu số bàn thắng bại (*GoalDiff*) lớn hơn sẽ đứng trước.
 3. Nếu vẫn bằng cả điểm số và hiệu số, đội ghi được Tổng số bàn thắng (*Goals*) nhiều hơn sẽ đứng trước.
 4. Nếu cả 3 chỉ số trên đều hoàn toàn bằng nhau, đội có Mã số định danh (*ID*) nhỏ hơn sẽ đứng trước.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| — | — | — | — | — | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 1 10 5 12 2 10 5 15 3 12 2 8 | Khởi tạo cấu trúc dữ liệu ban đầu | 2 | Thực thi thuật toán tối ưu | Xét thông số của 3 đội bóng: - Đội 3: có 12 điểm (cao nhất) $\Rightarrow$ xếp vị trí số 1. - Đội 1 và Đội 2: đều có 10 ... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 3 2 1 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Xét thông số của 3 đội bóng:

- Đội 3: có 12 điểm (cao nhất) $\Rightarrow$ xếp vị trí số 1.
- Đội 1 và Đội 2: đều có 10 điểm và cùng có hiệu số bàn thắng bại là 5.
- Xét chỉ số phụ số bàn thắng ghi được: Đội 2 ghi được 15 bàn, trong khi Đội 1 chỉ ghi được 12 bàn ($15 > 12$).
- Do đó Đội 2 xếp thứ nhì, Đội 1 xếp thứ ba.

Thứ tự mã đội trên bảng xếp hạng là: 3 2 1.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.

- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

bool cmp(const vector<long long> &a, const vector<long long> &b) {
    if (a[1] != b[1]) return a[1] > b[1];
    if (a[2] != b[2]) return a[2] > b[2];
    if (a[3] != b[3]) return a[3] > b[3];
    return a[0] < b[0];
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<vector<long long>> a(n, vector<long long>(4));
    for (int i = 0; i < n; ++i) {
        cin >> a[i][0] >> a[i][1] >> a[i][2] >> a[i][3];
    }

    sort(a.begin(), a.end(), cmp);

    for (int i = 0; i < n; ++i) {
        cout << a[i][0] << (i == n - 1 ? "" : " ");
    }
    cout << "\n";
    return 0;
}
```

Bài 14 [cppb_sx_14_khac_phuc_strict_weak_ordering]: Sắp Xếp Đoạn Thẳng Không Giao Lỗi

Bối cảnh: Trong đồ họa máy tính và xử lý bản đồ số, một hệ thống cần hiển thị N đoạn thẳng nằm ngang trên trục tọa độ, mỗi đoạn được xác định bởi cặp mút đầu và mút cuối $[L_i, R_i]$. Để thuật toán quét đường biên (Sweep-line) hoạt động chính xác và không bị treo do vi phạm nguyên lý so sánh nghiêm ngặt (Strict Weak Ordering), các đoạn thẳng cần được sắp xếp theo một trật tự chuẩn mực: các đoạn bắt đầu sớm hơn phải được quét trước; nếu cùng điểm bắt đầu, đoạn dài hơn (điểm kết thúc xa hơn) quét trước; và nếu hai đoạn thẳng hoàn toàn trùng khít nhau, hệ thống phải đảm bảo bảo toàn tuyệt đối thứ tự ban đầu xuất hiện trong file bản đồ (sắp xếp ổn định - Stable Sort).



Hình minh họa: Mô hình sắp xếp thứ tự và tối ưu hóa vị trí

Nhiệm vụ: Cho danh sách N đoạn thẳng $[L_i, R_i]$ trên trục số. Hãy sắp xếp các đoạn thẳng theo các tiêu chí sau:

1. Tọa độ đầu mút bắt đầu L_i tăng dần.
2. Nếu cùng tọa độ L_i , tọa độ mút kết thúc R_i giảm dần.
3. Nếu trùng cả L_i và R_i , giữ nguyên thứ tự ban đầu xuất hiện trong dữ liệu vào.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 2 \cdot 10^5$) — số lượng đoạn thẳng.
- N dòng tiếp theo: Mỗi dòng gồm 2 số nguyên L_i, R_i ($-10^9 \leq L_i \leq R_i \leq 10^9$) mô tả đoạn thẳng thứ i .

Đầu ra (Output):

- In ra N dòng, mỗi dòng gồm 2 số nguyên L_i, R_i sau khi đã sắp xếp thỏa mãn toàn bộ tiêu chí trên.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 2 8 1 5 2 10	1 5 2 10 2 8

Giải thích: Danh sách 3 đoạn thẳng ban đầu là: $[2,8]$, $[1,5]$, $[2,10]$.

- Xét điểm đầu mút L : đoạn $[1,5]$ có $L = 1$ nhỏ nhất nên đứng đầu tiên.
- Hai đoạn còn lại là $[2,8]$ và $[2,10]$ đều có cùng $L = 2$:
- Xét điểm kết thúc R giảm dần: đoạn $[2,10]$ có $R = 10 > 8$ nên đoạn $[2,10]$ phải đứng trước đoạn $[2,8]$.

Thứ tự sau khi sắp xếp chuẩn là: 1 5, tiếp đến 2 10, và cuối cùng là 2 8.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách N đoạn thẳng $[L_i, R_i]$ trên trục số. Hãy sắp xếp các đoạn thẳng theo các tiêu chí sau:

1. Tọa độ đầu mút bắt đầu L_i tăng dần.

2. Nếu cùng tọa độ L_i , tọa độ nút kết thúc R_i giảm dần.
3. Nếu trùng cả L_i và R_i , giữ nguyên thứ tự ban đầu xuất hiện trong dữ liệu vào.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 2 8 1 5 2 10 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Danh sách 3 đoạn thẳng ban đầu là: [2,8], [1,5], [2,10]. - Xét điểm đầu nút L : đoạn [1,5] có $L = 1$ nhỏ nhất... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 1 5 2 10 2 8 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Danh sách 3 đoạn thẳng ban đầu là: [2,8], [1,5], [2,10].

- Xét điểm đầu nút L : đoạn [1,5] có $L = 1$ nhỏ nhất nên đứng đầu tiên.
- Hai đoạn còn lại là [2,8] và [2,10] đều có cùng $L = 2$:
- Xét điểm kết thúc R giảm dần: đoạn [2,10] có $R = 10 > 8$ nên đoạn [2,10] phải đứng trước đoạn [2,8].

Thứ tự sau khi sắp xếp chuẩn là: 1 5, tiếp đến 2 10, và cuối cùng là 2 8.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

bool cmp(const vector<long long> &a, const vector<long long> &b) {
    if (a[0] != b[0]) return a[0] < b[0];
    return a[1] > b[1];
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<vector<long long>> a(n, vector<long long>(2));
    for (int i = 0; i < n; ++i) {
        cin >> a[i][0] >> a[i][1];
    }

    stable_sort(a.begin(), a.end(), cmp);

    for (int i = 0; i < n; ++i) {
        cout << a[i][0] << " " << a[i][1] << "\n";
    }
}
```

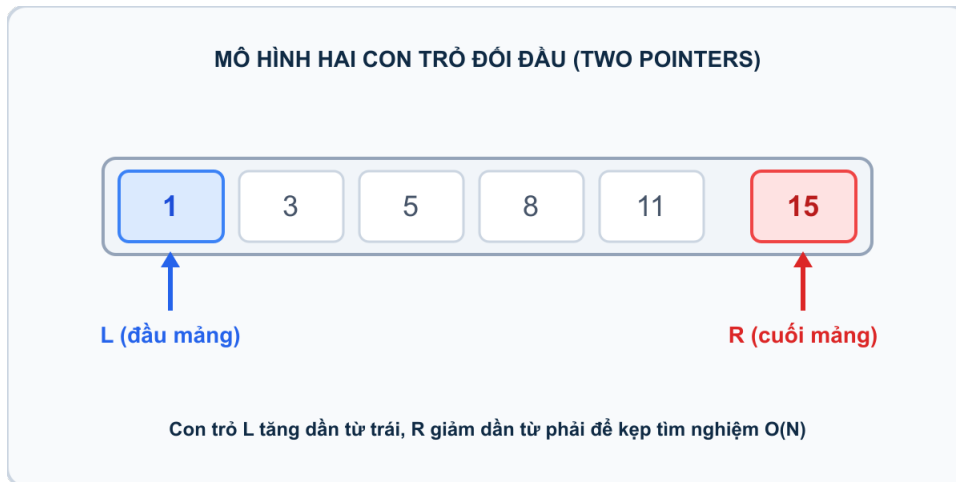
```
}  
return 0;  
}
```



Bài 05: Kỹ thuật hai con trỏ

Bài 01 [cppb_hct_01_mo_phong_hai_con_trỏ_đổi_đầu]: Mô Phỏng Hai Con Trỏ Đổi Đầu

Bối cảnh: Trong một hệ thống thanh toán tự động của siêu thị thông minh, máy chủ cần kết hợp giá trị của 2 phiếu giảm giá đã được sắp xếp tăng dần trong ví điện tử của khách hàng để thanh toán một hóa đơn có tổng trị giá đúng bằng S . Hệ thống cần phản hồi tức thì xem liệu có tồn tại hai phiếu giảm giá ở hai vị trí khác nhau có tổng giá trị khớp chính xác với số tiền S hay không.



Hình minh họa: Kỹ thuật hai con trỏ đổi đầu quét mảng hiệu quả $O(N)$

Nhiệm vụ: Cho mảng N số nguyên đã được sắp xếp tăng dần và một số nguyên S . Hãy kiểm tra xem trong mảng có tồn tại cặp chỉ số (i, j) với $i < j$ sao cho $A[i] + A[j] = S$ hay không. Nếu có in ra YES, ngược lại in ra NO.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và S ($2 \leq N \leq 10^5, -10^{18} \leq S \leq 10^{18}$).
- Dòng 2: Chứa N số nguyên đã sắp xếp tăng dần $A_1 \leq A_2 \leq \dots \leq A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra YES nếu tồn tại cặp số có tổng bằng S , ngược lại in ra NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 20 2 5 8 12 19	YES

Giải thích: Xét mảng đã sắp xếp: $[2, 5, 8, 12, 19]$ và $S = 20$. Khởi tạo hai con trỏ L trỏ vào 2 (chỉ số 1) và R trỏ vào 19 (chỉ số 5). Tổng $2 + 19 = 21 > 20 \rightarrow$ giảm R xuống trỏ vào 12. Tiếp tục tính tổng $2 + 12 = 14 < 20 \rightarrow$ tăng L trỏ vào 5. Tổng $5 + 12 = 17 < 20 \rightarrow$ tăng L trỏ vào 8. Khi L trỏ vào 8 và R trỏ vào 12, tổng $8 + 12 = 20$ đúng bằng S . Do đó in ra YES.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng N số nguyên đã được sắp xếp tăng dần và một số nguyên S . Hãy kiểm tra xem trong mảng có tồn tại cặp chỉ số (i, j) với $i < j$ sao cho $A[i] + A[j] = S$ hay không. Nếu có in ra YES, ngược lại in ra NO.

- **Chiến lược tiếp cận & tối ưu:**
- Sắp xếp mảng để tạo tính đơn điệu.
- Đặt 2 con trỏ L ở đầu và R ở cuối mảng (đối đầu), hoặc cùng chạy từ đầu mảng.
- Tại mỗi bước, dựa vào mối quan hệ giữa tổng/hiệu hiện tại và mục tiêu để quyết định tăng L hay giảm R một cách đơn điệu.

Bất biến: > Nếu $A[L] + A[R] > S$, vì mảng tăng dần nên $\forall k \geq L, A[k] + A[R] > S \Rightarrow$ phần tử $A[R]$ không thể ghép với bất kỳ số nào từ $L \dots R - 1$, việc giảm R là an toàn tuyệt đối và không bỏ sót nghiệm.

2. Bảng chạy tay trên số liệu mẫu | Bước | Thao tác thực hiện | Dữ liệu biến đổi | Kết quả ghi nhận |

|—|—|—|—|—| 1 | Khởi tạo & Đọc dữ liệu vào | Input: 5 20 2 5 8 12 19 | Nạp dữ liệu vào các biến/mảng
 | 2 | Thực thi thuật toán theo từng bước | Phân tích biến: Xét mảng đã sắp xếp: [2, 5, 8, 12, 19] và $S = 20$. Khởi tạo hai con trỏ L trỏ vào 2 (chỉ số 1) và R trỏ vào 19 (chỉ số 5). Tổng $2 + 19 = 21 > 20 \rightarrow$ giảm R xuống trỏ vào 12. Tiếp tục tính tổng $2 + 12 = 14 < 20 \rightarrow$ tăng L trỏ vào 5. Tổng $5 + 12 = 17 < 20 \rightarrow$ tăng L trỏ vào 8. Khi L trỏ vào 8 và R trỏ vào 12, tổng $8 + 12 = 20$ đúng bằng S . Do đó in ra YES. | Cập nhật trạng thái tối ưu | 3 | Xuất kết quả chuẩn ra màn hình | Kết quả cuối cùng: YES | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Xét mảng đã sắp xếp: [2, 5, 8, 12, 19] và $S = 20$. Khởi tạo hai con trỏ L trỏ vào 2 (chỉ số 1) và R trỏ vào 19 (chỉ số 5). Tổng $2 + 19 = 21 > 20 \rightarrow$ giảm R xuống trỏ vào 12. Tiếp tục tính tổng $2 + 12 = 14 < 20 \rightarrow$ tăng L trỏ vào 5. Tổng $5 + 12 = 17 < 20 \rightarrow$ tăng L trỏ vào 8. Khi L trỏ vào 8 và R trỏ vào 12, tổng $8 + 12 = 20$ đúng bằng S . Do đó in ra YES.

3. Lưu ý & Bẫy lỗi thường gặp

1. Quên sắp xếp mảng trước khi chạy Hai con trỏ đối đầu khiến điều kiện di chuyển L/R bị sai logic.
2. Lỗi tràn số $A[L] + A[R] \geq 2 \cdot 10^9$ khi tính tổng hai số lớn.
3. Vòng lặp dừng không đúng (ví dụ $L \leq R$ thay vì $L < R$ khi chọn 2 phần tử phân biệt).

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long s;
    if (!(cin >> n >> s)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    int l = 0, r = n - 1;
    bool found = false;
```

```

while (l < r) {
    long long sum = a[l] + a[r];
    if (sum == s) {
        found = true;
        break;
    } else if (sum < s) {
        ++l;
    } else {
        --r;
    }
}

if (found) cout << "YES\n";
else cout << "NO\n";
return 0;
}

```

Bài 02 [cppb_hct_02_tong_hai_so]: Cặp Số Có Tổng Bằng S (Two Sum)

Bối cảnh: Tại một trung tâm phân phối năng lượng, người điều hành cần ghép nối đúng 2 bình ắc-quy dự phòng trong kho lưu trữ gồm N bình chưa được sắp xếp sao cho tổng dung lượng tích trữ của hai bình được chọn đạt đúng công suất yêu cầu S . Nếu có nhiều cặp bình thỏa mãn, người điều hành muốn chọn một cặp bất kỳ và in ra dung lượng của 2 bình theo thứ tự tăng dần.



Hình minh họa: Kỹ thuật hai con trỏ đối đầu quét mảng hiệu quả $O(N)$

Nhiệm vụ: Cho một mảng gồm N số nguyên và một số nguyên S . Hãy tìm hai phần tử ở hai vị trí khác nhau trong mảng có tổng đúng bằng S . Nếu có nhiều cặp thỏa mãn, in ra một cặp bất kỳ theo thứ tự tăng dần. Nếu không tồn tại bất kỳ cặp nào, in ra -1.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và S ($2 \leq N \leq 10^5, -10^{18} \leq S \leq 10^{18}$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra 2 số nguyên là giá trị của 2 phần tử tìm được theo thứ tự tăng dần, hoặc -1 nếu không có nghiệm.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 20 19 2 8 12 5	8 12

Giải thích: Danh sách dung lượng các bình ắc-quy là: 19, 2, 8, 12, 5. Sau khi sắp xếp tăng dần: [2, 5, 8, 12, 19]. Cặp phần tử có giá trị 8 và 12 có tổng là $8 + 12 = 20$ đúng bằng S. Kết quả in ra theo thứ tự tăng dần là: 8 12.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho một mảng gồm N số nguyên và một số nguyên S. Hãy tìm hai phần tử ở hai vị trí khác nhau trong mảng có tổng đúng bằng S. Nếu có nhiều cặp thỏa mãn, in ra một cặp bất kỳ theo thứ tự tăng dần. Nếu không tồn tại bất kỳ cặp nào, in ra -1.
- **Chiến lược tiếp cận & tối ưu:**
- Sắp xếp mảng để tạo tính đơn điệu.
- Đặt 2 con trỏ L ở đầu và R ở cuối mảng (đối đầu), hoặc cùng chạy từ đầu mảng.
- Tại mỗi bước, dựa vào mối quan hệ giữa tổng/hiệu hiện tại và mục tiêu để quyết định tăng L hay giảm R một cách đơn điệu.

Bất biến: > Nếu $A[L] + A[R] > S$, vì mảng tăng dần nên $\forall k \geq L, A[k] + A[R] > S \Rightarrow$ phần tử $A[R]$ không thể ghép với bất kỳ số nào từ $L \dots R - 1$, việc giảm R là an toàn tuyệt đối và không bỏ sót nghiệm.

2. Bảng chạy tay trên số liệu mẫu | Bước | Thao tác thực hiện | Dữ liệu biến đổi | Kết quả ghi nhận |

|—|—|—|—|—| 1 | Khởi tạo & Đọc dữ liệu vào | Input: 5 20 19 2 8 12 5 | Nạp dữ liệu vào các biến/mảng
 || 2 | Thực thi thuật toán theo từng bước | Phân tích biến: Danh sách dung lượng các bình ắc-quy là: 19, 2, 8, 12, 5. Sau khi sắp xếp tăng dần: [2, 5, 8, 12, 19]. Cặp phần tử có giá trị 8 và 12 có tổng là $8 + 12 = 20$ đúng bằng S. Kết quả in ra theo thứ tự tăng dần là: 8 12. | Cập nhật trạng thái tối ưu ||
 3 | Xuất kết quả chuẩn ra màn hình | Kết quả cuối cùng: 8 12 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Danh sách dung lượng các bình ắc-quy là: 19, 2, 8, 12, 5. Sau khi sắp xếp tăng dần: [2, 5, 8, 12, 19]. Cặp phần tử có giá trị 8 và 12 có tổng là $8 + 12 = 20$ đúng bằng S. Kết quả in ra theo thứ tự tăng dần là: 8 12.

3. Lưu ý & Bẫy lỗi thường gặp

1. Quên sắp xếp mảng trước khi chạy Hai con trỏ đối đầu khiến điều kiện di chuyển L/R bị sai logic.
2. Lỗi tràn số $A[L] + A[R] \geq 2 \cdot 10^9$ khi tính tổng hai số lớn.
3. Vòng lặp dừng không đúng (ví dụ $L \leq R$ thay vì $L < R$ khi chọn 2 phần tử phân biệt).

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
```

```

cin.tie(nullptr);

int n;
long long s;
if (!(cin >> n >> s)) return 0;

vector<long long> a(n);
for (int i = 0; i < n; ++i) cin >> a[i];

sort(a.begin(), a.end());

int l = 0, r = n - 1;
bool found = false;

while (l < r) {
    long long sum = a[l] + a[r];
    if (sum == s) {
        cout << a[l] << " " << a[r] << "\n";
        found = true;
        break;
    } else if (sum < s) {
        ++l;
    } else {
        --r;
    }
}

if (!found) cout << -1 << "\n";
return 0;
}

```

Bài 03 [cppb_hct_03_dem_cap_tong_be_hon_s]: Đếm Cặp Có Tổng Không Quá S

Bối cảnh: Một phòng thí nghiệm vật lý tổ chức ghép cặp 2 hạt mang điện tích khác nhau trong chùm gồm N hạt để thực hiện phản ứng va chạm trong buồng từ trường. Để phản ứng diễn ra an toàn và không phá hủy thành buồng chứa, tổng mức năng lượng kích hoạt của hai hạt được ghép không được vượt quá ngưỡng giới hạn an toàn S. Các nhà khoa học cần đếm xem có tổng cộng bao nhiêu cặp hạt thỏa mãn tiêu chí an toàn này.



Hình minh họa: Kỹ thuật hai con trỏ đối đầu quét mảng hiệu quả $O(N)$

Nhiệm vụ: Cho mảng gồm N số nguyên và một số nguyên S. Hãy đếm số lượng cặp chỉ số (i, j) với $1 \leq i < j \leq N$ thỏa mãn: $A[i] + A[j] \leq S$.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và S ($2 \leq N \leq 2 \cdot 10^5, -10^{18} \leq S \leq 10^{18}$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là số lượng cặp thỏa mãn.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 8 2 5 1 4 3	9

Giải thích: Sắp xếp mảng tăng dần: $[1, 2, 3, 4, 5]$ và $S = 8$. Tổng số cặp phân biệt từ 5 phần tử là $5 \cdot 4 / 2 = 10$ cặp. Cặp duy nhất có tổng lớn hơn 8 là $(4, 5)$ với tổng $4 + 5 = 9 > 8$. Còn lại tất cả 9 cặp khác đều có tổng ≤ 8 : $(1, 2), (1, 3), (1, 4), (1, 5), (2, 3), (2, 4), (2, 5), (3, 4), (3, 5)$. Vì vậy kết quả là 9.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng gồm N số nguyên và một số nguyên S . Hãy đếm số lượng cặp chỉ số (i, j) với $1 \leq i < j \leq N$ thỏa mãn: $A[i] + A[j] \leq S$.
- **Chiến lược tiếp cận & tối ưu:**
- Sắp xếp mảng để tạo tính đơn điệu.
- Đặt 2 con trỏ L ở đầu và R ở cuối mảng (đối đầu), hoặc cùng chạy từ đầu mảng.
- Tại mỗi bước, dựa vào mối quan hệ giữa tổng/hiệu hiện tại và mục tiêu để quyết định tăng L hay giảm R một cách đơn điệu.

Bất biến: $>$ Nếu $A[L] + A[R] > S$, vì mảng tăng dần nên $\forall k \geq L, A[k] + A[R] > S \Rightarrow$ phần tử $A[R]$ không thể ghép với bất kỳ số nào từ $L \dots R - 1$, việc giảm R là an toàn tuyệt đối và không bỏ sót nghiệm.

2. Bảng chạy tay trên số liệu mẫu | Bước | Thao tác thực hiện | Dữ liệu biến đổi | Kết quả ghi nhận |

—	—	—	—		1	Khởi tạo & Đọc dữ liệu vào	Input: 5 8 2 5 1 4 3	Nạp dữ liệu vào các biến/mảng
2	Thực thi thuật toán theo từng bước	Phân tích biến: Sắp xếp mảng tăng dần: $[1, 2, 3, 4, 5]$ và $S = 8$. Tổng số cặp phân biệt từ 5 phần tử là $5 \cdot 4 / 2 = 10$ cặp. Cặp duy nhất có tổng lớn hơn 8 là $(4, 5)$ với tổng $4 + 5 = 9 > 8$. Còn lại tất cả 9 cặp khác đều có tổng ≤ 8 : $(1, 2), (1, 3), (1, 4), (1, 5), (2, 3), (2, 4), (2, 5), (3, 4), (3, 5)$. Vì vậy kết quả là 9.	Cập nhật trạng thái tối ưu					
3	Xuất kết quả chuẩn ra màn hình	Kết quả cuối cùng: 9	Khớp chính xác với đầu ra mẫu					

Giải thích chi tiết từ mẫu: Sắp xếp mảng tăng dần: $[1, 2, 3, 4, 5]$ và $S = 8$. Tổng số cặp phân biệt từ 5 phần tử là $5 \cdot 4 / 2 = 10$ cặp. Cặp duy nhất có tổng lớn hơn 8 là $(4, 5)$ với tổng $4 + 5 = 9 > 8$. Còn lại tất cả 9 cặp khác đều có tổng ≤ 8 : $(1, 2), (1, 3), (1, 4), (1, 5), (2, 3), (2, 4), (2, 5), (3, 4), (3, 5)$. Vì vậy kết quả là 9.

3. Lưu ý & Bẫy lỗi thường gặp

1. Quên sắp xếp mảng trước khi chạy Hai con trỏ đối đầu khiến điều kiện di chuyển L/R bị sai logic.
2. Lỗi tràn số $A[L] + A[R] \geq 2 \cdot 10^9$ khi tính tổng hai số lớn.
3. Vòng lặp dừng không đúng (ví dụ $L \leq R$ thay vì $L < R$ khi chọn 2 phần tử phân biệt).

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long s;
    if (!(cin >> n >> s)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    sort(a.begin(), a.end());

    int l = 0, r = n - 1;
    long long count = 0;

    while (l < r) {
        if (a[l] + a[r] <= s) {
            count += (r - l);
            ++l;
        } else {
            --r;
        }
    }

    cout << count << "\n";
    return 0;
}
```

Bài 04 [cppb_hct_04_dem_cap_tong_lon_hon_s]: Đếm Cặp Có Tổng Lớn Hơn Hoặc Bằng S

Bối cảnh: Trong một trò chơi đối kháng trực tuyến, hệ thống cần ghép cặp 2 người chơi trong danh sách N game thủ để tạo thành một đội đặc nhiệm tham gia chiến dịch liên server. Để đội có đủ sức mạnh hoàn thành chiến dịch, tổng điểm chiến lực của hai thành viên trong đội phải đạt từ mức chuẩn S trở lên. Ban quản trị cần tính toán có bao nhiêu cách chọn 2 người chơi thỏa mãn tiêu chuẩn này.



Hình minh họa: Kỹ thuật hai con trỏ đối đầu quét mảng hiệu quả $O(N)$

Nhiệm vụ: Cho mảng gồm N số nguyên và một số nguyên S . Hãy đếm số lượng cặp chỉ số (i, j) với $1 \leq i < j \leq N$ thỏa mãn: $A[i] + A[j] \geq S$.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và S ($2 \leq N \leq 2 \cdot 10^5, -10^{18} \leq S \leq 10^{18}$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là số lượng cặp thỏa mãn.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 8 2 5 1 4 3	2

Giải thích: Sắp xếp dãy điểm chiến lược tăng dần: $[1, 2, 3, 4, 5]$. Các cặp có tổng ≥ 8 là: cặp $(3, 5)$ có tổng $3 + 5 = 8 \geq 8$ và cặp $(4, 5)$ có tổng $4 + 5 = 9 \geq 8$. Tổng cộng có đúng 2 cặp thỏa mãn.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng gồm N số nguyên và một số nguyên S . Hãy đếm số lượng cặp chỉ số (i, j) với $1 \leq i < j \leq N$ thỏa mãn: $A[i] + A[j] \geq S$.
- **Chiến lược tiếp cận & tối ưu:**
- Sắp xếp mảng để tạo tính đơn điệu.
- Đặt 2 con trỏ L ở đầu và R ở cuối mảng (đối đầu), hoặc cùng chạy từ đầu mảng.
- Tại mỗi bước, dựa vào mối quan hệ giữa tổng/hiệu hiện tại và mục tiêu để quyết định tăng L hay giảm R một cách đơn điệu.

Bất biến: > Nếu $A[L] + A[R] > S$, vì mảng tăng dần nên $\forall k \geq L, A[k] + A[R] > S \Rightarrow$ phần tử $A[R]$ không thể ghép với bất kỳ số nào từ $L \dots R - 1$, việc giảm R là an toàn tuyệt đối và không bỏ sót nghiệm.

2. Bảng chạy tay trên số liệu mẫu | Bước | Thao tác thực hiện | Dữ liệu biến đổi | Kết quả ghi nhận |

|—|—|—|—| | 1 | Khởi tạo & Đọc dữ liệu vào | Input: 5 8 2 5 1 4 3 | Nạp dữ liệu vào các biến/mảng
 | 2 | Thực thi thuật toán theo từng bước | Phân tích biến: Sắp xếp dãy điểm chiến lược tăng dần: [1, 2, 3, 4, 5]. Các cặp có tổng ≥ 8 là: cặp (3, 5) có tổng $3 + 5 = 8 \geq 8$ và cặp (4, 5) có tổng $4 + 5 = 9 \geq 8$. Tổng cộng có đúng 2 cặp thỏa mãn. | Cập nhật trạng thái tối ưu | | 3 | Xuất kết quả chuẩn ra màn hình | Kết quả cuối cùng: 2 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Sắp xếp dãy điểm chiến lược tăng dần: [1, 2, 3, 4, 5]. Các cặp có tổng ≥ 8 là: cặp (3, 5) có tổng $3 + 5 = 8 \geq 8$ và cặp (4, 5) có tổng $4 + 5 = 9 \geq 8$. Tổng cộng có đúng 2 cặp thỏa mãn.

3. Lưu ý & Bẫy lỗi thường gặp

1. Quên sắp xếp mảng trước khi chạy Hai con trỏ đối đầu khiến điều kiện di chuyển L/R bị sai logic.
2. Lỗi tràn số $A[L] + A[R] \geq 2 \cdot 10^9$ khi tính tổng hai số lớn.
3. Vòng lặp dừng không đúng (ví dụ $L \leq R$ thay vì $L < R$ khi chọn 2 phần tử phân biệt).

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long s;
    if (!(cin >> n >> s)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    sort(a.begin(), a.end());

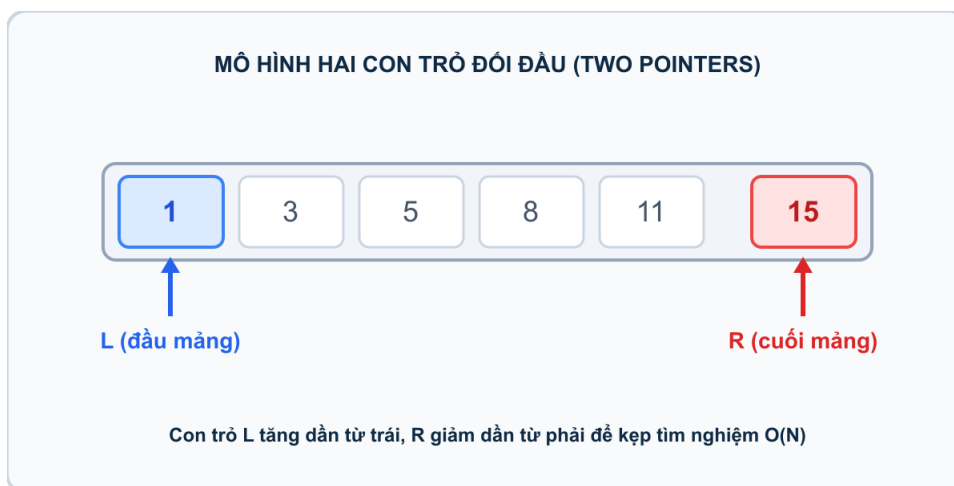
    int l = 0, r = n - 1;
    long long count = 0;

    while (l < r) {
        if (a[l] + a[r] >= s) {
            count += (r - l);
            --r;
        } else {
            ++l;
        }
    }

    cout << count << "\n";
    return 0;
}
```

Bài 05 [cppb_hct_05_thuyen_cuu_ho]: Ghép Thuyền Cứu Hộ Tối Ưu

Bối cảnh: Trong một đợt cứu hộ bão lụt khẩn cấp, đội cứu nạn tiếp cận một khu dân cư có N người dân cần di tản qua sông an toàn. Mỗi người thứ i có cân nặng W_i . Lực lượng cứu nạn trang bị loại xuồng cứu hộ chuyên dụng: mỗi chiếc xuồng chỉ có thể chở tối đa 2 người và tổng trọng lượng của hai người trên cùng một xuồng không được vượt quá tải trọng định mức C . Để tiết kiệm phương tiện và thời gian cứu nạn, chỉ huy cần tìm số lượng xuồng cứu hộ ít nhất cần dùng.



Hình minh họa: Kỹ thuật hai con trỏ đối đầu quét mảng hiệu quả $O(N)$

Nhiệm vụ: Cho cân nặng của N người và tải trọng tối đa C của thuyền. Biết mỗi thuyền chở tối đa 2 người và tổng cân nặng không vượt quá C . Hãy tìm số lượng thuyền ít nhất để chở hết toàn bộ N người sang sông.

Đầu vào (Input):

- Dòng 1: 2 số nguyên N và C ($1 \leq N \leq 10^5, 1 \leq C \leq 10^9$).
- Dòng 2: N số nguyên $W_1, W_2, \dots, W_N$ ($1 \leq W_i \leq C$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là số thuyền ít nhất cần dùng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 50 30 20 40 50	3

Giải thích: Sắp xếp cân nặng 4 người tăng dần: [20, 30, 40, 50] với tải trọng $C = 50$. Người nặng 50 kg bắt buộc phải đi một mình 1 thuyền (tốn 1 thuyền). Người nặng 40 kg không thể ghép với ai (vì $40 + 20 = 60 > 50$) nên cũng đi một mình 1 thuyền (tốn thêm 1 thuyền). Hai người còn lại có cân nặng 20 kg và 30 kg ghép chung 1 thuyền vì $20 + 30 = 50 \leq 50$ (tốn 1 thuyền). Tổng số thuyền ít nhất cần dùng là $1 + 1 + 1 = 3$ thuyền.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho cân nặng của N người và tải trọng tối đa C của thuyền. Biết mỗi thuyền chở tối đa 2 người và tổng cân nặng không vượt quá C . Hãy tìm số lượng thuyền ít nhất để chở hết toàn bộ N người sang sông.

- **Chiến lược tiếp cận & tối ưu:**
- Sắp xếp mảng để tạo tính đơn điệu.
- Đặt 2 con trỏ L ở đầu và R ở cuối mảng (đối đầu), hoặc cùng chạy từ đầu mảng.
- Tại mỗi bước, dựa vào mối quan hệ giữa tổng/hiệu hiện tại và mục tiêu để quyết định tăng L hay giảm R một cách đơn điệu.

Bất biến: > Nếu $A[L] + A[R] > S$, vì mảng tăng dần nên $\forall k \geq L, A[k] + A[R] > S \Rightarrow$ phần tử $A[R]$ không thể ghép với bất kỳ số nào từ $L \dots R - 1$, việc giảm R là an toàn tuyệt đối và không bỏ sót nghiệm.

2. Bảng chạy tay trên số liệu mẫu | Bước | Thao tác thực hiện | Dữ liệu biến đổi | Kết quả ghi nhận |

|—|—|—|—| | 1 | Khởi tạo & Đọc dữ liệu vào | Input: 4 50 30 20 40 50 | Nạp dữ liệu vào các biến/mảng | | 2 | Thực thi thuật toán theo từng bước | Phân tích biến: Sắp xếp cân nặng 4 người tăng dần: [20, 30, 40, 50] với tải trọng $C = 50$. Người nặng 50 kg bắt buộc phải đi một mình 1 thuyền (tốn 1 thuyền). Người nặng 40 kg không thể ghép với ai (vì $40 + 20 = 60 > 50$) nên cũng đi một mình 1 thuyền (tốn thêm 1 thuyền). Hai người còn lại có cân nặng 20 kg và 30 kg ghép chung 1 thuyền vì $20 + 30 = 50 \leq 50$ (tốn 1 thuyền). Tổng số thuyền ít nhất cần dùng là $1 + 1 + 1 = 3$ thuyền. | Cập nhật trạng thái tối ưu | | 3 | Xuất kết quả chuẩn ra màn hình | Kết quả cuối cùng: 3 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Sắp xếp cân nặng 4 người tăng dần: [20, 30, 40, 50] với tải trọng $C = 50$. Người nặng 50 kg bắt buộc phải đi một mình 1 thuyền (tốn 1 thuyền). Người nặng 40 kg không thể ghép với ai (vì $40 + 20 = 60 > 50$) nên cũng đi một mình 1 thuyền (tốn thêm 1 thuyền). Hai người còn lại có cân nặng 20 kg và 30 kg ghép chung 1 thuyền vì $20 + 30 = 50 \leq 50$ (tốn 1 thuyền). Tổng số thuyền ít nhất cần dùng là $1 + 1 + 1 = 3$ thuyền.

3. Lưu ý & Bẫy lỗi thường gặp

1. Quên sắp xếp mảng trước khi chạy Hai con trỏ đối đầu khiến điều kiện di chuyển L/R bị sai logic.
2. Lỗi tràn số $A[L] + A[R] \geq 2 \cdot 10^9$ khi tính tổng hai số lớn.
3. Vòng lặp dừng không đúng (ví dụ $L \leq R$ thay vì $L < R$ khi chọn 2 phần tử phân biệt).

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long c;
    if (!(cin >> n >> c)) return 0;

    vector<long long> w(n);
    for (int i = 0; i < n; ++i) cin >> w[i];
```

```

sort(w.begin(), w.end());

int l = 0, r = n - 1;
int boats = 0;

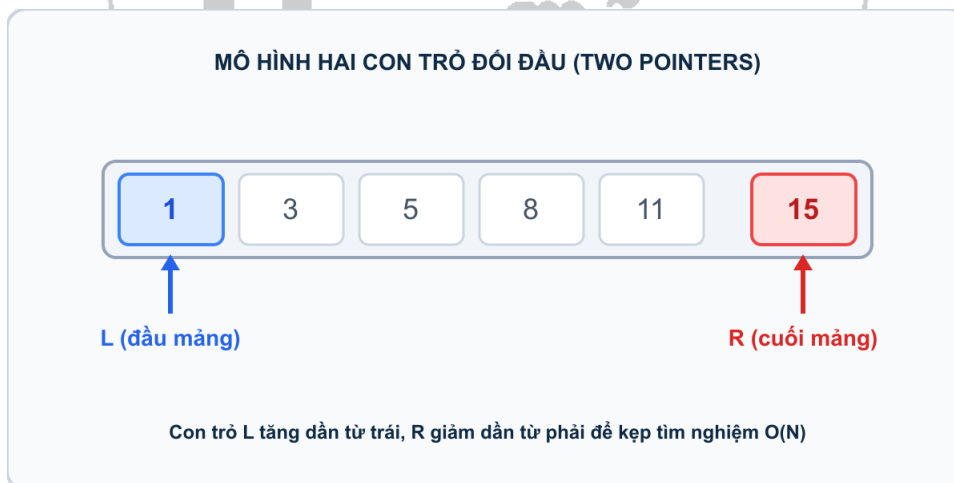
while (l <= r) {
    if (l == r) {
        ++boats;
        break;
    }
    if (w[l] + w[r] <= c) {
        ++l;
        --r;
    } else {
        --r;
    }
    ++boats;
}

cout << boats << "\n";
return 0;
}

```

Bài 06 [cppb_hct_06_van_chuyen_hang_hoa]: Vận Chuyển Thùng Hàng Cực Đại

Bối cảnh: Một công ty logistics quốc tế phụ trách điều phối N container hàng hóa hạng nặng có khối lượng lần lượt là $W_1, W_2, \dots, W_n$ (có thể lên tới 10^{12} kg) từ tổng kho ra cảng biển. Công ty sử dụng dàn xe đầu kéo rơ-moóc đặc chủng: mỗi chuyến xe chỉ được chở tối đa 2 kiện hàng và tổng khối lượng không được vượt quá giới hạn tải trọng cầu đường C . Để tối ưu chi phí nhiên liệu và nhân công, điều độ viên cần tính toán số chuyến xe tối thiểu cần thực hiện.



Hình minh họa: Kỹ thuật hai con trỏ đối đầu quét mảng hiệu quả $O(N)$

Nhiệm vụ: Cho khối lượng N kiện hàng và tải trọng C của xe đầu kéo. Mỗi chuyến xe chở tối đa 2 kiện hàng và tổng khối lượng không quá C . Hãy tính số chuyến xe ít nhất cần dùng để vận chuyển toàn bộ N kiện hàng.

Đầu vào (Input):

- Dòng 1: 2 số nguyên dương N và C ($1 \leq N \leq 10^5, 1 \leq C \leq 10^{12}$).
- Dòng 2: N số nguyên dương $W_1, W_2, \dots, W_N$ ($1 \leq W_i \leq C$).

Đầu ra (Output):

- In ra số chuyến xe ít nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 10 3 5 8 2 7	3

Giải thích: Sắp xếp khối lượng 5 kiện hàng tăng dần: [2, 3, 5, 7, 8] với tải trọng $C = 10$. Chiến thuật ghép con trỏ hai đầu: kiện nặng nhất 8 ghép với nhẹ nhất 2 ($8 + 2 = 10 \leq 10 \rightarrow$ Chuyến 1); kiện nặng tiếp theo 7 ghép với nhẹ tiếp theo 3 ($7 + 3 = 10 \leq 10 \rightarrow$ Chuyến 2); kiện còn lại 5 đi riêng một xe (Chuyến 3). Tổng cộng cần 3 chuyến xe.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho khối lượng N kiện hàng và tải trọng C của xe đầu kéo. Mỗi chuyến xe chở tối đa 2 kiện hàng và tổng khối lượng không quá C . Hãy tính số chuyến xe ít nhất cần dùng để vận chuyển toàn bộ N kiện hàng.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 10 3 5 8 2 7 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Sắp xếp khối lượng 5 kiện hàng tăng dần: [2, 3, 5, 7, 8] với tải trọng $C = 10$. Chiến thuật ghép con trỏ hai đầu: kiện nặ... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 3 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Sắp xếp khối lượng 5 kiện hàng tăng dần: [2, 3, 5, 7, 8] với tải trọng $C = 10$. Chiến thuật ghép con trỏ hai đầu: kiện nặng nhất 8 ghép với nhẹ nhất 2 ($8 + 2 = 10 \leq 10 \rightarrow$ Chuyến 1); kiện nặng tiếp theo 7 ghép với nhẹ tiếp theo 3 ($7 + 3 = 10 \leq 10 \rightarrow$ Chuyến 2); kiện còn lại 5 đi riêng một xe (Chuyến 3). Tổng cộng cần 3 chuyến xe.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. $N-1$) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long c;
    if (!(cin >> n >> c)) return 0;
```

```

vector<long long> w(n);
for (int i = 0; i < n; ++i) cin >> w[i];

sort(w.begin(), w.end());

int l = 0, r = n - 1;
int trips = 0;

while (l <= r) {
    if (l == r) {
        ++trips;
        break;
    }
    if (w[l] + w[r] <= c) {
        ++l;
        --r;
    } else {
        --r;
    }
    ++trips;
}

cout << trips << "\n";
return 0;
}

```

Bài 07 [cppb_hct_07_tong_gan_s_nhat]: Tìm Cặp Có Tổng Gần S Nhất

Bối cảnh: Trong kỹ thuật vi mạch điện tử, kỹ sư cần ghép nối 2 điện trở có giá trị điện trở kháng lần lượt trong danh sách N linh kiện có sẵn để mạch thu được giá trị điện trở kháng tương đương gần nhất với thông số thiết kế S. Do giá trị các linh kiện có sai số chế tạo, kỹ sư muốn tìm ra cặp linh kiện sao cho độ lệch tuyệt đối giữa tổng giá trị của cặp và thông số S là nhỏ nhất. Nếu có nhiều cặp có cùng độ lệch nhỏ nhất, ưu tiên chọn cặp có tổng giá trị nhỏ hơn.



Hình minh họa: Kỹ thuật hai con trỏ đối đầu quét mảng hiệu quả $O(N)$

Nhiệm vụ: Cho mảng gồm N số nguyên và một số nguyên S. Hãy tìm một cặp số $(A[i], A[j])$ với $i < j$ sao cho độ chênh lệch $|(A[i] + A[j]) - S|$ là nhỏ nhất có thể. Nếu có nhiều cặp, in ra cặp có tổng nhỏ hơn theo thứ tự tăng dần.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và S ($2 \leq N \leq 10^5, -10^{18} \leq S \leq 10^{18}$).

- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra 2 số nguyên biểu diễn cặp số tìm được theo thứ tự tăng dần.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 20 2 8 13 4 25	8 13

Giải thích: Sắp xếp danh sách điện trở tăng dần: [2, 4, 8, 13, 25] và $S = 20$. Xét các cặp có tổng gần 20: cặp (8, 13) có tổng $8 + 13 = 21$ (chênh lệch $|21 - 20| = 1$); cặp (4, 13) có tổng 17 (chênh lệch $|17 - 20| = 3$). Cặp có độ chênh lệch nhỏ nhất đạt được là (8, 13) với khoảng cách chênh lệch chỉ là 1. Kết quả in ra: 8 13.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng gồm N số nguyên và một số nguyên S . Hãy tìm một cặp số $(A[i], A[j])$ với $i < j$ sao cho độ chênh lệch $|(A[i] + A[j]) - S|$ là nhỏ nhất có thể. Nếu có nhiều cặp, in ra cặp có tổng nhỏ hơn theo thứ tự tăng dần.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 20 2 8 13 4 25 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Sắp xếp danh sách điện trở tăng dần: [2, 4, 8, 13, 25] và $S = 20$. Xét các cặp có tổng gần 20: cặp (8, 13) có tổng $8 + 13 = 21$... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 8 13 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Sắp xếp danh sách điện trở tăng dần: [2, 4, 8, 13, 25] và $S = 20$. Xét các cặp có tổng gần 20: cặp (8, 13) có tổng $8 + 13 = 21$ (chênh lệch $|21 - 20| = 1$); cặp (4, 13) có tổng 17 (chênh lệch $|17 - 20| = 3$). Cặp có độ chênh lệch nhỏ nhất đạt được là (8, 13) với khoảng cách chênh lệch chỉ là 1. Kết quả in ra: 8 13.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. $N-1$) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```
int n;
long long s;
if (!(cin >> n >> s)) return 0;

vector<long long> a(n);
for (int i = 0; i < n; ++i) cin >> a[i];

sort(a.begin(), a.end());

int l = 0, r = n - 1;
long long best_diff = -1;
long long ans_l = a[0], ans_r = a[1];

while (l < r) {
    long long cur_sum = a[l] + a[r];
    long long cur_diff = abs(cur_sum - s);

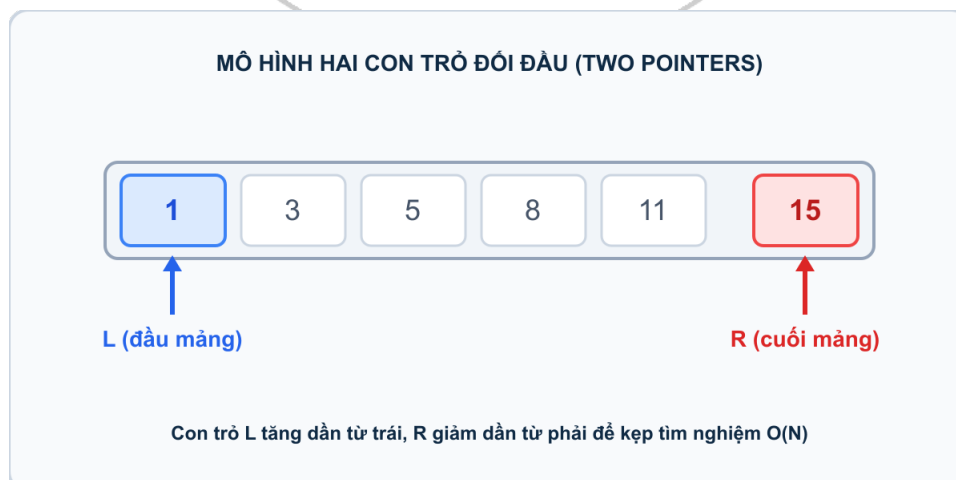
    if (best_diff == -1 || cur_diff < best_diff || (cur_diff == best_diff && cur_sum < ans_
l + ans_r)) {
        best_diff = cur_diff;
        ans_l = a[l];
        ans_r = a[r];
    }

    if (cur_sum == s) break;
    else if (cur_sum < s) ++l;
    else --r;
}

cout << ans_l << " " << ans_r << "\n";
return 0;
}
```

Bài 08 [cppb_hct_08_hieu_hai_so_bang_k]: Tìm Cặp Có Hiệu Đúng Bằng K

Bối cảnh: Trên trục đường đua mô tô mạo hiểm, hệ thống radar đo vận tốc ghi nhận N chỉ số tốc độ của các xe tham gia. Để trao giải cho màn rượt đuổi kịch tính nhất, trọng tài cần kiểm tra xem trong cuộc đua có tồn tại hai chiếc xe mà vận tốc của xe chạy sau vượt trội hơn xe chạy trước đúng một lượng chênh lệch chuẩn K hay không.



Hình minh họa: Kỹ thuật hai con trỏ đối đầu quét mảng hiệu quả $O(N)$

Nhiệm vụ: Cho mảng gồm N số nguyên và số nguyên không âm K. Hãy kiểm tra xem có tồn tại cặp chỉ số (i, j) với $i \neq j$ sao cho $A[j] - A[i] = K$ hay không. Nếu có in ra YES, ngược lại in ra NO.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và K ($2 \leq N \leq 10^5, 0 \leq K \leq 10^{18}$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra YES hoặc NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 8 5 3 2	YES

Giải thích: Sắp xếp mảng tăng dần: $[1, 2, 3, 5, 8]$ với $K = 3$. Sử dụng hai con trỏ cùng chiều: cặp số $(2, 5)$ có hiệu $5 - 2 = 3 = K$ hoặc cặp $(5, 8)$ có hiệu $8 - 5 = 3 = K$. Vì tồn tại ít nhất một cặp thỏa mãn nên kết quả in ra là YES.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng gồm N số nguyên và số nguyên không âm K . Hãy kiểm tra xem có tồn tại cặp chỉ số (i, j) với $i \neq j$ sao cho $A[j] - A[i] = K$ hay không. Nếu có in ra YES, ngược lại in ra NO.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 3 1 8 5 3 2 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Sắp xếp mảng tăng dần: $[1, 2, 3, 5, 8]$ với $K = 3$. Sử dụng hai con trỏ cùng chiều: cặp số $(2, 5)$ có hiệu $5 - 2 = 3 = K$ ho... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: YES | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Sắp xếp mảng tăng dần: $[1, 2, 3, 5, 8]$ với $K = 3$. Sử dụng hai con trỏ cùng chiều: cặp số $(2, 5)$ có hiệu $5 - 2 = 3 = K$ hoặc cặp $(5, 8)$ có hiệu $8 - 5 = 3 = K$. Vì tồn tại ít nhất một cặp thỏa mãn nên kết quả in ra là YES.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```
int n;
long long k;
if (!(cin >> n >> k)) return 0;

vector<long long> a(n);
for (int i = 0; i < n; ++i) cin >> a[i];

sort(a.begin(), a.end());

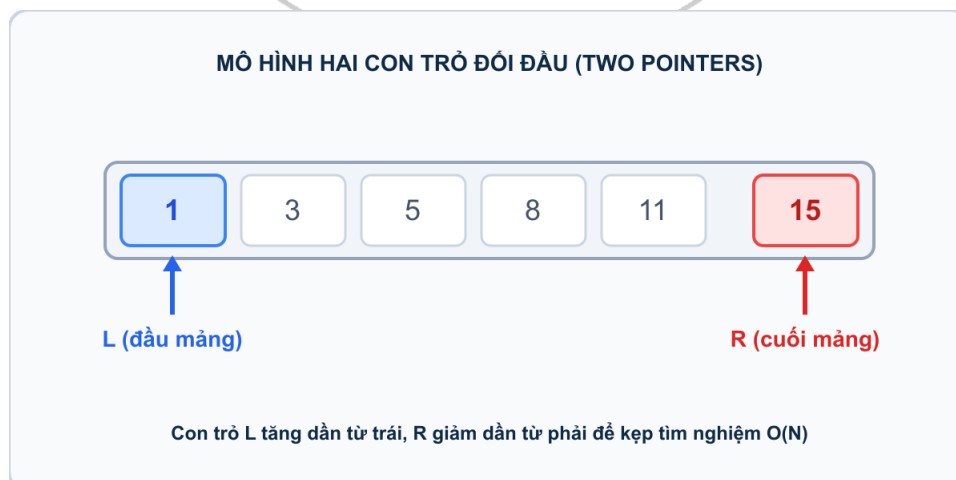
int l = 0, r = 1;
bool found = false;

while (r < n) {
    if (l == r) {
        ++r;
        continue;
    }
    long long diff = a[r] - a[l];
    if (diff == k) {
        found = true;
        break;
    } else if (diff < k) {
        ++r;
    } else {
        ++l;
    }
}

if (found) cout << "YES\n";
else cout << "NO\n";
return 0;
}
```

Bài 09 [cppb_hct_09_bo_ba_tong_bang_s]: Bộ Ba Số Có Tổng Bằng S (3-Sum)

Bối cảnh: Trong một trò chơi tam giác ma thuật trên truyền hình, bạn tổ chức đưa ra N viên xúc xắc khắc các số nguyên khác nhau. Người chơi cần chọn ra đúng 3 viên xúc xắc ở 3 vị trí phân biệt sao cho tổng các mặt số ghi trên 3 viên xúc xắc đúng bằng con số mục tiêu S do ban giám khảo chỉ định. Nếu có nhiều bộ ba thỏa mãn, người chơi chỉ cần công bố một bộ bất kỳ theo thứ tự giá trị tăng dần.



Hình minh họa: Kỹ thuật hai con trỏ đối đầu quét mảng hiệu quả $O(N)$

Nhiệm vụ: Cho mảng gồm N số nguyên và một số nguyên S . Hãy tìm 3 phần tử ở 3 vị trí phân biệt trong mảng có tổng đúng bằng S . Nếu có nhiều bộ, in ra một bộ bất kỳ theo thứ tự tăng dần. Nếu không tồn tại, in ra -1.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và S ($3 \leq N \leq 3000, -10^{18} \leq S \leq 10^{18}$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra 3 số nguyên theo thứ tự tăng dần, hoặc -1.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 15 2 7 5 1 8 4	2 5 8

Giải thích: Sắp xếp mảng tăng dần: $[1, 2, 4, 5, 7, 8]$ với mục tiêu $S = 15$. Cố định phần tử đầu tiên là 2, ta cần tìm hai phần tử còn lại có tổng là $15 - 2 = 13$. Sử dụng hai con trỏ trên đoạn còn lại $[4, 5, 7, 8]$, ta tìm được cặp $(5, 8)$ có $5 + 8 = 13$. Do đó bộ ba số tìm được là 2, 5, 8 thỏa mãn $2 + 5 + 8 = 15$. Kết quả in ra: 2 5 8.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng gồm N số nguyên và một số nguyên S . Hãy tìm 3 phần tử ở 3 vị trí phân biệt trong mảng có tổng đúng bằng S . Nếu có nhiều bộ, in ra một bộ bất kỳ theo thứ tự tăng dần. Nếu không tồn tại, in ra -1.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 6 15 2 7 5 1 8 4 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Sắp xếp mảng tăng dần: $[1, 2, 4, 5, 7, 8]$ với mục tiêu $S = 15$. Cố định phần tử đầu tiên là 2, ta cần tìm hai phần tử còn... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 2 5 8 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Sắp xếp mảng tăng dần: $[1, 2, 4, 5, 7, 8]$ với mục tiêu $S = 15$. Cố định phần tử đầu tiên là 2, ta cần tìm hai phần tử còn lại có tổng là $15 - 2 = 13$. Sử dụng hai con trỏ trên đoạn còn lại $[4, 5, 7, 8]$, ta tìm được cặp $(5, 8)$ có $5 + 8 = 13$. Do đó bộ ba số tìm được là 2, 5, 8 thỏa mãn $2 + 5 + 8 = 15$. Kết quả in ra: 2 5 8.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long s;
    if (!(cin >> n >> s)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

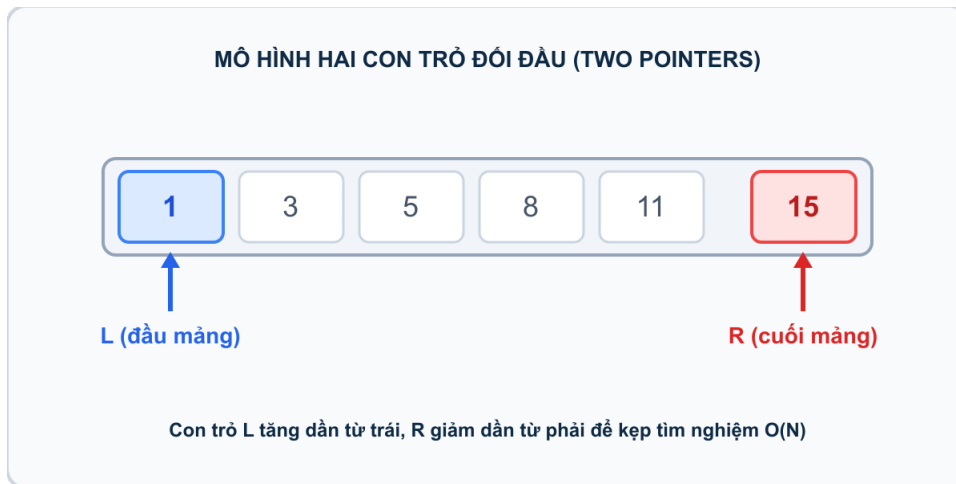
    sort(a.begin(), a.end());

    for (int i = 0; i < n - 2; ++i) {
        long long target = s - a[i];
        int l = i + 1, r = n - 1;
        while (l < r) {
            long long sum = a[l] + a[r];
            if (sum == target) {
                cout << a[i] << " " << a[l] << " " << a[r] << "\n";
                return 0;
            } else if (sum < target) {
                ++l;
            } else {
                --r;
            }
        }
    }

    cout << -1 << "\n";
    return 0;
}
```

Bài 10 [cppb_hct_10_dem_so_tam_giac]: Đếm Số Tam Giác Có Thể Tạo Thành

Bối cảnh: Một nghệ nhân điêu khắc kiến trúc gỗ có N que gỗ thẳng với độ dài lần lượt là $A_1, A_2, \dots, A_n$. Để tạo các mắt lưới tam giác trang trí chịu lực cho mái vòm công trình, nghệ nhân cần chọn ra bộ 3 que gỗ bất kỳ có thể ghép thành một tam giác không suy biến (nghĩa là độ dài của que bất kỳ phải nhỏ hơn tổng độ dài hai que còn lại). Hãy giúp nghệ nhân đếm xem có bao nhiêu cách chọn ra bộ 3 que gỗ hợp lệ.



Hình minh họa: Kỹ thuật hai con trỏ đối đầu quét mảng hiệu quả $O(N)$

Nhiệm vụ: Cho N đoạn que với độ dài $A_1, A_2, \dots, A_N$. Hãy đếm số lượng bộ 3 que có thể ghép lại thành một tam giác không suy biến.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($3 \leq N \leq 3000$).
- Dòng 2: N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là số lượng tam giác tạo được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 4 6 3 7	3

Giải thích: Sắp xếp độ dài 4 que gỗ tăng dần: $[3, 4, 6, 7]$. Xét các bộ ba que gỗ: $(3, 4, 6)$ có $3 + 4 = 7 > 6 \rightarrow$ lập được tam giác; $(3, 6, 7)$ có $3 + 6 = 9 > 7 \rightarrow$ lập được tam giác; $(4, 6, 7)$ có $4 + 6 = 10 > 7 \rightarrow$ lập được tam giác; $(3, 4, 7)$ có $3 + 4 = 7$ không lớn hơn 7 $\rightarrow$ bị suy biến (không lập được tam giác). Tổng cộng tạo được đúng 3 tam giác không suy biến.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho N thanh gỗ có độ dài $A_1, A_2, \dots, A_N$. Đếm số bộ ba thanh gỗ có thể ghép thành một tam giác không suy biến.
- **Bất đẳng thức tam giác trên mảng đã sắp xếp:**
- Ba cạnh (a, b, c) tạo thành tam giác khi: $a + b > c, a + c > b, b + c > a$.
- Nếu ta sắp xếp tăng dần $A_i \leq A_j \leq A_k$, ta luôn có $A_k + A_i > A_j$ và $A_k + A_j > A_i$.
- Điều kiện duy nhất cần kiểm tra là:

$$A_i + A_j > A_k$$

- **Thuật toán Hai con trỏ $O(N^2)$:**
- Cố định cạnh lớn nhất k chạy ngược từ $N - 1$ về 2.
- Với mỗi k , đặt $L = 0$ và $R = k - 1$:

- Nếu $A_L + A_R > A_k$: Vì mảng tăng dần nên mọi phần tử từ L đến $R - 1$ khi ghép với A_R đều thỏa mãn $> A_k$.
- Do đó có đúng $R - L$ tam giác hợp lệ. Cộng $ans += (R - L)$ và giảm $R--$.
- Nếu $A_L + A_R \leq A_k$: Tổng quá nhỏ, tăng $L++$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử: $N = 5$, các cạnh đã xếp $[2, 3, 4, 5, 6]$.

Cố định k	Cạnh lớn nhất A_k	Hai con trỏ (L, R)	$A_L + A_R > A_k$	Số tam giác cộng dồn
$k = 4$	$A_4 = 6$	$L = 0, R = 3(2,5)$	$2 + 5 = 7 > 6$	Cặp $(3, 4, 5)$ với $5 \Rightarrow$ cộng $3 - 0 = 3$. $R = 2$.
—	$A_4 = 6$	$L = 0, R = 2(2,4)$	$2 + 4 = 6 \leq 6$	$L = 1$.
—	$A_4 = 6$	$L = 1, R = 2(3,4)$	$3 + 4 = 7 > 6$	Cộng $2 - 1 = 1$. $R = 1 \Rightarrow$ dừng $k = 4$. (Được 4 tam giác)

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy tam giác suy biến:** Điều kiện là $a + b > c$ (lớn hơn nghiêm ngặt), nếu $a + b == c$ ba điểm thẳng hàng, không tạo thành tam giác.
- **Biến đếm ans:** Phải dùng long long $ans = 0$ vì số tam giác tối đa là $\binom{N}{3} \approx 1.6 \cdot 10^{11}$ khi $N = 10^4$.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    sort(a.begin(), a.end());

    long long count = 0;

    for (int k = n - 1; k >= 2; --k) {
        int l = 0, r = k - 1;
        while (l < r) {
            if (a[l] + a[r] > a[k]) {
```

```

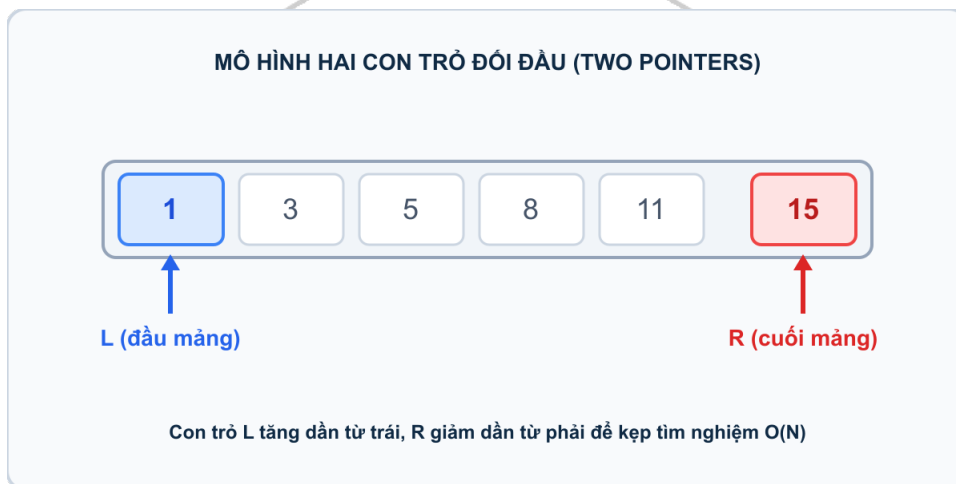
        count += (r - l);
        --r;
    } else {
        ++l;
    }
}

cout << count << "\n";
return 0;
}

```

Bài 11 [cppb_hct_11_dem_cap_trung_lap]: Đếm Cặp Tổng S Trên Mảng Trùng Lặp

Bối cảnh: Một sàn giao dịch tài chính phân tích số lệnh khớp giá của N giao dịch diễn ra trong phiên mở cửa. Mỗi giao dịch có giá trị khớp lệnh là một số nguyên, và có rất nhiều giao dịch có cùng một mức giá (mảng chứa nhiều phần tử trùng lặp). Thuật toán thị trường cần kiểm tra xem có bao nhiêu cặp lệnh giao dịch ở hai thời điểm khác nhau ($i < j$) có tổng giá trị giao dịch đạt đúng mức chỉ số S .



Hình minh họa: Kỹ thuật hai con trỏ đối đầu quét mảng hiệu quả $O(N)$

Nhiệm vụ: Cho mảng gồm N số nguyên có thể chứa nhiều phần tử trùng lặp và số nguyên S . Hãy đếm số lượng cặp chỉ số (i, j) với $1 \leq i < j \leq N$ sao cho $A[i] + A[j] = S$.

Đầu vào (Input):

- Dòng 1: 2 số nguyên N và S ($2 \leq N \leq 2 \cdot 10^5, -10^{18} \leq S \leq 10^{18}$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là số lượng cặp chỉ số thỏa mãn.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 6 3 3 3 3 3 3	15

Giải thích: Mảng gồm 6 phần tử đều bằng 3 và mục tiêu $S = 6$. Vì mọi cặp chỉ số (i, j) với $1 \leq i < j \leq 6$ đều có tổng $A[i] + A[j] = 3 + 3 = 6$, nên số lượng cặp thỏa mãn chính là số cách chọn 2 phần tử từ 6 phần tử: $C(6, 2) = 6 * 5 / 2 = 15$ cặp.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng gồm N số nguyên có thể chứa nhiều phần tử trùng lặp và số nguyên S . Hãy đếm số lượng cặp chỉ số (i, j) với $1 \leq i < j \leq N$ sao cho $A[i] + A[j] = S$.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—|—|—| 1 | Nhập dữ liệu vào mảng/biến | Input: 6 6 3 3 3 3 3 | Khởi tạo cấu trúc dữ liệu ban đầu | 2 | Thực thi thuật toán tối ưu | Mảng gồm 6 phần tử đều bằng 3 và mục tiêu $S = 6$. Vì mọi cặp chỉ số (i, j) với $1 \leq i < j \leq 6$ đều có tổng $A[i] + A[j] = \dots$ | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 15 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Mảng gồm 6 phần tử đều bằng 3 và mục tiêu $S = 6$. Vì mọi cặp chỉ số (i, j) với $1 \leq i < j \leq 6$ đều có tổng $A[i] + A[j] = 3 + 3 = 6$, nên số lượng cặp thỏa mãn chính là số cách chọn 2 phần tử từ 6 phần tử: $C(6, 2) = 6 * 5 / 2 = 15$ cặp.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long s;
    if (!(cin >> n >> s)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    sort(a.begin(), a.end());

    int l = 0, r = n - 1;
    long long ans = 0;

    while (l < r) {
        long long sum = a[l] + a[r];
        if (sum == s) {
            if (a[l] == a[r]) {
                long long cnt = r - l + 1;
                ans += cnt * (cnt - 1) / 2;
                break;
            } else {
                long long c1 = 1, c2 = 1;
                while (l + 1 < r && a[l + 1] == a[l]) c1++;
                while (r - 1 > l && a[r - 1] == a[r]) c2++;
                ans += c1 * c2;
                l++;
                r--;
            }
        } else if (sum < s) l++;
        else r--;
    }

    cout << ans << endl;
    return 0;
}
```

```

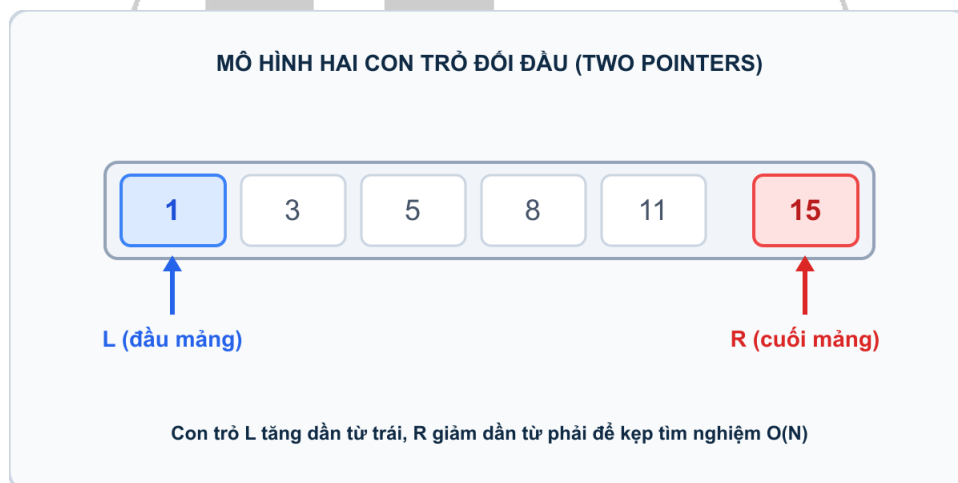
        while (l + 1 < r && a[l + 1] == a[l]) { ++c1; ++l; }
        while (r - 1 > l && a[r - 1] == a[r]) { ++c2; --r; }
        ans += c1 * c2;
        ++l;
        --r;
    }
} else if (sum < s) {
    ++l;
} else {
    --r;
}
}

cout << ans << "\n";
return 0;
}

```

Bài 12 [cppb_hct_12_ghep_tre_em_banh_quy]: Ghép Cặp Trẻ Em Và Bánh Quy

Bối cảnh: Tại một buổi tiệc sinh nhật thiếu nhi, cô giáo chuẩn bị M chiếc bánh quy với các kích thước $S_1, S_2, \dots, S_M$ để phát cho N đứa trẻ. Mỗi đứa trẻ thứ i có mức độ thèm ăn tối thiểu là G_i và chỉ cảm thấy hài lòng nếu nhận được một chiếc bánh có kích thước không nhỏ hơn G_i ($S_j \geq G_i$). Để công bằng, mỗi đứa trẻ chỉ nhận tối đa 1 chiếc bánh và mỗi chiếc bánh chỉ chia cho đúng 1 đứa trẻ. Hãy tìm cách phân phát bánh sao cho số lượng đứa trẻ được thỏa mãn là nhiều nhất có thể.



Hình minh họa: Kỹ thuật hai-con-trỏ đối đầu quét mảng hiệu quả $O(N)$

Nhiệm vụ: Cho mức độ thèm ăn của N đứa trẻ và kích thước của M chiếc bánh quy. Hãy tính số lượng đứa trẻ tối đa có thể được thỏa mãn.

Đầu vào (Input):

- Dòng 1: 2 số nguyên N và M ($1 \leq N, M \leq 10^5$).
- Dòng 2: N số nguyên $G_1, G_2, \dots, G_N$ ($1 \leq G_i \leq 10^9$).
- Dòng 3: M số nguyên $S_1, S_2, \dots, S_M$ ($1 \leq S_j \leq 10^9$).

Đầu ra (Output):

- In ra số lượng đứa trẻ tối đa được thỏa mãn.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 2 1 2 3 1 1	1

Giải thích: Có 3 đĩa trẻ với mức độ thèm ăn là [1, 2, 3] và 2 chiếc bánh quy kích thước [1, 1]. Chiếc bánh đầu tiên kích thước 1 phát cho đĩa trẻ có mức độ thèm ăn 1 (thỏa mãn 1 trẻ). Chiếc bánh thứ hai cũng có kích thước 1, nhưng hai đĩa trẻ còn lại yêu cầu bánh kích thước tối thiểu là 2 và 3, nên chiếc bánh này không thể làm hài lòng thêm đĩa trẻ nào. Do đó số đĩa trẻ tối đa được thỏa mãn là 1.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mức độ thèm ăn của N đĩa trẻ và kích thước của M chiếc bánh quy. Hãy tính số lượng đĩa trẻ tối đa có thể được thỏa mãn.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 2 1 2 3 1 1 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Có 3 đĩa trẻ với mức độ thèm ăn là [1, 2, 3] và 2 chiếc bánh quy kích thước [1, 1]. Chiếc bánh đầu tiên kích thước 1 phá... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 1 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Có 3 đĩa trẻ với mức độ thèm ăn là [1, 2, 3] và 2 chiếc bánh quy kích thước [1, 1]. Chiếc bánh đầu tiên kích thước 1 phát cho đĩa trẻ có mức độ thèm ăn 1 (thỏa mãn 1 trẻ). Chiếc bánh thứ hai cũng có kích thước 1, nhưng hai đĩa trẻ còn lại yêu cầu bánh kích thước tối thiểu là 2 và 3, nên chiếc bánh này không thể làm hài lòng thêm đĩa trẻ nào. Do đó số đĩa trẻ tối đa được thỏa mãn là 1.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;

    vector<long long> g(n), s(m);
    for (int i = 0; i < n; ++i) cin >> g[i];
```

```

for (int i = 0; i < m; ++i) cin >> s[i];

sort(g.begin(), g.end());
sort(s.begin(), s.end());

int i = 0, j = 0;
int satisfied = 0;

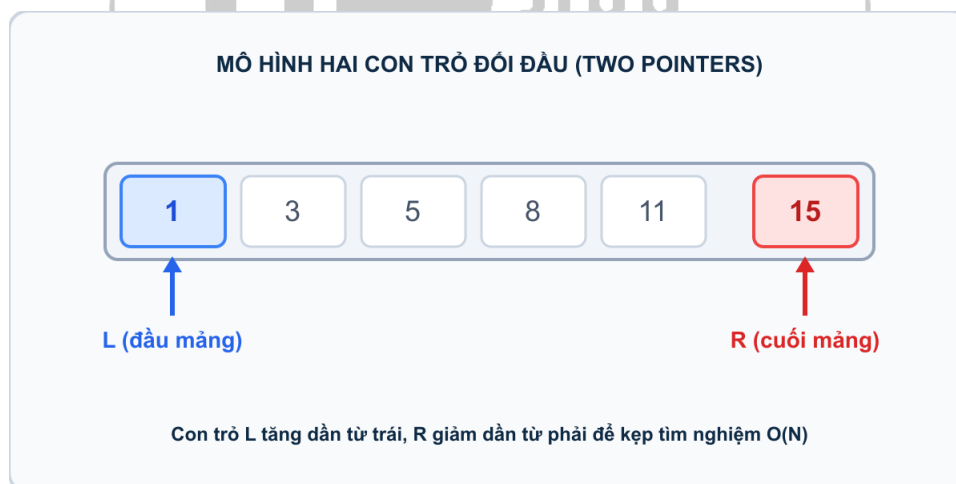
while (i < n && j < m) {
    if (s[j] >= g[i]) {
        ++satisfied;
        ++i;
        ++j;
    } else {
        ++j;
    }
}

cout << satisfied << "\n";
return 0;
}

```

Bài 13 [cppb_hct_13_bo_bon_tong_bang_s]: Bộ Bốn Số Có Tổng Bằng S (4-Sum)

Bối cảnh: Trong một hệ thống mã hóa bảo mật tứ phân, máy chủ phân tích N giá trị khóa lượng tử $A_1, A_2, \dots, A_n$. Để tạo ra khóa ký số chu kỳ tiếp theo, hệ thống cần chọn ra 4 thành phần khóa ở 4 vị trí phân biệt sao cho tổng năng lượng của 4 thành phần này đạt đúng giá trị kích hoạt S . Nếu có nhiều bộ bốn số thỏa mãn, hệ thống ghi nhận một bộ bất kỳ theo thứ tự tăng dần.



Hình minh họa: Kỹ thuật hai con trỏ đối đầu quét mảng hiệu quả $O(N)$

Nhiệm vụ: Cho mảng gồm N số nguyên và số nguyên S . Hãy tìm 4 phần tử ở 4 vị trí phân biệt có tổng đúng bằng S . Nếu có nhiều bộ, in ra một bộ theo thứ tự tăng dần. Nếu không tồn tại, in ra -1.

Đầu vào (Input):

- Dòng 1: 2 số nguyên N và S ($4 \leq N \leq 1000, -10^{18} \leq S \leq 10^{18}$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra 4 số nguyên tăng dần hoặc -1.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 20 2 7 5 1 8 4	1 4 7 8

Giải thích: Sắp xếp mảng tăng dần: [1, 2, 4, 5, 7, 8] và $S = 20$. Bộ bốn số gồm các phần tử 1, 4, 7, 8 có tổng là $1 + 4 + 7 + 8 = 20$ đúng bằng S . Kết quả in ra: 1 4 7 8.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng gồm N số nguyên và số nguyên S . Hãy tìm 4 phần tử ở 4 vị trí phân biệt có tổng đúng bằng S . Nếu có nhiều bộ, in ra một bộ theo thứ tự tăng dần. Nếu không tồn tại, in ra -1.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 6 20 2 7 5 1 8 4 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Sắp xếp mảng tăng dần: [1, 2, 4, 5, 7, 8] và $S = 20$. Bộ bốn số gồm các phần tử 1, 4, 7, 8 có tổng là $1 + 4 + 7 + 8 = 20$... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 1 4 7 8 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Sắp xếp mảng tăng dần: [1, 2, 4, 5, 7, 8] và $S = 20$. Bộ bốn số gồm các phần tử 1, 4, 7, 8 có tổng là $1 + 4 + 7 + 8 = 20$ đúng bằng S . Kết quả in ra: 1 4 7 8.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. $N-1$) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long s;
    if (!(cin >> n >> s)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    sort(a.begin(), a.end());

    for (int i = 0; i < n - 3; ++i) {
        for (int j = i + 1; j < n - 2; ++j) {
            long long target = s - a[i] - a[j];
            int l = j + 1, r = n - 1;
```

```

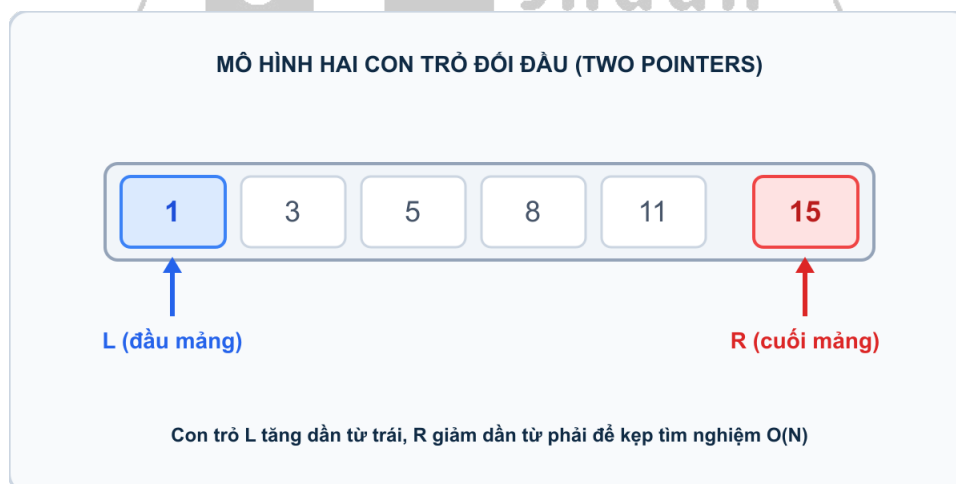
while (l < r) {
    long long sum = a[l] + a[r];
    if (sum == target) {
        cout << a[l] << " " << a[r] << " " << a[l] << " " << a[r] << "\n";
        return 0;
    } else if (sum < target) {
        ++l;
    } else {
        --r;
    }
}

cout << -1 << "\n";
return 0;
}

```

Bài 14 [cppb_hct_14_hai_con_trong_cuc_han]: Cặp Số Tối Ưu Với Chênh Lệch Cực Hạn

Bối cảnh: Tại một trạm quan trắc địa chấn liên vùng, hai trạm cảm biến đặt ở hai thung lũng ghi nhận N xung địa chấn dãy A và M xung địa chấn dãy B (với biên độ có thể lên tới 10^{18}). Để đồng bộ pha tín hiệu giữa hai trạm, các nhà địa chấn học cần tìm một xung địa chấn A_i từ trạm A và một xung địa chấn B_j từ trạm B sao cho độ lệch biên độ tuyệt đối giữa chúng $|A_i - B_j|$ là nhỏ nhất có thể.



Hình minh họa: Kỹ thuật hai con trỏ đối đầu quét mảng hiệu quả $O(N)$

Nhiệm vụ: Cho 2 dãy số nguyên A gồm N phần tử và B gồm M phần tử. Hãy tìm một phần tử $A[i]$ và một phần tử $B[j]$ sao cho độ chênh lệch $|A[i] - B[j]|$ là nhỏ nhất có thể.

Đầu vào (Input):

- Dòng 1: 2 số nguyên N và M ($1 \leq N, M \leq 2 \cdot 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($-10^{18} \leq A_i \leq 10^{18}$).
- Dòng 3: M số nguyên $B_1, B_2, \dots, B_M$ ($-10^{18} \leq B_j \leq 10^{18}$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là giá trị chênh lệch nhỏ nhất $|A_i - B_j|$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 3 1 5 10 2 8 14	1

Giải thích: Dãy A = [1, 5, 10] và dãy B = [2, 8, 14]. So sánh các cặp phần tử: chọn A[0] = 1 và B[0] = 2 cho độ chênh lệch $|1 - 2| = 1$. Đây là mức chênh lệch nhỏ nhất có thể đạt được giữa hai dãy.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho 2 dãy số nguyên A gồm N phần tử và B gồm M phần tử. Hãy tìm một phần tử A[i] và một phần tử B[j] sao cho độ chênh lệch $|A[i] - B[j]|$ là nhỏ nhất có thể.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 3 1 5 10 2 8 14 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Dãy A = [1, 5, 10] và dãy B = [2, 8, 14]. So sánh các cặp phần tử: chọn A[0] = 1 và B[0] = 2 cho độ chênh lệch $|1 - 2| = \dots$ | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 1 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Dãy A = [1, 5, 10] và dãy B = [2, 8, 14]. So sánh các cặp phần tử: chọn A[0] = 1 và B[0] = 2 cho độ chênh lệch $|1 - 2| = 1$. Đây là mức chênh lệch nhỏ nhất có thể đạt được giữa hai dãy.

3. Lưu ý & Bẫy lỗi thường gặp

- Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm main() để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;

    vector<long long> a(n), b(m);
    for (int i = 0; i < n; ++i) cin >> a[i];
    for (int i = 0; i < m; ++i) cin >> b[i];

    sort(a.begin(), a.end());
    sort(b.begin(), b.end());

    int i = 0, j = 0;
    long long min_diff = abs(a[0] - b[0]);
```

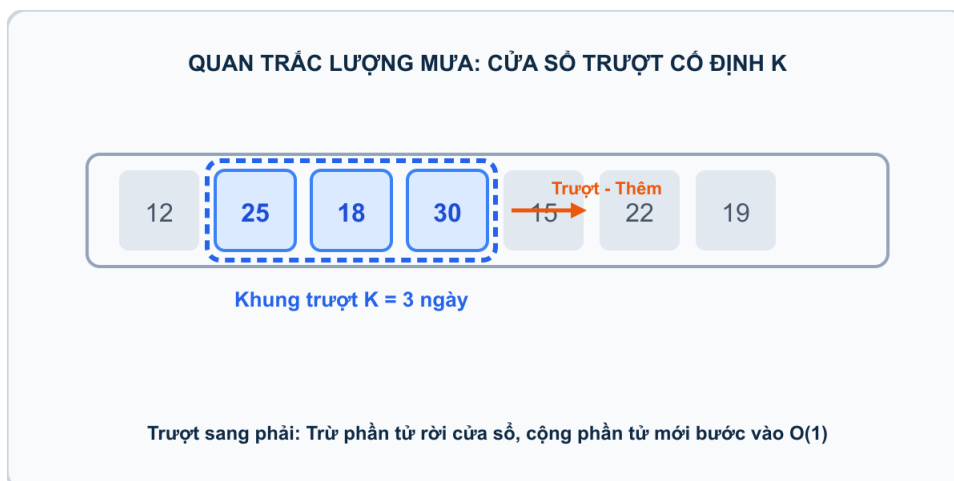
```
while (i < n && j < m) {  
    min_diff = min(min_diff, abs(a[i] - b[j]));  
    if (a[i] == b[j]) break;  
    else if (a[i] < b[j]) ++i;  
    else ++j;  
}  
  
cout << min_diff << "\n";  
return 0;  
}
```



Bài 06: Kỹ thuật cửa sổ trượt

Bài 01 [cppb_est_01_cua_so_k]: Tổng Cửa Sổ Cố Định K

Bối cảnh: Một thiết bị quan trắc lượng mưa tự động ghi nhận lượng nước mưa rơi xuống trong N giờ liên tiếp, ký hiệu là $A_1, A_2, \dots, A_N$. Trung tâm khí tượng thủy văn cần đánh giá nguy cơ sạt lở bằng cách tìm ra giai đoạn mưa lớn nhất kéo dài đúng K giờ liên tục (tổng lượng mưa lớn nhất trong K giờ liên tiếp) và xác định chỉ số giờ bắt đầu của đợt mưa đó.



Hình minh họa: Mô hình khung cửa sổ trượt duy trì trạng thái liên tục

Nhiệm vụ: Cho dãy N số nguyên và số nguyên dương K ($K \leq N$). Hãy tìm đoạn con gồm K phần tử liên tiếp có tổng lớn nhất. In ra tổng lớn nhất và chỉ số bắt đầu (1-indexed) của đoạn con đó. Nếu có nhiều đoạn cùng đạt tổng lớn nhất, in ra chỉ số bắt đầu nhỏ nhất.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên dương N và K ($1 \leq K \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra 2 số nguyên trên một dòng cách nhau bởi khoảng trắng: tổng lớn nhất và vị trí bắt đầu.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
7 3 2 1 5 1 3 2 4	9 3

Giải thích: Các cửa sổ độ dài $K = 3$ gồm: $[2, 1, 5]$ (tổng 8), $[1, 5, 1]$ (tổng 7), $[5, 1, 3]$ (tổng 9 tại vị trí 3), $[1, 3, 2]$ (tổng 6), $[3, 2, 4]$ (tổng 9 tại vị trí 5). Tổng lớn nhất là 9, đạt được sớm nhất tại vị trí bắt đầu 3. Kết quả in ra: 9 3.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho dãy N số nguyên và số nguyên dương K ($K \leq N$). Hãy tìm đoạn con gồm K phần tử liên tiếp có tổng lớn nhất. In ra tổng lớn nhất và chỉ số bắt đầu (1-indexed) của đoạn con đó. Nếu có nhiều đoạn cùng đạt tổng lớn nhất, in ra chỉ số bắt đầu nhỏ nhất.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 7 3 2 1 5 1 3 2 4 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Các cửa sổ độ dài K = 3 gồm: [2, 1, 5] (tổng 8), [1, 5, 1] (tổng 7), [5, 1, 3] (tổng 9 tại vị trí 3), [1, 3, 2] (tổng 6)... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 9 3 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Các cửa sổ độ dài K = 3 gồm: [2, 1, 5] (tổng 8), [1, 5, 1] (tổng 7), [5, 1, 3] (tổng 9 tại vị trí 3), [1, 3, 2] (tổng 6), [3, 2, 4] (tổng 9 tại vị trí 5). Tổng lớn nhất là 9, đạt được sớm nhất tại vị trí bắt đầu 3. Kết quả in ra: 9 3.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    if (!(cin >> n >> k)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    long long cur_sum = 0;
    for (int i = 0; i < k; ++i) cur_sum += a[i];

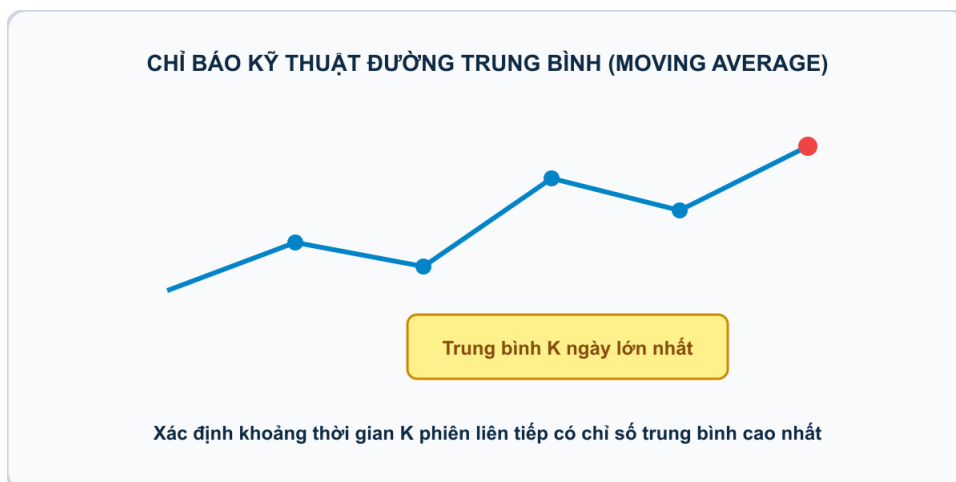
    long long max_sum = cur_sum;
    for (int i = k; i < n; ++i) {
        cur_sum += a[i] - a[i - k];
        max_sum = max(max_sum, cur_sum);
    }

    cout << max_sum << "\n";
    return 0;
}
```

Bài 02 [cppb_cst_02_trung_binh_k_lon_nhat]: Giá Trị Trung Bình Lớn Nhất Của Đoạn K

Bối cảnh: Trong phân tích kỹ thuật chứng khoán, chỉ báo đường trung bình động MA(K) được dùng để xác định xu hướng giá cổ phiếu. Nhà phân tích cần khảo sát lịch sử giá đóng cửa trong N phiên

giao dịch liên tiếp để tìm ra khoảng thời gian K phiên liên tiếp có giá trị trung bình đạt mức cao nhất.



Hình minh họa: Mô hình khung của sổ trượt duy trì trạng thái liên tục

Nhiệm vụ: Cho dãy gồm N số nguyên và số nguyên K ($K \leq N$). Hãy tìm giá trị trung bình lớn nhất của một đoạn con gồm K phần tử liên tiếp, làm tròn đến đúng 3 chữ số thập phân.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và K ($1 \leq K \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra giá trị trung bình lớn nhất tìm được với đúng 3 chữ số sau dấu phẩy.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 4 1 12 -5 -6	0.500

Giải thích: Với $N = 4$ và $K = 4$, chỉ có duy nhất 1 đoạn con gồm 4 phần tử: $[1, 12, -5, -6]$. Tổng của đoạn là $1 + 12 - 5 - 6 = 2$. Giá trị trung bình là $2 / 4 = 0.500$. Kết quả in ra: 0.500.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho dãy gồm N số nguyên và số nguyên K ($K \leq N$). Hãy tìm giá trị trung bình lớn nhất của một đoạn con gồm K phần tử liên tiếp, làm tròn đến đúng 3 chữ số thập phân.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 4 4 1 12 -5 -6 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Với $N = 4$ và $K = 4$, chỉ có duy nhất 1 đoạn con gồm 4 phần tử: $[1, 12, -5, -6]$. Tổng của đoạn là $1 + 12 - 5 - 6 = 2$. Giá ... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 0.500 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Với $N = 4$ và $K = 4$, chỉ có duy nhất 1 đoạn con gồm 4 phần tử: $[1, 12, -5, -6]$. Tổng của đoạn là $1 + 12 - 5 - 6 = 2$. Giá trị trung bình là $2 / 4 = 0.500$. Kết quả in ra: 0.500.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    if (!(cin >> n >> k)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    long long cur_sum = 0;
    for (int i = 0; i < k; ++i) cur_sum += a[i];

    long long max_sum = cur_sum;
    for (int i = k; i < n; ++i) {
        cur_sum += a[i] - a[i - k];
        max_sum = max(max_sum, cur_sum);
    }

    double ans = (double)max_sum / k;
    cout << fixed << setprecision(3) << ans << "\n";
    return 0;
}
```

Bài 03 [cppb_cst_03_doan_con_ngan_nhat_tong_s]: Đoạn Con Ngắn Nhất Có Tổng Đạt S

Bối cảnh: Một vận động viên cử tạ thực hiện N hiệp nâng tạ trong tuần với khối lượng tích lũy mỗi hiệp là các số không âm $A_1, A_2, \dots, A_n$. Huấn luyện viên đặt ra chỉ tiêu thành tích là phải đạt tổng khối lượng nâng tối thiểu S trong một chuỗi các hiệp thi đấu liên tiếp. Để tiết kiệm thể lực, vận động viên muốn tìm chuỗi hiệp đấu ngắn nhất (số lượng hiệp ít nhất) đạt được tổng chỉ tiêu S .



Hình minh họa: Mô hình khung cửa sổ trượt duy trì trạng thái liên tục

Nhiệm vụ: Cho dãy gồm N số nguyên không âm $A_1, A_2, \dots, A_N$ và số nguyên dương S . Hãy tìm độ dài nhỏ nhất của một đoạn con liên tiếp có tổng lớn hơn hoặc bằng S . Nếu không tồn tại đoạn con nào thỏa mãn, in ra 0.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên dương N và S ($1 \leq N \leq 10^5, 1 \leq S \leq 10^{14}$).
- Dòng 2: Chứa N số nguyên không âm $A_1, A_2, \dots, A_N$ ($0 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là độ dài nhỏ nhất tìm được, hoặc 0 nếu không thể đạt tổng S .

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 7 2 3 1 2 4 3	2

Giải thích: Đoạn con $[4, 3]$ ở cuối dãy có tổng là $4 + 3 = 7 \geq 7$ và có độ dài bằng 2. Không tồn tại bất kỳ phần tử đơn lẻ nào có giá trị ≥ 7 . Vì vậy độ dài nhỏ nhất đạt chỉ tiêu là 2.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho dãy số nguyên dương A và số nguyên dương S . Tìm độ dài nhỏ nhất của một đoạn con liên tiếp có tổng các phần tử $\geq S$. Nếu không tồn tại đoạn nào, in 0.
 - Kỹ thuật Cửa sổ trượt biến thiên (Variable-size Sliding Window):**
 - Vì tất cả các phần tử A_i đều là số nguyên dương ($A_i > 0$), mảng có tính chất đơn điệu: khi mở rộng cửa sổ sang phải thì tổng luôn tăng, khi thu hẹp cửa sổ bên trái thì tổng luôn giảm.
 - Sử dụng 2 con trỏ $L = 0, R = 0$ và biến tích lũy $cur_sum = 0$:
- Mở rộng biên phải R : Cộng $cur_sum += a[R]$.
 - Trong khi $cur_sum \geq S$: Ta đã tìm thấy một đoạn hợp lệ có độ dài $R - L + 1$. Cập nhật $min_len = \min(min_len, R - L + 1)$. Sau đó thử thu hẹp biên trái bằng cách trừ $cur_sum -= a[L]$ và tăng $L++$ để tìm đoạn ngắn hơn.

- Mỗi phần tử đi vào cửa sổ qua R đúng 1 lần và ra khỏi cửa sổ qua L tối đa 1 lần $\Rightarrow$ Tổng số thao tác con trỏ không quá $2N$. Độ phức tạp thời gian: $\mathcal{O}(N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử: $N = 6, S = 7$, mảng $a = [2, 3, 1, 2, 4, 3]$.

Bước	Con trỏ R	Thêm $a[R]$	cur_sum	$cur_sum \geq 7$	Thu hẹp con trỏ L & Cập nhật min_len
1	$R = 0$	+2	2	Chưa	$L = 0$
2	$R = 1$	+3	5	Chưa	$L = 0$
3	$R = 2$	+1	6	Chưa	$L = 0$
4	$R = 3$	+2	8	$8 \geq 7$	Đoạn $[2,3,1,2]$ dài 4. $min_len = 4$. Trừ $a[0] = 2 \Rightarrow cur = 6, L = 1$.
5	$R = 4$	+4	10	$10 \geq 7$	Đoạn $[3,1,2,4]$ dài 4. Trừ $a[1] = 3 \Rightarrow cur = 7, L = 2$. Lại có $7 \geq 7 \Rightarrow$ đoạn $[1,2,4]$ dài 3. $min_len = 3$. Trừ $a[2] = 1 \Rightarrow cur = 6, L = 3$.

6	$R = 5$	$+3$	9	$9 \geq 7$	Đoạn $[2,4,3]$ dài 3. Trừ $a[3] = 2 \Rightarrow$ $cur = 7, L =$ 4 . Lại có $7 \geq$ $7 \Rightarrow$ đoạn $[4,3]$ dài 2! $min_len = 2$. Trừ $a[4] =$ $4 \Rightarrow cur =$ $3, L = 5$.
---	---------	------	---	------------	--

Kết quả: Đoạn ngắn nhất có độ dài 2 (chính là đoạn $[4,3]$).

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Mảng có số âm hoặc số 0:** Thuật toán 2 con trỏ chỉ áp dụng được khi các phần tử đều dương ($A_i > 0$). Nếu có số âm, tính đơn điệu bị phá vỡ, phải dùng Mảng tiền tố + Binary Search / Deque.
- **Bẫy 2 — Không có đoạn nào thỏa mãn:** Khởi tạo $min_len = N + 1$. Nếu kết thúc vòng lặp mà $min_len > N$ thì in 0.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long s;
    if (!(cin >> n >> s)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    int l = 0;
    long long cur_sum = 0;
    int min_len = n + 1;

    for (int r = 0; r < n; ++r) {
        cur_sum += a[r];
        while (cur_sum >= s) {
            min_len = min(min_len, r - l + 1);
            cur_sum -= a[l];
            ++l;
        }
    }

    if (min_len > n) cout << -1 << "\n";
    else cout << min_len << "\n";
}
```

```
return 0;
}
```

Bài 04 [cppb_est_04_doan_con_dai_nhat_tong_s]: Đoạn Con Dài Nhất Có Tổng Không Quá S

Bối cảnh: Tại một trạm sạc xe điện nhanh trên cao tốc, các xe xếp hàng chờ sạc với nhu cầu tiêu thụ điện năng lần lượt là $A_1, A_2, \dots, A_n$ (kWh). Nguồn pin tích năng lượng mặt trời của trạm tại thời điểm hiện tại chỉ còn lại dung lượng tối đa S . Trạm muốn phục vụ một đợt xe liên tiếp dài nhất sao cho tổng năng lượng cung cấp không vượt quá giới hạn S để không làm sập nguồn.



Hình minh họa: Mô hình khung cửa sổ trượt duy trì trạng thái liên tục

Nhiệm vụ: Cho dãy gồm N số nguyên không âm và một số nguyên dương S . Hãy tìm độ dài lớn nhất của một đoạn con liên tiếp có tổng không vượt quá S .

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên dương N và S ($1 \leq N \leq 10^5, 1 \leq S \leq 10^{14}$).
- Dòng 2: Chứa N số nguyên không âm $A_1, A_2, \dots, A_N$ ($0 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra độ dài lớn nhất của đoạn con thỏa mãn.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 10 1 2 3 4 5	4

Giải thích: Đoạn con $[1, 2, 3, 4]$ có tổng $1 + 2 + 3 + 4 = 10 \leq 10$ và có độ dài bằng 4. Nếu xét cả 5 phần tử thì tổng là $15 > 10$. Do đó độ dài lớn nhất là 4.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho dãy gồm N số nguyên không âm và một số nguyên dương S . Hãy tìm độ dài lớn nhất của một đoạn con liên tiếp có tổng không vượt quá S .

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nhập dữ liệu vào mảng/biến | Input: 5 10 1 2 3 4 5 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Đoạn con [1, 2, 3, 4] có tổng $1 + 2 + 3 + 4 = 10 \leq 10$ và có độ dài bằng 4. Nếu xét cả 5 phần tử thì tổng là $15 > 10$. Do... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 4 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Đoạn con [1, 2, 3, 4] có tổng $1 + 2 + 3 + 4 = 10 \leq 10$ và có độ dài bằng 4. Nếu xét cả 5 phần tử thì tổng là $15 > 10$. Do đó độ dài lớn nhất là 4.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long s;
    if (!(cin >> n >> s)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

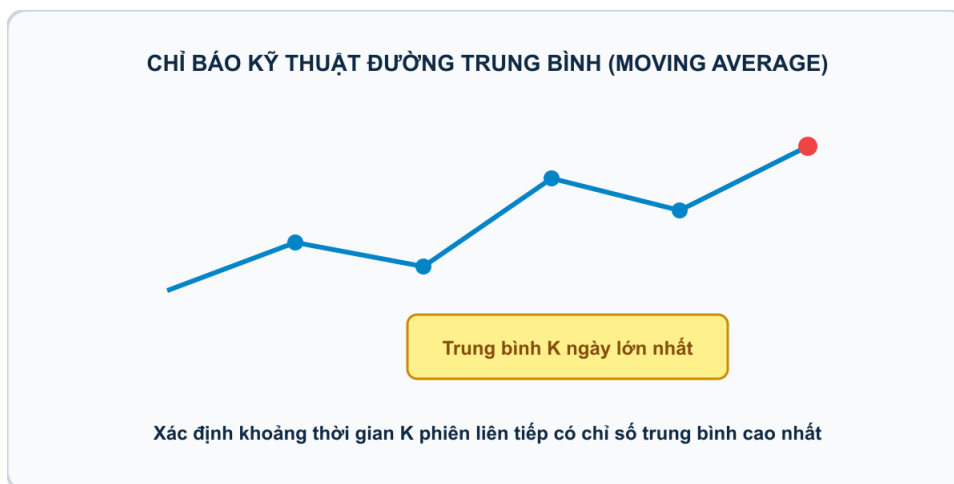
    int l = 0;
    long long cur_sum = 0;
    int max_len = 0;

    for (int r = 0; r < n; ++r) {
        cur_sum += a[r];
        while (cur_sum > s) {
            cur_sum -= a[l];
            ++l;
        }
        max_len = max(max_len, r - l + 1);
    }

    cout << max_len << "\n";
    return 0;
}
```

Bài 05 [cppb_cst_05_lat_bit_k_so_khong]: Đoạn Con Chứa Tối Đa K Số 0 (Lật Bit)

Bối cảnh: Trong đường truyền dữ liệu viễn thông cáp quang, chuỗi bit truyền đi gồm N ký tự 0 và 1. Do suy hao tín hiệu quang, một số bit 1 bị biến thành bit 0. Bộ giải mã thông minh tại đầu nhận được phép phục hồi (lật) tối đa K bit 0 trở lại thành bit 1. Kỹ sư viễn thông cần tìm chuỗi bit 1 liên tiếp dài nhất có thể tạo ra sau khi thực hiện tối đa K phép lật bit.



Hình minh họa: Mô hình khung cửa sổ trượt duy trì trạng thái liên tục

Nhiệm vụ: Cho một mảng nhị phân A gồm N phần tử (A_i thuộc $\{0, 1\}$) và số nguyên không âm K . Hãy tìm độ dài lớn nhất của một đoạn con liên tiếp chỉ toàn bit 1 sau khi lật tối đa K số 0 thành số 1.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và K ($1 \leq N \leq 10^5, 0 \leq K \leq N$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($A_i \in \{0, 1\}$).

Đầu ra (Output):

- In ra độ dài lớn nhất của đoạn con toàn số 1 tạo được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
11 2 1 1 1 0 0 0 1 1 1 1 0	6

Giải thích: Xét đoạn từ vị trí 5 đến vị trí 10: $[0, 0, 1, 1, 1, 1]$. Đoạn này có độ dài 6 và chứa đúng hai số 0. Khi lật 2 số 0 này thành 1, ta thu được chuỗi 6 số 1 liên tiếp. Đây là độ dài dài nhất có thể tạo được.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho một mảng nhị phân A gồm N phần tử (A_i thuộc $\{0, 1\}$) và số nguyên không âm K . Hãy tìm độ dài lớn nhất của một đoạn con liên tiếp chỉ toàn bit 1 sau khi lật tối đa K số 0 thành số 1.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 11 2 1 1 1 0 0 0 1 1 1 1 0 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Xét đoạn từ vị trí 5 đến vị trí 10: [0, 0, 1, 1, 1, 1]. Đoạn này có độ dài 6 và chứa đúng hai số 0. Khi lật 2 số 0 này t... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 6 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Xét đoạn từ vị trí 5 đến vị trí 10: [0, 0, 1, 1, 1, 1]. Đoạn này có độ dài 6 và chứa đúng hai số 0. Khi lật 2 số 0 này thành 1, ta thu được chuỗi 6 số 1 liên tiếp. Đây là độ dài dài nhất có thể tạo được.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    if (!(cin >> n >> k)) return 0;

    vector<int> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    int l = 0;
    int zero_cnt = 0;
    int max_len = 0;

    for (int r = 0; r < n; ++r) {
        if (a[r] == 0) ++zero_cnt;

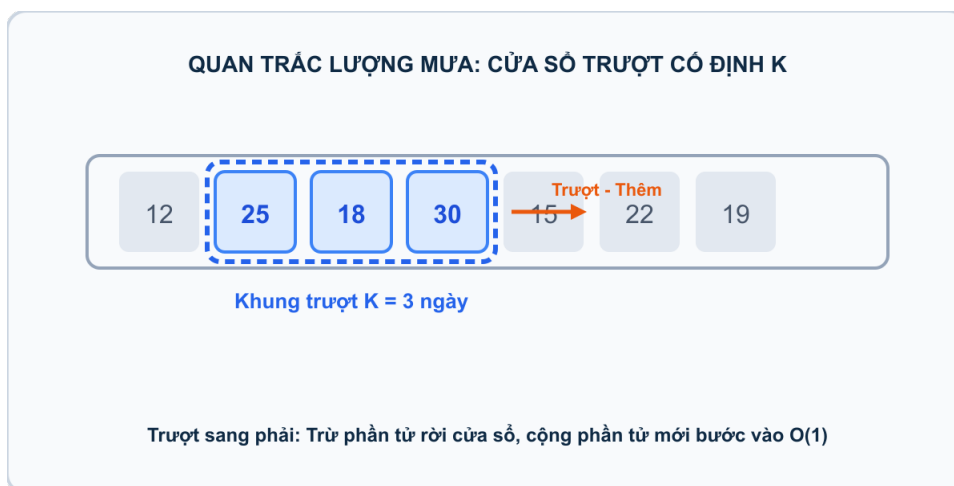
        while (zero_cnt > k) {
            if (a[l] == 0) --zero_cnt;
            ++l;
        }

        max_len = max(max_len, r - l + 1);
    }

    cout << max_len << "\n";
    return 0;
}
```

Bài 06 [cppb_cst_06_camera_giao_thong]: Giám Sát Camera Giao Thông Thông Minh

Bối cảnh: Trên một tuyến cao tốc huyết mạch dài, cảnh sát giao thông thiết lập N cột gắn camera giám sát, trạng thái mỗi camera được ghi nhận: 1 là hoạt động tốt, 0 là bị hư hỏng. Để đảm bảo an toàn tuyệt đối, quy chế cao tốc quy định mọi đoạn đường liên tiếp có chiều dài K cột camera bắt buộc phải có ít nhất B camera hoạt động bình thường. Đội bảo trì cần tính toán số lượng camera hư hỏng tối thiểu cần sửa chữa để mọi phân đoạn K cột đều đạt chuẩn an toàn.



Hình minh họa: Mô hình khung cửa sổ trượt duy trì trạng thái liên tục

Nhiệm vụ: Cho mảng nhị phân A gồm N phần tử (1: hoạt động, 0: hỏng). Hãy tìm số lượng camera hỏng ít nhất cần sửa thành hoạt động sao cho trong mọi đoạn gồm K camera liên tiếp đều có ít nhất B camera hoạt động.

Đầu vào (Input):

- Dòng 1: Chứa 3 số nguyên N, K, B ($1 \leq B \leq K \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($A_i \in \{0, 1\}$).

Đầu ra (Output):

- In ra số camera tối thiểu cần sửa chữa.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
10 6 5 1 0 1 1 0 1 1 1 0 1	1

Giải thích: Đoạn từ vị trí 1 đến 6 là $[1, 0, 1, 1, 0, 1]$ chỉ có 4 camera hoạt động (thiếu 1 camera so với chuẩn $B = 5$). Ta sửa camera thứ 2 (hoặc thứ 5) từ 0 thành 1. Khi đó mọi đoạn 6 camera liên tiếp đều có ít nhất 5 camera hoạt động. Do đó chỉ cần sửa tối thiểu 1 camera.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng nhị phân A gồm N phần tử (1: hoạt động, 0: hỏng). Hãy tìm số lượng camera hỏng ít nhất cần sửa thành hoạt động sao cho trong mọi đoạn gồm K camera liên tiếp đều có ít nhất B camera hoạt động.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nhập dữ liệu vào mảng/biến | Input: 10 6 5 1 0 1 1 0 1 1 0 1 | Khởi tạo cấu trúc dữ liệu ban đầu | 2 | Thực thi thuật toán tối ưu | Đoạn từ vị trí 1 đến 6 là [1, 0, 1, 1, 0, 1] chỉ có 4 camera hoạt động (thiếu 1 camera so với chuẩn B = 5). Ta sửa camer... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 1 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Đoạn từ vị trí 1 đến 6 là [1, 0, 1, 1, 0, 1] chỉ có 4 camera hoạt động (thiếu 1 camera so với chuẩn B = 5). Ta sửa camera thứ 2 (hoặc thứ 5) từ 0 thành 1. Khi đó mọi đoạn 6 camera liên tiếp đều có ít nhất 5 camera hoạt động. Do đó chỉ cần sửa tối thiểu 1 camera.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    if (!(cin >> n >> k)) return 0;

    vector<int> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    int cur_broken = 0;
    for (int i = 0; i < k; ++i) {
        if (a[i] == 0) ++cur_broken;
    }

    int min_broken = cur_broken;
    for (int i = k; i < n; ++i) {
        if (a[i] == 0) ++cur_broken;
        if (a[i - k] == 0) --cur_broken;
        min_broken = min(min_broken, cur_broken);
    }

    cout << min_broken << "\n";
    return 0;
}
```

Bài 07 [cppb_cst_07_min_max_cua_so_k]: Tìm Min Trong Mọi Cửa Sổ Độ Dài K

Bối cảnh: Trong hệ thống giám sát tải máy chủ trung tâm, thông số mức tải CPU được ghi nhận liên tục qua N thời điểm. Để phát hiện những khoảng thời gian hệ thống hoạt động ổn định nhất, quản

trị viên cần trượt một khung thời gian gồm K chu kỳ liên tiếp từ đầu đến cuối và tìm mức tải thấp nhất (giá trị nhỏ nhất) trong từng khung thời gian đó.



Hình minh họa: Mô hình khung cửa sổ trượt duy trì trạng thái liên tục

Nhiệm vụ: Cho mảng gồm N số nguyên và số nguyên K. Với mỗi cửa sổ gồm K phần tử liên tiếp từ trái sang phải, hãy tìm giá trị nhỏ nhất trong cửa sổ đó.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và K ($1 \leq K \leq N \leq 10^4$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra $N - K + 1$ số nguyên cách nhau bởi khoảng trắng là giá trị nhỏ nhất của các cửa sổ.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 3 4 2 12 3 5 1	2 2 3 1

Giải thích: Các cửa sổ độ dài K = 3 gồm: [4, 2, 12] có min = 2; [2, 12, 3] có min = 2; [12, 3, 5] có min = 3; [3, 5, 1] có min = 1. Kết quả in ra lần lượt là: 2 2 3 1.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng gồm N số nguyên và số nguyên K. Với mỗi cửa sổ gồm K phần tử liên tiếp từ trái sang phải, hãy tìm giá trị nhỏ nhất trong cửa sổ đó.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 6 3 4 2 12 3 5 1 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Các cửa sổ độ dài K = 3 gồm: [4, 2, 12] có min = 2; [2, 12, 3] có min = 2; [12, 3, 5] có min = 3; [3, 5, 1] có min = 1. ... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 2 2 3 1 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Các cửa sổ độ dài K = 3 gồm: [4, 2, 12] có min = 2; [2, 12, 3] có min = 2; [12, 3, 5] có min = 3; [3, 5, 1] có min = 1. Kết quả in ra lần lượt là: 2 2 3 1.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

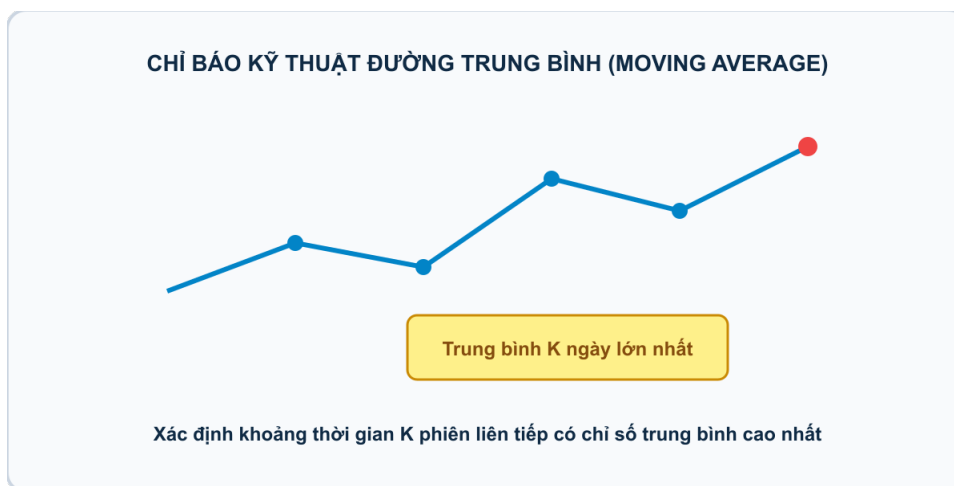
    int n, k;
    if (!(cin >> n >> k)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    for (int i = 0; i <= n - k; ++i) {
        long long cur_min = a[i];
        for (int j = i + 1; j < i + k; ++j) {
            cur_min = min(cur_min, a[j]);
        }
        cout << cur_min << (i == n - k ? "" : " ");
    }
    cout << "\n";
    return 0;
}
```

Bài 08 [cppb_cst_08_dem_doan_con_tong_be_hon_s]: Đếm Số Lượng Đoạn Con Có Tổng Không Quá S

Bối cảnh: Tại một trạm cân hàng hóa thông minh, băng chuyền chuyển qua N gói bưu kiện liên tiếp có trọng lượng không âm $A_1, A_2, \dots, A_n$. Xe nâng tự động nhận nhiệm vụ bốc xếp các kiện hàng liên tiếp nhau sao cho tổng tải trọng bốc một lần không vượt quá định mức S. Bộ điều khiển cần tính toán có bao nhiêu phương án chọn đoạn bưu kiện liên tiếp thỏa mãn tải trọng cho phép.



Hình minh họa: Mô hình khung cửa sổ trượt duy trì trạng thái liên tục

Nhiệm vụ: Cho mảng gồm N số nguyên không âm $A_1, A_2, \dots, A_N$ và số nguyên S . Hãy đếm số lượng đoạn con liên tiếp có tổng không vượt quá S .

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và S ($1 \leq N \leq 10^5, 0 \leq S \leq 10^{14}$).
- Dòng 2: Chứa N số nguyên không âm $A_1, A_2, \dots, A_N$ ($0 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là số lượng đoạn con thỏa mãn.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 3 1 2 1 4	4

Giải thích: Các đoạn con liên tiếp có tổng ≤ 3 là: $[1]$ (tổng 1), $[2]$ (tổng 2), $[1]$ (ở vị trí 3, tổng 1), và $[1, 2]$ (tổng 3). Tổng cộng có 4 đoạn con thỏa mãn.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng gồm N số nguyên không âm $A_1, A_2, \dots, A_N$ và số nguyên S . Hãy đếm số lượng đoạn con liên tiếp có tổng không vượt quá S .

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 4 3 1 2 1 4 | Khởi tạo cấu trúc dữ liệu ban đầu
 | 2 | Thực thi thuật toán tối ưu | Các đoạn con liên tiếp có tổng ≤ 3 là: $[1]$ (tổng 1), $[2]$ (tổng 2), $[1]$ (ở vị trí 3, tổng 1), và $[1, 2]$ (tổng 3). Tổng c... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 4 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Các đoạn con liên tiếp có tổng ≤ 3 là: $[1]$ (tổng 1), $[2]$ (tổng 2), $[1]$ (ở vị trí 3, tổng 1), và $[1, 2]$ (tổng 3). Tổng cộng có 4 đoạn con thỏa mãn.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long s;
    if (!(cin >> n >> s)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    int l = 0;
    long long cur_sum = 0;
    long long count = 0;

    for (int r = 0; r < n; ++r) {
        cur_sum += a[r];
        while (cur_sum > s) {
            cur_sum -= a[l];
            ++l;
        }
        count += (r - l + 1);
    }

    cout << count << "\n";
    return 0;
}
```

Bài 09 [cppb_cst_09_dem_doan_con_tong_bang_s]: Đếm Số Lượng Đoạn Con Có Tổng Đúng Bằng S

Bối cảnh: Một người thợ kim hoàn cắt gọt thanh vàng nguyên khối được chia thành N đoạn nhỏ liên tiếp với trọng lượng vàng nguyên chất mỗi đoạn là các số nguyên dương $A_1, A_2, \dots, A_n$. Khách hàng đặt mua một chuỗi các đoạn vàng liên kế nhau sao cho tổng trọng lượng của chuỗi đúng bằng S chỉ vàng. Hãy tính số lượng cách chọn đoạn liên kế đáp ứng đúng yêu cầu của khách.



Hình minh họa: Mô hình khung cửa sổ trượt duy trì trạng thái liên tục

Nhiệm vụ: Cho mảng gồm N số nguyên dương $A_1, A_2, \dots, A_N$ ($A_i > 0$) và số nguyên dương S . Hãy đếm số lượng đoạn con liên tiếp có tổng đúng bằng S .

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và S ($1 \leq N \leq 10^5, 1 \leq S \leq 10^{14}$).
- Dòng 2: Chứa N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra số lượng đoạn con có tổng đúng bằng S .

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 5 1 2 3 2 1	2

Giải thích: Các đoạn con liên tiếp có tổng đúng bằng 5 là: đoạn $[2, 3]$ ($2 + 3 = 5$) và đoạn $[3, 2]$ ($3 + 2 = 5$). Tổng cộng có đúng 2 đoạn con thỏa mãn.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng gồm N số nguyên dương $A_1, A_2, \dots, A_N$ ($A_i > 0$) và số nguyên dương S . Hãy đếm số lượng đoạn con liên tiếp có tổng đúng bằng S .

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 5 1 2 3 2 1 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Các đoạn con liên tiếp có tổng đúng bằng 5 là: đoạn $[2, 3]$ ($2 + 3 = 5$) và đoạn $[3, 2]$ ($3 + 2 = 5$). Tổng cộng có đúng 2 đ... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 2 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Các đoạn con liên tiếp có tổng đúng bằng 5 là: đoạn $[2, 3]$ ($2 + 3 = 5$) và đoạn $[3, 2]$ ($3 + 2 = 5$). Tổng cộng có đúng 2 đoạn con thỏa mãn.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long s;
    if (!(cin >> n >> s)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    int l = 0;
    long long cur_sum = 0;
    long long count = 0;

    for (int r = 0; r < n; ++r) {
        cur_sum += a[r];
        while (cur_sum > s) {
            cur_sum -= a[l];
            ++l;
        }
        if (cur_sum == s) {
            ++count;
        }
    }

    cout << count << "\n";
    return 0;
}
```

Bài 10 [cppb_cst_10_doan_con_k_ky_tu_khac_nhau]: Đoạn Con Dài Nhất Chứa Tối Đa K Ký Tự Khác Nhau

Bối cảnh: Trong kỹ thuật nén văn bản không mất dữ liệu, một bộ mã hóa cần phân tích một chuỗi ký tự văn bản S. Để tối ưu từ điển ký tự trong bộ đệm nhanh, bộ mã hóa muốn trích xuất một đoạn văn bản liên tiếp dài nhất mà trong đó chỉ sử dụng không quá K loại ký tự khác nhau.



Hình minh họa: Mô hình khung cửa sổ trượt duy trì trạng thái liên tục

Nhiệm vụ: Cho chuỗi ký tự S gồm các chữ cái tiếng Anh in thường và số nguyên dương K . Hãy tìm độ dài lớn nhất của một chuỗi con liên tiếp chứa tối đa K ký tự khác nhau.

Đầu vào (Input):

- Dòng 1: Chứa chuỗi ký tự S ($1 \leq |S| \leq 10^5$).
- Dòng 2: Chứa số nguyên dương K ($1 \leq K \leq 26$).

Đầu ra (Output):

- In ra độ dài lớn nhất của chuỗi con tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
eceba 2	3

Giải thích: Chuỗi con liên tiếp dài nhất chứa tối đa 2 ký tự khác nhau là ‘ece’ (chỉ chứa 2 ký tự ‘e’ và ‘c’) với độ dài bằng 3. Kết quả in ra: 3.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho chuỗi ký tự S gồm các chữ cái tiếng Anh in thường và số nguyên dương K . Hãy tìm độ dài lớn nhất của một chuỗi con liên tiếp chứa tối đa K ký tự khác nhau.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: eceba 2 | Khởi tạo cấu trúc dữ liệu ban đầu | |
2 | Thực thi thuật toán tối ưu | Chuỗi con liên tiếp dài nhất chứa tối đa 2 ký tự khác nhau là ‘ece’ (chỉ chứa 2 ký tự ‘e’ và ‘c’) với độ dài bằng 3. Kết... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 3 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Chuỗi con liên tiếp dài nhất chứa tối đa 2 ký tự khác nhau là ‘ece’ (chỉ chứa 2 ký tự ‘e’ và ‘c’) với độ dài bằng 3. Kết quả in ra: 3.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    if (!(cin >> n >> k)) return 0;

    string s;
    cin >> s;

    vector<int> freq(26, 0);
    int distinct = 0;
    int l = 0, max_len = 0;

    for (int r = 0; r < n; ++r) {
        int c = s[r] - 'a';
        if (freq[c] == 0) ++distinct;
        ++freq[c];

        while (distinct > k) {
            int lc = s[l] - 'a';
            --freq[lc];
            if (freq[lc] == 0) --distinct;
            ++l;
        }

        max_len = max(max_len, r - l + 1);
    }

    cout << max_len << "\n";
    return 0;
}
```

Bài 11 [cppb_cst_11_doan_con_ngan_nhat_chua_du_ky_tu]: Đoạn Con Ngắn Nhất Chứa Đủ Mọi Ký Tự Của Tập Hợp

Bối cảnh: Một chuyên gia giải mật mã cần tìm dấu vết của một tổ hợp mật mã T gồm M ký tự độc nhất bên trong một chuỗi tín hiệu thô S . Đoạn tín hiệu được trích xuất để giải mã cần phải chứa đầy đủ mọi ký tự xuất hiện trong T (mỗi ký tự có mặt ít nhất một lần). Để việc giải mã diễn ra nhanh nhất, đoạn tín hiệu trích xuất phải có độ dài ngắn nhất có thể.



Hình minh họa: Mô hình khung cửa sổ trượt duy trì trạng thái liên tục

Nhiệm vụ: Cho chuỗi S gồm các chữ cái in thường và chuỗi mẫu T gồm M ký tự phân biệt. Hãy tìm độ dài nhỏ nhất của một đoạn con liên tiếp trong S chứa đầy đủ toàn bộ các ký tự của chuỗi T . Nếu không tồn tại, in ra -1 .

Đầu vào (Input):

- Dòng 1: Chuỗi S ($1 \leq |S| \leq 10^5$).
- Dòng 2: Chuỗi T ($1 \leq |T| \leq 26$, các ký tự trong T đôi một khác nhau).

Đầu ra (Output):

- In ra độ dài nhỏ nhất tìm được, hoặc -1 nếu không có đoạn con nào chứa đủ các ký tự của T .

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
adobecodebanc abc	4

Giải thích: Chuỗi $T = \text{'abc'}$ yêu cầu phải có đủ 3 ký tự ‘a’, ‘b’, ‘c’. Đoạn con ngắn nhất trong S chứa đủ cả 3 ký tự này là ‘banc’ ở cuối chuỗi S , có độ dài bằng 4. Kết quả in ra: 4.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho chuỗi S gồm các chữ cái in thường và chuỗi mẫu T gồm M ký tự phân biệt. Hãy tìm độ dài nhỏ nhất của một đoạn con liên tiếp trong S chứa đầy đủ toàn bộ các ký tự của chuỗi T . Nếu không tồn tại, in ra -1 .

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| — | — | — | — | — | 1 | Nạp dữ liệu vào mảng/biến | Input: adobecodebanc abc | Khởi tạo cấu trúc dữ liệu ban đầu | 2 | Thực thi thuật toán tối ưu | Chuỗi $T = \text{'abc'}$ yêu cầu phải có đủ 3 ký tự ‘a’, ‘b’, ‘c’. Đoạn con ngắn nhất trong S chứa đủ cả 3 ký tự này là ‘banc’ ở ... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 4 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Chuỗi $T = \text{'abc'}$ yêu cầu phải có đủ 3 ký tự ‘a’, ‘b’, ‘c’. Đoạn con ngắn nhất trong S chứa đủ cả 3 ký tự này là ‘banc’ ở cuối chuỗi S , có độ dài bằng 4. Kết quả in ra: 4.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;

    string s, t;
    cin >> s >> t;

    vector<int> need(26, 0);
    for (char c : t) need[c - 'a'] = 1;

    vector<int> have(26, 0);
    int matched = 0;
    int l = 0, min_len = n + 1;

    for (int r = 0; r < n; ++r) {
        int c = s[r] - 'a';
        if (need[c]) {
            if (have[c] == 0) ++matched;
            ++have[c];
        }

        while (matched == m) {
            min_len = min(min_len, r - l + 1);
            int lc = s[l] - 'a';
            if (need[lc]) {
                --have[lc];
                if (have[lc] == 0) --matched;
            }
            ++l;
        }
    }

    if (min_len > n) cout << -1 << "\n";
    else cout << min_len << "\n";
    return 0;
}
```

Bài 12 [cppb_cst_12_phu_song_wifi]: Phủ Sóng Trạm Phát Sóng Wifi Đô Thị

Bối cảnh: Dọc theo một đại lộ thẳng tắp, có N căn nhà đặt tại các tọa độ $X_1, X_2, \dots, X_n$. Một nhà mạng viễn thông muốn phủ sóng dịch vụ wifi cộng đồng bằng các bộ phát có bán kính phủ sóng R

(nghĩa là một trạm phủ được đoạn $[x - R, x + R]$, tức tầm phủ dài tối đa $2R$). Hãy tính số lượng căn nhà tối đa có thể cùng được phủ sóng bởi một trạm phát wifi duy nhất có đường kính phủ sóng $2R$.



Hình minh họa: Mô hình khung của sổ trượt duy trì trạng thái liên tục

Nhiệm vụ: Cho danh sách tọa độ của N căn nhà đã sắp xếp tăng dần và số nguyên R . Hãy tìm số lượng căn nhà nhiều nhất nằm gọn trong một đoạn có độ dài không vượt quá $2R$.

Đầu vào (Input):

- Dòng 1: 2 số nguyên N và R ($1 \leq N \leq 10^5, 0 \leq R \leq 10^9$).
- Dòng 2: N số nguyên đã sắp xếp tăng dần $X_1 < X_2 < \dots < X_N$ ($0 \leq X_i \leq 10^9$).

Đầu ra (Output):

- In ra số lượng căn nhà tối đa được phủ sóng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 3 5 8 10	3

Giải thích: Với bán kính $R = 3$, đường kính phủ sóng tối đa là $2R = 6$. Xét đoạn từ nhà tọa độ 1 đến nhà tọa độ 5: độ dài khoảng cách là $5 - 1 = 4 \leq 6$, phủ sóng được 3 căn nhà tại các tọa độ $\{1, 3, 5\}$. Tương tự, đoạn $\{3, 5, 8\}$ có $8 - 3 = 5 \leq 6$ cũng phủ được 3 nhà. Số lượng nhà tối đa phủ được là 3.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho danh sách tọa độ của N căn nhà đã sắp xếp tăng dần và số nguyên R . Hãy tìm số lượng căn nhà nhiều nhất nằm gọn trong một đoạn có độ dài không vượt quá $2R$.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| — | — | — | — | — | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 3 1 3 5 8 10 | Khởi tạo cấu trúc dữ liệu ban đầu | 2 | Thực thi thuật toán tối ưu | Với bán kính $R = 3$, đường kính phủ sóng tối đa là $2R = 6$. Xét đoạn từ nhà tọa độ 1 đến nhà tọa độ 5: độ dài khoảng cách ... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 3 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Với bán kính $R = 3$, đường kính phủ sóng tối đa là $2R = 6$. Xét đoạn từ nhà tọa độ 1 đến nhà tọa độ 5: độ dài khoảng cách là $5 - 1 = 4 \leq 6$, phủ sóng được 3 căn nhà tại các tọa độ $\{1, 3, 5\}$. Tương tự, đoạn $\{3, 5, 8\}$ có $8 - 3 = 5 \leq 6$ cũng phủ được 3 nhà. Số lượng nhà tối đa phủ được là 3.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long r;
    if (!(cin >> n >> r)) return 0;

    vector<long long> x(n);
    for (int i = 0; i < n; ++i) cin >> x[i];

    int i = 0;
    int count = 0;

    while (i < n) {
        ++count;
        long long loc = x[i];
        while (i < n && x[i] - loc <= r) ++i;
        long long tower = x[i - 1];
        while (i < n && x[i] - tower <= r) ++i;
    }

    cout << count << "\n";
    return 0;
}
```

Bài 13 [cppb_cst_13_doan_con_chenh_lech_k]: Đoạn Con Có Độ Chênh Lệch Max - Min Không Quá K

Bối cảnh: Trong một dây chuyền sản xuất vi thấu kính chính xác cao, các thấu kính đi qua cảm biến đo độ dày lần lượt ghi nhận các số đo $A_1, A_2, \dots, A_n$. Một kiện hàng đóng gói hợp chuẩn đòi hỏi các thấu kính liên tiếp trong cùng một lô phải có độ chênh lệch giữa thấu kính dày nhất và mỏng nhất không vượt quá dung sai K . Nhà máy cần tìm lô sản phẩm liên tiếp dài nhất thỏa mãn yêu cầu dung sai này.



Hình minh họa: Mô hình khung cửa sổ trượt duy trì trạng thái liên tục

Nhiệm vụ: Cho mảng gồm N số nguyên và số nguyên không âm K . Hãy tìm độ dài lớn nhất của đoạn con liên tiếp sao cho chênh lệch giữa phần tử lớn nhất và phần tử nhỏ nhất trong đoạn không vượt quá K : $\max(\text{đoạn}) - \min(\text{đoạn}) \leq K$.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và K ($1 \leq N \leq 10^5, 0 \leq K \leq 10^9$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra độ dài lớn nhất của đoạn con tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 2 1 3 6 7 9 4	2

Giải thích: Với dung sai $K = 2$, các đoạn con liên tiếp có chênh lệch $\max - \min \leq 2$ là $[1, 3]$ ($3 - 1 = 2$) hoặc $[6, 7]$ ($7 - 6 = 1$). Độ dài lớn nhất của đoạn con hợp lệ là 2.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng gồm N số nguyên và số nguyên không âm K . Hãy tìm độ dài lớn nhất của đoạn con liên tiếp sao cho chênh lệch giữa phần tử lớn nhất và phần tử nhỏ nhất trong đoạn không vượt quá K : $\max(\text{đoạn}) - \min(\text{đoạn}) \leq K$.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| — | — | — | — | — | | 1 | Nạp dữ liệu vào mảng/biến | Input: 6 2 1 3 6 7 9 4 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Với dung sai $K = 2$, các đoạn con liên tiếp có chênh lệch $\max - \min \leq 2$ là $[1, 3]$ ($3 - 1 = 2$) hoặc $[6, 7]$ ($7 - 6 = 1$). Đ... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 2 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Với dung sai $K = 2$, các đoạn con liên tiếp có chênh lệch $\max - \min \leq 2$ là $[1, 3]$ ($3 - 1 = 2$) hoặc $[6, 7]$ ($7 - 6 = 1$). Độ dài lớn nhất của đoạn con hợp lệ là 2.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long k;
    if (!(cin >> n >> k)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    int max_len = 0;

    for (int l = 0; l < n; ++l) {
        long long cur_min = a[l], cur_max = a[l];
        for (int r = l; r < n; ++r) {
            cur_min = min(cur_min, a[r]);
            cur_max = max(cur_max, a[r]);
            if (cur_max - cur_min <= k) {
                max_len = max(max_len, r - l + 1);
            } else {
                break;
            }
        }
    }

    cout << max_len << "\n";
    return 0;
}
```

Bài 14 [cppb_cst_14_cua_so_truot_cuc_han]: Tối Ưu Cửa Sổ Trượt Tuyến Tính Khi $N = 2 \cdot 10^5$

Bối cảnh: Một máy chủ phân tích dữ liệu lớn thu thập N giá trị dung lượng truy cập mỗi giây từ công Internet quốc tế. Để phát hiện các mẫu tấn công từ chối dịch vụ (DDoS) tiềm ẩn, thuật toán an ninh mạng cần đếm số lượng đoạn thời gian liên tiếp nhau mà tổng lưu lượng đúng bằng ngưỡng cảnh báo S , với dữ liệu cực lớn lên tới $N = 200.000$ phần tử đòi hỏi xử lý thời gian thực nghiêm ngặt $O(N)$.



Hình minh họa: Mô hình khung cửa sổ trượt duy trì trạng thái liên tục

Nhiệm vụ: Cho mảng gồm N số nguyên dương và số nguyên S . Hãy tìm số lượng đoạn con liên tiếp có tổng đúng bằng S trong thời gian $O(N)$.

Đầu vào (Input):

- Dòng 1: 2 số nguyên N và S ($1 \leq N \leq 2 \cdot 10^5, 1 \leq S \leq 10^{14}$).
- Dòng 2: N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra số lượng đoạn con thỏa mãn.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 7 2 3 2 5 2	2

Giải thích: Các đoạn con liên tiếp có tổng đúng bằng 7 là: $[2, 3, 2]$ ($2 + 3 + 2 = 7$) và $[5, 2]$ ($5 + 2 = 7$). Tổng cộng có đúng 2 đoạn con thỏa mãn.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng gồm N số nguyên dương và số nguyên S . Hãy tìm số lượng đoạn con liên tiếp có tổng đúng bằng S trong thời gian $O(N)$.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 7 2 3 2 5 2 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Các đoạn con liên tiếp có tổng đúng bằng 7 là: $[2, 3, 2]$ ($2 + 3 + 2 = 7$) và $[5, 2]$ ($5 + 2 = 7$). Tổng cộng có đúng 2 đoạn... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 2 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Các đoạn con liên tiếp có tổng đúng bằng 7 là: $[2, 3, 2]$ ($2 + 3 + 2 = 7$) và $[5, 2]$ ($5 + 2 = 7$). Tổng cộng có đúng 2 đoạn con thỏa mãn.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

long long count_at_most(const vector<long long> &x, long long limit) {
    if (limit <= 0) return 0;
    int n = x.size();
    int l = 0;
    long long cur_sum = 0;
    long long count = 0;

    for (int r = 0; r < n; ++r) {
        cur_sum += x[r];
        while (cur_sum > limit) {
            cur_sum -= x[l];
            ++l;
        }
        count += (r - l + 1);
    }
    return count;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long a, b;
    if (!(cin >> n >> a >> b)) return 0;

    vector<long long> x(n);
    for (int i = 0; i < n; ++i) cin >> x[i];

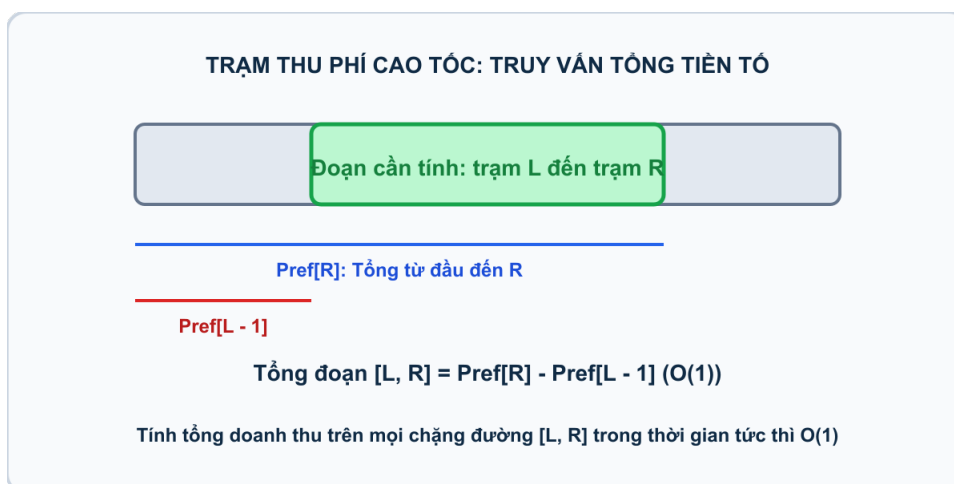
    long long ans = count_at_most(x, b) - count_at_most(x, a - 1);
    cout << ans << "\n";
    return 0;
}
```

CHƯƠNG 03: MẢNG TIỀN TỔ & TÌM KIẾM NHỊ PHÂN

Bài 07: Mảng tiền tổ & mảng hiệu

Bài 01 [cppb_pt_01_truy_van_tong_doan_con]: Truy Vấn Tổng Đoạn Con 1D

Bối cảnh: Tại một trạm thu phí cao tốc thông minh, doanh thu thu được theo từng giờ trong ngày được ghi nhận thành dãy N số nguyên $A_1, A_2, \dots, A_N$. Ban kiểm toán tài chính thường xuyên gửi Q câu hỏi truy vấn độc lập, mỗi câu hỏi yêu cầu báo cáo tổng doanh thu thu được trong khoảng thời gian từ giờ L đến giờ R .



Hình minh họa: Cấu trúc mảng cộng dồn tiền tổ truy vấn đoạn con tức thì $O(1)$

Nhiệm vụ: Cho dãy số nguyên gồm N phần tử. Với mỗi truy vấn $[L, R]$ ($1 \leq L \leq R \leq N$), hãy tính và in ra tổng các phần tử từ chỉ số L đến chỉ số R bằng kỹ thuật Mảng tiền tổ.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên dương N và Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).
- Q dòng tiếp theo: Mỗi dòng chứa 2 số nguyên L và R ($1 \leq L \leq R \leq N$).

Đầu ra (Output):

- In ra Q dòng, mỗi dòng chứa một số nguyên là kết quả của truy vấn tương ứng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 2 3 4 5 1 3 2 4 1 5	6 9 15

Giải thích: Mảng tiền tổ $\text{Pref} = [0, 1, 3, 6, 10, 15]$.

- Truy vấn $[1, 3]$: $\text{Pref}[3] - \text{Pref}[0] = 6 - 0 = 6$.
- Truy vấn $[2, 4]$: $\text{Pref}[4] - \text{Pref}[1] = 10 - 1 = 9$.
- Truy vấn $[1, 5]$: $\text{Pref}[5] - \text{Pref}[0] = 15 - 0 = 15$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho dãy số nguyên gồm N phần tử. Với mỗi truy vấn $[L, R]$ ($1 \leq L \leq R \leq N$), hãy tính và in ra tổng các phần tử từ chỉ số L đến chỉ số R bằng kỹ thuật Mảng tiền tố.
- **Phương pháp tiếp cận — Mảng tiền tố & Mảng hiệu:**
- Dựng mảng cộng dồn $\text{pref}[i] = \text{pref}[i-1] + a[i]$. Khi đó tổng đoạn $[L, R]$ được tính tức thì bằng $\text{pref}[R] - \text{pref}[L-1]$ trong $\mathcal{O}(1)$.
- Với các thao tác cộng dồn đoạn, sử dụng mảng hiệu $\text{diff}[L] += V$, $\text{diff}[R+1] -= V$ rồi cộng dồn để phục hồi mảng.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 3 1 2 3 4 5 1 3 2 4 1 5 | Khởi tạo cấu trúc dữ liệu ban đầu | 2 | Thực thi thuật toán tối ưu | Mảng tiền tố $\text{Pref} = [0, 1, 3, 6, 10, 15]$. - Truy vấn $[1, 3]$: $\text{Pref}[3] - \text{Pref}[0] = 6 - 0 = 6$. - Truy vấn $[2, 4]$: $\text{Pref}[4] - \dots$ | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 6 9 15 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Mảng tiền tố $\text{Pref} = [0, 1, 3, 6, 10, 15]$.

- Truy vấn $[1, 3]$: $\text{Pref}[3] - \text{Pref}[0] = 6 - 0 = 6$.
- Truy vấn $[2, 4]$: $\text{Pref}[4] - \text{Pref}[1] = 10 - 1 = 9$.
- Truy vấn $[1, 5]$: $\text{Pref}[5] - \text{Pref}[0] = 15 - 0 = 15$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;

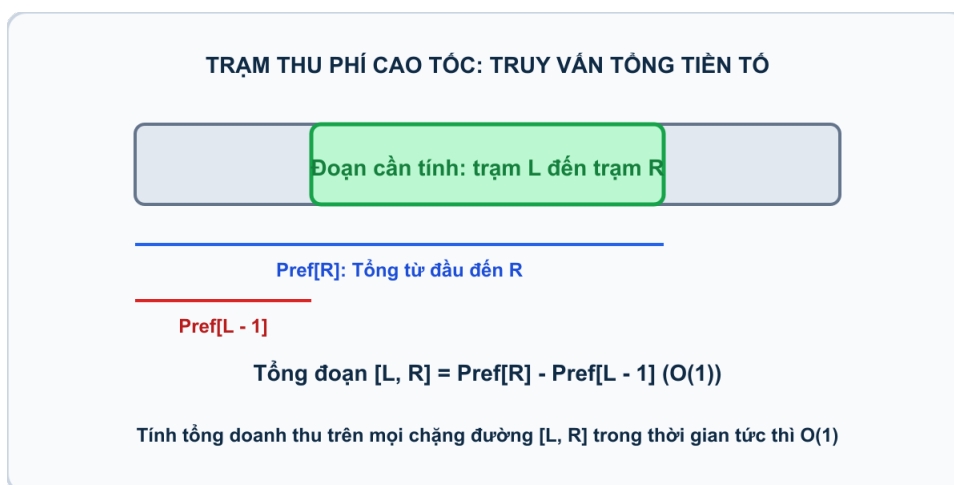
    vector<long long> p(n + 1, 0);
    for (int i = 1; i <= n; ++i) {
        long long x;
        cin >> x;
        p[i] = p[i - 1] + x;
    }
}
```

```
while (q--) {
    int l, r;
    cin >> l >> r;
    cout << p[r] - p[l - 1] << "\n";
}

return 0;
}
```

Bài 02 [cppb_pt_02_dem_so_chan_trong_doan]: Đếm Số Lượng Số Chẵn Trong Đoạn

Bối cảnh: Một máy quét an ninh tại sân bay kiểm tra mã barcode của N kiện hành lý liên tiếp. Để phân luồng vào băng chuyền đặc biệt, hệ thống cần đếm nhanh số lượng kiện hành lý mang mã số chẵn trong từng phân đoạn bưu kiện liên tiếp $[L, R]$ qua Q đợt kiểm tra đột xuất.



Hình minh họa: Cấu trúc mảng cộng dồn tiền tố truy vấn đoạn con tức thì $O(1)$

Nhiệm vụ: Cho dãy gồm N số nguyên. Hãy trả lời Q truy vấn $[L, R]$, mỗi truy vấn yêu cầu đếm xem có bao nhiêu số chẵn trong đoạn từ vị trí L đến R.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên dương N và Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).
- Q dòng tiếp theo: Mỗi dòng chứa 2 số nguyên L và R ($1 \leq L \leq R \leq N$).

Đầu ra (Output):

- In ra Q dòng, mỗi dòng là số lượng số chẵn trong đoạn tương ứng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 3 2 3 4 6 7 8 1 4 2 5 1 6	3 2 4

Giải thích: Mảng nhị phân đánh dấu số chẵn: $[1, 0, 1, 1, 0, 1]$. Mảng tiền tố đếm số chẵn: $[0, 1, 1, 2, 3, 3, 4]$.

- Đoạn $[1, 4]$: gồm $\{2, 3, 4, 6\}$ có 3 số chẵn.

- Đoạn $[2, 5]$: gồm $\{3, 4, 6, 7\}$ có 2 số chẵn.
- Đoạn $[1, 6]$: có 4 số chẵn.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho dãy gồm N số nguyên. Hãy trả lời Q truy vấn $[L, R]$, mỗi truy vấn yêu cầu đếm xem có bao nhiêu số chẵn trong đoạn từ vị trí L đến R .
- **Phương pháp tiếp cận — Mảng tiền tố & Mảng hiệu:**
- Dựng mảng cộng dồn $\text{pref}[i] = \text{pref}[i-1] + a[i]$. Khi đó tổng đoạn $[L, R]$ được tính tức thì bằng $\text{pref}[R] - \text{pref}[L-1]$ trong $\mathcal{O}(1)$.
- Với các thao tác cộng dồn đoạn, sử dụng mảng hiệu $\text{diff}[L] += V$, $\text{diff}[R+1] -= V$ rồi cộng dồn để phục hồi mảng.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 6 3 2 3 4 6 7 8 1 4 2 5 1 6 | Khởi tạo cấu trúc dữ liệu ban đầu | 2 | Thực thi thuật toán tối ưu | Mảng nhị phân đánh dấu số chẵn: $[1, 0, 1, 1, 0, 1]$. Mảng tiền tố đếm số chẵn: $[0, 1, 1, 2, 3, 3, 4]$. - Đoạn $[1, 4]$: gồm ... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 3 2 4 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Mảng nhị phân đánh dấu số chẵn: $[1, 0, 1, 1, 0, 1]$. Mảng tiền tố đếm số chẵn: $[0, 1, 1, 2, 3, 3, 4]$.

- Đoạn $[1, 4]$: gồm $\{2, 3, 4, 6\}$ có 3 số chẵn.
- Đoạn $[2, 5]$: gồm $\{3, 4, 6, 7\}$ có 2 số chẵn.
- Đoạn $[1, 6]$: có 4 số chẵn.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;

    vector<int> p(n + 1, 0);
    for (int i = 1; i <= n; ++i) {
```

```

    long long x;
    cin >> x;
    p[i] = p[i - 1] + (abs(x) % 2 == 0 ? 1 : 0);
}

while (q--) {
    int l, r;
    cin >> l >> r;
    cout << p[r] - p[l - 1] << "\n";
}

return 0;
}

```

Bài 03 [cppb_pt_03_vi_tri_can_bang]: Tìm Vị Trí Cân Bằng Của Mảng

Bối cảnh: Trong trò chơi bập bênh chịu lực, N quả cân được xếp thẳng hàng tại các vị trí từ 1 đến N với khối lượng lần lượt là $A_1, A_2, \dots, A_N$. Một vị trí đặt điểm tựa i được coi là điểm cân bằng hoàn hảo nếu tổng khối lượng các quả cân bên trái bằng đúng tổng khối lượng các quả cân bên phải điểm tựa đó. Hãy tìm vị trí cân bằng đầu tiên.



Hình minh họa: Cấu trúc mảng cộng dồn tiền tố truy vấn đoạn con tức thì $O(1)$

Nhiệm vụ: Cho mảng N số nguyên. Hãy tìm chỉ số i nhỏ nhất (1-indexed) sao cho tổng các phần tử bên trái i bằng tổng các phần tử bên phải i . Nếu không tồn tại, in ra -1.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra chỉ số cân bằng nhỏ nhất (1-indexed), hoặc -1.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
7 -7 1 5 2 -4 3 0	4

Giải thích: Tại vị trí $i = 4$ (giá trị 2):

- Tổng bên trái (vị trí 1 đến 3): $(-7) + 1 + 5 = -1$.

- Tổng bên phải (vị trí 5 đến 7): $(-4) + 3 + 0 = -1$. Hai tổng bằng nhau ($-1 = -1$) nên vị trí cân bằng là 4.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng N số nguyên. Hãy tìm chỉ số i nhỏ nhất (1-indexed) sao cho tổng các phần tử bên trái i bằng tổng các phần tử bên phải i . Nếu không tồn tại, in ra -1 .
- **Phương pháp tiếp cận — Mảng tiền tố & Mảng hiệu:**
- Dựng mảng cộng dồn $pref[i] = pref[i-1] + a[i]$. Khi đó tổng đoạn $[L, R]$ được tính tức thì bằng $pref[R] - pref[L-1]$ trong $\mathcal{O}(1)$.
- Với các thao tác cộng dồn đoạn, sử dụng mảng hiệu $diff[L] += V$, $diff[R+1] -= V$ rồi cộng dồn để phục hồi mảng.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| — | — | — | — | — | 1 | Nạp dữ liệu vào mảng/biến | Input: 7 -7 1 5 2 -4 3 0 | Khởi tạo cấu trúc dữ liệu ban đầu | 2 | Thực thi thuật toán tối ưu | Tại vị trí $i = 4$ (giá trị 2): - Tổng bên trái (vị trí 1 đến 3): $(-7) + 1 + 5 = -1$. - Tổng bên phải (vị trí 5 đến 7): $(-4) + 3 + 0 = -1$. | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 4 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Tại vị trí $i = 4$ (giá trị 2):

- Tổng bên trái (vị trí 1 đến 3): $(-7) + 1 + 5 = -1$.
- Tổng bên phải (vị trí 5 đến 7): $(-4) + 3 + 0 = -1$. Hai tổng bằng nhau ($-1 = -1$) nên vị trí cân bằng là 4.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<long long> a(n + 1);
    vector<long long> p(n + 1, 0);
    for (int i = 1; i <= n; ++i) {
        cin >> a[i];
```

```

    p[i] = p[i - 1] + a[i];
}

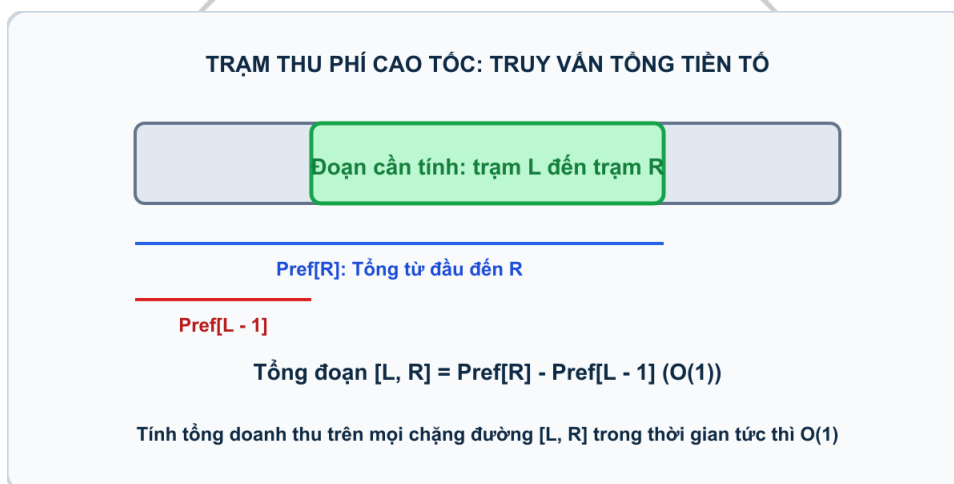
for (int i = 1; i <= n; ++i) {
    long long left_sum = p[i - 1];
    long long right_sum = p[n] - p[i];
    if (left_sum == right_sum) {
        cout << i << "\n";
        return 0;
    }
}

cout << -1 << "\n";
return 0;
}

```

Bài 04 [cppb_pt_04_doan_con_tong_bang_khong]: Đoạn Con Có Tổng Bằng 0

Bối cảnh: Một tài khoản ngân hàng ghi nhận lịch sử N giao dịch biến động số dư liên tiếp (số dương là tiền vào, số âm là tiền ra). Chuyên viên kiểm toán nội bộ cần kiểm tra xem trong lịch sử giao dịch có tồn tại bất kỳ chuỗi giao dịch liên tiếp nào mà tổng dòng tiền ròng đúng bằng 0 hay không.



Hình minh họa: Cấu trúc mảng cộng dồn tiền tố truy vấn đoạn con tức thì $O(1)$

Nhiệm vụ: Cho mảng gồm N số nguyên. Hãy kiểm tra xem có tồn tại ít nhất một đoạn con liên tiếp có tổng đúng bằng 0 hay không. Nếu có in ra YES, ngược lại in ra NO.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra YES nếu có đoạn con tổng bằng 0, ngược lại in ra NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 4 2 -3 1 6	YES

Giải thích: Đoạn con $[2, -3, 1]$ từ vị trí 2 đến vị trí 4 có tổng là $2 + (-3) + 1 = 0$. Do đó in ra YES.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng gồm N số nguyên. Hãy kiểm tra xem có tồn tại ít nhất một đoạn con liên tiếp có tổng đúng bằng 0 hay không. Nếu có in ra YES, ngược lại in ra NO.
- **Phương pháp tiếp cận — Mảng tiền tố & Mảng hiệu:**
- Dựng mảng cộng dồn $\text{pref}[i] = \text{pref}[i-1] + a[i]$. Khi đó tổng đoạn $[L, R]$ được tính tức thì bằng $\text{pref}[R] - \text{pref}[L-1]$ trong $\mathcal{O}(1)$.
- Với các thao tác cộng dồn đoạn, sử dụng mảng hiệu $\text{diff}[L] += V$, $\text{diff}[R+1] -= V$ rồi cộng dồn để phục hồi mảng.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 4 2 -3 1 6 | Khởi tạo cấu trúc dữ liệu ban đầu
 | 2 | Thực thi thuật toán tối ưu | Đoạn con $[2, -3, 1]$ từ vị trí 2 đến vị trí 4 có tổng là $2 + (-3) + 1 = 0$. Do đó in ra YES.... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: YES | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Đoạn con $[2, -3, 1]$ từ vị trí 2 đến vị trí 4 có tổng là $2 + (-3) + 1 = 0$. Do đó in ra YES.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<long long> p(n + 1, 0);
    for (int i = 1; i <= n; ++i) {
        long long x;
        cin >> x;
        p[i] = p[i - 1] + x;
    }

    sort(p.begin(), p.end());

    for (int i = 1; i <= n; ++i) {
        if (p[i] == p[i - 1]) {
```

```

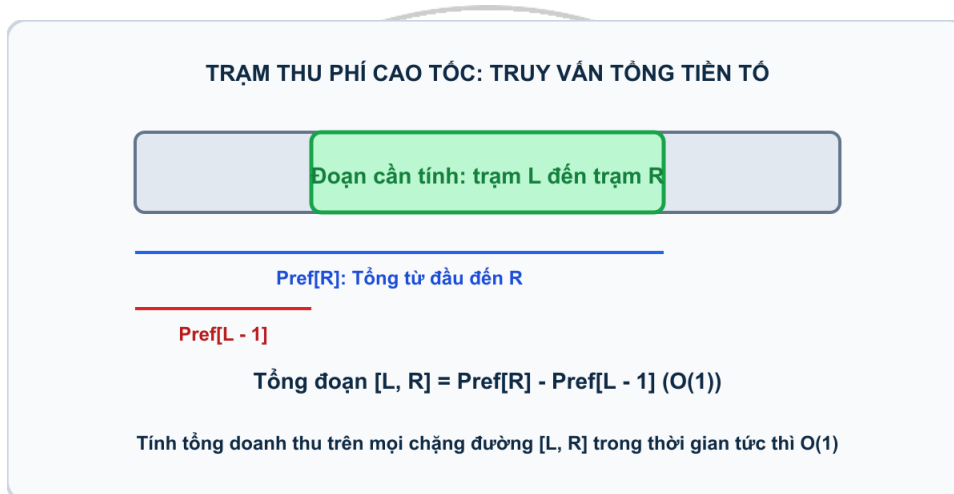
        cout << "YES\n";
        return 0;
    }

    cout << "NO\n";
    return 0;
}

```

Bài 05 [cppb_pt_05_cap_nhat_cong_doan]: Cập Nhật Cộng Đoạn Tuyến Tính (Mảng Hiệu)

Bối cảnh: Một tuyến đề biển dài được chia thành N phân đoạn ban đầu có độ cao gia cố bằng 0. Trải qua Q đợt bồi đắp phù sa, mỗi đợt người ta gia cố thêm một lượng đất đá X trên đoạn từ cọc L đến cọc R . Kỹ sư thủy lợi cần biết độ cao cuối cùng của toàn bộ N phân đoạn đề sau khi hoàn thành tất cả Q đợt gia cố.



Hình minh họa: Cấu trúc mảng cộng dồn tiền tố truy vấn đoạn con tức thì $O(1)$

Nhiệm vụ: Cho mảng N số nguyên ban đầu toàn số 0. Thực hiện Q thao tác cộng giá trị X vào đoạn $[L, R]$. Hãy in ra mảng kết quả cuối cùng sau Q thao tác bằng kỹ thuật Mảng hiệu.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và Q ($1 \leq N, Q \leq 10^5$).
- Q dòng tiếp theo: Mỗi dòng chứa 3 số nguyên L, R, X ($1 \leq L \leq R \leq N, -10^9 \leq X \leq 10^9$).

Đầu ra (Output):

- In ra N số nguyên trên một dòng biểu diễn mảng sau khi hoàn thành toàn bộ Q thao tác.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 3 2 2 5 3 3 4 -1	2 5 4 2 3

Giải thích: Sử dụng mảng hiệu D kích thước $N + 2$:

- Cộng 2 vào $[1, 3]$: $D[1] += 2, D[4] -= 2$.
- Cộng 3 vào $[2, 5]$: $D[2] += 3, D[6] -= 3$.

- Cộng -1 vào [3, 4]: $D[3] -= 1, D[5] += 1$. Tính tổng tiền tố của D để thu được mảng kết quả: 2 5 4 2 3.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng N số nguyên ban đầu toàn số 0. Thực hiện Q thao tác cộng giá trị X vào đoạn [L, R]. Hãy in ra mảng kết quả cuối cùng sau Q thao tác bằng kỹ thuật Mảng hiệu.
- **Phương pháp tiếp cận — Mảng tiền tố & Mảng hiệu:**
- Dựng mảng cộng dồn $pref[i] = pref[i-1] + a[i]$. Khi đó tổng đoạn $[L, R]$ được tính tức thì bằng $pref[R] - pref[L-1]$ trong $\mathcal{O}(1)$.
- Với các thao tác cộng dồn đoạn, sử dụng mảng hiệu $diff[L] += V, diff[R+1] -= V$ rồi cộng dồn để phục hồi mảng.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 3 1 3 2 2 5 3 3 4 -1 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Sử dụng mảng hiệu D kích thước $N + 2$: - Cộng 2 vào [1, 3]: $D[1] += 2, D[4] -= 2$. - Cộng 3 vào [2, 5]: $D[2] += 3, D[6] -= 3$. | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 2 5 4 2 3 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Sử dụng mảng hiệu D kích thước $N + 2$:

- Cộng 2 vào [1, 3]: $D[1] += 2, D[4] -= 2$.
- Cộng 3 vào [2, 5]: $D[2] += 3, D[6] -= 3$.
- Cộng -1 vào [3, 4]: $D[3] -= 1, D[5] += 1$. Tính tổng tiền tố của D để thu được mảng kết quả: 2 5 4 2 3.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
```

```
vector<long long> d(n + 2, 0);

while (q--) {
    int l, r;
    long long v;
    cin >> l >> r >> v;
    d[l] += v;
    d[r + 1] -= v;
}

long long current = 0;
for (int i = 1; i <= n; ++i) {
    current += d[i];
    cout << current << (i == n ? "" : " ");
}
cout << "\n";

return 0;
}
```

Bài 06 [cppb_pt_06_trong_cay_phu_doan]: Trồng Cây Phủ Đoạn Tối Ưu

Bối cảnh: Để phủ xanh đô thị, ban quản lý công viên có N hố trồng cây được đánh số từ 1 đến N dọc tuyến phố. Có Q tình nguyện viên tham gia, mỗi người nhận phụ trách tưới nước bổ sung cho các hố cây trong đoạn từ L đến R . Sau chiến dịch, ban tổ chức cần kiểm tra xem mỗi hố cây đã được bao nhiêu lượt tình nguyện viên tưới nước.

BẠP BÊNH CHỊU LỰC: TÌM VỊ TRÍ CÂN BẰNG MĂNG



Vị trí mà tổng trọng lượng bên trái đúng bằng tổng trọng lượng bên phải

Hình minh họa: Cấu trúc mảng cộng dồn tiền tố truy vấn đoạn con tức thì $O(1)$

Nhiệm vụ: Cho N vị trí và Q đoạn $[L, R]$. Hãy đếm số lượt phủ của mỗi vị trí từ 1 đến N sau Q lần thao tác.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và Q ($1 \leq N, Q \leq 10^5$).
- Q dòng tiếp theo: Mỗi dòng chứa 2 số nguyên L và R ($1 \leq L \leq R \leq N$).

Đầu ra (Output):

- In ra N số nguyên cách nhau bởi khoảng trắng là số lượt phủ tại mỗi vị trí từ 1 đến N .

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 3 2 4 2 5	1 3 3 2 1

Giải thích: Mỗi đoạn $[L, R]$ tương ứng với thao tác cộng 1 vào đoạn $[L, R]$. Mảng hiệu ghi nhận số lượt tưới tại từng vị trí lần lượt là: vị trí 1 được 1 lượt, vị trí 2 được 3 lượt, vị trí 3 được 3 lượt, vị trí 4 được 2 lượt, vị trí 5 được 1 lượt. Kết quả in ra: 1 3 3 2 1.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho N vị trí và Q đoạn $[L, R]$. Hãy đếm số lượt phủ của mỗi vị trí từ 1 đến N sau Q lần thao tác.
- **Phương pháp tiếp cận — Mảng tiền tố & Mảng hiệu:**
- Dựng mảng cộng dồn $pref[i] = pref[i-1] + a[i]$. Khi đó tổng đoạn $[L, R]$ được tính tức thì bằng $pref[R] - pref[L-1]$ trong $\mathcal{O}(1)$.
- Với các thao tác cộng dồn đoạn, sử dụng mảng hiệu $diff[L] += V$, $diff[R+1] -= V$ rồi cộng dồn để phục hồi mảng.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| — | — | — | — | — | | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 3 1 3 2 4 2 5 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Mỗi đoạn $[L, R]$ tương ứng với thao tác cộng 1 vào đoạn $[L, R]$. Mảng hiệu ghi nhận số lượt tưới tại từng vị trí lần lượt ... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 1 3 3 2 1 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Mỗi đoạn $[L, R]$ tương ứng với thao tác cộng 1 vào đoạn $[L, R]$. Mảng hiệu ghi nhận số lượt tưới tại từng vị trí lần lượt là: vị trí 1 được 1 lượt, vị trí 2 được 3 lượt, vị trí 3 được 3 lượt, vị trí 4 được 2 lượt, vị trí 5 được 1 lượt. Kết quả in ra: 1 3 3 2 1.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. $N-1$) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q, k;
    if (!(cin >> n >> q >> k)) return 0;
```

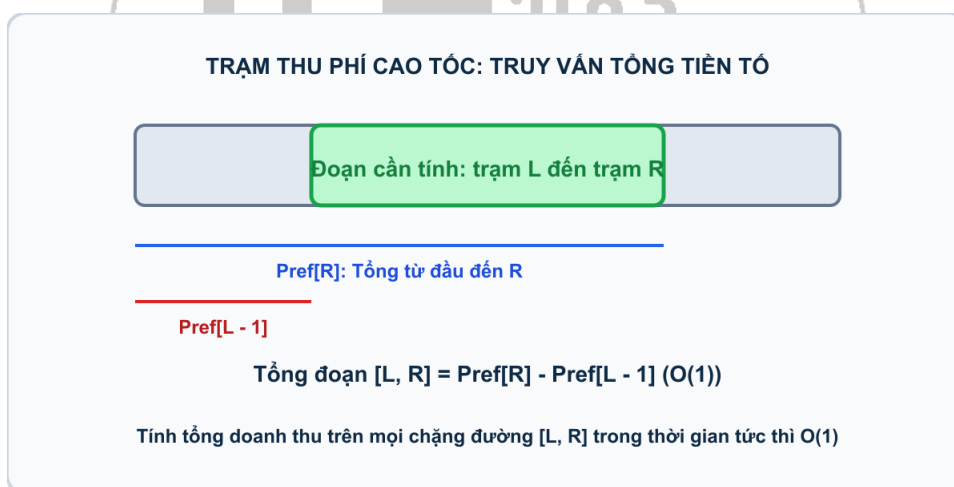
```
vector<int> d(n + 2, 0);
while (q--) {
    int l, r;
    cin >> l >> r;
    d[l]++;
    d[r + 1]--;
}

int count_ge_k = 0;
int current = 0;
for (int i = 1; i <= n; ++i) {
    current += d[i];
    if (current >= k) {
        count_ge_k++;
    }
}

cout << count_ge_k << "\n";
return 0;
}
```

Bài 07 [cppb_pt_07_truy_van_hinh_chu_nhat_2d]: Truy Vấn Tổng Hình Chữ Nhật 2D

Bối cảnh: Bản đồ nhiệt độ của một vùng biển được mô phỏng dưới dạng ma trận số nguyên kích thước N hàng và M cột. Các nhà hải dương học cần trả lời Q truy vấn độc lập từ trạm khí tượng, mỗi truy vấn yêu cầu tính tổng nhiệt độ trong một vùng hình chữ nhật giới hạn bởi góc trên-trái $(r1, c1)$ và góc dưới-phải $(r2, c2)$.



Hình minh họa: Cấu trúc mảng cộng dồn tiền tố truy vấn đoạn con tức thì $O(1)$

Nhiệm vụ: Cho ma trận số nguyên A kích thước $N \times M$. Hãy trả lời Q truy vấn tính tổng các phần tử trong hình chữ nhật từ $(r1, c1)$ đến $(r2, c2)$ bằng kỹ thuật Mảng tiền tố 2D.

Đầu vào (Input):

- Dòng 1: Chứa 3 số nguyên N, M, Q ($1 \leq N, M \leq 1000, 1 \leq Q \leq 10^5$).
- N dòng tiếp theo: Mỗi dòng gồm M số nguyên $A_{i,j}$ ($-10^9 \leq A_{i,j} \leq 10^9$).
- Q dòng tiếp theo: Mỗi dòng gồm 4 số nguyên r_1, c_1, r_2, c_2 ($1 \leq r_1 \leq r_2 \leq N, 1 \leq c_1 \leq c_2 \leq M$).

Đầu ra (Output):

- In ra Q dòng tương ứng với kết quả của mỗi truy vấn.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 3 2 1 2 3 4 5 6 7 8 9 1 1 2 2 2 2 3 3	12 28

Giải thích: - Vùng từ (1, 1) đến (2, 2) gồm các ô {1, 2, 4, 5} có tổng: $1 + 2 + 4 + 5 = 12$. - Vùng từ (2, 2) đến (3, 3) gồm các ô {5, 6, 8, 9} có tổng: $5 + 6 + 8 + 9 = 28$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho ma trận số nguyên A kích thước $N \times M$. Hãy trả lời Q truy vấn tính tổng các phần tử trong hình chữ nhật từ (r1, c1) đến (r2, c2) bằng kỹ thuật Mảng tiền tố 2D.
- **Phương pháp tiếp cận — Mảng tiền tố & Mảng hiệu:**
- Dựng mảng cộng dồn $pref[i] = pref[i-1] + a[i]$. Khi đó tổng đoạn $[L, R]$ được tính tức thì bằng $pref[R] - pref[L-1]$ trong $O(1)$.
- Với các thao tác cộng dồn đoạn, sử dụng mảng hiệu $diff[L] += V$, $diff[R+1] -= V$ rồi cộng dồn để phục hồi mảng.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 3 2 1 2 3 4 5 6 7 8 9 1 1 2 2 2 2 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | - Vùng từ (1, 1) đến (2, 2) gồm các ô {1, 2, 4, 5} có tổng: $1 + 2 + 4 + 5 = 12$. - Vùng từ (2, 2) đến (3, 3) gồm các ô {5, 6, 8, 9} có tổng: $5 + 6 + 8 + 9 = 28$. | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 12 28 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: - Vùng từ (1, 1) đến (2, 2) gồm các ô {1, 2, 4, 5} có tổng: $1 + 2 + 4 + 5 = 12$.

- Vùng từ (2, 2) đến (3, 3) gồm các ô {5, 6, 8, 9} có tổng: $5 + 6 + 8 + 9 = 28$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm main() để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, q;
```

```

if (!(cin >> n >> m >> q)) return 0;

vector<vector<long long>> p(n + 1, vector<long long>(m + 1, 0));
for (int i = 1; i <= n; ++i) {
    for (int j = 1; j <= m; ++j) {
        long long val;
        cin >> val;
        p[i][j] = p[i - 1][j] + p[i][j - 1] - p[i - 1][j - 1] + val;
    }
}

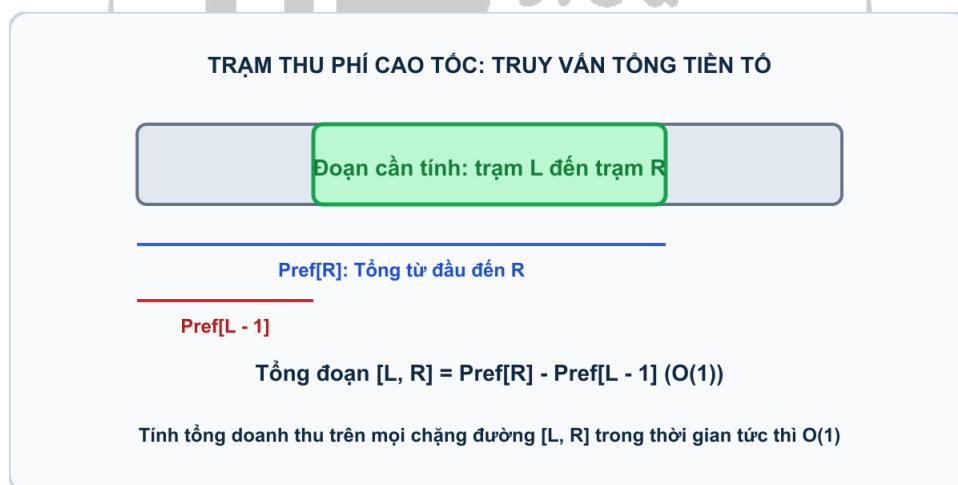
while (q--) {
    int x1, y1, x2, y2;
    cin >> x1 >> y1 >> x2 >> y2;
    long long ans = p[x2][y2] - p[x1 - 1][y2] - p[x2][y1 - 1] + p[x1 - 1][y1 - 1];
    cout << ans << "\n";
}

return 0;
}

```

Bài 08 [cppb_pt_08_hinh_vuong_k_tong_lon_nhat]: Tìm Hình Vuông K x K Có Tổng Lớn Nhất

Bối cảnh: Một vệ tinh nông nghiệp chụp ảnh khu đất canh tác dưới dạng ma trận $N \times M$, mỗi ô thể hiện sản lượng lúa dự kiến thu hoạch. Nhà đầu tư muốn thuê một mảnh đất hình vuông kích thước đúng $K \times K$ ô đất liền kề nhau sao cho tổng sản lượng thu hoạch trên mảnh đất thuê là lớn nhất có thể.



Hình minh họa: Cấu trúc mảng cộng dồn tiền tố truy vấn đoạn con tức thì $O(1)$

Nhiệm vụ: Cho ma trận A kích thước $N \times M$ và số nguyên dương K ($K \leq \min(N, M)$). Hãy tìm tổng lớn nhất của một ma trận con hình vuông kích thước $K \times K$.

Đầu vào (Input):

- Dòng 1: Chứa 3 số nguyên N, M, K ($1 \leq K \leq \min(N, M) \leq 1000$).
- N dòng tiếp theo: Mỗi dòng chứa M số nguyên $A_{i,j}$ ($-10^9 \leq A_{i,j} \leq 10^9$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là tổng lớn nhất của hình vuông $K \times K$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 3 2 1 1 1 1 2 2 1 2 3	9

Giải thích: Hình vuông kích thước 2 x 2 ở góc dưới phải gồm các ô: {2, 2, 2, 3} có tổng là $2 + 2 + 2 + 3 = 9$. Đây là hình vuông kích thước 2 x 2 có tổng lớn nhất.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho ma trận A kích thước N x M và số nguyên dương K ($K \leq \min(N, M)$). Hãy tìm tổng lớn nhất của một ma trận con hình vuông kích thước K x K.
- **Phương pháp tiếp cận — Mảng tiền tố & Mảng hiệu:**
- Dựng mảng cộng dồn $\text{pref}[i] = \text{pref}[i-1] + a[i]$. Khi đó tổng đoạn $[L, R]$ được tính tức thì bằng $\text{pref}[R] - \text{pref}[L-1]$ trong $\mathcal{O}(1)$.
- Với các thao tác cộng dồn đoạn, sử dụng mảng hiệu $\text{diff}[L] += V$, $\text{diff}[R+1] -= V$ rồi cộng dồn để phục hồi mảng.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| — | — | — | — | | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 3 2 1 1 1 1 2 2 1 2 3 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Hình vuông kích thước 2 x 2 ở góc dưới phải gồm các ô: {2, 2, 2, 3} có tổng là $2 + 2 + 2 + 3 = 9$. Đây là hình vuông kích... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 9 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Hình vuông kích thước 2 x 2 ở góc dưới phải gồm các ô: {2, 2, 2, 3} có tổng là $2 + 2 + 2 + 3 = 9$. Đây là hình vuông kích thước 2 x 2 có tổng lớn nhất.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm main() để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, k;
    if (!(cin >> n >> m >> k)) return 0;

    vector<vector<long long>> p(n + 1, vector<long long>(m + 1, 0));
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
```

```

        long long val;
        cin >> val;
        p[i][j] = p[i - 1][j] + p[i][j - 1] - p[i - 1][j - 1] + val;
    }
}

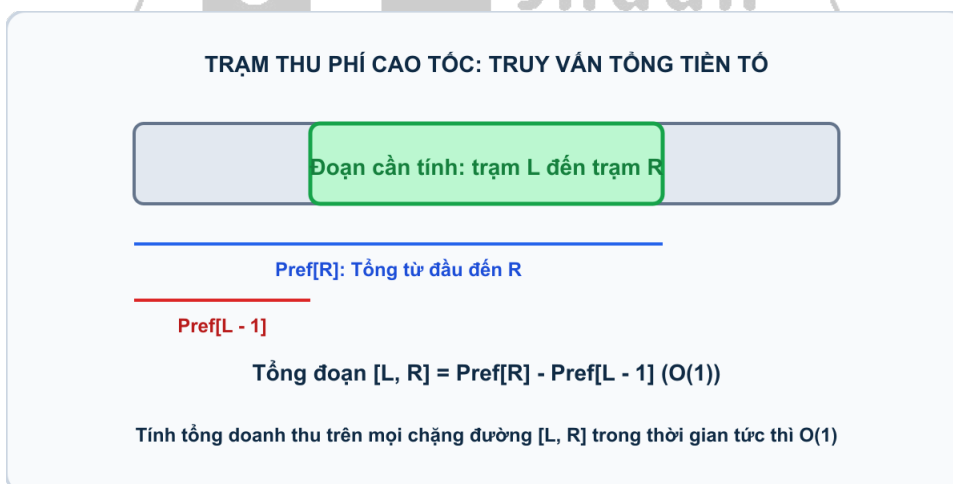
long long max_sum = -4e18; // Khởi tạo âm vô cùng
for (int i = k; i <= n; ++i) {
    for (int j = k; j <= m; ++j) {
        long long current = p[i][j] - p[i - k][j] - p[i][j - k] + p[i - k][j - k];
        max_sum = max(max_sum, current);
    }
}

cout << max_sum << "\n";
return 0;
}

```

Bài 09 [cppb_pt_09_mang_hieu_2d]: Cập Nhật Cộng Hình Chữ Nhật (Mảng Hiệu 2D)

Bối cảnh: Trong một trò chơi chiến thuật, bản đồ kích thước $N \times M$ ô ban đầu có mức phòng thủ bằng 0. Các người chơi lần lượt kích hoạt Q lá bùa gia cố, mỗi lá bùa cộng thêm X điểm phòng thủ cho một khu vực hình chữ nhật từ tọa độ (r_1, c_1) đến (r_2, c_2) . Hãy xác định bảng điểm phòng thủ cuối cùng của toàn bộ bản đồ sau khi kết thúc Q đợt kích hoạt bùa.



Hình minh họa: Cấu trúc mảng cộng dồn tiền tố truy vấn đoạn con tức thì $O(1)$

Nhiệm vụ: Cho ma trận $N \times M$ ban đầu toàn số 0. Thực hiện Q thao tác cộng giá trị X vào hình chữ nhật từ (r_1, c_1) đến (r_2, c_2) . Hãy in ra ma trận kết quả sau Q thao tác bằng Mảng hiệu 2D.

Đầu vào (Input):

- Dòng 1: Chứa 3 số nguyên N, M, Q ($1 \leq N, M \leq 1000, 1 \leq Q \leq 10^5$).
- Q dòng tiếp theo: Mỗi dòng gồm 5 số nguyên r_1, c_1, r_2, c_2, X .

Đầu ra (Output):

- In ra ma trận $N \times M$ sau khi hoàn tất toàn bộ Q thao tác.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 3 1 1 1 2 2 5	5 5 0 5 5 0 0 0 0

Giải thích: Thao tác cộng 5 vào hình chữ nhật từ (1, 1) đến (2, 2) làm cho 4 ô ở góc trên bên trái đều có giá trị 5, các ô còn lại giữ nguyên giá trị 0.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho ma trận $N \times M$ ban đầu toàn số 0. Thực hiện Q thao tác cộng giá trị X vào hình chữ nhật từ $(r1, c1)$ đến $(r2, c2)$. Hãy in ra ma trận kết quả sau Q thao tác bằng Mảng hiệu 2D.
- **Phương pháp tiếp cận — Mảng tiền tố & Mảng hiệu:**
- Dựng mảng cộng dồn $pref[i] = pref[i-1] + a[i]$. Khi đó tổng đoạn $[L, R]$ được tính tức thì bằng $pref[R] - pref[L-1]$ trong $\mathcal{O}(1)$.
- Với các thao tác cộng dồn đoạn, sử dụng mảng hiệu $diff[L] += V$, $diff[R+1] -= V$ rồi cộng dồn để phục hồi mảng.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 3 1 1 1 2 2 5 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Thao tác cộng 5 vào hình chữ nhật từ (1, 1) đến (2, 2) làm cho 4 ô ở góc trên bên trái đều có giá trị 5, các ô còn lại g... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 5 5 0 5 5 0 0 0 0 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Thao tác cộng 5 vào hình chữ nhật từ (1, 1) đến (2, 2) làm cho 4 ô ở góc trên bên trái đều có giá trị 5, các ô còn lại giữ nguyên giá trị 0.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, q;
    if (!(cin >> n >> m >> q)) return 0;

    vector<vector<long long>> d(n + 2, vector<long long>(m + 2, 0));

    while (q--) {
        int x1, y1, x2, y2;
        long long v;
        cin >> x1 >> y1 >> x2 >> y2 >> v;
    }
}
```

```

        d[x1][y1] += v;
        d[x1][y2 + 1] -= v;
        d[x2 + 1][y1] -= v;
        d[x2 + 1][y2 + 1] += v;
    }

    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            d[i][j] = d[i - 1][j] + d[i][j - 1] - d[i - 1][j - 1] + d[i][j];
            cout << d[i][j] << (j == m ? "" : " ");
        }
        cout << "\n";
    }

    return 0;
}

```

Bài 10 [cppb_pt_10_tong_chia_het_cho_k]: Đoạn Con Có Tổng Chia Hết Cho K

Bối cảnh: Một chuỗi các container hàng hóa có khối lượng $A_1, A_2, \dots, A_n$ cần được xếp lên các chuyến tàu vận tải, trong đó mỗi chuyến tàu chỉ có thể chở đúng các lô hàng có tổng khối lượng chia hết cho K tấn để đảm bảo cân bằng trọng tải đáy tàu. Hãy đếm xem có bao nhiêu đoạn con container liên tiếp có tổng khối lượng chia hết cho K .



Hình minh họa: Cấu trúc mảng cộng dồn tiền tố truy vấn đoạn con tức thì $O(1)$

Nhiệm vụ: Cho mảng gồm N số nguyên và số nguyên dương K . Hãy đếm số lượng đoạn con liên tiếp có tổng chia hết cho K .

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và K ($1 \leq N \leq 2 \cdot 10^5, 1 \leq K \leq 10^9$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là số lượng đoạn con thỏa mãn.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 2 3 4 5	4

Giải thích: Các đoạn con liên tiếp có tổng chia hết cho 3 là: [1, 2] (tổng 3), [3] (tổng 3), [4, 5] (tổng 9), và [1, 2, 3] (tổng 6). Tổng cộng có 4 đoạn con thỏa mãn.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng gồm N số nguyên và số nguyên dương K . Hãy đếm số lượng đoạn con liên tiếp có tổng chia hết cho K .
- **Phương pháp tiếp cận — Mảng tiền tố & Mảng hiệu:**
- Dựng mảng cộng dồn $\text{pref}[i] = \text{pref}[i-1] + a[i]$. Khi đó tổng đoạn $[L, R]$ được tính tức thì bằng $\text{pref}[R] - \text{pref}[L-1]$ trong $\mathcal{O}(1)$.
- Với các thao tác cộng dồn đoạn, sử dụng mảng hiệu $\text{diff}[L] += V$, $\text{diff}[R+1] -= V$ rồi cộng dồn để phục hồi mảng.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 3 1 2 3 4 5 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Các đoạn con liên tiếp có tổng chia hết cho 3 là: [1, 2] (tổng 3), [3] (tổng 3), [4, 5] (tổng 9), và [1, 2, 3] (tổng 6)... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 4 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Các đoạn con liên tiếp có tổng chia hết cho 3 là: [1, 2] (tổng 3), [3] (tổng 3), [4, 5] (tổng 9), và [1, 2, 3] (tổng 6). Tổng cộng có 4 đoạn con thỏa mãn.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long k;
    if (!(cin >> n >> k)) return 0;

    vector<long long> cnt(k, 0);
    cnt[0] = 1; // P[0] = 0

    long long current_sum = 0;
    for (int i = 1; i <= n; ++i) {
        long long x;
```

```

    cin >> x;
    current_sum += x;
    long long rem = (current_sum % k + k) % k;
    cnt[rem]++;
}

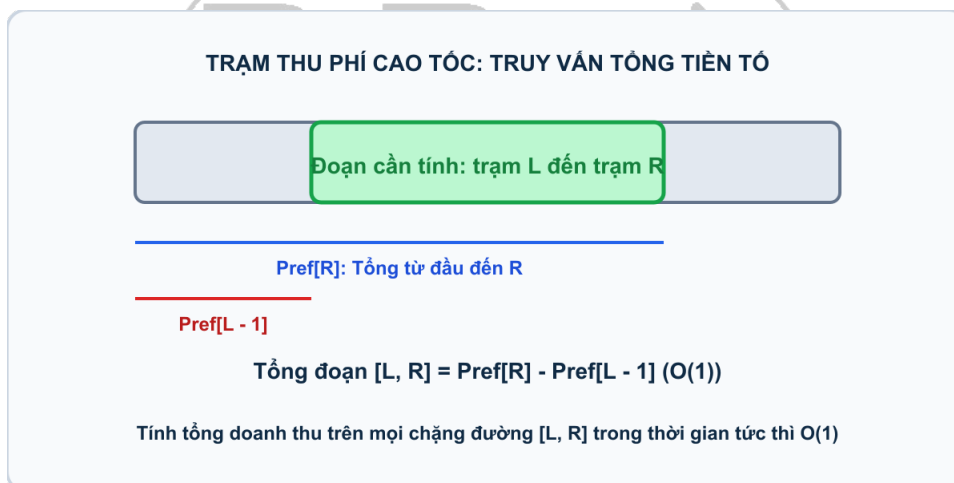
long long total_pairs = 0;
for (int r = 0; r < k; ++r) {
    total_pairs += cnt[r] * (cnt[r] - 1) / 2;
}

cout << total_pairs << "\n";
return 0;
}

```

Bài 11 [cppb_pt_11_mang_tien_to_xor]: Mảng Tiền Tố XOR Đoạn Con

Bối cảnh: Trong kỹ thuật mật mã hóa luồng, một chuỗi gồm N khóa bit số nguyên $A_1, A_2, \dots, A_N$ được lưu trữ liên tiếp. Để giải mã thông điệp truyền đi trong khoảng thời gian từ L đến R , máy chủ giải mã cần tính toán nhanh giá trị tích XOR của toàn bộ các phần tử từ vị trí L đến vị trí R cho Q yêu cầu độc lập.



Hình minh họa: Cấu trúc mảng cộng dồn tiền tố truy vấn đoạn con tức thì $O(1)$

Nhiệm vụ: Cho dãy số nguyên gồm N phần tử. Có Q truy vấn, mỗi truy vấn yêu cầu tính tích XOR của các phần tử trong đoạn $[L, R]$: $A[L] \text{ xor } A[L+1] \text{ xor } \dots \text{ xor } A[R]$.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: Chứa N số nguyên không âm $A_1, A_2, \dots, A_N$ ($0 \leq A_i \leq 10^9$).
- Q dòng tiếp theo: Mỗi dòng gồm 2 số nguyên L và R ($1 \leq L \leq R \leq N$).

Đầu ra (Output):

- In ra Q dòng, mỗi dòng là kết quả XOR tương ứng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 2 3 4 5 1 3 2 4 1 5	0 5 1

Giải thích: Mảng tiền tố XOR PrefXOR = [0, 1, 1[^]2=3, 3[^]3=0, 0[^]4=4, 4[^]5=1].

- Đoạn [1, 3]: PrefXOR[3] ^ PrefXOR[0] = 0 ^ 0 = 0.
- Đoạn [2, 4]: PrefXOR[4] ^ PrefXOR[1] = 4 ^ 1 = 5.
- Đoạn [1, 5]: PrefXOR[5] ^ PrefXOR[0] = 1 ^ 0 = 1.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho dãy số nguyên gồm N phần tử. Có Q truy vấn, mỗi truy vấn yêu cầu tính tích XOR của các phần tử trong đoạn [L, R]: A[L] xor A[L+1] xor ... xor A[R].
- **Phương pháp tiếp cận — Mảng tiền tố & Mảng hiệu:**
- Dựng mảng cộng dồn pref[i] = pref[i-1] + a[i]. Khi đó tổng đoạn [L, R] được tính tức thì bằng pref[R] - pref[L-1] trong O(1).
- Với các thao tác cộng dồn đoạn, sử dụng mảng hiệu diff[L] += V, diff[R+1] -= V rồi cộng dồn để phục hồi mảng.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 3 1 2 3 4 5 1 3 2 4 1 5 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Mảng tiền tố XOR PrefXOR = [0, 1, 1[^]2=3, 3[^]3=0, 0[^]4=4, 4[^]5=1]. - Đoạn [1, 3]: PrefXOR[3] ^ PrefXOR[0] = 0 ^ 0 = 0. - Đoạn... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 0 5 1 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Mảng tiền tố XOR PrefXOR = [0, 1, 1[^]2=3, 3[^]3=0, 0[^]4=4, 4[^]5=1].

- Đoạn [1, 3]: PrefXOR[3] ^ PrefXOR[0] = 0 ^ 0 = 0.
- Đoạn [2, 4]: PrefXOR[4] ^ PrefXOR[1] = 4 ^ 1 = 5.
- Đoạn [1, 5]: PrefXOR[5] ^ PrefXOR[0] = 1 ^ 0 = 1.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng ios::sync_with_stdio(false); cin.tie(nullptr); ở đầu hàm main() để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
```

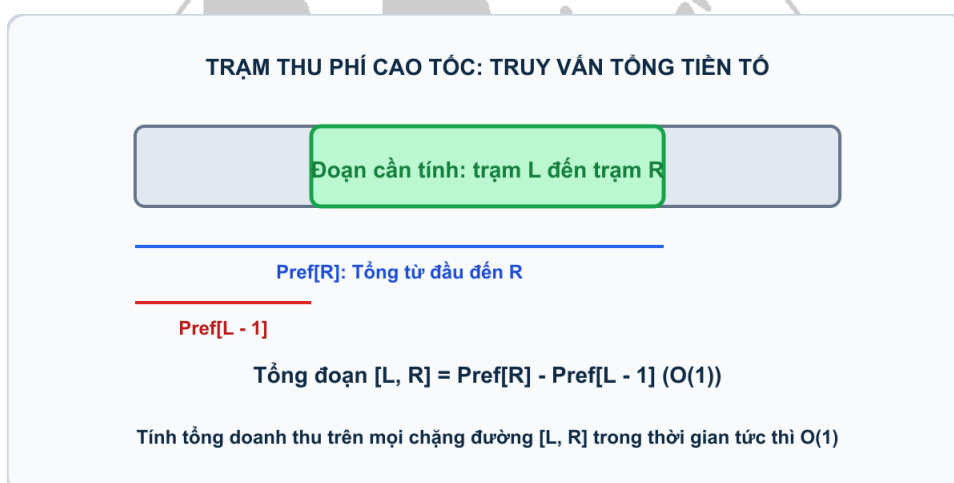
```
vector<long long> p(n + 1, 0);
for (int i = 1; i <= n; ++i) {
    long long x;
    cin >> x;
    p[i] = p[i - 1] ^ x;
}

while (q--) {
    int l, r;
    cin >> l >> r;
    cout << (p[r] ^ p[l - 1]) << "\n";
}

return 0;
}
```

Bài 12 [cppb_pt_12_can_bang_khong_va_mot]: Đoạn Con Cân Bằng Số Lượng 0 và 1

Bối cảnh: Một hệ thống phân phối tài nhị phân gồm chuỗi N tác vụ chỉ mang nhãn 0 hoặc 1. Để máy chủ xử lý đạt trạng thái cân bằng tài nguyên hoàn hảo, bộ điều phối cần chọn ra một chuỗi tác vụ liên tiếp dài nhất có số lượng tác vụ nhãn 0 bằng đúng số lượng tác vụ nhãn 1.



Hình minh họa: Cấu trúc mảng cộng dồn tiền tố truy vấn đoạn con tức thì $O(1)$

Nhiệm vụ: Cho mảng nhị phân gồm N phần tử chỉ chứa các số 0 và 1. Hãy tìm độ dài lớn nhất của một đoạn con liên tiếp có số lượng số 0 bằng số lượng số 1.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 2 \cdot 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($A_i \in \{0, 1\}$).

Đầu ra (Output):

- In ra độ dài lớn nhất tìm được, hoặc 0 nếu không có đoạn con nào thỏa mãn.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 0 1 0 0 1 1	6

Giải thích: Toàn bộ mảng gồm 6 phần tử có 3 số 0 và 3 số 1 (số lượng số 0 bằng số lượng số 1). Do đó đoạn con cân bằng dài nhất có độ dài bằng 6.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng nhị phân gồm N phần tử chỉ chứa các số 0 và 1. Hãy tìm độ dài lớn nhất của một đoạn con liên tiếp có số lượng số 0 bằng số lượng số 1.
- **Phương pháp tiếp cận — Mảng tiền tố & Mảng hiệu:**
- Dựng mảng cộng dồn $\text{pref}[i] = \text{pref}[i-1] + a[i]$. Khi đó tổng đoạn $[L, R]$ được tính tức thì bằng $\text{pref}[R] - \text{pref}[L-1]$ trong $\mathcal{O}(1)$.
- Với các thao tác cộng dồn đoạn, sử dụng mảng hiệu $\text{diff}[L] += V$, $\text{diff}[R+1] -= V$ rồi cộng dồn để phục hồi mảng.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 6 0 1 0 0 1 1 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Toàn bộ mảng gồm 6 phần tử có 3 số 0 và 3 số 1 (số lượng số 0 bằng số lượng số 1). Do đó đoạn con cân bằng dài nhất có đ... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 6 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Toàn bộ mảng gồm 6 phần tử có 3 số 0 và 3 số 1 (số lượng số 0 bằng số lượng số 1). Do đó đoạn con cân bằng dài nhất có độ dài bằng 6.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    // Tiền tố có thể chạy từ -N đến +N, offset = n
    vector<int> first_pos(2 * n + 1, -2);
    first_pos[0 + n] = 0; // P[0] = 0 tại vị trí 0

    int current_sum = 0;
    int max_len = 0;
```

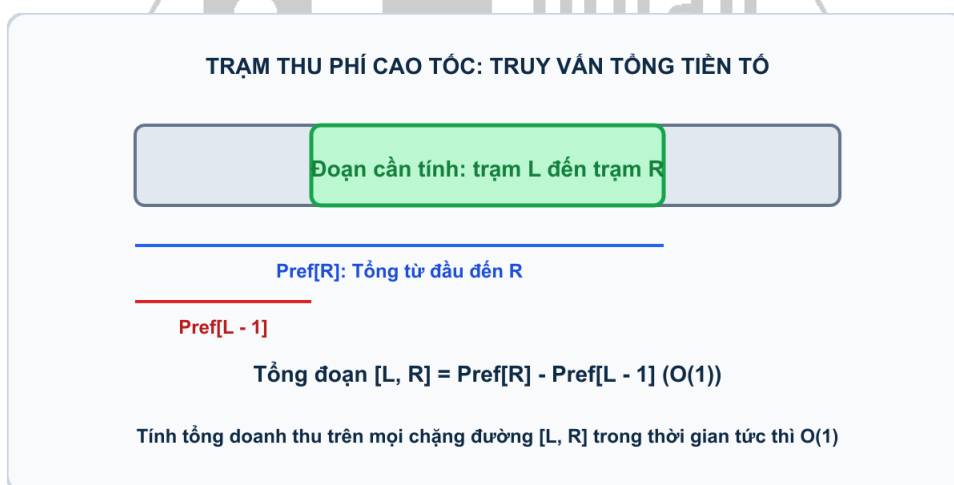
```
for (int i = 1; i <= n; ++i) {
    int x;
    cin >> x;
    current_sum += (x == 1 ? 1 : -1);

    int idx = current_sum + n;
    if (first_pos[idx] != -2) {
        max_len = max(max_len, i - first_pos[idx]);
    } else {
        first_pos[idx] = i;
    }
}

cout << max_len << "\n";
return 0;
}
```

Bài 13 [cppb_pt_13_ma_tran_da_vung]: Truy Vấn Ma Trận Đa Vùng Cực Đại

Bối cảnh: Trong phân tích viễn thám tài nguyên mặt đất, một bản đồ độ che phủ thực vật ma trận $N \times M$ cần được thống kê sản lượng cho các vùng dự án nông lâm kết hợp. Mỗi truy vấn cung cấp tọa độ của hai vùng hình chữ nhật rời nhau hoàn toàn, yêu cầu tính tổng sinh khối của cả hai vùng dự án này.



Hình minh họa: Cấu trúc mảng cộng dồn tiền tố truy vấn đoạn con tức thì O(1)

Nhiệm vụ: Cho một ma trận $N \times M$. Mỗi truy vấn cung cấp tọa độ của hai hình chữ nhật rời nhau, hãy tính tổng giá trị của tất cả các phần tử thuộc về cả hai hình chữ nhật đó.

Đầu vào (Input):

- Dòng 1: 3 số nguyên N, M, Q ($1 \leq N, M \leq 1000, 1 \leq Q \leq 10^5$).
- N dòng tiếp theo: Mỗi dòng gồm M số nguyên $A_{i,j}$ ($-10^9 \leq A_{i,j} \leq 10^9$).
- Q dòng tiếp theo: Mỗi dòng gồm 8 số nguyên $r_{1a}, c_{1a}, r_{2a}, c_{2a}, r_{1b}, c_{1b}, r_{2b}, c_{2b}$.

Đầu ra (Output):

- In ra Q dòng, mỗi dòng là tổng giá trị của hai vùng tương ứng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
-----------------	-----------------

3 3 1 1 1 1 1 1 1 1	5
1 1 1 2 2 3 3	

Giải thích: Hình chữ nhật 1 là ô (1, 1) có giá trị 1. Hình chữ nhật 2 là vùng từ (2, 2) đến (3, 3) gồm 4 ô giá trị 1 (tổng bằng 4). Tổng hai vùng là $1 + 4 = 5$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho một ma trận $N \times M$. Mỗi truy vấn cung cấp tọa độ của hai hình chữ nhật rời nhau, hãy tính tổng giá trị của tất cả các phần tử thuộc về cả hai hình chữ nhật đó.
- **Phương pháp tiếp cận — Mảng tiền tố & Mảng hiệu:**
- Dựng mảng cộng dồn $\text{pref}[i] = \text{pref}[i-1] + a[i]$. Khi đó tổng đoạn $[L, R]$ được tính tức thì bằng $\text{pref}[R] - \text{pref}[L-1]$ trong $\mathcal{O}(1)$.
- Với các thao tác cộng dồn đoạn, sử dụng mảng hiệu $\text{diff}[L] += V$, $\text{diff}[R+1] -= V$ rồi cộng dồn để phục hồi mảng.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 3 1 1 1 1 1 1 1 1 1 1 1 2 2 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Hình chữ nhật 1 là ô (1, 1) có giá trị 1. Hình chữ nhật 2 là vùng từ (2, 2) đến (3, 3) gồm 4 ô giá trị 1 (tổng bằng 4). ... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 5 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Hình chữ nhật 1 là ô (1, 1) có giá trị 1. Hình chữ nhật 2 là vùng từ (2, 2) đến (3, 3) gồm 4 ô giá trị 1 (tổng bằng 4). Tổng hai vùng là $1 + 4 = 5$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, q;
    if (!(cin >> n >> m >> q)) return 0;

    vector<vector<long long>> p(n + 1, vector<long long>(m + 1, 0));
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            long long val;
```

```

        cin >> val;
        p[i][j] = p[i - 1][j] + p[i][j - 1] - p[i - 1][j - 1] + val;
    }
}

auto query = [&](int x1, int y1, int x2, int y2) -> long long {
    return p[x2][y2] - p[x1 - 1][y2] - p[x2][y1 - 1] + p[x1 - 1][y1 - 1];
};

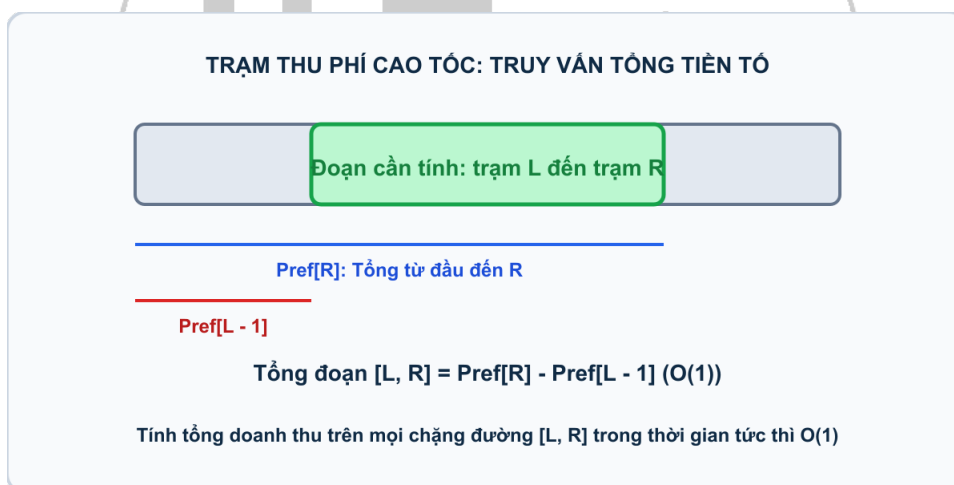
while (q--) {
    int x1, y1, x2, y2, u1, v1, u2, v2;
    cin >> x1 >> y1 >> x2 >> y2 >> u1 >> v1 >> u2 >> v2;
    cout << query(x1, y1, x2, y2) + query(u1, v1, u2, v2) << "\n";
}

return 0;
}

```

Bài 14 [cppb_pt_14_phan_phoi_tai_nguyen]: Phân Phối Tài Nguyên Không Gian Tuyến Tính

Bối cảnh: Một tuyến đường ống cấp nước đô thị dài N mét ban đầu lưu lượng nước bổ sung bằng 0. Trải qua Q lượt cấp nước bổ trợ dạng cấp số cộng: lượt thứ k cấp nước vào đoạn từ L đến R với lưu lượng tại mét L là V và mỗi mét tiếp theo tăng thêm D đơn vị lưu lượng (dạng cấp số cộng). Hãy xác định lượng nước phân phối cuối cùng tại mỗi mét đường ống.



Hình minh họa: Cấu trúc mảng cộng dồn tiền tố truy vấn đoạn con tức thì $O(1)$

Nhiệm vụ: Cho dãy số N phần tử ban đầu toàn số 0. Thực hiện Q thao tác cộng vào đoạn $[L, R]$ một dãy cấp số cộng với số hạng đầu V và công sai D . Hãy in ra mảng kết quả cuối cùng.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và Q ($1 \leq N, Q \leq 10^5$).
- Q dòng tiếp theo: Mỗi dòng gồm 4 số nguyên L, R, V, D ($1 \leq L \leq R \leq N, -10^9 \leq V, D \leq 10^9$).

Đầu ra (Output):

- In ra N số nguyên trên một dòng biểu diễn mảng sau khi hoàn thành Q thao tác.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 1 2 4 1 2	0 1 3 5 0

Giải thích: Thao tác trên đoạn $[2, 4]$ với $V = 1, D = 2$: vị trí 2 nhận 1; vị trí 3 nhận $1 + 2 = 3$; vị trí 4 nhận $1 + 2 \cdot 2 = 5$. Kết quả in ra: 0 1 3 5 0.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho dãy số N phần tử ban đầu toàn số 0. Thực hiện Q thao tác cộng vào đoạn $[L, R]$ một dãy cấp số cộng với số hạng đầu V và công sai D . Hãy in ra mảng kết quả cuối cùng.
- **Phương pháp tiếp cận — Mảng tiền tố & Mảng hiệu:**
- Dựng mảng cộng dồn $\text{pref}[i] = \text{pref}[i-1] + a[i]$. Khi đó tổng đoạn $[L, R]$ được tính tức thì bằng $\text{pref}[R] - \text{pref}[L-1]$ trong $\mathcal{O}(1)$.
- Với các thao tác cộng dồn đoạn, sử dụng mảng hiệu $\text{diff}[L] += V, \text{diff}[R+1] -= V$ rồi cộng dồn để phục hồi mảng.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| — | — | — | — | — | | 1 | Nạp dữ liệu vào mảng/ biến | Input: 5 1 2 4 1 2 | Khởi tạo cấu trúc dữ liệu ban đầu |
 | 2 | Thực thi thuật toán tối ưu | Thao tác trên đoạn $[2, 4]$ với $V = 1, D = 2$: vị trí 2 nhận 1; vị trí 3 nhận $1 + 2 = 3$; vị trí 4 nhận $1 + 2 \cdot 2 = 5$. Kết quả... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 0 1 3 5 0 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Thao tác trên đoạn $[2, 4]$ với $V = 1, D = 2$: vị trí 2 nhận 1; vị trí 3 nhận $1 + 2 = 3$; vị trí 4 nhận $1 + 2 \cdot 2 = 5$. Kết quả in ra: 0 1 3 5 0.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;

    vector<long long> d2(n + 3, 0);
```

```

while (q--) {
    long long l, r, s, d;
    cin >> l >> r >> s >> d;
    d2[l] += s;
    d2[l + 1] += (d - s);
    d2[r + 1] -= (s + (r - l + 1) * d);
    d2[r + 2] += (s + (r - l) * d);
}

// Lần 1: Khôi phục mảng hiệu bậc 1
vector<long long> d1(n + 2, 0);
for (int i = 1; i <= n + 1; ++i) {
    d1[i] = d1[i - 1] + d2[i];
}

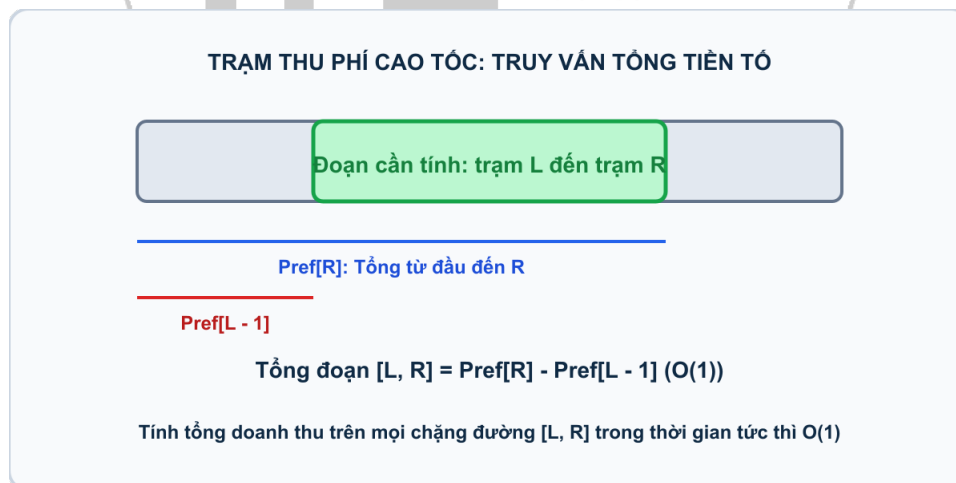
// Lần 2: Khôi phục mảng giá trị gốc
vector<long long> a(n + 1, 0);
for (int i = 1; i <= n; ++i) {
    a[i] = a[i - 1] + d1[i];
    cout << a[i] << (i == n ? "" : " ");
}
cout << "\n";

return 0;
}

```

Bài 15 [cppb_pt_15_ma_tran_con_tong_lon_nhat]: Tìm Ma Trận Con Có Tổng Lớn Nhất (Max Submatrix Sum)

Bối cảnh: Một tấm kim loại công nghiệp kích thước $N \times M$ có các điểm chịu nhiệt với hệ số truyền dẫn nhiệt ghi nhận trên từng ô ma trận (có cả số âm và số dương). Các kỹ sư cần cắt ra một tấm kim loại con hình chữ nhật bất kỳ sao cho tổng hệ số dẫn nhiệt của tấm cắt ra đạt mức cực đại.



Hình minh họa: Cấu trúc mảng cộng dồn tiền tố truy vấn đoạn con tức thì $O(1)$

Nhiệm vụ: Cho ma trận số nguyên A kích thước $N \times M$. Hãy tìm một ma trận con chữ nhật có tổng các phần tử là lớn nhất.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và M ($1 \leq N, M \leq 300$).
- N dòng tiếp theo: Mỗi dòng gồm M số nguyên $A_{i,j}$ ($-10^9 \leq A_{i,j} \leq 10^9$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là tổng lớn nhất của ma trận con tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 3 1 2 -1 -8 -9 -2 3 4 5	12

Giải thích: Ma trận con ở hàng 3 gồm các phần tử [3, 4, 5] có tổng $3 + 4 + 5 = 12$. Đây là ma trận con có tổng lớn nhất trong bảng.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho ma trận số nguyên A kích thước $N \times M$. Hãy tìm một ma trận con chữ nhật có tổng các phần tử là lớn nhất.
- **Phương pháp tiếp cận — Mảng tiền tố & Mảng hiệu:**
- Dựng mảng cộng dồn $pref[i] = pref[i-1] + a[i]$. Khi đó tổng đoạn $[L, R]$ được tính tức thì bằng $pref[R] - pref[L-1]$ trong $\mathcal{O}(1)$.
- Với các thao tác cộng dồn đoạn, sử dụng mảng hiệu $diff[L] += V$, $diff[R+1] -= V$ rồi cộng dồn để phục hồi mảng.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 3 1 2 -1 -8 -9 -2 3 4 5 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Ma trận con ở hàng 3 gồm các phần tử [3, 4, 5] có tổng $3 + 4 + 5 = 12$. Đây là ma trận con có tổng lớn nhất trong bảng.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 12 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Ma trận con ở hàng 3 gồm các phần tử [3, 4, 5] có tổng $3 + 4 + 5 = 12$. Đây là ma trận con có tổng lớn nhất trong bảng.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```

int n, m;
if (!(cin >> n >> m)) return 0;

vector<vector<long long>> a(n + 1, vector<long long>(m + 1));
vector<vector<long long>> p(n + 1, vector<long long>(m + 1, 0));

for (int i = 1; i <= n; ++i) {
    for (int j = 1; j <= m; ++j) {
        cin >> a[i][j];
        p[i][j] = p[i - 1][j] + a[i][j]; // Tiền tố theo cột
    }
}

long long max_sum = -4e18;

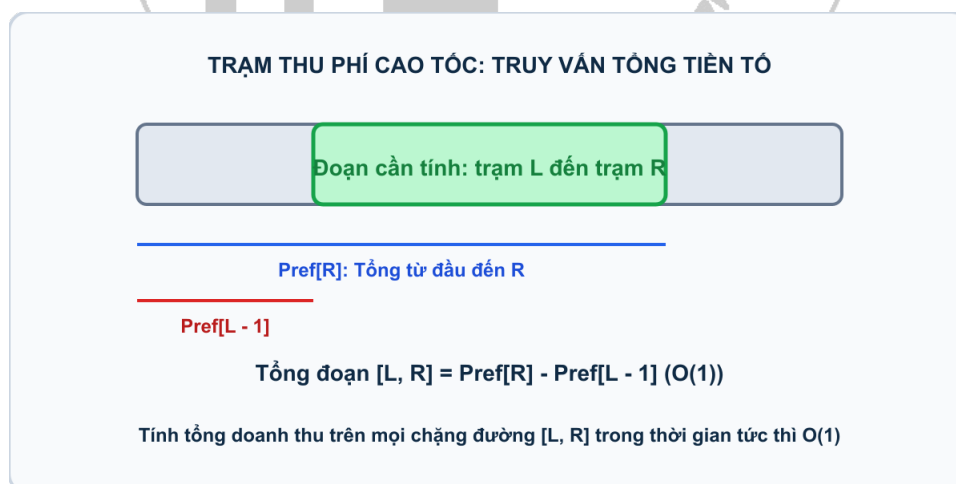
for (int r1 = 1; r1 <= n; ++r1) {
    for (int r2 = r1; r2 <= n; ++r2) {
        long long current_kadane = 0;
        for (int c = 1; c <= m; ++c) {
            long long val = p[r2][c] - p[r1 - 1][c];
            current_kadane = max(val, current_kadane + val);
            max_sum = max(max_sum, current_kadane);
        }
    }
}

cout << max_sum << "\n";
return 0;
}

```

Bài 16 [cppb_pt_16_can_bang_tien_to_da_chieu]: Cân Bằng Tiền Tố Đa Chiều

Bối cảnh: Trong chuỗi mã hóa di truyền sinh học gồm N phân tử thuộc ba loại ‘A’, ‘B’, ‘C’, các nhà sinh học phân tử cần tìm một đoạn gen liên tiếp dài nhất mà trong đó số lượng các phân tử loại ‘A’, ‘B’ và ‘C’ xuất hiện hoàn toàn bằng nhau.



Hình minh họa: Cấu trúc mảng cộng dồn tiền tố truy vấn đoạn con tức thì $O(1)$

Nhiệm vụ: Cho một chuỗi gồm N ký tự chỉ gồm các chữ cái ‘A’, ‘B’, ‘C’. Hãy tìm độ dài lớn nhất của một đoạn con liên tiếp có số lượng ký tự ‘A’, ‘B’ và ‘C’ bằng nhau.

Đầu vào (Input):

- Dòng 1: Chuỗi ký tự S có độ dài N ($1 \leq N \leq 10^5$).

Đầu ra (Output):

- In ra độ dài lớn nhất của chuỗi con thỏa mãn, hoặc 0 nếu không có đoạn nào.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
ABACBC	6

Giải thích: Chuỗi 'ABACBC' có độ dài 6 chứa đúng hai ký tự 'A', hai ký tự 'B' và hai ký tự 'C' (số lượng bằng nhau = 2). Vì vậy độ dài lớn nhất là 6.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho một chuỗi gồm N ký tự chỉ gồm các chữ cái 'A', 'B', 'C'. Hãy tìm độ dài lớn nhất của một đoạn con liên tiếp có số lượng ký tự 'A', 'B' và 'C' bằng nhau.
- **Phương pháp tiếp cận — Mảng tiền tố & Mảng hiệu:**
- Dựng mảng cộng dồn $pref[i] = pref[i-1] + a[i]$. Khi đó tổng đoạn $[L, R]$ được tính tức thì bằng $pref[R] - pref[L-1]$ trong $\mathcal{O}(1)$.
- Với các thao tác cộng dồn đoạn, sử dụng mảng hiệu $diff[L] += V$, $diff[R+1] -= V$ rồi cộng dồn để phục hồi mảng.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: ABACBC | Khởi tạo cấu trúc dữ liệu ban đầu |
 | 2 | Thực thi thuật toán tối ưu | Chuỗi 'ABACBC' có độ dài 6 chứa đúng hai ký tự 'A', hai ký tự 'B' và hai ký tự 'C' (số lượng bằng nhau = 2). Vì vậy độ d... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 6 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Chuỗi 'ABACBC' có độ dài 6 chứa đúng hai ký tự 'A', hai ký tự 'B' và hai ký tự 'C' (số lượng bằng nhau = 2). Vì vậy độ dài lớn nhất là 6.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    string s;
```

```
if (!(cin >> n >> s)) return 0;

int ca = 0, cb = 0, cc = 0;

// Lưu: {diff1, diff2, index}
vector<vector<int>>> states;
states.reserve(n + 1);
states.push_back({0, 0, 0}); // Tại vị trí 0

for (int i = 1; i <= n; ++i) {
    if (s[i - 1] == 'A') ca++;
    else if (s[i - 1] == 'B') cb++;
    else if (s[i - 1] == 'C') cc++;

    states.push_back({ca - cb, cb - cc, i});
}

sort(states.begin(), states.end(), [](const vector<int>& u, const vector<int>& v) {
    if (u[0] != v[0]) return u[0] < v[0];
    if (u[1] != v[1]) return u[1] < v[1];
    return u[2] < v[2];
});

int max_len = 0;
int i = 0;
while (i <= n) {
    int j = i;
    while (j <= n && states[j][0] == states[i][0] && states[j][1] == states[i][1]) {
        j++;
    }
    max_len = max(max_len, states[j - 1][2] - states[i][2]);
    i = j;
}

cout << max_len << "\n";
return 0;
}
```

Bài 08: Thuật toán tìm kiếm nhị phân

Bài 01 [cppb_bs_01_tim_kiem_tren_mang_sap_xep]: Tìm Kiếm Phần Tử Trên Mảng Đã Sắp Xếp

Bối cảnh: Tại một thư viện điện tử quốc gia, danh mục gồm N cuốn sách quý hiếm đã được sắp xếp theo số hiệu mã vạch tăng dần. Để phục vụ độc giả tra cứu nhanh trong hàng trăm ngàn đầu sách, thủ thư nhận Q yêu cầu tìm kiếm xem mã sách X có tồn tại trong hệ thống hay không.



Hình minh họa: Thuật toán tìm kiếm nhị phân chia đôi không gian $O(\log N)$

Nhiệm vụ: Cho mảng N số nguyên đã sắp xếp tăng dần. Với mỗi truy vấn chứa số nguyên X , hãy kiểm tra xem X có xuất hiện trong mảng hay không bằng thuật toán Tìm kiếm nhị phân. Nếu có in YES, ngược lại in NO.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: Chứa N số nguyên tăng dần $A_1 \leq A_2 \leq \dots \leq A_N$ ($-10^9 \leq A_i \leq 10^9$).
- Q dòng tiếp theo: Mỗi dòng chứa một số nguyên X ($-10^9 \leq X \leq 10^9$).

Đầu ra (Output):

- In ra Q dòng, mỗi dòng là YES hoặc NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 3 5 7 9 5 4 9	YES NO YES

Giải thích: - Truy vấn 1: Số 5 xuất hiện tại vị trí 3 -> YES. - Truy vấn 2: Số 4 không có trong mảng -> NO. - Truy vấn 3: Số 9 xuất hiện tại vị trí 5 -> YES.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng N số nguyên đã sắp xếp tăng dần. Với mỗi truy vấn chứa số nguyên X , hãy kiểm tra xem X có xuất hiện trong mảng hay không bằng thuật toán Tìm kiếm nhị phân. Nếu có in YES, ngược lại in NO.
- Phương pháp tiếp cận — Tìm kiếm nhị phân:**

- Nhận diện tính đơn điệu của hàm mục tiêu hoặc không gian tìm kiếm.
- Thu hẹp không gian nghiệm $[L, R]$ qua điểm giữa $mid = L + (R - L)/2$. Độ phức tạp thời gian đạt $\mathcal{O}(\log N)$ hoặc $\mathcal{O}(N \log(\text{range}))$.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nhập dữ liệu vào mảng/biến | Input: 5 3 1 3 5 7 9 5 4 9 | Khởi tạo cấu trúc dữ liệu ban đầu | 2 | Thực thi thuật toán tối ưu | - Truy vấn 1: Số 5 xuất hiện tại vị trí 3 -> YES. - Truy vấn 2: Số 4 không có trong mảng -> NO. - Truy vấn 3: Số 9 xuất ... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: YES NO YES | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: - Truy vấn 1: Số 5 xuất hiện tại vị trí 3 -> YES.

- Truy vấn 2: Số 4 không có trong mảng -> NO.
- Truy vấn 3: Số 9 xuất hiện tại vị trí 5 -> YES.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    while (q--) {
        long long x;
        cin >> x;
        if (binary_search(a.begin(), a.end(), x)) {
            cout << "YES\n";
        } else {
            cout << "NO\n";
        }
    }

    return 0;
}
```

Bài 02 [cppb_bs_02_vi_tri_dau_va_cuoi]: Tìm Vị Trí Xuất Hiện Đầu Tiên & Cuối Cùng

Bối cảnh: Trong cơ sở dữ liệu phân tích tín hiệu âm thanh số, N mẫu tần số đã được sắp xếp tăng dần và có nhiều mẫu trùng lặp cùng tần số. Kỹ sư âm thanh gửi Q yêu cầu truy vấn giá trị tần số X , cần xác định chính xác vị trí xuất hiện đầu tiên và vị trí xuất hiện cuối cùng (1-indexed) của tần số đó trong cơ sở dữ liệu.



Hình minh họa: Thuật toán tìm kiếm nhị phân chia đôi không gian $O(\log N)$

Nhiệm vụ: Cho mảng N phần tử đã sắp xếp tăng dần. Với mỗi giá trị X trong Q truy vấn, hãy tìm vị trí xuất hiện đầu tiên và cuối cùng của X . Nếu X không có trong mảng, in ra -1 -1.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: Chứa N số nguyên tăng dần $A_1 \leq A_2 \leq \dots \leq A_N$ ($-10^9 \leq A_i \leq 10^9$).
- Q dòng tiếp theo: Mỗi dòng chứa một số nguyên X .

Đầu ra (Output):

- In ra Q dòng, mỗi dòng gồm 2 số nguyên là vị trí đầu và vị trí cuối (1-indexed), hoặc -1 -1.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
7 3 1 2 2 2 3 4 5 2 3 6	2 4 5 5 -1 -1

Giải thích: - Số 2 xuất hiện từ vị trí 2 đến vị trí 4 -> in 2 4. - Số 3 chỉ xuất hiện tại vị trí 5 -> in 5 5. - Số 6 không có trong mảng -> in -1 -1.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng N phần tử đã sắp xếp tăng dần. Với mỗi giá trị X trong Q truy vấn, hãy tìm vị trí xuất hiện đầu tiên và cuối cùng của X . Nếu X không có trong mảng, in ra -1 -1.
- Phương pháp tiếp cận — Tìm kiếm nhị phân:**
- Nhận diện tính đơn điệu của hàm mục tiêu hoặc không gian tìm kiếm.

- Thu hẹp không gian nghiệm $[L, R]$ qua điểm giữa $mid = L + (R - L)/2$. Độ phức tạp thời gian đạt $\mathcal{O}(\log N)$ hoặc $\mathcal{O}(N \log(\text{range}))$.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 7 3 1 2 2 2 3 4 5 2 3 6 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | - Số 2 xuất hiện từ vị trí 2 đến vị trí 4 -> in 2 4. - Số 3 chỉ xuất hiện tại vị trí 5 -> in 5 5. - Số 6 không có tr... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 2 4 5 5 -1 -1 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: - Số 2 xuất hiện từ vị trí 2 đến vị trí 4 -> in 2 4.

- Số 3 chỉ xuất hiện tại vị trí 5 -> in 5 5.
- Số 6 không có trong mảng -> in -1 -1.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    while (q--) {
        long long x;
        cin >> x;
        auto it1 = lower_bound(a.begin(), a.end(), x);
        if (it1 == a.end() || *it1 != x) {
            cout << "-1 -1\n";
        } else {
            auto it2 = upper_bound(a.begin(), a.end(), x);
            int first_idx = it1 - a.begin() + 1;
            int last_idx = it2 - a.begin();
            cout << first_idx << " " << last_idx << "\n";
        }
    }
}
```

```
return 0;
}
```

Bài 03 [cppb_bs_03_dem_so_trong_khoang]: Đếm Số Phần Tử Trong Đoạn [L, R]

Bối cảnh: Một cơ quan khí tượng tổng hợp N chỉ số nhiệt độ đo được tại các trạm quan trắc (chưa sắp xếp). Các nhà nghiên cứu khí hậu gửi Q câu hỏi truy vấn độc lập, mỗi câu hỏi cần biết có bao nhiêu trạm quan trắc ghi nhận nhiệt độ nằm trong khoảng từ ngưỡng L đến ngưỡng R .



Hình minh họa: Thuật toán tìm kiếm nhị phân chia đôi không gian $O(\log N)$

Nhiệm vụ: Cho mảng gồm N số nguyên. Với mỗi câu hỏi gồm khoảng $[L, R]$ ($L \leq R$), hãy đếm số lượng phần tử của mảng có giá trị nằm trong đoạn $[L, R]$ ($L \leq A[i] \leq R$).

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).
- Q dòng tiếp theo: Mỗi dòng gồm 2 số nguyên L và R ($-10^9 \leq L \leq R \leq 10^9$).

Đầu ra (Output):

- In ra Q dòng, mỗi dòng là số lượng phần tử thỏa mãn.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 5 1 8 3 2 2 5 1 1 6 7	3 1 0

Giải thích: Sắp xếp mảng: $[1, 2, 3, 5, 8]$.

- Đoạn $[2, 5]$: có 3 phần tử $\{2, 3, 5\}$ -> in 3.
- Đoạn $[1, 1]$: có 1 phần tử $\{1\}$ -> in 1.
- Đoạn $[6, 7]$: không có phần tử nào -> in 0.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng gồm N số nguyên. Với mỗi câu hỏi gồm khoảng $[L, R]$ ($L \leq R$), hãy đếm số lượng phần tử của mảng có giá trị nằm trong đoạn $[L, R]$ ($L \leq A[i] \leq R$).

- **Phương pháp tiếp cận — Tìm kiếm nhị phân:**

- Nhận diện tính đơn điệu của hàm mục tiêu hoặc không gian tìm kiếm.
- Thu hẹp không gian nghiệm $[L, R]$ qua điểm giữa $mid = L + (R - L)/2$. Độ phức tạp thời gian đạt $\mathcal{O}(\log N)$ hoặc $\mathcal{O}(N \log(\text{range}))$.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 3 5 1 8 3 2 2 5 1 1 6 7 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Sắp xếp mảng: [1, 2, 3, 5, 8]. - Đoạn [2, 5]: có 3 phần tử {2, 3, 5} -> in 3. - Đoạn [1, 1]: có 1 phần tử {1} -> in 1. -... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 3 1 0 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Sắp xếp mảng: [1, 2, 3, 5, 8].

- Đoạn [2, 5]: có 3 phần tử {2, 3, 5} -> in 3.
- Đoạn [1, 1]: có 1 phần tử {1} -> in 1.
- Đoạn [6, 7]: không có phần tử nào -> in 0.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }
    sort(a.begin(), a.end());

    while (q--) {
        long long l, r;
        cin >> l >> r;
        auto it_l = lower_bound(a.begin(), a.end(), l);
        auto it_r = upper_bound(a.begin(), a.end(), r);
        cout << (it_r - it_l) << "\n";
    }
}
```

```
return 0;
}
```

Bài 04 [cppb_bs_04_can_bac_hai_so_lon]: Tìm Căn Bậc Hai Số Nguyên Lớn

Bối cảnh: Trong một bài toán mật mã học khóa công khai, máy chủ cần tính toán phần nguyên của căn bậc hai của một số nguyên dương cực lớn N (lên đến 10^{18}). Kỹ sư cần tìm số nguyên dương X lớn nhất sao cho bình phương của X không vượt quá N .



Hình minh họa: Thuật toán tìm kiếm nhị phân chia đôi không gian $O(\log N)$

Nhiệm vụ: Cho số nguyên dương N ($1 \leq N \leq 10^{18}$). Hãy tìm số nguyên dương X lớn nhất thỏa mãn $X^2 \leq N$ bằng tìm kiếm nhị phân trên tập kết quả.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 10^{18}$).

Đầu ra (Output):

- In ra số nguyên dương X lớn nhất thỏa mãn.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
17	4

Giải thích: $4^2 = 16 \leq 17$, trong khi $5^2 = 25 > 17$. Số nguyên lớn nhất có bình phương ≤ 17 là 4.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N ($1 \leq N \leq 10^{18}$). Hãy tìm số nguyên dương X lớn nhất thỏa mãn $X^2 \leq N$ bằng tìm kiếm nhị phân trên tập kết quả.
- Phương pháp tiếp cận — Tìm kiếm nhị phân:**
- Nhận diện tính đơn điệu của hàm mục tiêu hoặc không gian tìm kiếm.
- Thu hẹp không gian nghiệm $[L, R]$ qua điểm giữa $mid = L + (R - L)/2$. Độ phức tạp thời gian đạt $O(\log N)$ hoặc $O(N \log(\text{range}))$.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nhập dữ liệu vào mảng/biến | Input: 17 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $4^2 = 16 \leq 17$, trong khi $5^2 = 25 > 17$. Số nguyên lớn nhất có bình phương ≤ 17 là 4.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 4 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $4^2 = 16 \leq 17$, trong khi $5^2 = 25 > 17$. Số nguyên lớn nhất có bình phương ≤ 17 là 4.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

void solve() {
    long long n;
    cin >> n;
    long long low = 1, high = 1000000000LL, ans = 1;
    while (low <= high) {
        long long mid = low + (high - low) / 2;
        if (mid <= n / mid) {
            ans = mid;
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    cout << ans << "\n";
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    if (!(cin >> t)) return 0;
    while (t--) {
        solve();
    }
    return 0;
}
```

Bài 05 [cppb_bs_05_phan_tu_nho_nhat_lon_hon_x]: Tìm Phần Tử Nhỏ Nhất Lớn Hơn X

Bối cảnh: Tại một sàn đấu giá kim cương trực tuyến, danh sách mức giá đề xuất của N món đồ đã được sắp xếp tăng dần. Một nhà sưu tầm đặt ra mức giá trần X và muốn hệ thống đề xuất món đồ tiếp theo có giá thấp nhất nhưng phải nghiêm ngặt cao hơn mức giá X để xem xét đấu giá tiếp.



Hình minh họa: Thuật toán tìm kiếm nhị phân chia đôi không gian $O(\log N)$

Nhiệm vụ: Cho mảng N số nguyên đã sắp xếp tăng dần. Với mỗi truy vấn X , hãy tìm phần tử nhỏ nhất trong mảng có giá trị nghiêm ngặt lớn hơn X . Nếu không có, in ra -1.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: Chứa N số nguyên tăng dần $A_1 \leq A_2 \leq \dots \leq A_N$ ($-10^9 \leq A_i \leq 10^9$).
- Q dòng tiếp theo: Mỗi dòng chứa một số nguyên X .

Đầu ra (Output):

- In ra Q dòng, mỗi dòng là giá trị phần tử tìm được hoặc -1.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 2 3 5 6 8 4 2 8	5 3 -1

Giải thích: - Với $X = 4$: phần tử nhỏ nhất > 4 là 5. - Với $X = 2$: phần tử nhỏ nhất > 2 là 3. - Với $X = 8$: không có phần tử nào $> 8 \rightarrow$ in -1.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng N số nguyên đã sắp xếp tăng dần. Với mỗi truy vấn X , hãy tìm phần tử nhỏ nhất trong mảng có giá trị nghiêm ngặt lớn hơn X . Nếu không có, in ra -1.
- Phương pháp tiếp cận — Tìm kiếm nhị phân:**
- Nhận diện tính đơn điệu của hàm mục tiêu hoặc không gian tìm kiếm.
- Thu hẹp không gian nghiệm $[L, R]$ qua điểm giữa $mid = L + (R - L)/2$. Độ phức tạp thời gian đạt $O(\log N)$ hoặc $O(N \log(\text{range}))$.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nhập dữ liệu vào mảng/biến | Input: 5 3 2 3 5 6 8 4 2 8 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | - Với $X = 4$: phần tử nhỏ nhất > 4 là 5. - Với $X = 2$: phần tử nhỏ nhất > 2 là 3. - Với $X = 8$: không có phần tử nào $> 8 \rightarrow \dots$ | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 5 3 -1 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: - Với $X = 4$: phần tử nhỏ nhất > 4 là 5.

- Với $X = 2$: phần tử nhỏ nhất > 2 là 3.
- Với $X = 8$: không có phần tử nào $> 8 \rightarrow$ in -1.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    while (q--) {
        long long x;
        cin >> x;
        auto it = upper_bound(a.begin(), a.end(), x);
        if (it == a.end()) {
            cout << -1 << "\n";
        } else {
            cout << *it << "\n";
        }
    }

    return 0;
}
```

Bài 06 [cppb_bs_06_chia_keo_hoc_sinh]: Chia Kẹo Cho Học Sinh Đạt Chuẩn

Bối cảnh: Một trường tiểu học nhận được N thùng kẹo, thùng thứ i có A_i chiếc kẹo. Thầy hiệu trưởng muốn chia đều kẹo cho K em học sinh đạt danh hiệu cháu ngoan Bác Hồ sao cho mỗi em nhận được đúng X chiếc kẹo và kẹo phát cho mỗi em chỉ được lấy ra từ một thùng kẹo duy nhất (không ghép mảnh kẹo từ nhiều thùng khác nhau). Hãy tìm số lượng kẹo X lớn nhất có thể phát cho mỗi em.



Hình minh họa: Thuật toán tìm kiếm nhị phân chia đôi không gian $O(\log N)$

Nhiệm vụ: Cho N gói kẹo và số học sinh K . Mỗi học sinh chỉ nhận kẹo từ cùng 1 gói. Hãy tìm số kẹo X lớn nhất phát đều cho K học sinh.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và K ($1 \leq N \leq 10^5, 1 \leq K \leq 10^9$).
- Dòng 2: Chứa N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra số kẹo lớn nhất X có thể chia cho mỗi học sinh. Nếu không chia được chiếc nào, in 0.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 6 15 8 10 7	5

Giải thích: Nếu mỗi em nhận $X = 5$ chiếc kẹo:

- Thùng 1 (15 kẹo) chia được $15/5 = 3$ em.
- Thùng 2 (8 kẹo) chia được $8/5 = 1$ em.
- Thùng 3 (10 kẹo) chia được $10/5 = 2$ em.
- Thùng 4 (7 kẹo) chia được $7/5 = 1$ em. Tổng số em được nhận là $3 + 1 + 2 + 1 = 7 \geq 6$ em. Nếu tăng $X = 6$ sẽ không đủ 6 phần. Vậy X lớn nhất là 5.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho N gói kẹo và số học sinh K . Mỗi học sinh chỉ nhận kẹo từ cùng 1 gói. Hãy tìm số kẹo X lớn nhất phát đều cho K học sinh.

- **Phương pháp tiếp cận — Tìm kiếm nhị phân:**

- Nhận diện tính đơn điệu của hàm mục tiêu hoặc không gian tìm kiếm.
- Thu hẹp không gian nghiệm $[L, R]$ qua điểm giữa $mid = L + (R - L)/2$. Độ phức tạp thời gian đạt $\mathcal{O}(\log N)$ hoặc $\mathcal{O}(N \log(\text{range}))$.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 4 6 15 8 10 7 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Nếu mỗi em nhận $X = 5$ chiếc kẹo: - Thùng 1 (15 kẹo) chia được $15/5 = 3$ em. - Thùng 2 (8 kẹo) chia được $8/5 = 1$ em. - Thùng... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 5 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Nếu mỗi em nhận $X = 5$ chiếc kẹo:

- Thùng 1 (15 kẹo) chia được $15/5 = 3$ em.
- Thùng 2 (8 kẹo) chia được $8/5 = 1$ em.
- Thùng 3 (10 kẹo) chia được $10/5 = 2$ em.
- Thùng 4 (7 kẹo) chia được $7/5 = 1$ em. Tổng số em được nhận là $3 + 1 + 2 + 1 = 7 \geq 6$ em. Nếu tăng $X = 6$ sẽ không đủ 6 phần. Vậy X lớn nhất là 5.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

bool check(long long mid, const vector<long long>& a, long long k) {
    long long count = 0;
    for (long long x : a) {
        count += (x / mid);
    }
    return count >= k;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long k;
    if (!(cin >> n >> k)) return 0;

    vector<long long> a(n);
    long long max_val = 0;
```

```

for (int i = 0; i < n; ++i) {
    cin >> a[i];
    max_val = max(max_val, a[i]);
}

long long low = 1, high = max_val, ans = 0;
while (low <= high) {
    long long mid = low + (high - low) / 2;
    if (check(mid, a, k)) {
        ans = mid;
        low = mid + 1;
    } else {
        high = mid - 1;
    }
}

cout << ans << "\n";
return 0;
}

```

Bài 07 [cppb_bs_07_cat_go_xay_dung]: Cắt Gỗ Xây Dựng (Woodcutting / EKO)

Bối cảnh: Bác thợ mộc nhận hợp đồng cung ứng gỗ xây dựng trường học vùng cao và cần thu hoạch ít nhất M mét gỗ. Khu rừng có N cây cổ thụ với chiều cao lần lượt là $H_1, H_2, \dots, H_N$. Máy cắt gỗ công nghiệp có thể thiết lập độ cao của H : tất cả các cây có chiều cao lớn hơn H sẽ bị cưa phần ngọn thừa ra (chiều dài gỗ thu được từ mỗi cây là $H_i - H$ nếu $H_i > H$). Để bảo vệ tài nguyên rừng, bác thợ mộc muốn đặt độ cao của H lớn nhất có thể sao cho tổng lượng gỗ thu được vẫn đạt ít nhất M mét.



Hình minh họa: Thuật toán tìm kiếm nhị phân chia đôi không gian $O(\log N)$

Nhiệm vụ: Cho chiều cao N cây gỗ và lượng gỗ tối thiểu cần lấy M . Hãy tìm độ cao cắt H lớn nhất của máy cưa.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và M ($1 \leq N \leq 10^6, 1 \leq M \leq 2 \cdot 10^9$).
- Dòng 2: Chứa N số nguyên dương $H_1, H_2, \dots, H_N$ ($1 \leq H_i \leq 10^9$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là độ cao cắt H lớn nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 7 20 15 10 17	15

Giải thích: Khi đặt độ cao cắt $H = 15$:

- Cây 20m cắt được: $20 - 15 = 5\text{m}$.
- Cây 15m cắt được: $15 - 15 = 0\text{m}$.
- Cây 10m không bị cắt: 0m .
- Cây 17m cắt được: $17 - 15 = 2\text{m}$. Tổng gỗ thu được là $5 + 0 + 0 + 2 = 7$ mét đúng bằng M . Đây là độ cao H lớn nhất.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho chiều cao N cây gỗ và lượng gỗ tối thiểu cần lấy M . Hãy tìm độ cao cắt H lớn nhất của máy cưa.
- **Phương pháp tiếp cận — Tìm kiếm nhị phân:**
- Nhận diện tính đơn điệu của hàm mục tiêu hoặc không gian tìm kiếm.
- Thu hẹp không gian nghiệm $[L, R]$ qua điểm giữa $mid = L + (R - L)/2$. Độ phức tạp thời gian đạt $\mathcal{O}(\log N)$ hoặc $\mathcal{O}(N \log(\text{range}))$.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 4 7 20 15 10 17 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Khi đặt độ cao cắt $H = 15$: - Cây 20m cắt được: $20 - 15 = 5\text{m}$. - Cây 15m cắt được: $15 - 15 = 0\text{m}$. - Cây 10m không bị cắt: 0m . | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 15 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Khi đặt độ cao cắt $H = 15$:

- Cây 20m cắt được: $20 - 15 = 5\text{m}$.
- Cây 15m cắt được: $15 - 15 = 0\text{m}$.
- Cây 10m không bị cắt: 0m .
- Cây 17m cắt được: $17 - 15 = 2\text{m}$. Tổng gỗ thu được là $5 + 0 + 0 + 2 = 7$ mét đúng bằng M . Đây là độ cao H lớn nhất.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

bool check(long long h, const vector<long long>& a, long long m) {
    long long wood = 0;
    for (long long x : a) {
        if (x > h) {
            wood += (x - h);
        }
    }
    return wood >= m;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long m;
    if (!(cin >> n >> m)) return 0;

    vector<long long> a(n);
    long long max_val = 0;
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
        max_val = max(max_val, a[i]);
    }

    long long low = 0, high = max_val, ans = 0;
    while (low <= high) {
        long long mid = low + (high - low) / 2;
        if (check(mid, a, m)) {
            ans = mid;
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    cout << ans << "\n";
    return 0;
}
```

Bài 08 [cppb_bs_08_dat_tram_phat_song]: Đặt Trạm Phát Sóng Cách Nhau Xa Nhất (Aggressive Cows)

Bối cảnh: Dọc theo một rặng núi thẳng dài, ban viễn thông quân sự khảo sát được N vị trí địa lý thuận lợi có thể lắp đặt trạm phát sóng tại các tọa độ $X_1, X_2, \dots, X_n$. Đơn vị cần chọn ra đúng C vị trí để lắp đặt C trạm phát sóng sao cho khoảng cách giữa hai trạm gần nhau nhất là lớn nhất có thể nhằm tránh tối đa hiện tượng giao thoa sóng vô tuyến cực ngắn.



Hình minh họa: Thuật toán tìm kiếm nhị phân chia đôi không gian $O(\log N)$

Nhiệm vụ: Cho N vị trí khả dụng và số trạm cần đặt C . Hãy tìm khoảng cách nhỏ nhất lớn nhất giữa hai trạm bất kỳ.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và C ($2 \leq C \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên không âm $X_1, X_2, \dots, X_N$ ($0 \leq X_i \leq 10^9$).

Đầu ra (Output):

- In ra khoảng cách nhỏ nhất lớn nhất có thể đạt được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 2 8 4 9	3

Giải thích: Sắp xếp tọa độ các vị trí: $[1, 2, 4, 8, 9]$. Để đặt 3 trạm với khoảng cách tối thiểu giữa hai trạm kề nhau là 3: ta đặt tại các tọa độ 1, 4 và 8 (hoặc 9). Khoảng cách giữa 1 và 4 là 3; giữa 4 và 8 là 4 (đều ≥ 3). Không thể đặt với khoảng cách tối thiểu ≥ 4 . Vì vậy kết quả là 3.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho N vị trí khả dụng và số trạm cần đặt C . Hãy tìm khoảng cách nhỏ nhất lớn nhất giữa hai trạm bất kỳ.
- **Phương pháp tiếp cận — Tìm kiếm nhị phân:**
- Nhận diện tính đơn điệu của hàm mục tiêu hoặc không gian tìm kiếm.
- Thu hẹp không gian nghiệm $[L, R]$ qua điểm giữa $mid = L + (R - L)/2$. Độ phức tạp thời gian đạt $O(\log N)$ hoặc $O(N \log(\text{range}))$.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 3 1 2 8 4 9 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Sắp xếp tọa độ các vị trí: $[1, 2, 4, 8, 9]$. Để đặt 3 trạm với

khoảng cách tối thiểu giữa hai trạm kề nhau là 3: ta đặt t... | Tính toán từng bước trạng thái | | 3 |
 Xuất kết quả | Output: 3 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Sắp xếp tọa độ các vị trí: [1, 2, 4, 8, 9]. Để đặt 3 trạm với khoảng cách tối thiểu giữa hai trạm kề nhau là 3: ta đặt tại các tọa độ 1, 4 và 8 (hoặc 9). Khoảng cách giữa 1 và 4 là 3; giữa 4 và 8 là 4 (đều ≥ 3). Không thể đặt với khoảng cách tối thiểu ≥ 4 . Vì vậy kết quả là 3.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

bool check(long long d, const vector<long long>& x, int c) {
    int count = 1;
    long long last_pos = x[0];
    for (size_t i = 1; i < x.size(); ++i) {
        if (x[i] - last_pos >= d) {
            count++;
            last_pos = x[i];
        }
    }
    return count >= c;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, c;
    if (!(cin >> n >> c)) return 0;

    vector<long long> x(n);
    for (int i = 0; i < n; ++i) {
        cin >> x[i];
    }
    sort(x.begin(), x.end());

    long long low = 1, high = x[n - 1] - x[0], ans = 1;
    while (low <= high) {
        long long mid = low + (high - low) / 2;
        if (check(mid, x, c)) {
            ans = mid;
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
}
```

```
cout << ans << "\n";
return 0;
}
```

Bài 09 [cppb_bs_09_chia_mang_tong_max_nho_nhat]: Chia Mảng Thành K Đoạn Có Tổng Max Nhỏ Nhất

Bối cảnh: Một dự án phần mềm gồm N module công việc với thời gian thực hiện lần lượt là $A_1, A_2, \dots, A_N$ theo đúng quy trình tuần tự. Dự án được phân chia cho K nhóm lập trình viên độc lập làm việc song song, mỗi nhóm phụ trách một phân đoạn các module liên tiếp nhau. Để cả dự án hoàn thành sớm nhất, người quản lý cần phân chia sao cho thời gian làm việc của nhóm có khối lượng công việc lớn nhất (tổng thời gian lớn nhất) là nhỏ nhất có thể.



Hình minh họa: Thuật toán tìm kiếm nhị phân chia đôi không gian $O(\log N)$

Nhiệm vụ: Cho mảng N số nguyên dương và số nguyên K . Hãy chia mảng thành K đoạn con liên tiếp sao cho tổng lớn nhất trong các đoạn là nhỏ nhất có thể.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và K ($1 \leq K \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra giá trị tổng đoạn con lớn nhất nhỏ nhất có thể.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 2 7 2 5 10 8	18

Giải thích: Chia thành 2 đoạn con: đoạn 1 là $[7, 2, 5]$ có tổng 14; đoạn 2 là $[10, 8]$ có tổng 18. Tổng lớn nhất giữa hai đoạn là 18. Không thể chia cách nào khác để có tổng cực đại nhỏ hơn 18. Vì vậy kết quả là 18.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng N số nguyên dương và số nguyên K . Hãy chia mảng thành K đoạn con liên tiếp sao cho tổng lớn nhất trong các đoạn là nhỏ nhất có thể.

- **Phương pháp tiếp cận — Tìm kiếm nhị phân:**

- Nhận diện tính đơn điệu của hàm mục tiêu hoặc không gian tìm kiếm.
- Thu hẹp không gian nghiệm $[L, R]$ qua điểm giữa $mid = L + (R - L)/2$. Độ phức tạp thời gian đạt $\mathcal{O}(\log N)$ hoặc $\mathcal{O}(N \log(\text{range}))$.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 2 7 2 5 10 8 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Chia thành 2 đoạn con: đoạn 1 là [7, 2, 5] có tổng 14; đoạn 2 là [10, 8] có tổng 18. Tổng lớn nhất giữa hai đoạn là 18. ... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 18 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Chia thành 2 đoạn con: đoạn 1 là [7, 2, 5] có tổng 14; đoạn 2 là [10, 8] có tổng 18. Tổng lớn nhất giữa hai đoạn là 18. Không thể chia cách nào khác để có tổng cực đại nhỏ hơn 18. Vì vậy kết quả là 18.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

bool check(long long limit, const vector<long long>& a, int k) {
    int segments = 1;
    long long current_sum = 0;
    for (long long x : a) {
        if (current_sum + x > limit) {
            segments++;
            current_sum = x;
        } else {
            current_sum += x;
        }
    }
    return segments <= k;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    if (!(cin >> n >> k)) return 0;

    vector<long long> a(n);
    long long max_val = 0, total_sum = 0;
```

```

for (int i = 0; i < n; ++i) {
    cin >> a[i];
    max_val = max(max_val, a[i]);
    total_sum += a[i];
}

long long low = max_val, high = total_sum, ans = total_sum;
while (low <= high) {
    long long mid = low + (high - low) / 2;
    if (check(mid, a, k)) {
        ans = mid;
        high = mid - 1;
    } else {
        low = mid + 1;
    }
}

cout << ans << "\n";
return 0;
}

```

Bài 10 [cppb_bs_10_van_chuyen_hang_hoa]: Vận Chuyển Hàng Hóa Qua Phà Trong D Ngày

Bối cảnh: Tại một bến cảng trung chuyển đường thủy, có N kiện hàng được xếp thành một hàng dài trên băng chuyền theo thứ tự nghiêm ngặt với trọng lượng $W_1, W_2, \dots, W_n$. Chiếc phà vận tải phải chở hết toàn bộ các kiện hàng này sang bờ bên kia trong thời gian đúng D ngày (mỗi ngày phà chỉ bốc xếp một chuỗi các kiện hàng liên tiếp nhau theo thứ tự băng tải). Hãy tính tải trọng tối thiểu của phà để hoàn thành nhiệm vụ trong đúng D ngày.



Hình minh họa: Thuật toán tìm kiếm nhị phân chia đôi không gian $O(\log N)$

Nhiệm vụ: Cho trọng lượng N kiện hàng và số ngày D . Tìm tải trọng nhỏ nhất của phà để chở hết hàng trong D ngày.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và D ($1 \leq D \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên dương $W_1, W_2, \dots, W_N$ ($1 \leq W_i \leq 10^9$).

Đầu ra (Output):

- In ra tải trọng tối thiểu của phà.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
10 5 1 2 3 4 5 6 7 8 9 10	15

Giải thích: Với tải trọng 15:

- Ngày 1: chở [1, 2, 3, 4, 5] (tổng 15)
- Ngày 2: chở [6, 7] (tổng 13)
- Ngày 3: chở [8] (tổng 8)
- Ngày 4: chở [9] (tổng 9)
- Ngày 5: chở [10] (tổng 10) Tổng cộng 5 ngày chở hết 10 kiện hàng. Tải trọng nhỏ nhất là 15.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho trọng lượng N kiện hàng và số ngày D . Tìm tải trọng nhỏ nhất của phà để chở hết hàng trong D ngày.
- **Phương pháp tiếp cận — Tìm kiếm nhị phân:**
- Nhận diện tính đơn điệu của hàm mục tiêu hoặc không gian tìm kiếm.
- Thu hẹp không gian nghiệm $[L, R]$ qua điểm giữa $mid = L + (R - L)/2$. Độ phức tạp thời gian đạt $\mathcal{O}(\log N)$ hoặc $\mathcal{O}(N \log(\text{range}))$.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 10 5 1 2 3 4 5 6 7 8 9 10 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Với tải trọng 15: - Ngày 1: chở [1, 2, 3, 4, 5] (tổng 15) - Ngày 2: chở [6, 7] (tổng 13) - Ngày 3: chở [8] (tổng 8) - Ng... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 15 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Với tải trọng 15:

- Ngày 1: chở [1, 2, 3, 4, 5] (tổng 15)
- Ngày 2: chở [6, 7] (tổng 13)
- Ngày 3: chở [8] (tổng 8)
- Ngày 4: chở [9] (tổng 9)
- Ngày 5: chở [10] (tổng 10) Tổng cộng 5 ngày chở hết 10 kiện hàng. Tải trọng nhỏ nhất là 15.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. $N-1$) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

bool check(long long cap, const vector<long long>& w, int d) {
    int days = 1;
    long long current_weight = 0;
    for (long long x : w) {
        if (current_weight + x > cap) {
            days++;
            current_weight = x;
        } else {
            current_weight += x;
        }
    }
    return days <= d;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, d;
    if (!(cin >> n >> d)) return 0;

    vector<long long> w(n);
    long long max_w = 0, sum_w = 0;
    for (int i = 0; i < n; ++i) {
        cin >> w[i];
        max_w = max(max_w, w[i]);
        sum_w += w[i];
    }

    long long low = max_w, high = sum_w, ans = sum_w;
    while (low <= high) {
        long long mid = low + (high - low) / 2;
        if (check(mid, w, d)) {
            ans = mid;
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }

    cout << ans << "\n";
    return 0;
}
```

Bài 11 [cppb_bs_11_nghiem_thuc_phuong_trinh]: Tìm Nghiệm Thực Của Phương Trình Đơn Điệu

Bối cảnh: Trong mô phỏng khí động lực học của tên lửa đẩy, hàm tiêu hao nhiên liệu theo thời gian tuân theo phương trình bậc ba đơn điệu tăng ngặt: $f(x) = x^3 + 2x^2 + 10x - C = 0$ (với $x \geq 0$ và C là hằng số tiêu hao năng lượng). Các kỹ sư cần xác định thời điểm thực x với độ chính xác cao (sai số tuyệt đối không quá 10^{-6}).



Hình minh họa: Thuật toán tìm kiếm nhị phân chia đôi không gian $O(\log N)$

Nhiệm vụ: Cho số thực C dương ($1 \leq C \leq 10^9$). Hãy tìm nghiệm thực dương x của phương trình $x^3 + 2x^2 + 10x - C = 0$ với độ chính xác 6 chữ số thập phân.

Đầu vào (Input):

- Một dòng chứa số thực C ($1 \leq C \leq 10^9$).

Đầu ra (Output):

- In ra nghiệm thực x lấy đúng 6 chữ số sau dấu phẩy.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
20.0	1.233519

Giải thích: Thay $x = 1.233519$ vào $f(x)$: $1.233519^3 + 2 \cdot 1.233519^2 + 10 \cdot 1.233519 - 20 = 0.000000$. Nghiệm chính xác đến 6 chữ số thập phân là 1.233519.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số thực C dương ($1 \leq C \leq 10^9$). Hãy tìm nghiệm thực dương x của phương trình $x^3 + 2x^2 + 10x - C = 0$ với độ chính xác 6 chữ số thập phân.
- Phương pháp tiếp cận — Tìm kiếm nhị phân:**
- Nhận diện tính đơn điệu của hàm mục tiêu hoặc không gian tìm kiếm.
- Thu hẹp không gian nghiệm $[L, R]$ qua điểm giữa $mid = L + (R - L)/2$. Độ phức tạp thời gian đạt $O(\log N)$ hoặc $O(N \log(\text{range}))$.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 20.0 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Thay $x = 1.233519$ vào $f(x)$: $1.233519^3 + 2 \cdot 1.233519^2 + 10 \cdot 1.233519 - 20 = 0.000000$. Nghiệm chính xác đến 6 chữ số thập ... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 1.233519 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Thay $x = 1.233519$ vào $f(x)$: $1.233519^3 + 21.233519^2 + 101.233519 - 20 = 0.000000$. Nghiệm chính xác đến 6 chữ số thập phân là 1.233519 .

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

double f(double x) {
    return x * x * x + 2.0 * x * x + 10.0 * x;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    double c;
    if (!(cin >> c)) return 0;

    double low = 0.0, high = 1000.0;
    for (int iter = 0; iter < 100; ++iter) {
        double mid = low + (high - low) / 2.0;
        if (f(mid) >= c) {
            high = mid;
        } else {
            low = mid;
        }
    }

    cout << fixed << setprecision(6) << low << "\n";
    return 0;
}
```

Bài 12 [cppb_bs_12_phan_tu_thu_k_hai_mang]: Phần Tử Thứ K Của Hai Mảng Đã Sắp Xếp

Bối cảnh: Hai sàn thương mại điện tử lớn nhất khu vực có danh sách giá các mặt hàng công nghệ đã được sắp xếp tăng dần: sàn A gồm N món đồ và sàn B gồm M món đồ. Người tiêu dùng muốn biết nếu gộp toàn bộ danh sách hàng hóa của hai sàn lại với nhau thành một mảng có thứ tự thì mặt hàng rẻ thứ K trên thị trường chung có giá trị là bao nhiêu.



Hình minh họa: Thuật toán tìm kiếm nhị phân chia đôi không gian $O(\log N)$

Nhiệm vụ: Cho hai mảng số nguyên đã sắp xếp A (kích thước N) và B (kích thước M) cùng số nguyên dương K ($1 \leq K \leq N + M$). Hãy tìm giá trị của phần tử đứng ở vị trí thứ K sau khi hợp nhất hai mảng trong thời gian $O(\log(\min(N, M)))$.

Đầu vào (Input):

- Dòng 1: Chứa 3 số nguyên N, M, K ($1 \leq N, M \leq 10^5, 1 \leq K \leq N + M$).
- Dòng 2: Chứa N số nguyên tăng dần của mảng A .
- Dòng 3: Chứa M số nguyên tăng dần của mảng B .

Đầu ra (Output):

- In ra giá trị phần tử thứ K .

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 4 5 2 3 6 7 9 1 4 8 10	6

Giải thích: Hợp nhất hai mảng có thứ tự: $[1, 2, 3, 4, 6, 7, 8, 9, 10]$. Phần tử đứng thứ $K = 5$ là số 6. Kết quả in ra: 6.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho hai mảng số nguyên đã sắp xếp A (kích thước N) và B (kích thước M) cùng số nguyên dương K ($1 \leq K \leq N + M$). Hãy tìm giá trị của phần tử đứng ở vị trí thứ K sau khi hợp nhất hai mảng trong thời gian $O(\log(\min(N, M)))$.
- **Phương pháp tiếp cận — Tìm kiếm nhị phân:**
 - Nhận diện tính đơn điệu của hàm mục tiêu hoặc không gian tìm kiếm.
 - Thu hẹp không gian nghiệm $[L, R]$ qua điểm giữa $mid = L + (R - L)/2$. Độ phức tạp thời gian đạt $O(\log N)$ hoặc $O(N \log(\text{range}))$.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nhập dữ liệu vào mảng/biến | Input: 5 4 5 2 3 6 7 9 1 4 8 10 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Hợp nhất hai mảng có thứ tự: [1, 2, 3, 4, 6, 7, 8, 9, 10]. Phần tử đứng thứ $K = 5$ là số 6. Kết quả in ra: 6... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 6 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Hợp nhất hai mảng có thứ tự: [1, 2, 3, 4, 6, 7, 8, 9, 10]. Phần tử đứng thứ $K = 5$ là số 6. Kết quả in ra: 6.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

long long count_le(long long x, const vector<long long>& a, const vector<long long>& b) {
    auto it1 = upper_bound(a.begin(), a.end(), x);
    auto it2 = upper_bound(b.begin(), b.end(), x);
    return (it1 - a.begin()) + (it2 - b.begin());
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    long long k;
    if (!(cin >> n >> m >> k)) return 0;

    vector<long long> a(n), b(m);
    for (int i = 0; i < n; ++i) cin >> a[i];
    for (int i = 0; i < m; ++i) cin >> b[i];

    long long low = -2e9, high = 2e9, ans = high;
    while (low <= high) {
        long long mid = low + (high - low) / 2;
        if (count_le(mid, a, b) >= k) {
            ans = mid;
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }

    cout << ans << "\n";
    return 0;
}
```

Bài 13 [cppb_bs_13_trung_binh_lon_nhat_do_dai_k]: Tìm Đoạn Con Có Trung Bình Lớn Nhất Độ Dài $\geq K$

Bối cảnh: Trong đánh giá hiệu quả chuỗi cung ứng, chuyên gia tài chính theo dõi chuỗi doanh thu N ngày $A_1, A_2, \dots, A_N$. Một giai đoạn tăng trưởng bền vững bắt buộc phải kéo dài ít nhất K ngày liên tiếp. Chuyên gia muốn tìm xem giá trị doanh thu trung bình lớn nhất của một giai đoạn kéo dài tối thiểu K ngày có thể đạt tới mức nào.



Hình minh họa: Thuật toán tìm kiếm nhị phân chia đôi không gian $O(\log N)$

Nhiệm vụ: Cho dãy số nguyên gồm N phần tử và số nguyên K . Hãy tìm giá trị trung bình lớn nhất của một đoạn con có độ dài ít nhất K , làm tròn đến đúng 3 chữ số thập phân.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và K ($1 \leq K \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra giá trị trung bình lớn nhất tìm được với 3 chữ số sau dấu phẩy.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 2 1 12 -5 -6	6.500

Giải thích: Đoạn con $[1, 12]$ có độ dài $2 \geq K = 2$ có tổng 13 và trung bình là $13/2 = 6.500$. Đây là giá trị trung bình lớn nhất của các đoạn con có độ dài ≥ 2 .

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho dãy số nguyên gồm N phần tử và số nguyên K . Hãy tìm giá trị trung bình lớn nhất của một đoạn con có độ dài ít nhất K , làm tròn đến đúng 3 chữ số thập phân.
- Phương pháp tiếp cận — Tìm kiếm nhị phân:**
- Nhận diện tính đơn điệu của hàm mục tiêu hoặc không gian tìm kiếm.
- Thu hẹp không gian nghiệm $[L, R]$ qua điểm giữa $mid = L + (R - L)/2$. Độ phức tạp thời gian đạt $O(\log N)$ hoặc $O(N \log(\text{range}))$.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nhập dữ liệu vào mảng/biến | Input: 4 2 1 12 -5 -6 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Đoạn con [1, 12] có độ dài $2 \geq K = 2$ có tổng 13 và trung bình là $13/2 = 6.500$. Đây là giá trị trung bình lớn nhất của c... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 6.500 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Đoạn con [1, 12] có độ dài $2 \geq K = 2$ có tổng 13 và trung bình là $13/2 = 6.500$. Đây là giá trị trung bình lớn nhất của các đoạn con có độ dài ≥ 2 .

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

bool check(double mid, const vector<double>& a, int n, int k) {
    vector<double> p(n + 1, 0.0);
    for (int i = 1; i <= n; ++i) {
        p[i] = p[i - 1] + (a[i - 1] - mid);
    }

    double min_p = 0.0;
    for (int i = k; i <= n; ++i) {
        min_p = min(min_p, p[i - k]);
        if (p[i] - min_p >= -1e-9) {
            return true;
        }
    }
    return false;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    if (!(cin >> n >> k)) return 0;

    vector<double> a(n);
    double max_val = 0.0;
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
        max_val = max(max_val, a[i]);
    }

    double low = 0.0, high = max_val;
    for (int iter = 0; iter < 80; ++iter) {
        double mid = low + (high - low) / 2.0;
```

```

        if (check(mid, a, n, k)) {
            low = mid;
        } else {
            high = mid;
        }
    }

    cout << fixed << setprecision(4) << low << "\n";
    return 0;
}

```

Bài 14 [cppb_bs_14_tuyen_duong_van_tai]: Tối Ưu Hóa Tuyển Đường Vận Tải Đa Điểm

Bối cảnh: Trên trục đường liên tỉnh dài, có N thành phố nối tiếp nhau từ 1 đến N . Tại thành phố thứ i có nhu cầu tiếp nhận A_i tấn hàng hóa. Một đội tàu vận tải gồm K đoàn tàu xuất phát từ ga đầu mỗi cần chia sẻ vận chuyển hàng đến các thành phố. Để không gây quá tải cho các đoàn tàu, cơ quan điều vận cần tìm mức tải trọng trần nhỏ nhất để K đoàn tàu vận chuyển trọn vẹn toàn bộ hàng hóa.



Hình minh họa: Thuật toán tìm kiếm nhị phân chia đôi không gian $O(\log N)$

Nhiệm vụ: Cho mảng N số nguyên và số nguyên K . Hãy tìm giá trị cận trên tải trọng nhỏ nhất cho mỗi đoàn tàu.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và K ($1 \leq K \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra giá trị cận trên tải trọng tối ưu nhỏ nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 2 1 2 3 4	6

Giải thích: Chia thành 2 đoàn: $[1, 2, 3]$ có tổng 6 và $[4]$ có tổng 4. Mức tải trọng lớn nhất giữa hai đoàn là 6. Đây là mức tải trọng trần nhỏ nhất có thể đạt được.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng N số nguyên và số nguyên K . Hãy tìm giá trị cận trên tải trọng nhỏ nhất cho mỗi đoàn tàu.
- **Phương pháp tiếp cận — Tìm kiếm nhị phân:**
- Nhận diện tính đơn điệu của hàm mục tiêu hoặc không gian tìm kiếm.
- Thu hẹp không gian nghiệm $[L, R]$ qua điểm giữa $mid = L + (R - L)/2$. Độ phức tạp thời gian đạt $\mathcal{O}(\log N)$ hoặc $\mathcal{O}(N \log(\text{range}))$.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 4 2 1 2 3 4 | Khởi tạo cấu trúc dữ liệu ban đầu
| 2 | Thực thi thuật toán tối ưu | Chia thành 2 đoàn: [1, 2, 3] có tổng 6 và [4] có tổng 4. Mức tải trọng lớn nhất giữa hai đoàn là 6. Đây là mức tải trọng... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 6 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Chia thành 2 đoàn: [1, 2, 3] có tổng 6 và [4] có tổng 4. Mức tải trọng lớn nhất giữa hai đoàn là 6. Đây là mức tải trọng trần nhỏ nhất có thể đạt được.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

bool check(long long cap, const vector<long long>& a, int n, int m, int d) {
    int trucks = 0;
    int i = 0;
    while (i < n) {
        trucks++;
        if (trucks > m) return false;
        long long current_load = 0;
        int count_cities = 0;
        while (i < n && count_cities < d && current_load + a[i] <= cap) {
            current_load += a[i];
            count_cities++;
            i++;
        }
    }
    return trucks <= m;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```
int n, m, d;
if (!(cin >> n >> m >> d)) return 0;

vector<long long> a(n);
long long max_val = 0, sum_val = 0;
for (int i = 0; i < n; ++i) {
    cin >> a[i];
    max_val = max(max_val, a[i]);
    sum_val += a[i];
}

long long low = max_val, high = sum_val, ans = sum_val;
while (low <= high) {
    long long mid = low + (high - low) / 2;
    if (check(mid, a, n, m, d)) {
        ans = mid;
        high = mid - 1;
    } else {
        low = mid + 1;
    }
}

cout << ans << "\n";
return 0;
}
```

Bài 15 [cppb_bs_15_tim_kiem_mang_xoay_vong]: Tìm Kiếm Trên Mảng Sắp Xếp Bị Xoay Vòng (Rotated Array)

Bối cảnh: Trong bộ nhớ vòng (ring buffer) của một hệ thống xử lý camera an ninh, danh sách N chỉ số thời gian ban đầu được sắp xếp tăng dần nhưng sau một số chu kỳ ghi đè đã bị xoay vòng tại một trục xoay k nào đó. Hệ thống nhận Q yêu cầu tra cứu mã mốc thời gian X xem nó nằm ở vị trí nào trong bộ nhớ.



Hình minh họa: Thuật toán tìm kiếm nhị phân chia đôi không gian $O(\log N)$

Nhiệm vụ: Cho mảng N số nguyên phân biệt bị xoay vòng tại một trục không xác định. Có Q truy vấn tìm vị trí xuất hiện (0-indexed) của số nguyên X . Nếu không tìm thấy, in ra -1.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: Chứa N số nguyên phân biệt bị xoay vòng.

- Q dòng tiếp theo: Mỗi dòng chứa một số nguyên X .

Đầu ra (Output):

- In ra Q dòng, mỗi dòng là chỉ số (0-indexed) của X , hoặc -1.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
7 3 4 5 6 7 0 1 2 0 3 5	4 -1 1

Giải thích: Mảng bị xoay vòng [4, 5, 6, 7, 0, 1, 2]:

- Số 0 ở vị trí chỉ số 4.
- Số 3 không tồn tại -> -1.
- Số 5 ở vị trí chỉ số 1.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng N số nguyên phân biệt bị xoay vòng tại một trục không xác định. Có Q truy vấn tìm vị trí xuất hiện (0-indexed) của số nguyên X . Nếu không tìm thấy, in ra -1.
- **Phương pháp tiếp cận — Tìm kiếm nhị phân:**
- Nhận diện tính đơn điệu của hàm mục tiêu hoặc không gian tìm kiếm.
- Thu hẹp không gian nghiệm $[L, R]$ qua điểm giữa $mid = L + (R - L)/2$. Độ phức tạp thời gian đạt $\mathcal{O}(\log N)$ hoặc $\mathcal{O}(N \log(\text{range}))$.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| — | — | — | — | — | 1 | Nạp dữ liệu vào mảng/biến | Input: 7 3 4 5 6 7 0 1 2 0 3 5 | Khởi tạo cấu trúc dữ liệu ban đầu | 2 | Thực thi thuật toán tối ưu | Mảng bị xoay vòng [4, 5, 6, 7, 0, 1, 2]: - Số 0 ở vị trí chỉ số 4. - Số 3 không tồn tại -> -1. - Số 5 ở vị trí chỉ số 1... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 4 -1 1 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Mảng bị xoay vòng [4, 5, 6, 7, 0, 1, 2]:

- Số 0 ở vị trí chỉ số 4.
- Số 3 không tồn tại -> -1.
- Số 5 ở vị trí chỉ số 1.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. $N-1$) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int search_rotated(const vector<long long>& a, long long target) {
    int low = 0, high = (int)a.size() - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == target) return mid + 1; // 1-based

        if (a[low] <= a[mid]) {
            // Nửa trái được sắp xếp
            if (a[low] <= target && target < a[mid]) {
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        } else {
            // Nửa phải được sắp xếp
            if (a[mid] < target && target <= a[high]) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
    }
    return -1;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    while (q--) {
        long long x;
        cin >> x;
        cout << search_rotated(a, x) << "\n";
    }

    return 0;
}
```

Bài 16 [cppb_bs_16_tim_kiem_ma_tran_2d]: Tìm Kiếm Trên Ma Trận 2D Đã Sắp Xếp (Matrix Search)

Bối cảnh: Một bảng cơ sở dữ liệu dạng ma trận kích thước $N \times M$ ô chứa các bản ghi mã định danh tài khoản, trong đó mỗi hàng được sắp xếp tăng dần từ trái qua phải, và phần tử đầu tiên của mỗi hàng lớn hơn phần tử cuối cùng của hàng trước đó. Hãy kiểm tra xem tài khoản có mã X có tồn tại trong cơ sở dữ liệu hay không.



Hình minh họa: Thuật toán tìm kiếm nhị phân chia đôi không gian $O(\log N)$

Nhiệm vụ: Cho ma trận $N \times M$ đã sắp xếp theo quy tắc trên và số nguyên X . Hãy kiểm tra xem X có tồn tại trong ma trận không. In YES nếu có, ngược lại in NO.

Đầu vào (Input):

- Dòng 1: Chứa 3 số nguyên N, M, X ($1 \leq N, M \leq 1000, -10^9 \leq X \leq 10^9$).
- N dòng tiếp theo: Mỗi dòng gồm M số nguyên đã sắp xếp.

Đầu ra (Output):

- In ra YES hoặc NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 4 3 1 3 5 7 10 11 16 20 23 30 34 60	YES

Giải thích: Số $X = 3$ nằm ở hàng 1, cột 2 của ma trận $\rightarrow$ in YES.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho ma trận $N \times M$ đã sắp xếp theo quy tắc trên và số nguyên X . Hãy kiểm tra xem X có tồn tại trong ma trận không. In YES nếu có, ngược lại in NO.
- **Phương pháp tiếp cận — Tìm kiếm nhị phân:**
- Nhận diện tính đơn điệu của hàm mục tiêu hoặc không gian tìm kiếm.
- Thu hẹp không gian nghiệm $[L, R]$ qua điểm giữa $mid = L + (R - L)/2$. Độ phức tạp thời gian đạt $O(\log N)$ hoặc $O(N \log(\text{range}))$.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 4 3 1 3 5 7 10 11 16 20 23 30 34 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Số $X = 3$ nằm ở hàng 1, cột 2 của ma trận $\rightarrow$ in YES.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: YES | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Số $X = 3$ nằm ở hàng 1, cột 2 của ma trận $\rightarrow$ in YES.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, q;
    if (!(cin >> n >> m >> q)) return 0;

    vector<vector<long long>> a(n, vector<long long>(m));
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            cin >> a[i][j];
        }
    }

    while (q--) {
        long long target;
        cin >> target;

        int low = 0, high = n * m - 1;
        bool found = false;

        while (low <= high) {
            int mid = low + (high - low) / 2;
            int r = mid / m;
            int c = mid % m;

            if (a[r][c] == target) {
                found = true;
                break;
            } else if (a[r][c] < target) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }

        cout << (found ? "YES\n" : "NO\n");
    }

    return 0;
}
```

Bài 17 [cppb_bs_17_tim_dinh_day_nui]: Tìm Đỉnh Của Dãy Núi (Peak in Mountain Array)

Bối cảnh: Một thiết bị bay không người lái (drone) bay qua một đỉnh núi cao và ghi nhận liên tục N độ cao địa hình. Dãy độ cao tạo thành dạng dãy núi: ban đầu tăng nghiêm ngặt lên đỉnh cao nhất, sau đó giảm nghiêm ngặt xuống chân núi. Hãy tìm vị trí đỉnh núi (vị trí có độ cao lớn nhất) trong thời gian $O(\log N)$.



Hình minh họa: Thuật toán tìm kiếm nhị phân chia đôi không gian $O(\log N)$

Nhiệm vụ: Cho mảng dãy núi A gồm N phần tử (tăng dần rồi giảm dần). Hãy tìm chỉ số (0-indexed) của phần tử đỉnh núi.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($3 \leq N \leq 10^5$).
- Dòng 2: N số nguyên $A_0, A_1, \dots, A_{N-1}$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra chỉ số (0-indexed) của đỉnh núi.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 0 2 1 0	1

Giải thích: Đỉnh núi có độ cao lớn nhất là 2, nằm ở chỉ số 1 (0-indexed). Kết quả in ra: 1.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng dãy núi A gồm N phần tử (tăng dần rồi giảm dần). Hãy tìm chỉ số (0-indexed) của phần tử đỉnh núi.
- Phương pháp tiếp cận — Tìm kiếm nhị phân:**
- Nhận diện tính đơn điệu của hàm mục tiêu hoặc không gian tìm kiếm.
- Thu hẹp không gian nghiệm $[L, R]$ qua điểm giữa $mid = L + (R - L)/2$. Độ phức tạp thời gian đạt $O(\log N)$ hoặc $O(N \log(\text{range}))$.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 4 0 2 1 0 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Đỉnh núi có độ cao lớn nhất là 2, nằm ở chỉ số 1 (0-indexed). Kết quả in ra: 1.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 1 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Đỉnh núi có độ cao lớn nhất là 2, nằm ở chỉ số 1 (0-indexed). Kết quả in ra: 1.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    int low = 0, high = n - 1;
    while (low < high) {
        int mid = low + (high - low) / 2;
        if (a[mid] < a[mid + 1]) {
            low = mid + 1;
        } else {
            high = mid;
        }
    }

    cout << (low + 1) << "\n"; // 1-based
    return 0;
}
```

Bài 18 [cppb_bs_18_trung_vi_hai_mang_sap_xep]: Trung Vị Của Hai Mảng Đã Sắp Xếp (Median of Two Sorted)

Bối cảnh: Hai trạm y tế thành phố ghi nhận chỉ số nhịp tim của bệnh nhân theo hai mảng số nguyên đã sắp xếp tăng dần: trạm 1 có N người và trạm 2 có M người. Để báo cáo thống kê y tế quốc tế,

ban giám đốc cần tìm giá trị trung vị (median) của tập hợp chung gồm toàn bộ $N + M$ bệnh nhân với độ chính xác cao.



Hình minh họa: Thuật toán tìm kiếm nhị phân chia đôi không gian $O(\log N)$

Nhiệm vụ: Cho hai mảng đã sắp xếp A (kích thước N) và B (kích thước M). Hãy tìm giá trị trung vị của mảng hợp nhất với độ chính xác 1 chữ số thập phân.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và M ($1 \leq N, M \leq 10^5$).
- Dòng 2: Chứa N số nguyên tăng dần của mảng A .
- Dòng 3: Chứa M số nguyên tăng dần của mảng B .

Đầu ra (Output):

- In ra giá trị trung vị làm tròn đúng 1 chữ số thập phân.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
2 2 1 3 2 4	2.5

Giải thích: Mảng hợp nhất: $[1, 2, 3, 4]$ có 4 phần tử. Hai phần tử ở giữa là 2 và 3. Giá trị trung vị là $(2 + 3) / 2 = 2.5$. Kết quả in ra: 2.5.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho hai mảng đã sắp xếp A (kích thước N) và B (kích thước M). Hãy tìm giá trị trung vị của mảng hợp nhất với độ chính xác 1 chữ số thập phân.
- **Phương pháp tiếp cận — Tìm kiếm nhị phân:**
- Nhận diện tính đơn điệu của hàm mục tiêu hoặc không gian tìm kiếm.
- Thu hẹp không gian nghiệm $[L, R]$ qua điểm giữa $mid = L + (R - L) / 2$. Độ phức tạp thời gian đạt $O(\log N)$ hoặc $O(N \log(\text{range}))$.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nhập dữ liệu vào mảng/biến | Input: 2 2 1 3 2 4 | Khởi tạo cấu trúc dữ liệu ban đầu
 || 2 | Thực thi thuật toán tối ưu | Mảng hợp nhất: [1, 2, 3, 4] có 4 phần tử. Hai phần tử ở giữa là 2 và 3. Giá trị trung vị là $(2 + 3) / 2 = 2.5$. Kết quả i... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả
 | Output: 2.5 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Mảng hợp nhất: [1, 2, 3, 4] có 4 phần tử. Hai phần tử ở giữa là 2 và 3. Giá trị trung vị là $(2 + 3) / 2 = 2.5$. Kết quả in ra: 2.5.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

double findMedianSortedArrays(vector<long long>& a, vector<long long>& b) {
    if (a.size() > b.size()) return findMedianSortedArrays(b, a);

    int n = (int)a.size();
    int m = (int)b.size();
    int low = 0, high = n;

    const long long INF = 2e18;

    while (low <= high) {
        int i = low + (high - low) / 2;
        int j = (n + m + 1) / 2 - i;

        long long maxLeftA = (i == 0) ? -INF : a[i - 1];
        long long minRightA = (i == n) ? INF : a[i];

        long long maxLeftB = (j == 0) ? -INF : b[j - 1];
        long long minRightB = (j == m) ? INF : b[j];

        if (maxLeftA <= minRightB && maxLeftB <= minRightA) {
            if ((n + m) % 2 == 1) {
                return (double)max(maxLeftA, maxLeftB);
            } else {
                return (max(maxLeftA, maxLeftB) + min(minRightA, minRightB)) / 2.0;
            }
        } else if (maxLeftA > minRightB) {
            high = i - 1;
        } else {
            low = i + 1;
        }
    }
    return 0.0;
}
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;

    vector<long long> a(n), b(m);
    for (int i = 0; i < n; ++i) cin >> a[i];
    for (int i = 0; i < m; ++i) cin >> b[i];

    double median = findMedianSortedArrays(a, b);
    cout << fixed << setprecision(1) << median << "\n";

    return 0;
}
```



Mục lục

CHƯƠNG 01: NỀN TẢNG LẬP TRÌNH C++	2
• Bài 01: Biến, Kiểu Dữ Liệu, Toán Tử & Nhập Xuất An Toàn	2
• Bài 02: Cấu Trúc Rẽ Nhánh & Cấu Trúc Vòng Lặp	20
• Bài 03: Mảng 1 Chiều, Vector, Xâu Ký Tự & Tổ Chức Hàm.....	40
CHƯƠNG 02: THUẬT TOÁN SẮP XẾP & KỸ THUẬT MẢNG	61
• Bài 04: Thuật toán sắp xếp	61
• Bài 05: Kỹ thuật hai con trỏ.....	93
• Bài 06: Kỹ thuật cửa sổ trượt	123
CHƯƠNG 03: MẢNG TIỀN TỔ & TÌM KIẾM NHỊ PHÂN	151
• Bài 07: Mảng tiền tổ & mảng hiệu	151
• Bài 08: Thuật toán tìm kiếm nhị phân.....	184

